

Core Banking Solution

Scripting Userhooks

10.2.25

Contents

Scripting Userhooks

Repositories and Classes

Userhooks

Account Related Userhooks

Account Balance as on Given Date

Advance Details of an Account in Bancs
Repository

BACID to Account ID and Name

Fetch Account Type

Fetch Multi Currency Account Details

Get Account Number

Get Account Status

Get Account Turnover Details

Get Average Balance of Account

Get FFD Available Amount for Account

Get Full Available Amount

Issue Cheque Leaves

Sum Credit and Debit Transaction Amount
Over a Period for an Account

Sum Transaction Amount for a Specified Account on a Account Label

Getting Account Details into Bancs Repository

Get the Foracid from Acid

Get the Acid from Foracid

Update Lien

Loan Account Interest Details

Get the Float and the Unclear Balance

Update the ECS Enabled Flag as Y at Account Level in the GAC Table

Fetch the Last Charge Date

Get the APY Details of the Account

Get the APYE Details of the Account

Cust Type Validation

Interest Calculation Details for Account in Bancs Repository

Current and Previous Year Contribution for an ISA Account

EMTD Record for an Account

Update Mandate Ref No in LAM

Current APR of an Open Account

Updation of Tax Event Status of Accounts
for a Deceased CIF ID

Get the APR Details of the Account/Scheme
Code

Get the APY Details of Account or Scheme
Fetch Valid Scheme Codes

Update Account Status to Active

Check maturity for an Account

Check maximum Withdrawal for an
Account

Get Bump Up Frequency Status

Get Bump Up Period Status

Get No. of Bump Up Allowed Utilized

Populate and Insert Into BURHT

Third Party Account Close for Package
Account

Get Net Limit Accounts

Clearing Userhooks

Get Details from CTC Table for Transaction
Code

Get Inward Clearing Zone Header Details

Get Outward Clearing Zone Header Details

Get Outward Clearing Instrument Details

Get Outward Clearing Part Transaction
Details

Raise Exception for Clearing Transactions

Process ACH Nonfinancial Message

Check Duplicate Field

Currency Calendar Userhooks

Calculate Next Currency Working Date

Input Date Coincides with Currency
Holiday

Custom Batch Job Userhooks

Get Field Value for Service ID and Field
Name

Put Field Value with Service ID and Field
Name

Pre/Post Process Scripts for Batches

Customer Related Userhooks

Get All Address Details of a Customer

Get Customer Details

Get Document Details of a Customer

Customer Details in Bancs Repository

Get CRM Address of a Customer

Date Related Userhooks

Adding Days and Months in a Given Date

Compute Next Execution Date

Convert CalBase Date to Gregorian Date

Convert Gregorian Date to CalBase Date

Date Add

Difference between Two Dates

Get Call Map Parameters

Get Currency Working Date for Given
Number of Days

Get Date from Mnemonic

Get Four Digit Year

Get Local Calendar General Information

Get Next Frequency Date

Get Next Working Date

Given a Date get Date Difference from
BOD

Register for Date Shift Operation

De-register for Date Shift Operation

Getting Holiday Status for Given Date

Get the Last Successful DC

Check Whether it is a DC Holiday

Check whether it is a ACH Holiday or Working Day

Get the Next ACH Working Date

Difference between Two Dates in Months and Days

Get Difference between Two Frequencies

Get the Time Zone Description

Demand Draft Userhooks

Change DD Status

Get DD Credit Details

Get Scheme Details for DD Scheme Code and Currency

DSA Related Userhooks

Get DSA Effective Available Limit

Get DSA Master Details

Get DSA Parameter Details

Get DSA Product Details

Get DSA Turnover Details

Get DSA Turnover Details for Scheme Type

Get Past Due Cycles

Update DSA Turnover Details

Fetch Data from DB

DB Cursor Close

DB Cursor Fetch

DB Cursor Open

DB Cursor Open with Bind Variables

Get Details from CTC Table for Transaction Code

Selection of Fields from Database

Selection of Fields from Database using Bind Variables

Selection of Fields from Database using Bind Variables for Larger Column Length

Using SQL Statement in Script

Using SQL Statement with Bind Variables in Script

Get Interpool Amounts for a Financial Year

Get Details of RPC Disbursements

Get List of Accounts Based on Instrument Number

Get Past Due Cycles

Get Pool Available Amount

Get Tranche Outstanding

Get User Additional Details

Getting Scheme Details into Bancs Repository

Fetch the Event Type Description

Get Conversion Rate

Get Tax Rate

Get Destination Bank ID

Get Fee Transaction Details during Commercial Loan Payment

Get Loan Scheme Details Repository

Get Fee Transaction Details during Loan Payment

Get the Bank Identifier for a given Paysys ID/Bank Code/Branch Code Combination

Get the Bank Code/Branch Code for the given Paysys ID/Bank Identifier Combination

Get the Nostro and the Vostro Account Details for the Bank Code/Branch Code/Currency Code Combination

General Scripting Userhooks

Populate Pool Details for a Specified Cycle

Call Batch Executable from Script

Check for Pipe Character in the Input String

Execute Service for Template

Execute Template SRV

Execute a Service Function

FXBILL_GETTRAN

FXBILL_INITTRAN

Get Amount Calculated Based on Setup in
Amount Code Table

Accept the Amount String

Get Cash Account Foracid

Get Interest between Dates

Get Module Information

Get Orchestration Transaction ID Sequence
Number

Get Syn Agent Upfront Fee

Get Total Profit and Loss Amount for Pool

IBR Userhooks

Initiate Audit for Logical Unit of Work

Get Total Sanction Limit of All Accounts
Linked to DSA

Interest Rate for an Account as on Given
Date in Bancs Repository

Next Number Generation

Populate Collateral Information

Put Environment Variable

Print Repository Fields

Raise Exception for Transaction Created
through HTM

Raising Exceptions

Round Off Amount

Save User Additional Details

Update BJM

User Defined TOD

Userhook -

GetFldValueWithSrvIdAndFldName

Userhook -

PutFldValueWithSrvIdAndFldName

Check Pending Verification for Security

Execute Web Based External Application in
Web Instance for Finacle

Mask Input

Enabling Debug Logs

Discretionary Limit Set for the User

Get additional details for HLAPSP

Amount in LAKH/MILLION Format using
Commas

Get Limit B2kId

Get the Cust ID

Get B2k ID for the Given Event ID

Userhook for the mrbx Report Function

Get Error Description for the Error Code

Put Custom Exception

Put Exceptions into Repository

Insert Lien Details in Repository

Write into a File

Inserts a Record in the Negative TD LL

Insert Records into PQT Table

Generate Past Due Details Inquiry and Report

Generate Past Due Report

Get Past Due Display or Print String

Get Past Due Record

IBAN Userhooks

Fetch Account Number from IBAN

Fetch IBAN from Account Number
Generate the Check Digits
Validation of IBAN
Insert Record into Tables
 Create DCTI Record
 Create Entry in BJM
 Create FEL Entry
 Insert Audit Records for Custom Table
 Update Part Transaction Details in
 Repository
 Insert into CFATbls
 updateAADFrmPORA
 Insert into GPC Table
 Insert Notification Data into NOTFN Table
 Update Notification Data to NOTFN Table
Interest Related Userhooks
 Get Derived Interest Amount
MTT Userhooks
 Check if Customer Instructions Exists
 Define Part Transaction
 Define Transaction

Distribute Amount as per Customer
Instruction

End Definition of a Transaction or Part
Transaction

Get Additional Input Details in MTT

MTTS Define Charge Part Transaction

MTTS Get PTT Records

MTTS Initialise PTT

MTTS HandleExceptions

Define Loan Part Tran

Generate Charge Part Transaction

Populate Input Details

Report Error Condition

Initialize MTTS

Close MTTS

Get Document Details of a Customer

Get CIF ID

Payment System Userhooks

ACH Userhooks

Handling Operations from AED Table
from Script

Populate Reason Code for ACH

Fund Validation for ACH

Update Entry Status for ACH

SWIFT Userhooks

Create SMH Record

Get SMH Record

Update SMH Record

Outward Swift Transaction Creation

Populate Amount Field

ECS Userhooks

Get Address Details for Bic Bank and Branch

Fetch Unique Bic for PaysysId

Fetch Value for Bic, PaysysId and Additional Details

Fetch Directory Reference Number for Bic and PaysysId

Fetch Additional Details for Directory Reference Number

Remaining Value of Prepayment Permitted in a Year

Updates Blacklist Status of a Payment

Overdraft Userhooks

Check If Tran Used COD

Referral Userhooks

Raising an Amount Based Exception

Setting Referral Data

Loan Related Userhooks

Get SMH Record

Get Loans Demand Satisfaction Fee
Transactions Details

Get Loans Pay Fee Transaction Details

Get Additional Details for H LAPSP

Get Loans Pay Fee Transaction Details

Get Loans Demand Satisfaction Fee
Transactions Details

Get Loan Account Outstandings

Lien Related Userhooks

Trade Finance Related Userhooks

FXBILL_GETTRAN

FXBILL_INITTRAN

Get Additional Input Details in MTT

Get Details of RPC Disbursements

Get User Additional Details

Raising Exceptions

Save User Additional Details

Userhooks for Connect24

Determine if Connect24 Online Fees Must
be Treated as Charge Income

Generate Hashed Number for Connect24
Messages

Create DCTI Record

Userhooks used for Conversions

Convert Lower String to Upper Case

B2k_Convert Amount

Userhooks used within the Operating System

Get PID of the Process

Get Value of Environment Variable

Getting the Location of a File in the Path

Sleep

Validation Userhooks

Check Pending Verification for Security

Check Whether Account is Repeatedly
Overdrawn

Check Whether Account is Cash Account

Check Whether Account is FCNR Account

Check Whether Account is HO Account

Check Whether Account is System Only Account

Validate Amount with Currency

Validate Bank and Branch Code

Validate Bank Code

Validate Checksum

Validate Collateral Code with Report Type

Validate Currency Code

Validate Currency for Pool Code

Validate Date

Validate Foracid for Mudarabah Pool Product Account

Validate HO Transaction Category Code

Validate if Record Already Exists for WMS Type in ASM Table

Validate Instrument Date

Validate Instrument for Account

Validate Job Group

Validate Mudarabah Pool Product Account

Validate Mudarabah Pool Product Scheme Code

Validate Numeric

Validate Pool Code for Inquiry

Validate Reference Code

Validate SOL ID

Validate Transaction Report Code for Account

Validating the Account

Validate GL Subhead Desc Userhooks

Validate BIC

Validate BIC For PaysysId

Validate Multi Currency Account

Validate Multi-Currency/Master Account

Validate Channel ID For Origination

Validate Channel ID For Services

Validation Originator ID

Check Whether Account is Repeatedly Overdrawn

SIS Userhooks

Get Account Details from SIS DB

Get Customer Details from SIS DB

Selection of Fields from CSIS Database
using Bind Variables

Updating the values in CSIS Database using
Bind Variables

FI Related Userhooks

Validate for Duplicate Transaction

Fetching Channel audit information from
Finacle Core

Make Outbound Calls to FI Services using
LIMOC APIs and Connection Caching

Others

Enabling Debug Logs

To Move Files using Scripts

ProcessOutboundMessage

FIJCA Connectivity Userhook

Online Scripting Userhooks

To Move the Data to the Frontend

To Set the Environment Variables for Use
in the com Script

To Execute the com Script in Background

SetOrbError

SetOrbWarning

Referential Exchange Rate Userhooks

Get Tax Rate

GetFxRate

Userhook to Fetch and Update UK_WTR Table

Userhook for Select Queries

Update Statements

Get APY Earned with Compounding

Insert Record into DTD

Insert into CFATbls

Addendum

Addendum for 10.2.15

urhk_changeAcctOpnDate.uhk

urhk_getSSAMiscDtlsForAcct

urhk_performCleanUp

Addendum for 10.2.16

User hooks Added

Scripting Userhooks About this Module

Finacle Core Banking Solution supports a number of deployment topologies from fully distributed to fully centralized and implementation scenarios like single currency, multiple currency, various retail and trade finance banking products, interest methods and so on. To support these features, it is essential that Finacle is open to customization by the bank so that it can implement features that it requires and in the way that it desires. The bank also needs to interface some of their peripheral applications like treasury, consumer finance and others which are not supported directly by Finacle, with Finacle in various ways. Another area of customizability is in the interface to various special purpose equipment like ATM switches, VRUs, MICR encoders and so on which are supplied by many vendors with some differences in the actual implementation.

Finacle had addressed this issue by defining parameters set up by the bank depending upon its requirements. However, this has the limitation of requiring that the developers of Finacle know all the possible business logic beforehand and then implement them using parameters. However, a far more powerful approach is to allow the bank to take control of certain events, apply their own logic on the data associated with that event and be able to either influence the outcome of that event or communicate the occurrence of the event to other custom applications. The Finacle Script Engine based customization allows the bank to do just this.

The **Script Engine Based Customization** (SEBC) consists of various components. It include:

- The Script Engine

This is an interpretive language processor which allows a wide range of key programming constructs, while allowing the bank to custom develop additional functions that can be called from the scripts.

- The Finacle functions

Certain standard functions and certain module-specific functions have been developed by Finacle developers which can be called from the scripts.

- Finacle script events

Right across Finacle, there are several events that have been opened up by defining script events. For any script-enabled event, Finacle checks for the existence of a defined script and if present, transfers control to the script engine to execute that script after populating all the data required by that event. It is the scripts' responsibility to apply a specific logic as required and provide necessary output for Finacle to continue its processing.

Finacle Userhooks

Some functions provided in Finacle, used while writing a Finacle script are general and can be used in the context of any of the script events. However, certain functions are specific either to the workflow context or to a specific event. In such a case it is documented along with its description as:

- Brief description of the function and context in which it can be used

Scripting Userhooks About this Module

Finacle Core Banking Solution supports a number of deployment topologies from fully distributed to fully centralized and implementation scenarios like single currency, multiple currency, various retail and trade finance banking products, interest methods and so on. To support these features, it is essential that Finacle is open to customization by the bank so that it can implement features that it requires and in the way that it desires. The bank also needs to interface some of their peripheral applications like treasury, consumer finance and others which are not supported directly by Finacle, with Finacle in various ways. Another area of customizability is in the interface to various special purpose equipment like ATM switches, VRUs, MICR encoders and so on which are supplied by many vendors with some differences in the actual implementation.

Finacle had addressed this issue by defining parameters set up by the bank depending upon its requirements. However, this has the limitation of requiring that the developers of Finacle know all the possible business logic beforehand and then implement them using parameters. However, a far more powerful approach is to allow the bank to take control of certain events, apply their own logic on the data associated with that event and be able to either influence the outcome of that event or communicate the occurrence of the event to other custom applications. The Finacle Script Engine based customization allows the bank to do just this.

The **Script Engine Based Customization** (SEBC) consists of various components. It include:

- The Script Engine

This is an interpretive language processor which allows a wide range of key programming constructs, while allowing the bank to custom develop additional functions that can be called from the scripts.

- The Finacle functions

Certain standard functions and certain module-specific functions have been developed by Finacle developers which can be called from the scripts.

- Finacle script events

Right across Finacle, there are several events that have been opened up by defining script events. For any script-enabled event, Finacle checks for the existence of a defined script and if present, transfers control to the script engine to execute that script after populating all the data required by that event. It is the scripts' responsibility to apply a specific logic as required and provide necessary output for Finacle to continue its processing.

Finacle Userhooks

Some functions provided in Finacle, used while writing a Finacle script are general and can be used in the context of any of the script events. However, certain functions are specific either to the workflow context or to a specific event. In such a case it is documented along with its description as:

- Brief description of the function and context in which it can be used
- Function syntax – (Also gives change log between Finacle versions)
- Detailed functionality
- Description of the input fields and output fields
- Example of the usage of the function

Repositories and Classes

All Finacle userhooks, except for workflow related functions and MTT related functions, interface with the scripts using a standard repository called BANCS. There are two classes predefined in this repository - INPARAM and OUTPARAM each of which hold **String** type of fields. INPARAM is the class used to populate specified fields, depending upon the function, in the script so that the function can access those as parameters. All the fields, depending upon the function, that are output by the function are populated in the OUTPARAM class.

In addition to INPARAM class, the input to the functions can also be provided as arguments to the function.

Note:

The INPARAM class is cleared out once the userhook is executed and must be repopulated by the script when calling another userhook or the same userhook in a loop.

Return Status of the Function

All functions return a value of either SUCCESS (0) or FAILURE (1).

©Copyright Infosys Ltd.

Scripting Userhooks

This section discusses:

[Account Related Userhooks](#)

[Clearing Userhooks](#)

[Currency Calendar Userhooks](#)

[Custom Batch Job Userhooks](#)

[Customer Related Userhooks](#)

[Date Related Userhooks](#)

[Demand Draft Userhooks](#)

[DSA Related Userhooks](#)

■ [Fetch Data from DB](#)

[General Scripting Userhooks](#)

[Generate Past Due Details Inquiry and Report](#)

[IBAN Userhooks](#)

[Insert Record into Tables](#)

[Interest Related Userhooks](#)

[MTT Userhooks](#)

[Payment System Userhooks](#)

[Overdraft Userhooks](#)

[Referral Userhooks](#)

[Loan Related Userhooks](#)

[Lien Related Userhooks](#)

[Trade Finance Related Userhooks](#)

[Userhooks for Connect24](#)

[Userhooks used for Conversions](#)

[Userhooks used within the Operating System](#)

[Validation Userhooks](#)

[SIS Userhooks](#)

[FI Related Userhooks](#)

[Others](#)

[Online Scripting Userhook](#)

[Referential Exchange Rate Userhooks](#)

[Userhook to Fetch and Update UK_WTR Table](#)

[Userhook for Select Queries](#)

[Update Statements](#)

[Get APY Earned with Compounding](#)

[Insert Record into DTD](#)

[Insert into CFATbls](#)

©Copyright Infosys Ltd.

Account Related Userhooks

Finacle provides userhooks for performing different operations on Customer Accounts.

These userhooks helps us in getting various details like Account Status, Account Type, Account Balance, Average Balance, FFD Available Amount, Full Available Amount, Turn Over details, and so on. Also there are userhooks which calculate the sum of transactions on an account. Using a scripting userhook it is also possible to issue cheque leaves automatically.

This section discusses:

[Account Balance as on Given Date](#)

[Advance Details of an Account in Bance Repository](#)

[BACID to Account ID and Name](#)

[Fetch Account Type](#)

[Fetch Multi Currency Account Details](#)

[Get Account Number](#)

[Get Account Status](#)

[Get Account Turnover Details](#)

[Get Average Balance of Account](#)

[Get FFD Available Amount for Account](#)

[Get Full Available Amount](#)

[Issue Cheque Leaves](#)

[Sum Credit and Debit Transaction Amount Over a Period for an Account](#)

[Sum Transaction Amount for a Specified Account on a Account Label](#)

[Getting Account Details into Bancs Repository](#)

[Get the Foracid from Acid](#)

[Get the Acid from Foracid](#)

[Update Lien](#)

[Get the Float and the Unclear Balance](#)

[Interest Calculation Details for Account in Bancs Repository](#)

[Current and Previous Year Contribution for an ISA Account](#)

[EMTD Record for an Account](#)

[Update Mandate Ref No in LAM](#)

[Current APR of an Open Account](#)

[Updation of Tax Event Status of Accounts for a Deceased CIF ID](#)

[Get the APR Details of the Account/Scheme Code](#)

[Get the APY Details of Account or Scheme](#)

[Update Account Status to Active](#)

[Check Maturity for an Account](#)

[Check Maximum Withdrawal for an Account](#)

[Get Bump Up Frequency Status](#)

[Get Bump Up Period Status](#)

[Get No. of Bump Up Allowed Utilized](#)

[Populate and Insert Into BURHT](#)

[Third Party Account Close for Package Account](#)

[Get Net Limit Accounts](#)

©Copyright Infosys Ltd.

Finacle Copyright

Account Balance as on Given Date

This userhook returns end of day account balance as on a given date from Finacle. This userhook can be called from any script event and validates account number and as on date. Returns failure if account number is invalid or if date format is not correct.

If inputs are valid, then the end of day account balance is populates into Finacle repository.

Syntax:

```
sv_a = urhk_getAcctBalanceAsOnDate("")
```

sv_a: scratch pad variable to store the return value.

Input Arguments:

Valid account number

As on Date

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctId	Valid Account Number in GAM table
BANCS.INPARAM.asOnDate	As on Date(in DD-MM-YYYY format) in GAM table

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.acctBal	Account balance in GAM table

Return Value:

If the input values are invalid then the return value is 1.

End of day account balance based on transaction date is populated into the repository.

For Example:

To obtain account balance as on BOD date for an account, the steps to be are:

```
<--start
# Populate BOD date and into as on date and account number
BANCS.INPARAM.acctId = BANCS.INPUT.acctId
BANCS.INPARAM.asOnDate = BANCS.STDIN.BODDate
sv_a = urhk_getAcctBalanceAsOnDate("")
if ( sv_a == 1 ) then
  exitscript
endif
sv_i = cdouble(BANCS.OUTPARAM.balance)
end-->
```

©Copyright Infosys Ltd.

Advance Details of an Account in Bancs Repository

This userhook returns advance details specified through HACM menu option for a given account number. This userhook can be called from any script event.

See Maintain Customer Account in the Cash Credit and Overdraft Account Module for HACM menu option details.

Checks for validity of account number. Unverified and closed accounts are considered as valid.

If the account number passed is found to be valid, the advance details of the are populated into repository fields.

Syntax:

`sv_a = urhk_getAdvanceDetailsForAccount(Variable)`

`sv_a` -> scratch pad variable to store the return value.

Variable -> This variable can be a scratch pad variable (`sv_b`) or a string ("SBGEN1").

Input Arguments:

Valid account number.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctId	Valid Account Number in GAM table

Output Repository Fields:

Field Name	Description
------------	-------------

BANCS.OUTPUTPARAM.SectCode	Sector Code in GAC table
BANCS.OUTPUTPARAM.SubSectCode	Sub Sector Code in GAC Table
BANCS.OUTPUTPARAM.GuarCoverCode	Guarantee Cover Code in GAC Table
BANCS.OUTPUTPARAM.NatureOfAdvn	Nature of Advance in GAC Table
BANCS.OUTPUTPARAM.TypeOfAdvn	Type of Advance in GAC Table
BANCS.OUTPUTPARAM.ModeOfAdvn	Mode of Advance in GAC Table
BANCS.OUTPUTPARAM.PurposeOfAdvn	Purpose of Advance in GAC Table
BANCS.OUTPUTPARAM.FreeCode1	Free code 1 in GAC Table
BANCS.OUTPUTPARAM.FreeCode2	Free code 2 in GAC Table
BANCS.OUTPUTPARAM.FreeCode3	Free code 3 in GAC Table
BANCS.OUTPUTPARAM.FreeCode4	Free code 4 in GAC Table
BANCS.OUTPUTPARAM.FreeCode5	Free code 5 in GAC Table
BANCS.OUTPUTPARAM.FreeCode6	Free code 6 in GAC Table
BANCS.OUTPUTPARAM.FreeCode7	Free code 7 in GAC Table
BANCS.OUTPUTPARAM.FreeCode8	Free code 8 in GAC Table
BANCS.OUTPUTPARAM.FreeCode9	Free code 9 in GAC Table
BANCS.OUTPUTPARAM.FreeCode10	Free code 10 in GAC Table
BANCS.OUTPUTPARAM.IndustryType	Industry Type in GAC Table
BANCS.OUTPUTPARAM.AcctOccpCode	Account Occupation Code in GAC Table
BANCS.OUTPUTPARAM.BorwrCtgCode	Borrower Category Code in GAC Table
BANCS.OUTPUTPARAM.PastDueFlag	Past Due Flag in GAC Table
BANCS.OUTPUTPARAM.PastDueXferDate	Past Due Transfer Date in GAC Table
BANCS.OUTPUTPARAM.PastDueReXferDate	Past Due Re-Transfer Date in GAC Table
BANCS.OUTPUTPARAM.PrvToPastDueGLSubHeadCode	Previous To Past Due GL Sub Head Code in GAC Table
BANCS.OUTPUTPARAM.IntCrAcctFlag	Interest Credit Account Flag in GAC

	Table
BANCS.OUTPARAM.IntDrAcctFlag	Interest Debit Account Flag in GAC Table
BANCS.OUTPARAM.IntDrAcctId	Interest Debit Account ID in GAC Table
BANCS.OUTPARAM.IntCrAcctId	Interest Credit Account ID in GAC Table
BANCS.OUTPARAM.gacFreeText1	Free Text 1 in GAC Table
BANCS.OUTPARAM.gacFreeText2	Free Text 2 in GAC Table
BANCS.OUTPARAM.gacFreeText3	Free Text 3 in GAC Table
BANCS.OUTPARAM.gacFreeText4	Free Text 4 in GAC Table
BANCS.OUTPARAM.gacFreeText5	Free Text 5 in GAC Table
BANCS.OUTPARAM.gacFreeText6	Free Text 6 in GAC Table
BANCS.OUTPARAM.gacFreeText7	Free Text 7 in GAC Table
BANCS.OUTPARAM.gacFreeText8	Free Text 8 in GAC Table
BANCS.OUTPARAM.gacFreeText9	Free Text 9 in GAC Table
BANCS.OUTPARAM.gacFreeText10	Free Text 10 in GAC Table
BANCS.OUTPARAM.gacFreeText11	Free Text 11 in GAC Table
BANCS.OUTPARAM.gacFreeText12	Free Text 12 in GAC Table
BANCS.OUTPARAM.gacFreeText13	Free Text 13 in GAC Table
BANCS.OUTPARAM.gacFreeText14	Free Text 14 in GAC Table
BANCS.OUTPARAM.gacFreeText15	Free Text 15 in GAC Table

Return Value:

If the account number specified is valid and advance details exist for account, then the return value is '0'. Else return value is 1.

The advance details is populated into class OUTPARAM of BANCS repository. Individual repository field names are mentioned against the corresponding field name in the Table.

For Example:

Advance details of an account may be required during generation of reports on Past Due accounts. Asset type, category and sector codes specified at account level may be obtained as below.

```
<--start
```

```
sv_a = BANCS.INPUT.acctId
```

```
# sv_a (account id) is the input to user hook
```

```
sv_b = urhk_getAdvanceDetailsForAccount(sv_a)
```

```
# If urhk_getAdvanceDetailsForAccount function returns '1'
```

```
# if account number is invalid.
```

```
# If the return value is '1' then populate error message and
```

```
# exit from script.
```

```
if( sv_b == 1 ) then
```

```
  exitscript
```

```
endif
```

```
sv_i = BANCS.OUTPARAM.IndustryType
```

```
sv_j = BANCS.OUTPARAM.SectCode
```

```
sv_k = BANCS.OUTPARAM.SubSectCode
```

```
end-->
```

©Copyright Infosys Ltd.

BACID to Account ID and Name

This function returns the Account ID and name of the account that is identified through a BACID, Currency code and SOL ID, if such an account exists. This is a subset of getAcctDetailsInRepository and may be removed in future versions. It is usually used as a part of a script where the Bacid, Currency and SOL ID are known (or are obtained) before this userhook is called. Get the account ID for passed Bacid, Currency and Sol ID.

Syntax:

The syntax for calling the function is:

```
sv_r = urhk_B2k_BacidToAcctId (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("PURCH|INR|123456").

Input Arguments:

Input string contains three parts separated by a "|"

Bacid

Currency code

Sol ID

For Example:

```
PURCH|INR|123456
```

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.acctSolId	Service Outlet ID in GAM Table
BANCS.OUTPARAM.bacid	Bacid in GAM Table
BANCS.OUTPARAM.crncyCode	Currency Code in GAM Table
BANCS.OUTPARAM.acctName	Account Name in GAM Table
BANCS.OUTPARAM.acctId	Account Id in GAM Table

Return Value:

The return value is '1' if no account is found, else it is '0' and fields AcctName and AcctId of OUTPARAM class of BANCS repository is populated for the deduced account.

For Example:

This userhook is a subset of getAcctDetailsInRepository userhook. It is used to manipulate only acctName and acctId (Foracid). In the example below, the bank has named all system cash accounts as "SYSTEM CASH" and wants these accounts to be skipped in the script.

```
<--start
```

```
# The above statements set the context of Bacid, Currency  
Code and Foracid
```

```
# in variables sv_b, sv_c and sv_f respectively for this  
example.
```

```
sv_a = sv_b + "|" + sv_c + "|" sv_f
```

```
# -----  
-----
```

```
# - sv_a (see Input section above) is the input to user hook
```

```
# B2k_BacidToAcctId
```

```
# - Call user hook B2k_BacidToAcctId to put the account name
```

```

and foracid
# in the repository
# -----
-----

sv_r = urhk_B2k_BacidToAcctId (sv_a)
# if B2k_BacidToAcctId function returns failure exit out of
script.
if( sv_r == 1 ) then
exitscript
endif

sv_s ="SYSTEM CASH"
# All cash accounts in various currencies have this name
# -----
-----

# acctName is compared
# If system cash, the script skips the account
# -----
-----

if (BANCS.OUTPARAM.acctName == sv_s) then
GOTO lblSkipSystemCash
else
# do the rest of the processing as desired
endif
end-->

```

Fetch Account Type

This userhook fetches the type of input account. The various types returned are: Multi Currency account, Master account, Operative account and invalid account.

Syntax:

```
sv_b = urhk_ scrFetchAccountType ("")
```

Input Arguments:

Account ID

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctId	Valid Account ID in GAM Table

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.acctType	Type of the Account

Return Value:

Account Type (whether Master,Multi currency ,Normal Operative or invalid account).

For Example:

```
<--start
```

```
#trace on
```

```
BANCS.INPARAM. acctId
```



```
sv_z = urhk_ scrFetchAccountType ("")
```

```
BANCS.OUTPARAM. acctType
```

```
#trace off
```

```
end-->
```

©Copyright Infosys Ltd.

Fetch Multi Currency Account Details

This userhook fetches the currency account for a combination of multi currency account and currency code.

Syntax:

```
sv_b = urhk_scrFetchMultiCcyCrncyAcctDtls ("" )
```

Input Arguments:

Input Multi Currency Account

Currency Code

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.multiCcyAcctNum	Valid Multi Currency Account ID
BANCS.INPARAM.crncyCode	Currency Code of the Account

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.foracid	Account ID in GAM Table
BANCS.OUTPARAM.acctName	Account Name
BANCS.OUTPARAM.schmType	Scheme Type of the Account

Return Value:

Foracid, Account Name and Scheme Type are the output.

For Example:

```
<--start
```

```
#trace on
BANCS.INPARAM. multiCcyAcctNum
BANCS.INPARAM. crncyCode
sv_z = urhk_ scrFetchMultiCcyCrncyAcctDtls (""")
BANCS.OUTPARAM. foracid
BANCS.OUTPARAM. acctName
BANCS.OUTPARAM. schmType
#trace off
end-->
```

©Copyright Infosys Ltd.

Get Account Number

A userhook is required to give the Full Account number with check digit, prefix and suffix if any after taking scheme code and serial number part of the account number.

Syntax:

```
sv_b = urhk_getAcctNum ("")
```

Input Arguments:

BANCS.INPARAM.schmCode= (scheme code, this is mandatory)

BANCS.INPARAM.serialNum= (serial number)

BANCS.INPARAM.solld= (sol id)

BANCS.INPARAM.glSubHeadCode="" (gl sub head code, this will be defaulted from scheme if not given)

BANCS.INPARAM.crncyCode="" (currency code)

BANCS.INPARAM.cifld="" (cif id)

BANCS.INPARAM.acctOwnership="" (acct ownership to denote if it is customer a/c or office a/c. this will be defaulted to customer's account if not given.)

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.schmCode	Scheme Code of the Account
BANCS.INPARAM.serialNum	Serial Number
BANCS.INPARAM.solld	Service Outlet ID
BANCS.INPARAM.glSubHeadCode	GL Sub Head Code

BANCS.INPARAM.crncyCode	Currency Code
BANCS.INPARAM.cifld	Customer Information File ID
BANCS.INPARAM.acctOwnership	Ownership of the Account

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.acctNum	Account Number
BANCS.OUTPARAM.errorMesg	Error Message
BANCS.OUTPUT.successOrFailure	Success Or Failure - Status

Return Value:

The output from this userhook is the Account Number in case of Success and Error Message in case of failure.

BANCS.OUTPARAM.acctNum

BANCS.OUTPARAM.errorMesg

BANCS.OUTPUT.successOrFailure

For Example:

```

<--start
trace on
BANCS.INPARAM.schmCode="ODAYU"
BANCS.INPARAM.serialNum="3582"
BANCS.INPARAM.solId="000000"
BANCS.INPARAM.glSubHeadCode=""
BANCS.INPARAM.crncyCode="INR"
BANCS.INPARAM.cifId=""
BANCS.INPARAM.acctOwnership="C"

```

```
sv_r = urhk_getAcctNum("")
print(sv_r)
if (sv_r==0) then
sv_a=BANCS.OUTPUTPARAM.acctNum
BANCS.OUTPUT.successOrFailure= "S"
else
sv_a=BANCS.OUTPUTPARAM.errorMesg
BANCS.OUTPUT.successOrFailure = "F"
endif
print(sv_a)
trace off
end-->
```

©Copyright Infosys Ltd.

Get Account Status

This userhook is provided to find out the status of the given account ID.

Syntax:

```
sv_a = urhk_getAcctStatus()
```

Input Arguments:

Input to this userhook is Account ID (acid).

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acid	Acid of the Account

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.acctStatus	Status of the Account

Return Value:

Output of this userhook is the status of the account (acctStatus).

For Example:

```
<--start
#trace on
BANCS.INPARAM.acid = sv_a
if (sv_a != "") then
```

```
sv_e = urhk_getAcctStatus()
if (sv_e == 0) then
sv_f = BANCS.OUTPUTPARAM.acctStatus
endif
else
sv_f = ""
endif
BANCS.OUTPUT.acct_status = sv_f
#trace off
end-->
```

©Copyright Infosys Ltd.

Get Account Turnover Details

This user hook provides the account turnover details for the given foracid as on date. The input to this user hook is the foracid and as on date separated by a delimiter.

Syntax:

```
sv_a = urhk_getAcctTurnoverDetails ()
```

Input Arguments:

Account ID (foracid)

As on Date

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctId	Valid Account Number
BANCS.INPARAM.asOnDate	As on Date(in DD-MM-YYYY format)

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.ATOInd	ATO Indicator
BANCS.OUTPARAM.startDate	Account opening date or start date for the period.
BANCS.OUTPARAM.endDate	Period end date or last run date.
BANCS.OUTPARAM.totDrBalDay	No. of Days A/c in debit (that is, Debit Balance) for the period. Period is the frequency at which the turnover is computed.
BANCS.OUTPARAM.totCrBalDay	No. of Days A/c in Credit (that is, Credit Balance) for the period. Period is the frequency at which the turnover is computed.

BANCS.OUTPARAM.totDrBal	Total debit balance of the account for the period.
BANCS.OUTPARAM.totCrBal	Total credit balance of the account for the period.
BANCS.OUTPARAM.highCrBal	Highest debit balance for the period.
BANCS.OUTPARAM.highDrBal	Highest credit balance for the period.
BANCS.OUTPARAM.drTot	Total amount of all debits made into the account.
BANCS.OUTPARAM.crTot	Total amount of all credits.
BANCS.OUTPARAM.clgCrTot	Total amount of all credits made into the account through clearing.
BANCS.OUTPARAM.clgNbrOfInstrmnts	Total Number of clearing instruments used.
BANCS.OUTPARAM.totNbrOfTodsGranted	Total number of TOD granted on the account.
BANCS.OUTPARAM.intCollAmt	Total interest collected from the account during the period.
BANCS.OUTPARAM.intPaidAmt	Total interest paid to the account during the period.
BANCS.OUTPARAM.timeTakenToClrTods	Total time taken to clear TODs.
BANCS.OUTPARAM.numOfTodsReglr	Number of regular TOD granted to the account.
BANCS.OUTPARAM.numTimesChqRetIn	Number of Inward Cheques Returned.
BANCS.OUTPARAM.numTimesChqRetOut	Number of Outward Cheques Returned.
BANCS.OUTPARAM.numOfDrTrans	Number of Debit Transactions.
BANCS.OUTPARAM.numOfCrTrans	Number of Credit Transaction.
BANCS.OUTPARAM.intAccuredCrAmt	Credit Interest Accrued.
BANCS.OUTPARAM.intAccuredDrAmt	Debit Interest Accrued.
BANCS.OUTPARAM.lowCrBal	Lowest debit Balance for the period.
BANCS.OUTPARAM.lowDrBal	Lowest Credit Balance for the period.
BANCS.OUTPARAM.chqRetAmtInwClg	Total Cheque Return Amount for Inward Clearing.
BANCS.OUTPARAM.chqRetAmtOutClg	Total Cheque Return Amount for Outward Clearing.

Return Value:

Account Turnover details for the input passed.

The account turnover details is available if ATOInd is YES

For Example:

```
# Get the inputs from repository
sv_a = BANCS.INPUT.acid
sv_b = BANCS.INPUT.asOnDate
sv_c = sv_a + "|" + sv_b
#call the user hook for getting Account turnover details
sv_n = urhk_ getAcctTurnoverDetails (sv_c)
IF (sv_n != 0) THEN
BANCS.OUTPUT.successOrFailure="F"
BANCS.OUTPUT.error= "Account Turnover details not found"
ENDIF
```

©Copyright Infosys Ltd.

Get Average Balance of Account

Syntax:

```
sv_a = urhk_getAvBalDetailsForAcct("")
```

Where sv_a is a scratch pad variable to store the return value.

Input Arguments:

- The input is the account number.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The output is available in the BANCS.OUTPARAM.FullAvailableAmt and BANCS.OUTPARAM.FxFullAvailableAmt fields. If the system encounters an error, the error message is available in the BANCS.OUTPARAM.errorMesg field.

©Copyright Infosys Ltd.

Get FFD Available Amount for Account

Syntax:

```
sv_a = urhk_getFFDAvailAmt("account number")
```

Input Arguments:

Input is a operative account number.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.InBuff	Account ID in Buffer

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.FullAvailableAmt	Full Available Amount
BANCS.OUTPARAM.FxFullAvailableAmt	Full Available Forex Amount
BANCS.OUTPARAM.errorMesg	Error Message

Return Value:

The output is available in the BANCS.OUTPARAM.FFDAvailAmt and BANCS.OUTPARAM.FxFFDAvailAmt fields. In case the system encounters an error, the error message is available in the BANCS.OUTPARAM.errorMesg field.

For Example:

```
< --start
```

```
BANCS.INPARAM.InBuff ="BMLAA"
```

```
sv_c = BANCS.INPARAM.InBuff
sv_a = urhk_getFFDAvailAmt(sv_c)
if (sv_a == 1) then
sv_b = BANCS.OUTPARAM.errorMesg
print(sv_b)
else
sv_b = BANCS.OUTPARAM.FFDAvailAmt
print(sv_b)
sv_c = BANCS.OUTPARAM.FxFFDAvailAmt
print(sv_c)
endif
exitscript
end-- >
```

©Copyright Infosys Ltd.

Get Full Available Amount

Syntax:

```
sv_a = urhk_getFullAvailAmtForAcct("accountnum");
```

Input Arguments:

■ The input is the account number.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.InBuff	Account ID in Buffer

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.FFDAvailableAmt	FFD Available Amount
BANCS.OUTPARAM.FxFFDAvailableAmt	FFD Available Forex Amount
BANCS.OUTPARAM.errorMesg	Error Message

Return Value:

The output is available in the BANCS.OUTPARAM.FullAvailableAmt and BANCS.OUTPARAM.FxFullAvailableAmt fields. If the system encounters an error, the error message is available in the BANCS.OUTPARAM.errorMesg field.

For Example:

```
< --start
```

```
BANCS.INPARAM.InBuff ="BMLAA"
```

```
sv_c = BANCS.INPARAM.InBuff
sv_a = urhk_getFullAvailAmtForAcct(sv_c)
if (sv_a == 1) then
sv_b = BANCS.OUTPARAM.errorMesg
print(sv_b)
else
```

©Copyright Infosys Ltd.

Issue Cheque Leaves

This userhook returns '0' on success and '1' on failure. The userhook takes the AcctNum, BeginChqNum, BeginChqAlpha, NoOfChqLvs as input in repository. It validates whether the account number is an existing one or not. It accepts the NoOfChqLvs between 1 and 100 including both, and the corresponding cheque leaves (with that numbers) must not be issued. After validation an entry is made in the CBT table.

Syntax:

```
sv_a = urhk_ IssueChequeLeaves ("" )
```

Input Arguments:

Input to userhook is through the input repository.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.AcctNum	Account Number
BANCS.INPARAM.BeginChqNum	Begin Cheque Number
BANCS.INPARAM.BeginChqAlpha	Alpha portion of the Cheque Number
BANCS.INPARAM.NoOfChqLvs	Total Number of Leaves
BANCS.INPARAM.chqLvsStat	Status of the Cheque Leaf

Output Repository Fields:

Output not present.

Return Value:

This userhook returns '0' on success and '1' on failure.

For Example:

Check the sample scripts named
IssueCheckLeafForPayment.sscr

©Copyright Infosys Ltd.

Sum Credit and Debit Transaction Amount Over a Period for an Account

This userhook returns the sum of credit and debit transaction amount over a period for a given account. This userhook can be called from any script event. The userhook validates account number, from date and to date specified as inputs and also checks for data range (that is, begin date must be lesser than or equal to the end date). Returns failure if any of the inputs is found invalid.

If inputs are valid, populates sum credit and debit balances into FINACLE repositories. Only posted transactions is considered. Transaction amount is based on the transaction date and not value date of transaction.

Syntax:

```
sv_a = urhk_getSumCrDrTranAmtsForPeriod("")
```

sv_a: scratch pad variable to store the return value.

Input Arguments:

Valid Account Number

From Date

To Date

These inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctId	Valid Account Number
BANCS.INPARAM.fromDate	From Date(in DD-MM-YYYY format)

BANCS.INPARAM.toDate	To Date(in DD-MM-YYYY format)
----------------------	-------------------------------

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.sumCreditAmt	Sum Credit transaction amount
BANCS.OUTPARAM.sumDebitAmt	Sum Debit transaction amount

Return Value:

If the input values are invalid return value is '1'.

Sum credit and debit transaction amount for period is populated into the repositories.

For Example:

To obtain sum credits made towards a past due account from the date of transfer to past due status, the mentioned code is to be incorporated.

```
<--start
# Get past due transfer date
sv_b = BANCS.INPUT.acctId
sv_a = urhk_getAdvanceDetailsForAccount(sv_b)
#Populate from date as past due transfer date. To date as BOD
#date and call user hook.
BANCS.INPARAM.acctId = sv_b
BANCS.INPARAM.fromDate =
BANCS.OUTPARAM.PastDueXferDate

BANCS.INPARAM.toDate = BANCS.STDIN.BODDate
sv_a = getSumCrDrTranAmtsForPeriod ("" )
if ( sv_a == 1 ) then
```

exitscript

endif

sv_i = cdouble(BANCS.OUTPARAM.sumCreditAmt)

end-->

©Copyright Infosys Ltd.

Sum Transaction Amount for a Specified Account on a Account Label

This userhook returns the sum transaction amount on a account label for a given account. This userhook can be called from any script event. The userhook validates the account number and account label. Returns failure ('1') if the inputs are invalid.

If inputs are valid, it populates sum transaction amount on account label for account into the Finacle repository. Only posted transactions is considered.

Syntax:

```
sv_a = urhk_getSumTranAmtForAcctLabel ("" )
```

sv_a: scratch pad variable to store the return value.

Input Arguments:

Valid account number

Valid account label

These inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctId	Valid Account Number
BANCS.INPARAM.acctLabel	Account label

Output Repository Fields:

Field Name	Description

BANCS.OUTPARAM.sumTranAmt	Sum transaction amount
---------------------------	------------------------

Return Value:

If the input values are invalid return value is '1'.

Sum transaction amount is populated into the repository.

For Example:

Provision amount made for a customer account can be obtained as:

```
<--start
```

```
BANCS.INPARAM.acctId = BANCS.INPUT.acctId
```

```
BANCS.INPARAM.acctLabel = "PROVISION"
```

```
sv_a = getSumCrDrTranAmtsForPeriod("")
```

```
if ( sv_a == 1 ) then
```

```
  exitscript
```

```
endif
```

```
sv_i = cdouble(BANCS.OUTPARAM. sumTranAmt)
```

```
end-->
```

©Copyright Infosys Ltd.

Getting Account Details into Bancs Repository

This function returns information about a given account number in the Finacle database. It can be used in the context of any Script Event or Workflow. This function checks whether the given account number exists in the data center database. If the account exists (irrespective of the state of the account), then the various components of the account balance such as available balance, effective available balance, lien amount, etc., are returned in repository fields.

Syntax:

```
sv_a = urhk_getAcctDetailsInRepository(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("SBGEN1").

Input Arguments:

The input to this function is an account number (FORACID).

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.acctSolId	sol_id in GAM Table
BANCS.OUTPARAM.acctPlaceholder	Bacid in GAM Table
BANCS.OUTPARAM.acctName	acct_name in GAM Table
BANCS.OUTPARAM.custId	cust_id in GAM Table
BANCS.OUTPARAM.empId	emp_id in GAM Table
BANCS.OUTPARAM.glSubHeadCode	gl_sub_head_code in GAM Table

BANCS.OUTPARAM.acctOwnerShip	acct_ownership(C'ustomer, E'mployee, O'ffice account) in GAM Table
BANCS.OUTPARAM.schmCode	schm_code in GAM Table
BANCS.OUTPARAM.schmType	schm_type in GAM Table
BANCS.OUTPARAM.acctOpenDate	acct_opn_date in GAM Table
BANCS.OUTPARAM.acctCloseDate	acct_cls_date in GAM Table
BANCS.OUTPARAM.acctCloseflg	acct_cls_flg(Y/N) in GAM Table
BANCS.OUTPARAM.modeOprnCode	mode_of_oper_code in GAM Table
BANCS.OUTPARAM.acctLocnCode	acct_locn_code in GAM Table
BANCS.OUTPARAM.acctCrncyCode	acct_crncy_code in GAM Table
BANCS.OUTPARAM.systemOnlyAcctFlg	system_only_acct_flg ('Y/N') in GAM Table
BANCS.OUTPARAM.AcctId	Foracid in GAM Table
BANCS.OUTPARAM.debitBalanceLimit	dr_bal_lim in GAM Table
BANCS.OUTPARAM.freezeCode	frez_code in GAM Table
BANCS.OUTPARAM. freezeReasonCode	frez_reason_code in GAM Table
BANCS.OUTPARAM.clearBalance	clr_bal_amt in GAM Table
BANCS.OUTPARAM.unclearBalance	un_clr_bal_amt in GAM Table
BANCS.OUTPARAM.ledgerNumber	ledg_num in GAM Table
BANCS.OUTPARAM.drawingPower	drwng_power in GAM Table
BANCS.OUTPARAM.sanctionLimit	sanct_lim in GAM Table
BANCS.OUTPARAM.adhocLimit	adhoc_lim in GAM Table
BANCS.OUTPARAM.emergencyAdvance	emer_advn in GAM Table
BANCS.OUTPARAM.DACCLimit	dacc_lim in GAM Table
BANCS.OUTPARAM.systemReservedAmt	system_reserved_amt in GAM Table
BANCS.OUTPARAM.singleTranLimit	single_tran_lim in GAM Table
BANCS.OUTPARAM. cleanAdhocLimit	clean_adhoc_lim in GAM Table
BANCS.OUTPARAM. cleanEmergencyAdvance	clean_emer_advn in GAM Table

BANCS.OUTPARAM.cleanSingleTranLimit	clean_single_tran_lim in GAM Table
BANCS.OUTPARAM.systemGeneratedLimit	system_gen_lim in GAM Table
BANCS.OUTPARAM.chequeAllowed	chq_alwd_flg in GAM Table
BANCS.OUTPARAM.cashExceptionAmtLimit	cash_excp_amt_lim in GAM Table
BANCS.OUTPARAM.clearingExceptionAmtLimit	clg_excp_amt_lim in GAM Table
BANCS.OUTPARAM.transferExceptionAmtLimit	xfer_excp_amt_lim in GAM Table
BANCS.OUTPARAM.cashCrExceptionAmtLimit	cash_cr_excp_amt_lim in GAM Table
BANCS.OUTPARAM.clearingCrExceptionAmtLimit	clg_cr_excp_amt_lim in GAM Table
BANCS.OUTPARAM.transferCrExceptionAmtLimit	xfer_cr_excp_amt_lim in GAM Table
BANCS.OUTPARAM.cashAbnormalAmtLimit	cash_abnrml_amt_lim in GAM Table
BANCS.OUTPARAM.clearingAbnormalAmtLimit	clg_abnrml_amt_lim in GAM Table
BANCS.OUTPARAM.TransferAbnormalAmtLimit	xfer_abnrml_amt_lim in GAM Table
BANCS.OUTPARAM.accruedCreditAmt	acrd_cr_amt in GAM Table
BANCS.OUTPARAM.passbookOrPasssheet	pb_ps_code in GAM Table
BANCS.OUTPARAM.serviceChrgCollectedFlg	serv_chrg_coll_flg in GAM Table
BANCS.OUTPARAM.interestPaidFlaginterestPaidFlag	int_paid_flg in GAM Table
BANCS.OUTPARAM.interestCollectedFlag	int_coll_flg in GAM Table
BANCS.OUTPARAM. in GAM Table	limit_prefix in GAM Table
BANCS.OUTPARAM.limitSuffix	limit_suffix in GAM Table
BANCS.OUTPARAM.drawingPowerIndicator	drwng_power_ind in GAM Table
BANCS.OUTPARAM.drawingPowerPercentage	drwng_power_pcmt in GAM Table
BANCS.OUTPARAM.notionalRate	notional_rate in GAM Table
BANCS.OUTPARAM.notionalRateCode	notional_rate_code in GAM Table
BANCS.OUTPARAM.FXClearBalance	fx_clr_bal_amt in GAM Table
BANCS.OUTPARAM.FCNRCrncyCode	crncy_code in GAM Table
BANCS.OUTPARAM.TaxFlg	wtax_flg in GAM Table

BANCS.OUTPARAM.TaxAmtScopeFlg	wtax_amount_scope_flg in GAM Table
BANCS.OUTPARAM.lienAmt	lien_amt in GAM Table
BANCS.OUTPARAM.accountManagerUserId	acct_mgr_user_id in GAM Table
BANCS.OUTPARAM.schemeType	schm_type in GAM Table
BANCS.OUTPARAM.partitionedFlg	Partitioned_flg in GAM Table
BANCS.OUTPARAM.partitionedType	Partitioned_type in GAM Table
BANCS.OUTPARAM.AvailableAmt	Not in GAM table (Sum of different limits)
BANCS.OUTPARAM.FxAvailableAmt	Not in GAM table (Sum of different limits)
BANCS.OUTPARAM.FFDAvailableAmt	Not in GAM table (Sum of different limits)
BANCS.OUTPARAM.FxFFDAvailableAmt	Not in GAM table (Sum of different limits)
BANCS.OUTPARAM.EffAvailableAmt	Not in GAM table (Sum of different limits)
BANCS.OUTPARAM. FxEffAvailableAmt	Not in GAM table (Sum of different limits)
BANCS.OUTPARAM.FullAvailableAmt	Not in GAM table (Sum of different limits)
BANCS.OUTPARAM.FxFullAvailableAmt	Not in GAM table (Sum of different limits)
BANCS.OUTPARAM.FullEffAvailableAmt	Not in GAM table (Sum of different limits)
BANCS.OUTPARAM. FxFullEffAvailableAmt	Not in GAM table (Sum of different limits)

BANCS.OUTPARAM. dispBlockAmt

This field contains the total amount held by all the liens of BLIEN type which have been applied on the account.

Note:

This is the disp_block_amt field of the GAM table.

Return Value:

Return value is '0', if account found, else '1'.

This function returns the account details into OUTPARAM class of BANCS repository if the account exists in the data center database.

For Example:

Let us say that the bank does not want the account balance of 'Office accounts' to be displayed in the lower block of the Transaction Maintenance screen when doing a transaction. This information must not be available to the Teller. Then the CheckAndDispAcctBal.scr script can be coded as:

```
<--start
sv_a = BANCS.INPUT.AcctId
# -----
# - sv_a (account id) is the input to user hook
  getAcctDetailsInRepository
# - Call user hook getAcctDetailsInRepository to put the
  Account details into BANCS repository.
# -----

sv_b = urhk_getAcctDetailsInRepository(sv_a)
# if getAcctDetailsInRepository function returns failure exit out
  of script.
# This condition should not occur since BANCS.INPUT.AcctId
  would
# already have been validated to exists when this script is
  invoked.
if( sv_b == 1 ) then
  exitscript
endif
```

```

sv_c="O"
# acctOwnership flag is"O" for office accounts
# -----
# If the acctOwnerShip flag of ACCOUNT class is equal to"O"
# then don?t display dispblk2.acct_bal field.
# -----

if (BANCS.OUTPARAM.acctOwnerShip == sv_c) then
sv_z = "dispblk2.acct_bal|H"
sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)
else
sv_z = "dispblk2.acct_bal|U"
sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)
endif
exitscript
end-->

```

Note:

Refer to urhk_TBAF_ChangeFieldAttrib given separately for is usage

```

# -----
# If the tod amt changes at the time of posting (InstantTodAmt
# and TodLvIntFlg are inputs to the script),
# raise exception if the difference is more than 100
# -----

sv_e = cdouble(BANCS.INPUT.InstantTodAmt)
if (sv_e != 0.00 ) then

```

```
sv_a = BANCS.INPUT.AcctNum
sv_b = urhk_getAcctDetailsInRepository(sv_a)
if (sv_b == 1) then
BANCS.OUTPUT.OutString = "Account number is invalid"
BANCS.OUTPUT.successOrFailure="F"
exitscript
endif
sv_c = cdouble(BANCS.OUTPARAM.AvailableAmt)
sv_d = cdouble(BANCS.OUTPARAM.FullEffAvailableAmt)
if ( sv_d < 0 ) then
sv_d = 0
endif
sv_f = cdouble(BANCS.OUTPARAM.FullAvailableAmt)
sv_g = BANCS.INPUT.TodLvlIntFlg
sv_h = cdouble(BANCS.INPUT.TranAmt)
sv_i = ( sv_h - sv_d )
sv_j = ( sv_e - 100 )
sv_k = ( sv_e + 100 )
if ( ( sv_i < sv_j ) OR ( sv_i > sv_k ) ) then
BANCS.OUTPUT.successOrFailure = "F"
BANCS.OUTPUT.ErrorDesc = "Error: TOD Amt. is different from
that of specified value"
endif
endif
```

Get the Foracid from Acid

This function is used to get the Foracid corresponding to the Acid entered in the front end, from the table.

Syntax:

The syntax for the calling function is

```
sv_q = urhk_B2k_AcidToForacid ("" )
```

Input Arguments:

Acid

Input Repository Fields:

Input repository fields not required

Output Repository Fields:

BANCS.OUTPARAM.Foracid.

Return Value:

Return value is 0, if Foracid is returned, else 1.

For Example:

```
sv_a= urhk_B2k_AcidToForacid("PA220")  
print(BANCS.OUTPARAM.Foracid)
```

Get the Acid from Foracid

This function is used to get the Acid corresponding to the Foracid entered in the front end, from the table.

Syntax:

The syntax for the calling function is

```
sv_q = urhk_B2k_ForacidToAcid("")
```

Input Arguments:

Foracid

Input Repository Fields:

Input repository fields not required

Output Repository Fields:

BANCS.OUTPUTPARAM.Foracid.

Return Value:

Return value is 0, if acid is returned, else 1.

For Example:

```
sv_a= urhk_B2k_ForacidToAcid("10393564403101")  
print(BANCS.OUTPUTPARAM.Acid)
```

Update Lien

This userhook updates a given line amount to the given account. The other required data is also passed from the script.

Syntax:

```
sv_a = urhk_updateLien("")
```

Input Arguments:

Input Arguments not required.

Input Repository Fields:

BANCS.INPARAM.acctNum

BANCS.INPARAM.blockMode

BANCS.INPARAM.lienAmt

BANCS.INPARAM.remarks

BANCS.INPARAM.expiryDate

BANCS.INPARAM.reasonCode

BANCS.INPARAM.lienValueDate

BANCS.INPARAM.b2kId

Output Repository Fields:

Output Repository Field not required.

Return Value:

This user hook returns 0 on successful completion and the success and failure result is stored in BANCS.OUTPARAM.successOrFailure.

This will store 'S' for success and 'F' for failure

For Example:

```
BANCS.INPARAM.acctNum="BPMKS-INR"  
BANCS.INPARAM.blockMode="B"  
BANCS.INPARAM.lienAmt="3067"  
BANCS.INPARAM.remarks="NO REMARKS"  
BANCS.INPARAM.expiryDate="21-12-1998"  
BANCS.INPARAM.reasonCode="RSN"  
BANCS.INPARAM.lienValueDate="22-12-1998"  
BANCS.INPARAM.b2kId="AA3"  
sv_e = urhk_updateLien("")  
print(sv_e)  
print(BANCS.OUTPARAM.successOrFailure)
```

©Copyright Infosys Ltd.

Loan Account Interest Details

This userhook returns interest related details for an account based on account ID passed.

Syntax:

```
sv_a = urhk_getLoanAcctInterestDetails("")
```

Input:

Valid Loan Account Number

Input Repository Fields:

Field Name	Description
BANCS.INPUT.foracid	Valid Loan Account Number in GAM table.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.intrate	Interest rate applicable for the account.
BANCS.OUTPARAM.intAccrYTD	Interest Accrued year to date.
BANCS.OUTPARAM.intAccrDrYTD	Interest Accrued Debit year to date.
BANCS.OUTPARAM.intAccrCrYTD	Interest Accrued Credit year to date.
BANCS.OUTPARAM.intCollyTD	Interest Collected year to date.
BANCS.OUTPARAM.penalIntRate	Valid Penal interest Rate.
BANCS.OUTPARAM.intTableCode	Valid interest table code attached to account.
BANCS.OUTPARAM.custDrPcnt	Customer debit percentage.
BANCS.OUTPARAM.idDrPcnt	ID debit percentage.
BANCS.OUTPARAM.negotiatedDrPcnt	Negotiated debit percentage.
BANCS.OUTPARAM.diffRateFlg	Differential rate flag.

BANCS.OUTPARAM.intAccruedDate	Gives Interest accrued date.
BANCS.OUTPARAM.intAccruedTillDate	Interest accrued amount till date for the account.
BANCS.OUTPARAM.intCollectedTillDate	Interest collected till date for the account .
BANCS.OUTPARAM.intBookedTillDate	Interest booked till date for the account.
BANCS.OUTPARAM.penalIntCollTillDate	Penal interest amount collected till date.
BANCS.OUTPARAM.penalIntAccrTillDate	Penal interest amount accrued till date.
BANCS.OUTPARAM.intDemandSatTillDate	Interest demand satisfied till date for the account.
BANCS.OUTPARAM.intDemandSatYTD	Interest demand satisfied year to date.
BANCS.OUTPARAM.penalIntTblcode	Penal Interest table code if separate penal interest table code attached.
BANCS.OUTPARAM.penalPrefPcnt	Penal Preferential percentage if separate penal interest table code attached.
BANCS.OUTPARAM.penalIntWaivedTillDate	Penal Interest waived for the account till date.
BANCS.OUTPARAM.intWaivedTillDateExclude	Interest waived for the account till date.
BANCS.OUTPARAM.markUpPcnt	Markup percentage if loan is linked to Collateral.
BANCS.OUTPARAM.effPcnt	Effective percentage if loan is linked to collateral.

Output:

Returns 1 if the input values are invalid.

©Copyright Infosys Ltd.

Get the Float and the Unclear Balance

This user hook is used to get the float balance and the unclear balance for a given account ID.

Syntax:

```
sv_a = urhk_getFloatBalanceAmountAndUnclearBalance(<acid>)
```

Input:

Acid of the account.

Output Repository Fields:

The following output fields are available for customizing this user hook

Output Field	Description
BANCS.OUTPUTPARAM.float_bal	Float balance for the given account.
BANCS.OUTPUTPARAM.funds_in_clearing	Unclear balance

Output:

Success

For Example:

```
<--start
sv_s  =  BANCS.OUTPUTPARAM.acid

                                sv_t
urhk_getFloatBalanceAmountAndUnclearBalance(sv_s)  =

    sv_m = BANCS.OUTPUTPARAM.funds_in_clearing
sv_y= BANCS.OUTPUTPARAM.float_bal
end-->
```

©Copyright Infosys Ltd.

Finacle Copyright

Update the ECS Enabled Flag as Y at Account Level in the GAC Table

UpdateECSFlg:

This user hook is used to update the ECS enabled flag as Y at account level in the GAC table.

Syntax:

```
sv_q = urhk_updateECSFlg("")
```

Input:

Following is the input for this user hook: AcctNum

Output:

None

For Example:

```
BANCS.INPARAM. AcctNum = sv_a
```

```
sv_q = urhk_updateECSFlg ("" )
```

©Copyright Infosys Ltd.

Fetch the Last Charge Date

getCXLLastChargeDate:

This user hook is added to fetch the last Charge date. Acid & Event type is passed as input and Last charge date is the output.

Syntax:

```
sv_b = urhk_getCXLLastChargeDate("")
```

Input:

BANCS.INPPARAM.acid

Output:

BANCS.OUTPARAM.chrgDate

For Example:

NA

©Copyright Infosys Ltd.

Get the APY Details of the Account

getAcctApy:

This user hook gets the APY details of the account.

Syntax:

```
sv_a = urhk_getAcctApy("")
```

Input:

BANCS.INPARAM.acctNum

BANCS.INPARAM.crncyCode

BANCS.INPARAM.schmCode

BANCS.INPARAM.EnteredPrefInt

BANCS.INPARAM.cifld

Output:

BANCS.OUTPUT.APY

For Example:

NA

Get the APYE Details of the Account

getAcctApye:

This user hook gets the APYE details of the account.

Syntax:

```
sv_a = urhk_getAcctApye ("")
```

Input:

BANCS.INPUT.acctNum

Output:

BANCS.OUTPUT.APYE

For Example:

NA

©Copyright Infosys Ltd.

Cust Type Validation

This user hook is added to validate the account open matrix setup for a given combination of Scheme & Customer.

Syntax

`sv_b = urhk_custTypeValidation ("")` (With no optional inputs)

`sv_m=urhk_custTypeValidation ("MC|Y|Y||GC|||MAC|PAC")` (With optional inputs)

Input Arguments:

Scheme Code

Cifld

Input string (optional) containing the below fields:

- Mode Of Operation
- Cheque Allowed Flag
- Account turnover details flag
- Account ownership
- Guarantee Cover code
- Nature of Advance
- Type of Advance
- Mode of Advance
- Purpose of Advance

Input Repository Fields:

Field Name	Description
------------	-------------

BANCS.INPARAM.CifId Cif Id
BANCS.INPARAM.SchemeCode Scheme Code

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.matrixExcpVal	This variable will be set based on the validity of the customer type.

valid/invalid indicator for AOM record	AOM record & A/c field data comparison indicator	Value of matrixExcpVal
INVALID	ALL MATCHED	YES
VALID	NOT ALL MATCHED	YES
INVALID	NOT ALL MATCHED	NO
VALID	ALL MATCHED	NO

matrixExcpVal = YES – indicates that the customer type is invalid & exception needs to be raised.

matrixExcpVal = NO – indicates that the customer type is valid & no need to raise an exception.

Userhook Output:

Returns Success in case all userhook validations are through else returns failure.

©Copyright Infosys Ltd.

Interest Calculation Details for Account in Bancs Repository

This userhook returns interest calculation details for a given account number from Finacle. This userhook can be called from any script event. Checks for validity of account number specified. Checks for existence of account level interest entries.

Syntax:

sv_a = urhk_getInterestDetailsForAccount (Variable)

sv_a: scratch pad variable to store the return value.

Variable: Can be a scratch pad variable (sv_b) or a string ("EMILN1").

Input Arguments:

Valid account number

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.AccrdUptoDateCr	accrued_upto_date_cr in EIT table
BANCS.OUTPARAM.AccrdUptoDateDr	accrued_upto_date_dr in EIT table
BANCS.OUTPARAM.LastAccrlRunDateCr	last_accrual_run_date_cr in EIT table
BANCS.OUTPARAM.LastAccrlRunDateDr	last_accrual_run_date_dr in EIT table
BANCS.OUTPARAM.BookedUptoDateCr	booked_upto_date_cr in EIT table
BANCS.OUTPARAM.BookedUptoDateDr	booked_upto_date_dr in EIT table
BANCS.OUTPARAM.LastBookRunDateCr	last_book_run_date_cr in EIT table

BANCS.OUTPARAM.LastBookRunDateDr	last_book_run_date_dr in EIT table
BANCS.OUTPARAM.BookForRevDateCr	book_for_rev_date_cr in EIT table
BANCS.OUTPARAM.BookForRevDateDr	book_for_rev_date_dr in EIT table
BANCS.OUTPARAM.LastBookCrTranId	last_book_cr_tran_id in EIT table
BANCS.OUTPARAM.LastBookDrTranId	last_book_dr_tran_id in EIT table
BANCS.OUTPARAM.LastBookCrTranDate	last_book_cr_tran_date in EIT table
BANCS.OUTPARAM.LastBookDrTranDate	last_book_dr_tran_date in EIT table
BANCS.OUTPARAM.IntCalcUptoDateCr	interest_calc_upto_date_cr in EIT table
BANCS.OUTPARAM.IntCalcUptoDateDr	interest_calc_upto_date_dr in EIT table
BANCS.OUTPARAM.LastIntRunDateCr	last_interest_run_date_cr in EIT table
BANCS.OUTPARAM.LastIntRunDateDr	last_interest_run_date_dr in EIT table
BANCS.OUTPARAM.LastIntCrTranId	last_int_cr_tran_id in EIT table
BANCS.OUTPARAM.LastIntDrTranId	last_int_dr_tran_id in EIT table
BANCS.OUTPARAM.LastIntCrTranDate	last_int_cr_tran_date in EIT table
BANCS.OUTPARAM.LastIntDrTranDate	last_int_dr_tran_date in EIT table
BANCS.OUTPARAM.CummCrIntAdjAmt	cumm_cr_int_adj_amt in EIT table
BANCS.OUTPARAM.CummDrIntAdjAmt	cumm_dr_int_adj_amt in EIT table
BANCS.OUTPARAM.XferIntAmtCr	xfer_int_amt_cr in EIT table
BANCS.OUTPARAM.XferIntAmtDr	xfer_int_amt_dr in EIT table
BANCS.OUTPARAM.XferMinBal	xfer_min_bal in EIT table
BANCS.OUTPARAM.XferMinBalDate	xfer_min_bal_date in EIT table
BANCS.OUTPARAM.IntFreqTypeCr	int_freq_type_cr in EIT table
BANCS.OUTPARAM.IntFreqWeekNumCr	int_freq_week_num_cr in EIT table
BANCS.OUTPARAM.IntFreqWeekDayCr	int_freq_week_day_cr in EIT table
BANCS.OUTPARAM.IntFreqStartDdCr	int_freq_start_dd_cr in EIT table
BANCS.OUTPARAM.IntFreqHldyStatCr	int_freq_hldy_stat_cr in EIT table
BANCS.OUTPARAM.NextIntRunDateCr	next_int_run_date_cr in EIT table
BANCS.OUTPARAM.IntFreqTypeDr	int_freq_type_dr
BANCS.OUTPARAM.IntFreqWeekNumDr	int_freq_week_num_dr in EIT table

BANCS.OUTPUTPARAM.IntFreqWeekDayDr	int_freq_week_day_dr in EIT table
BANCS.OUTPUTPARAM.IntFreqStartDdDr	int_freq_start_dd_dr in EIT table
BANCS.OUTPUTPARAM.IntFreqHldyStatDr	int_freq_hldy_stat_dr in EIT table
BANCS.OUTPUTPARAM.NextIntRunDateDr	next_int_run_date_dr in EIT table

Return Value:

If specified account number is valid and interest details exist for the same at account level then return value is '0'. Else return value is '1'.

The details is populated into class OUTPUTPARAM of BANCS repository. Individual repository field names are specified against the table field name.

For Example:

Last debit interest calculation date, last debit interest accrued up to date of a past due account may be used for reporting purposes. These may be obtained as:

```
<--start
```

```
sv_a = BANCS.INPUT.AcctId
```

```
# -----
```

```
# - sv_a (Account Number) is the input to user hook
```

```
# getInterestDetailsForAccount
```

```
# -----
```

```
sv_b = urhk_getInterestDetailsForAccount (sv_a)
```

```
# User Hook will return failure if Account number passed is
```

```
# invalid or interest details are not maintained at account
```

```
# level for the account (BIA and FBA type).
```

```
# If the return value is '1' then populate error message and
```

```
# exit from script.
```

```
if( sv_b == 1 ) then
exitscript
endif
sv_i = BANCS.OUTPUTPARAM.AccrdUptoDateDr
sv_j = BANCS.OUTPUTPARAM.IntCalcUptoDateDr
sv_k = BANCS.OUTPUTPARAM.BookedUptoDateDr
sv_l = BANCS.OUTPUTPARAM.LastIntDrTranId
end
```

©Copyright Infosys Ltd.

Current and Previous Year Contribution for an ISA Account

This user hook is required to fetch the current and previous year contribution for an ISA account. This will help in any customization to validate previous or current year amounts. The user hook will also dump the contribution or limit cap defined at the sub-category level.

Syntax:

```
sv_b = urhk_getAccountContribution("")
```

Input arguments:

User will have to populate following field, before calling script hook.

BANCS.INPARAM.AcctId

Input Repository Fields:

Input not present

Output Repository Fields:

Output not present

Return value:

Following are the outputs from the user hook.

BANCS.OUTPARAM.successOrFailure

BANCS.OUTPARAM.CurrYearContribution

BANCS.OUTPARAM.PrevYearContribution

BANCS.OUTPARAM.contributionCap

BANCS.OUTPUTPARAM.withdrawalCap

BANCS.OUTPUTPARAM.prevYrContributionCap

Example:

```
<--start
trace on
BANCS.INPUTPARAM.AcctId = "ISATAX2"
sv_b = urhk_getAccoutContribution("")
#sv_d = BANCS.OUTPUTPARAM.successOrFailure
#sv_d = BANCS.OUTPUTPARAM.errorMesg
sv_e = BANCS.OUTPUTPARAM.contributionCap
sv_f =BANCS.OUTPUTPARAM.withdrawalCap
sv_g =BANCS.OUTPUTPARAM.prevYrContributionCap
sv_h =BANCS.OUTPUTPARAM.CurrYearContribution
sv_i =BANCS.OUTPUTPARAM.PrevYearContribution
trace off
end-->
```

©Copyright Infosys Ltd.

EMTD Record for an Account

This user hook takes the `excess_acct_id` and `excess_date` from the user and fetches the record from EMDT table for account ID and excess date combination. If record exists, then user hook returns all the others fields from the table of the record

Syntax:

```
sv_a = urhk_GetEMDTRecordForAcctId ("")  
sv_a: scratch pad variable to store the return value.
```

Input Arguments:

- `Excess_acct_id`
- `Excess_date`

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
EXCESS. GENERAL.EXCESS_ACCT_ID	Excess account for which emdt records are requested.
EXCESS. GENERAL.EXCESS_DATE	The excess date for which the emdt records are requested.
EXCESS. GENERAL.TOTAL_DUMPED_EMDT_RECORDS	Total dumped records

Output Repository Fields:

Field Name	Description
EXCESS. GENERAL.TOTAL_DUMPED_EMDT_RECORDS	Total dumped records

©Copyright Infosys Ltd.

Finacle Copyright

Update Mandate Ref No in LAM

This user hook is used to update mandate ref no in LAM table during UMD upload.

Syntax:

```
sv_a = urhk_updateMandateRefNoInLAM("")
```

sv_a: scratch pad variable to store the return value.

Input Arguments:

- Loan account
- mandate ref number

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.MandateRefNum	Mandate Ref Number
BANCS.INPARAM.LoanAcct	Corresponding acid value of the account number in GAM table

Output :

None

Example:

This user hook is used to update mandate ref no in LAM table during UMD upload.

For Example:

```
BANCS.INPARAM.MandateRefNum = BANCS.INPARAM.MANIRN
```

```
BANCS.INPARAM.LoanAcct = ONS.CUSTOM.foracid  
sv_r = urhk_updateMandateRefNoInLAM("")  
if(sv_r == 0) then  
    print("LAM updated Successfully")  
endif
```

©Copyright Infosys Ltd.

Current APR of an Open Account

Current APR of an Open Account userhook returns the current APR of an open account as on a given date from Finacle. This userhook can be called from any script event and validates account number. Returns failure if account number is invalid.

If input is valid, then the current APR is populated into Finacle repository.

Syntax:

```
sv_a = urhk_getCurrentAprForOpenLoanAcc("")
```

sv_a: scratch pad variable to store the account number.

Input Argument:

■Valid account number

The above input must be populated into the scratch pad variables.

Input Repository Fields:

None

Output Repository Field:

Field Name	Description
BANCS.OUTPARAM.currentApr	Current APR of the account.

Return Value:

If the input values are invalid then the return value is 1.

Current APR is populated into the repository.

For Example:

To obtain the current APR for an account, the steps are:

```
<--start
```

```
sv_a = BANCS.INPUT.foracid
```

```
sv_b = "clsflg | select acct_cls_flg from gam where foracid = ?  
SVAR "
```

```
BANCS.INPARAM.BINDVARS = sv_a
```

```
sv_c = urhk_dbSelectWithBind(sv_b)
```

```
if(sv_c == 0) then
```

```
if(BANCS.OUTPARAM.clsflg == "N") then
```

```
sv_d = urhk_getCurrentAprForOpenLoanAcc(sv_a)
```

```
if(fieldexists(BANCS.OUTPARAM.currentApr)) then
```

```
BANCS.OUTPUT.current_apr = BANCS.OUTPARAM.currentApr
```

```
endif
```

```
endif
```

```
endif
```

```
end-->
```

©Copyright Infosys Ltd.

Updation of Tax Event Status of Accounts for a Deceased CIF ID

The Updation of Tax Event Status of Accounts for a Deceased CIF ID userhook is used to support updation of all the accounts to mark the tax event status to 'Deregistration' for a deceased CIF ID.

Syntax:

```
sv_e      =      BANCS.INPUT.CIF_ID      +      "|"      +  
BANCS.INPUT.REASON_CODE      +      "|"      +  
BANCS.INPUT.DATE_OF_DECEASED      +      "|"      +  
BANCS.INPUT.DATE_OF_NOTIFICATION
```

```
sv_f = urhk_deRegisterTaxStatus(sv_e)
```

Input Arguments:

- CIF ID
- Reason Code
- DATE of Deceased
- DATE of Notification

Input Repository Fields:

Field Name	Description
BANCS.INPUT.CIF_ID	CIF_ID for which the tax status update to be done
BANCS.INPUT.REASON_CODE	RRCDM reason code for tax status updation
BANCS.INPUT.DATE_OF_DECEAED	Date on which customer deceased
BANCS.INPUT.DATE_OF_NOTIFICATION	Date of notification

Output Repository Field:

Field Name	Description
BANCS.OUTPUT.errorMsg	In case of any issues with updating the status – error message will be returned

Return Value:

- NF_FAILURE – This will be returned in case of functionality failure
- USRHK_FAILURE – This will be returned in case of user hook failure
- NF_SUCCESS – In case of successful updation this value will be returned.

For Example:

```
sv_a = BANCS.INPUT.CIF_ID
sv_b = BANCS.INPUT.REASON_CODE
sv_c = BANCS.INPUT.DATE_OF_DECEASED
sv_d = BANCS.INPUT.DATE_OF_NOTIFICATION
print(sv_a)
print(sv_b)
print(sv_c)
print(sv_d)
sv_e = sv_a + "|" + sv_b + "|" + sv_c + "|" + sv_d
sv_f = urhk_deRegisterTaxStatus(sv_e)
```

Get the APR Details of the Account/Scheme Code

The Get the APR Details of the Account/Scheme Code userhook gets the APR details of the account or scheme code. This userhook can be used to get the current APR value for various overdraft facilities for an existing account or to get the various combinations of APR values applicable for various overdraft facilities for a scheme code. Hence, this userhook can be used by existing customers and prospects to compare between various products of the bank. There is also a facility to override the existing Interest rate set up in Finacle with a new interest rate which helps the bank user to foresee new APR values without making changes to existing Product setup.

Syntax:

```
sv_a=urhk_getAcctApr("")
```

sv_a: scratch pad variable to store the return value.

Input Arguments:

- Scheme code
- Account ID
- Currency code
- CIF ID
- Preferential rates
- Overdraft facility (PL/SL/TOD/ALL)
- Interest Rate
- Balance

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctNum	Foracid
BANCS.INPARAM.crncyCode	crncyCode is optional. acct crncy code is acctNum is not null or crncyCode related to scheme code if acctNum is NULL.
BANCS.INPARAM.schmCode	Scheme Code
BANCS.INPARAM.EnteredPrefInt	EnteredPrefInt is added in net interest rate and further used in APR calculation
BANCS.INPARAM.cifld	If cifld is entered, customer preferential is considered in APR calculation.
BANCS.INPARAM.InterestRate	InterestRate
BANCS.INPARAM.inputBalance	The balance on which interest rate should be fetched

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.APR	APR
BANCS.OUTPARAM.APR_1	APR_1, 2, 3 will be output if APR is calculated for scheme code. This count basically indicates the slab present in interest table code.
BANCS.OUTPARAM.SLAB_CNT	Number of slab present for interest table code
BANCS.OUTPARAM.INT_RATE_1	INT_RATE_1 - will be output if APR is calculated for scheme code. This count basically indicates the slab present in interest table code.
BANCS.OUTPARAM.BEGIN_SLAB_AMT_1	BEGIN_SLAB_AMT_1 - For SBA/ODA/CAA schemeType.
BANCS.OUTPARAM.END_SLAB_AMT_1	END_SLAB_AMT_1 for SBA/ODA/CAA scheme Type
BANCS.OUTPARAM.INT_METHOD_1	INT_METHOD_1,2,3 ... Full or Differential for SBA/ODA and CAA type of scheme.

Return Value:

Returns the APR of the Account if found, else returns 0.

■ If A/c ID is present as one of the Inputs. The following will be the output

- APR if account is input

■ If Scheme code + CCY is given but A/c ID is not present as one of the Inputs. The following will be the output

- APR and slabs of interest table code if scheme is input

- APR (TOD) and slabs of interest table code if scheme is input

- A/c level APR and slabs of interest table code if scheme is input

For Example:

```
<--start
```

```
BANCS.INPARAM.acctNum = sv_a
```

```
BANCS.INPARAM.crncyCode = "INR"
```

```
BANCS.INPARAM.schmCode = "SBA1"
```

```
BANCS.INPARAM.EnteredPrefInt = ""
```

```
BANCS.INPARAM.cifId = ""
```

```
sv_v = urhk_getAcctApr ("" )
```

```
sv_v = cdouble (BANCS.OUTPARAM.APR)
```

```
BANCS.OUTPUT.APR = Format$(sv_v, "%.6f")
```

```
end-->
```

Get the APY Details of Account or Scheme

Syntax:

```
sv_a=urhk_getApyWithSlabs("")
```

Functionality:

This userhook is used for getting value of APY with inputs and output in multiple slabs in case of scheme code as input.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctNum	Foracid
BANCS.INPARAM.crncyCode	CrncyCode is optional. acct crncy code is acctNum is not null or crncyCode related to scheme code fi acctNum is NULL.
BANCS.INPARAM.schmCode	Scheme Code
BANCS.INPARAM.EnteredPrefInt	EnteredPrefInt is added in net interest rate and further used in APY calculation
BANCS.INPARAM.cifld	If cifld is entered, customer preferential is considered in APY calculation.
BANCS.INPARAM.interestRate	InterestRate
BANCS.INPARAM.inputBalance	The balance on which interest rate should be fetched.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.APY	APY
BANCS.OUTPARAM.APY_1	APY_1, 2,3 will be output if APY is calculated for scheme code. This count basically indicates the slab present in interest table code.
BANCS.OUTPARAM.SLAB_CNT	Number of slab present for interest table code

BANCS.OUTPARAM.INT_RATE_1	INT_RATE_1 - will be output if APY is calculated for scheme code. This count basically indicates the slab present in interest table code.
BANCS.OUTPARAM.BEGIN_SLAB_AMT_1	BEGIN_SLAB_AMT_1 - For SBA/ODA & CAA schemeType.
BANCS.OUTPARAM.END_SLAB_AMT_1	END_SLAB_AMT_1 for SBA/ODA/CAA scheme Type
BANCS.OUTPARAM.INT_METHOD_1	INT_METHOD_1,2,3 ... Full or Differential for SBA/ODA and CAA type of scheme.
BANCS.OUTPARAM.MAX_CONTRACT_MNTH_1	MAX_CONTRACT_MNTH_1 - For TDA/TUA scheme Type.
BANCS.OUTPARAM.MAX_CONTRACT_DAY_1	MAX_CONTRACT_DAY_1 - For TDA/TUA scheme Type
BANCS.OUTPARAM.BEGIN_RUN_MNTH_1	BEGIN_RUN_MNTH_1 - For TDA and TUA scheme type
BANCS.OUTPARAM.BEGIN_RUN_DAY_1	BEGIN_RUN_DAY_1 - For TDA/TUA scheme type

However, there are multiple scenarios which drives input and output values.

Scenario 1:

Input: If Input contains A/c ID as one of the Inputs

Output: APY

Input: A/c ID + Interest Rate combinations

Output: APY (The new Interest Rate given in Input is taken as r for APY calculation)

Input: A/c ID + Balance combinations

Output: APY

Input: A/c ID + Interest Rate + Balance combinations

Output: APY (The new Interest Rate given in Input is taken as r for APY calculation and Balance will be ignored)

Scenario 2:

Input: If Input contains Scheme code + CCY as one of the Inputs and does not contain A/c ID.

Output (For Operative A/c): Int. Rate, APY, Int. Method (Full/Differential/None), Start/End Slab

Output (For TD/TU A/c): Contracted Period, Run Period, Max. Slab Amt., Int. Rate, APY

(If Scheme Code + CCY then Int. Rate = Normal Cr. Rate,

Scheme Code + CCY + CIF ID then Int. Rate = Normal Cr. Rate + Cust. Preferentials,

Scheme Code + CCY + CIF ID + Preferential rates then Int. Rate = Normal Cr. Rate + Cust. Preferentials + Preferential rates,

Scheme Code + CCY + Preferential rates then Int. Rate = Normal Cr. Rate + Preferential rates)

CALLING_FUNC_NAME - Updates the calling_func_name field in the ADT table for the Audit record

AUDIT_ENTITY_ID - Updates the entity_id field in the ADT table for the Audit record

For Example:

This is an example for scheme code input case.

```
<--start
```

```
trace on
```

```
print("Inside 2.scr")
```

```
BANCS.INPARAM.acctNum = ""
```

```
BANCS.INPARAM.crncyCode = "INR"
```

```
BANCS.INPARAM.schmCode = "AAH1"
```



```

BANCS.INPARAM.EnteredPrefInt = "2.00"
BANCS.INPARAM.cifId = "410000"
BANCS.INPARAM.interestRate = "1"
BANCS.INPARAM.inputBalance = "2"
sv_a = urhk_getApyWithSlabs("")
sv_e = BANCS.OUTPARAM.ErrorMessage
print(sv_e)
sv_i = 1
sv_c = 0
if(sv_e == "") then
    sv_c = BANCS.OUTPARAM.SLAB_CNT
endif
while(sv_i <= sv_c)
#{
    sv_f = FORMAT$(sv_i, "%d")
    sv_g = ("BANCS").("OUTPARAM").("APY_" + sv_f)
    sv_h = ("BANCS").("OUTPARAM").("INT_RATE_" + sv_f)
    sv_k = ("BANCS").("OUTPARAM").
("BEGIN_SLAB_AMT_" + sv_f)
    sv_l = ("BANCS").("OUTPARAM").("END_SLAB_AMT_" + sv_f)
    sv_m = ("BANCS").("OUTPARAM").("INT_METHOD_" + sv_f)
    print(sv_i)
    print("Begin Slab Amt: " + sv_k)
    print("End Slab Amt: " + sv_l)
    print("Int Method: " + sv_m)
    print("INT Rate: " + sv_h)

```

```
print("APY: " + sv_g)
sv_i = sv_i + 1
#}
do
print(sv_c)
trace off
exitscript
end-->
```

©Copyright Infosys Ltd.

Fetch Valid Scheme Codes

This user hook is used for fetching valid scheme codes based upon AOM parameters.

Syntax

```
sv_a=urhk_getSchmDtlsFromAOM ("")
```

Input Repository Fields

Field Name	Description
BANCS.INPARAM.schmType	The valid scheme type as input SBA, CAA, ODA, TUA. Either Scheme Type or Product Group field has to be entered mandatorily.
BANCS.INPARAM.prodGroup	The field contains the Product Group specified at scheme level. The field can have multiple values separated by space. Maximum of 5 different product groups can be given as input. Either Scheme Type or Product Group field has to be entered mandatorily.
BANCS.INPARAM.crncyCode	The currency code of the given scheme.
BANCS.INPARAM.schmCode	Scheme Code (Optional).
BANCS.INPARAM.schmName	Scheme Name (Optional).
BANCS.INPARAM.staffSchm	Staff Scheme (Optional).
BANCS.INPARAM.solId	SOL ID (Mandatory).
BANCS.INPARAM.channelId	The Channel ID for the scheme.
BANCS.INPARAM.applicability	This field indicates about the channel applicability. The possible value for this field are O(origination), S(services) and B(Both).
BANCS.INPARAM.productType	This field is applicable only for SBA Scheme Type. The valid input values are: Regular Savings, Notice Savings, Normal Savings and ALL. (By default taken as ALL). (For other Scheme Types this field is taken as Null).

BANCS.INPARAM.ssaFlag	This field is applicable only for SBA and TUA Scheme types. The valid input values are: Yes (IRA Accounts only) or No (Non IRA Accounts only) or both (IRA and Non IRA Accounts) or ALL. (By default taken as ALL). (For other Scheme Types this field is Null).
BANCS.INPARAM.custStatus	Customer Status.
BANCS.INPARAM.custSector	Customer Sector.
BANCS.INPARAM.custSubSector	Customer Sub-sector.
BANCS.INPARAM.custType	Customer Type Code.
BANCS.INPARAM.custConst	Customer Constitution.
BANCS.INPARAM.custMinor	Customer Minor.
BANCS.INPARAM.custStaffId	Customer Staff ID.
BANCS.INPARAM.custOtherBank	Customer Other Bank.
BANCS.INPARAM.modOfOper	Mode of Operation
BANCS.INPARAM.chqAlwdFlg	Cheque Allowed.
BANCS.INPARAM.guaranteeCoverCode	Guarantee Cover.
BANCS.INPARAM.acctTurnOverDtIfFlg	Turnover required.
BANCS.INPARAM.acctOwnership	A/c. ownership.
BANCS.INPARAM.natureOfAdvn	Nature of Advance.
BANCS.INPARAM.typeOfAdvn	Advance type.
BANCS.INPARAM.modeOfAdvn	Mode of Advance.
BANCS.INPARAM.purposeOfAdvn	Purpose of Advance.
BANCS.INPARAM.freeText1	Free Text 1.
BANCS.INPARAM.freeText2	Free Text 2.
BANCS.INPARAM.freeText3	Free Text 3.
BANCS.INPARAM.freeText4	Free Text 4.
BANCS.INPARAM.perfMatch	Perfect Match (Y/N) – defaulted to YES, unless provided.

BANCS.INPARAM.effDate	Effective Date.
BANCS.INPARAM.depositPrdMnth	Deposit Period in months (MMM) - applicable for TUA type of schemes only.
BANCS.INPARAM.depositPrdDays	Deposit Period in days (DDD) - applicable for TUA type of schemes only.
BANCS.INPARAM.depAmt	Deposit Amount.

Output Repository Fields

Field Name	Description
BANCS.OUTPARAM. Product_Count	The total number of records returned.
BANCS.OUTPARAM.schmType_n	Scheme Type.
BANCS.OUTPARAM.schmCode_n	Scheme Code.
BANCS.OUTPARAM.prodGroup_n	Product Group.
BANCS.OUTPARAM.schmName_n	Scheme Name.
BANCS.OUTPARAM.currencyCode_n	Currency Code.
BANCS.OUTPARAM.schmExpDate_n	Scheme Code Expiry Date.
BANCS.OUTPARAM.minDepAmt_n	Minimum Deposit Amount.
BANCS.OUTPARAM.prodType_n	Product Type. The Product Type Output field is populated for SBA Scheme Type only and displays the Product Type value as Regular Savings or Notice Savings or Normal Savings (For other Scheme Types this field is Null).
BANCS.OUTPARAM.ssaFlag_n	SSA Flag The SSA flag Output field is populated for SBA and TUA Scheme types only and displays the SSA flag value as Y (IRA Accounts only) or O (Non IRA Accounts only) or B (IRA and NON IRA Accounts). (For other Scheme Types this field is Null).
BANCS.OUTPARAM.intTblCode_n	Interest Table Code.
BANCS.OUTPARAM.intRate1_n	Interest Rate (from range).

BANCS.OUTPARAM.intRate2_n	Interest Rate (to range).
BANCS.OUTPARAM.APY1_n	APY value (from range).
BANCS.OUTPARAM.APY2_n	APY value (to range).
BANCS.OUTPARAM.minAmtSlab_n	Minimum Amount Slab.
BANCS.OUTPARAM.maxAmtSlab_n	Maximum Amount Slab.
BANCS.OUTPARAM.startDate_n	Start Date.
BANCS.OUTPARAM.endDate_n	End Date.
BANCS.OUTPARAM.minDepPrdSlab_n	Minimum Deposit Period Slab.
BANCS.OUTPARAM.maxDepPrdSlab_n	Maximum Deposit Period Slab.
BANCS.OUTPARAM.succFailFlg	This field is used internally in the <i>prodSelector.scr</i> file to check if the process is successfully or failed.
BANCS.OUTPARAM.errorMessage	Error Message (in case any error message is returned).

©Copyright Infosys Ltd.

Update Account Status to Active

This userhook has been provided in posting module for updating the account status to Active. This user hook is available only in FinTran.scr script. This user hook expects two inputs in BANCS.INPARAM: AcctNum and Scheme Code. This userhook returns '0' on success and '1' on failure.

This User hook will return failure for below cases:

- Invalid account number.
- Scheme Type not in SB/CA/CC/OD
- If record not found in SMT for SB/CA type of scheme for the given account.
- If record not found in CAM for CC/OD type of scheme for the given account.

In case of above checks being success, the user hook will do the following:

- Update the status of the account to Active.
- Add an audit record with remark as "ACCOUNT ACTIVATED" and modified fields data as "account_status|CurrStatus|A"

Syntax:

```
sv_a=urhk_FTS_ActivateAccount("")
```

Input Arguments:

- Input to userhook is through the input repository.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.AcctNum	Account Number available in BANCS.INPUT.AcctNum
BANCS.INPARAM.SchemeType	Scheme Type available in BANCS.INPUT.SchemeType

Output Repository Fields:

Output is not present.

Return Value:

This userhook returns '0' on success and '1' on failure.

Note:

1. User hook doesn't check if the current status of account is active or not. It always updates the status as Active and Status date as current date.
2. Table name in Audit record will be SMT(or SB/CA) or CAM(CC/OD) based on the scheme type.
3. BANCS.INPUT. AccountStatus and BANCS.INPUT.AccountStatusDate are also available as inputs.

For Example:

BANCS.INPARAM.AcctNum = BANCS.INPUT.AcctNum

BANCS.INPARAM.SchemeType = BANCS.INPUT.SchemeType

sv_a = urhk_FTS_ActivateAccount("")

Check Maturity For An Account

This user hook is used to Validate Maturity date w.r.t execution date. On success it returns value "0".

Syntax:

```
sv_a = urhk_chkMaturityForAcnt( )
```

Input Arguments:

Valid account number

Request Date

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctId	Valid Account Number in GAM table
BANCS.INPARAM.RegDate	Date to be validated

Return Value:

It returns integer value as:

- 0(Zero): BumpUP Request date is beyond A/c maturity date.
- Non-Zero: BumpUp request date is after account maturity date.

For Example:

```
<--start
```

```
BANCS.INPARAM.acctId = BANCS.INPUT.acctId
```

BANCS.INPARAM.RegDate = BANCS.INPUT.RegDate

sv_a = Urhk_chkMaturityForAcnt()

end-->

©Copyright Infosys Ltd.

Check Maximum Withdrawal For An Account

This user hook is used to Validation for Max withdrawal Amount. On success it returns value "0".

Syntax:

```
sv_a = urhk_chkMaxWithdrawForAcnt( )
```

Input Arguments:

Valid account number

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
------------	-------------

BANCS.INPARAM.acctId	Valid Account Number in GAM table
----------------------	-----------------------------------

Return Value:

It returns integer value as:

- 0(Zero) : validation successful for Max withdrawal Amount
- Non-Zero: Total withdrawn/part closed amt. exceeds the Max. Withdrawal limits configured.

For Example:

```
<--start
```

```
BANCS.INPARAM.acctId = BANCS.INPUT.acctId
```

```
sv_a = urhk_chkMaxWithdrawForAcnt( )
```

end-->

©Copyright Infosys Ltd.

Finacle Copyright

Get Bump Up Frequency Status

Checks Bump Up Frequency status for an Account

Syntax:

```
sv_a = urhk_getBumpUpFreqStatus( )
```

Input Arguments:

- Valid account number
- Request Date
- Scheme Code
- Currency Code

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.accountId	Valid Account Number in GAM table
BANCS.INPARAM ReqDate	Request Date
BANCS.INPARAM.schm_code	Valid Scheme Code
BANCS.INPARAM.crncy_code	Valid Currency Code

Return Value:

It returns value as:

- S: Success
- F: Failure

For Example:

```
<--start
```

```
BANCS.INPARAM.acctId = BANCS.INPUT.acctId
```

```
BANCS.INPARAM.RegDate = BANCS.INPUT.RegDate
```

```
BANCS.INPARAM.schm_code = BANCS.INPUT.schm_code
```

```
BANCS.INPARAM.crncy_code = BANCS.INPUT.crncy_code
```

```
sv_a = urhk_getBumpUpFreqStatus( )
```

```
end-->
```

©Copyright Infosys Ltd.

Get Bump Up Period Status

Checks Minimum Period for First Bump Up for an Account

Syntax:

```
sv_a = urhk_getBumpUpPeriodStatus( )
```

Input Arguments:

- Valid account number
- Request Date
- Scheme Code
- Currency Code

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.accountId	Valid Account Number in GAM table
BANCS.INPARAM.RegDate	Request Date
BANCS.INPARAM.schm_code	Valid Scheme Code
BANCS.INPARAM.crncy_code	Valid Currency Code

Return Value:

It returns value as:

- S: Success
- F: Failure

For Example:

<--start

BANCS.INPARAM.acctId = BANCS.INPUT.acctId

BANCS.INPARAM.RegDate = BANCS.INPUT.RegDate

BANCS.INPARAM.schm_code = BANCS.INPUT.schm_code

BANCS.INPARAM.crncy_code = BANCS.INPUT.crncy_code

sv_a = urhk_getBumpUpPeriodStatus()

end-->

©Copyright Infosys Ltd.

Get Number of Bump Up Allowed Utilized

Populate the Number of Bump Up Allowed/Utilized Details.

Syntax:

```
sv_a = urhk_getNoOfBumpUpAllwdUtlzd( )
```

Input Arguments:

Valid account number

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctId	Valid Account Number in GAM table

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.bumpUpAllwd	No of Bump Up allowed
BANCS.OUTPARAM.bumpUpUtlzd	No of Bump Up utilised

Return Value:

It returns integer value as:

- Zero: Success
- Non-Zero: Failure

For Example:

```
<--start  
BANCS.INPARAM.acctId = BANCS.INPUT.acctId  
Sv_a = urhk_getNoOfBumpUpAllwdUtlzd( )  
sv_b = BANCS.OUTPARAM.bumpUpAllwd  
sv_c = BANCS.OUTPARAM.bumpUpUtlId  
end-->
```

©Copyright Infosys Ltd.

Populate and Insert Into BURHT

This userhook deletes entry from BURT table and insert into BURHT table.

Syntax:

```
sv_a = urhk_PopulateAndInsertIntoBURHT( )
```

Input Arguments:

Valid account number

Bump up status

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.accountId	Valid Account Number in GAM table
BANCS.INPARAM.bump_up_status	Bump Up Status

Return Value:

It returns integer value as:

- Zero: Insert or update into BURHT table successful.
- Non-Zero: Insert or update into BURHT table failed.

For Example:

```
<--start
```

```
BANCS.INPARAM.accountId = BANCS.INPUT.accountId
```

```
BANCS.INPARAM.bump_up_status      =  
BANCS.INPUT.bump_up_status  
  
sv_a = urhk_PopulateAndInsertIntoBURHT( )  
end-->
```

©Copyright Infosys Ltd.

Third Party Account Close For Package Account

This user hook is used to close the link third party account of a valid package. On success it returns “0”.

Syntax:

```
sv_a = urhk_trdPrtyAcctClsForPkgAcct( )
```

Input Arguments:

Third Party Account Reference Id

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.trdPrtyAcctRefId	Valid Third Party Account Reference Id.

Return Value:

It returns integer value as:

- 0(Zero) : if closure of third party account happen successfully
- Non-Zero : if closure of third party account not done.

For Example:

```
<--start
```

```
BANCS.INPARAM.trdPrtyAcctRefId =  
BANCS.INPUT.trdPrtyAcctRefId
```

```
sv_a = urhk_trdPrtyAcctClsForPkgAcct( )
```

```
end-->
```

©Copyright Infosys Ltd.

Finacle Copyright

Get Net Limit Accounts

The user hook gets the net limit amounts from limits library via the function "getAmounts". This is applicable in case of only UK locale.

Syntax:

```
sv_a = urhk_getNetLimitAmounts( )
```

Input Arguments:

B2K Id of Limit

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.limit_b2kid	Valid B2K Id of limit

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.net_ledger_balance	Net Ledger Balance amount
BANCS.OUTPARAM.gross_ledger_balance	Gross Ledger Balance Amount
BANCS.OUTPARAM.net_amt_balance	Net Amount Balance

Return Value:

It returns integer value as:

- 0(Zero) : Success
- Non-Zero : Failure.

For Example:

<--start

BANCS.INPARAM.limit_b2kid = BANCS.INPUT.limit_b2kid

sv_a = urhk_getNetLimitAmounts()

BANCS.OUTPUT.net_ledger_balance =
BANCS.OUTPARAM.net_ledger_balance

BANCS.OUTPUT.gross_ledger_balance =
BANCS.OUTPARAM.gross_ledger_balance

BANCS.OUTPUT.net_amt_balance =
BANCS.OUTPARAM.net_amt_balance

end-->

©Copyright Infosys Ltd.

Clearing Userhooks

Clearing is one of the activities a bank undertakes as a part of its business. Finacle™ system provides facility for handling clearing activities of a Bank. Clearing activities can be divided into inward and outward clearing.

Inward clearing is a process whereby the bank receives various types of negotiable instruments that are drawn on it by its customers and parts the resultant proceeds to the presenter of the instruments.

In case of Outward clearing, the bank gets the proceeds since it acts as a collecting agent on behalf of its customers who have tendered the instruments for credit of their accounts.

There are scripting userhooks available which help us in fetching various details related to Clearing from the database.

This section discusses:

[Get Details from CTC Table for Transaction Code](#)

[Get Inward Clearing Zone Header Details](#)

[Get Outward Clearing Zone Header Details](#)

[Get Outward Clearing Instrument Details](#)

[Get Outward Clearing Part Transaction Details](#)

[Raise Exception for Clearing Transactions](#)

[Process ACH Nonfinancial Message](#)

[Check Duplicate Field](#)

Get Details from CTC Table for Transaction Code

This function gets all the attributes of a 'Clearing transaction code' (as printed on the instrument) that is maintained in the CTC table of Finacle. Clearing transaction codes are used in India to distinguish between various types of clearing.

Syntax:

The syntax for calling the function is

```
sv_r = urhk_GetCTCDetailsFromTranCode(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("10").

Input Arguments:

The input to the function is Clearing transaction code

Example - "10"

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.SysArrivedAcctFlg	SysArrivedAcctFlg
BANCS.OUTPARAM.DrBacid	DrBacid
BANCS.OUTPARAM.InstrmntType	InstrmntType
BANCS.OUTPARAM.MediaLoadFlg	MediaLoadFlg
BANCS.OUTPARAM.ApplicabilityInd	ApplicabilityInd
BANCS.OUTPARAM.ConsolidationFlg	ConsolidationFlg
BANCS.OUTPARAM.DefaultUpldAcctPrefix	DefaultUpldAcctPrefix

BANCS.OUTPARAM.SearchSchmType	SearchSchmType
-------------------------------	----------------

Return Value:

Returns '1' as return value if the transaction code does not exists in the CTC table, else returns '0'.

©Copyright Infosys Ltd.

Get Inward Clearing Zone Header Details

This function returns the attributes of the Inward clearing zone from the IZH table.

Syntax:

The syntax for calling the function is

```
sv_r = urhk_GetZoneHeaderDetails(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("AM|12-12-1999 00:00:00|123456")

Input Arguments:

Input String contains three parts separated by a '|'

Zone Code

Zone Date (DD-MM-YYYY 00:00:00)

Col ID

For Example:

AM|12-12-1999 00:00:00|123456

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.ValueDate	ValueDate
BANCS.OUTPARAM.TranDate	TranDate

BANCS.OUTPARAM.MICRCIgFlg	MICRCIgFlg
BANCS.OUTPARAM.MedialnpAlwdFlg	MedialnpAlwdFlg
BANCS.OUTPARAM.DefaultCarveFlg	DefaultCarveFlg
BANCS.OUTPARAM.TrancodeMandatory	TrancodeMandatory
BANCS.OUTPARAM.InstTypeMandatory	InstTypeMandatory

Return Value:

Return value is '1' if no zone is found, else it is '0'.

©Copyright Infosys Ltd.

Get Outward Clearing Zone Header Details

This function returns the attributes of the Outward clearing zone from the OZH table.

Syntax:

The syntax for calling the function is

```
sv_r = urhk_getOutZoneHeaderDetails (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("SOL100|12-12-1999 00:00:00|ZONE10").

Input Arguments:

Input String contains three parts separated by a '|':

Sol ID to which the zone belongs.

Zone Date (DD-MM-YYYY 00:00:00)

Zone Code

For Example:

SOL100|12-12-1999 00:00:00|ZONE10

This zone must be a valid zone. If the zone does not exist, the user gets a fatal error.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
------------	-------------

BANCS.OUTPUTPARAM.ClgSolId	sol_id in OZH table
BANCS.OUTPUTPARAM.ClgZoneDate	clg_zone_date in OZH table
BANCS.OUTPUTPARAM.ClgZoneCode	clg_zone_code in OZH table
BANCS.OUTPUTPARAM.ParkingCrAcid	cr_acid in OZH table
BANCS.OUTPUTPARAM.ClgDrAcid	dr_acid in OZH table
BANCS.OUTPUTPARAM.HomeClgFlg	home_clg_flg in OZH table
BANCS.OUTPUTPARAM.ShadowBalFlg	shadow_bal_flg in OZH table
BANCS.OUTPUTPARAM.MicrClgFlg	micr_clg_flg in OZH table
BANCS.OUTPUTPARAM.EncodeFlg	encode_flg in OZH table
BANCS.OUTPUTPARAM.ZoneMicrChrgCode	zone_micr_chrg_code in OZH table
BANCS.OUTPUTPARAM.ClgSecDpCode	clg_sec_dp_code in OZH table
BANCS.OUTPUTPARAM.OZHTranDate1	tran_date1 in OZH table
BANCS.OUTPUTPARAM.OZHTranDate2	tran_date2 in OZH table
BANCS.OUTPUTPARAM.ZoneStat	zone_stat in OZH table
BANCS.OUTPUTPARAM. OZHValueDate1	value_date1 in OZH table
BANCS.OUTPUTPARAM.OZHValueDate2	value_date2 in OZH table
BANCS.OUTPUTPARAM.BarInstAmt	bar_inst_amtin OZH table
BANCS.OUTPUTPARAM.BarInstNum	bar_inst_num in OZH table
BANCS.OUTPUTPARAM.BarNum	bar_num in OZH table
BANCS.OUTPUTPARAM.BarDate	bar_date in OZH table
BANCS.OUTPUTPARAM.FrgnCrcncyClgFlg	frgn_crcncy_clg_flg in OZH table
BANCS.OUTPUTPARAM.SttlmntInInstCrcncyFlg	sttlmnt_in_inst_crcncy_flg in OZH table
BANCS.OUTPUTPARAM.SttlmntCrcncyCode	sttlmnt_crcncy_code in OZH table
BANCS.OUTPUTPARAM.ZoneCrcncyCode	zone_crcncy_code in OZH table
BANCS.OUTPUTPARAM.ZoneToSttlmntCrcncyRate	zone_to_sttlmnt_crcncy_rate in OZH table
BANCS.OUTPUTPARAM.ZoneToSttlmntCrcncyRateCode	rate_code in OZH table
BANCS.OUTPUTPARAM. EventId	event_id in OZH table

BANCS.OUTPARAM.EventType	event_type in OZH table
BANCS.OUTPARAM.DefRateCode1	def_rate_code1 in OZH table
BANCS.OUTPARAM.DefRateCode2	def_rate_code2 in OZH table
BANCS.OUTPARAM.ClgSecBankCode	clg_sec_bank_code in OZH table
BANCS.OUTPARAM.InstOrAcctPtranCreFlg	inst_or_acct_ptran_cre_flg in OZH table
BANCS.OUTPARAM.set_rel_reg_flg	set_rel_reg_flg in OZH table
BANCS.OUTPARAM.SetRelDrAcid	set_rel_dr_acid in OZH table
BANCS.OUTPARAM.ZoneLatency	zone_latency in OZH table
BANCS.OUTPARAM.OutRejEventId	out_rej_event_id in OZH table
BANCS.OUTPARAM.CrAcidForOutRej	cr_acid_for_out_rej in OZH table
BANCS.OUTPARAM.ControlSolId	control_sol_id in OZH table

Return Value:

Return value is '0' when zone is found. If zone is not found, then user gets a fatal error.

The attributes of the zone stored in the OZH table is returned in the fields of BANCS repository and OUTPARAM class.

©Copyright Infosys Ltd.

Get Outward Clearing Instrument Details

This function returns the details of the Outward clearing instrument from the OCI table.

Syntax:

The syntax for calling the function is

`sv_r = urhk_getOutClgInstDetails (Variable)`

The variable can be a scratch pad variable (sv_b) or a string ("SOL100|12-12-1999 00:00:00|ZONE10| 1| 10").

Input Arguments:

Input string contains five parts separated by a '|'

SOL ID to which the zone belongs.

Zone Date (DD-MM-YYYY 00:00:00)

Zone Code

Set Number

Instrument Srl Num.

For Example:

"SOL100|12-12-1999 00:00:00|ZONE10| 1| 10"

This instrument should exist in the OCI table. If the instrument does not exist, the user gets a fatal error.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.ClgSolId	ClgSolId
BANCS.OUTPARAM.ClgZoneDate	ClgZoneDate
BANCS.OUTPARAM.ClgZoneCode	ClgZoneCode
BANCS.OUTPARAM.SetNum	SetNum
BANCS.OUTPARAM.InstrmntSrlNum	InstrmntSrlNum
BANCS.OUTPARAM.ShdBalCode	ShdBalCode
BANCS.OUTPARAM.TranCode	TranCode
BANCS.OUTPARAM.InstrmntId	InstrmntId
BANCS.OUTPARAM.InstrmntAmt	InstrmntAmt
BANCS.OUTPARAM.NcodeFlg	NcodeFlg
BANCS.OUTPARAM.RepCode	RepCode
BANCS.OUTPARAM.BankCode	BankCode
BANCS.OUTPARAM.BrCode	BrCode
BANCS.OUTPARAM.SortCode	SortCode
BANCS.OUTPARAM.EncodeSeqNum	EncodeSeqNum
BANCS.OUTPARAM.StatusFlg	StatusFlg
BANCS.OUTPARAM.InwRejZoneCode	InwRejZoneCode
BANCS.OUTPARAM.InwRejZoneDate	InwRejZoneDate
BANCS.OUTPARAM.InwRejInstNum	InwRejInstNum
BANCS.OUTPARAM.InwRejCtrlKeyNum	InwRejCtrlKeyNum
BANCS.OUTPARAM.AcctSolId	AcctSolId
BANCS.OUTPARAM.InstValueDate	InstValueDate
BANCS.OUTPARAM.RejType	RejType
BANCS.OUTPARAM.RejCode_1	RejCode_1
BANCS.OUTPARAM.RejCode_2	RejCode_2
BANCS.OUTPARAM.RejCode_3	RejCode_3

BANCS.OUTPARAM.RejCode_4	RejCode_4
BANCS.OUTPARAM.RejCode_5	RejCode_5
BANCS.OUTPARAM.RejDate	RejDate
BANCS.OUTPARAM.InwRejZoneSrlNum	InwRejZoneSrlNum
BANCS.OUTPARAM.InwRejSolId	InwRejSolId

Return Value:

Return value is '0' when Instrument is found. If instrument is not found, then user gets a fatal error.

©Copyright Infosys Ltd.

Get Outward Clearing Part Transaction Details

This function returns the details of the Outward clearing part transaction from the OCP table.

Syntax:

The syntax for calling the function is

```
sv_r = urhk_getOutClgPtranDetails (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("SOL100|12-12-1999 00:00:00|ZONE10| 1| 10").

Input Arguments:

Input string contains five parts separated by a '|':

Sol ID to which the zone belongs

Zone Date (DD-MM-YYYY 00:00:00)

Zone Code

Set Number

Part Tran Srl Num

For Example:

```
SOL100|12-12-1999 00:00:00|ZONE10| 1| 10
```

This part transaction must exist in the OCP table. If the part transaction does not exist, the user gets a fatal error.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.sol_id	ClgSolId in OCP table
BANCS.OUTPARAM.clg_zone_date	ClgZoneDate in OCP table
BANCS.OUTPARAM.clg_zone_code	ClgZoneCode in OCP table
BANCS.OUTPARAM.set_num	SetNum in OCP table
BANCS.OUTPARAM.part_tran_srl_num	PtranSrlNum in OCP table
BANCS.OUTPARAM.Acid	PtranAcid in OCP table
BANCS.OUTPARAM.tran_amt	TranAmt in OCP table
BANCS.OUTPARAM.part_tran_type	PartTranType in OCP table
BANCS.OUTPARAM.prnt_advclg_flg	PrntAdvclgFlg in OCP table
BANCS.OUTPARAM.pend_amt	PendAmt in OCP table
BANCS.OUTPARAM.partcls	Partcls in OCP table
BANCS.OUTPARAM.tran_rpt_code	TranRptCode in OCP table
BANCS.OUTPARAM.tran_rmks	TranRmks in OCP table
BANCS.OUTPARAM.shd_bal_rem_amt	ShdBalRemAmt in OCP table
BANCS.OUTPARAM.status_flg	PtranStatusFlg in OCP table
BANCS.OUTPARAM.navigation_flg	NavigationFlg in OCP table
BANCS.OUTPARAM.acct_crncy_code	AcctCrncyCode in OCP table
BANCS.OUTPARAM.part_tran_amt_in_inst_crncy	PartTranAmtInInstCrncy in OCP table
BANCS.OUTPARAM.sttl_inst_to_hm_crncy_rate	SttlInstToHmCrncyRate in OCP table
BANCS.OUTPARAM.hm_to_acct_crncy_rate	HmToAcctCrncyRate in OCP table
BANCS.OUTPARAM.sttl_inst_to_acc_crncy_rate	SttlInstToAccCrncyRate in OCP table
BANCS.OUTPARAM.sttl_to_hm_rate_code	SttlToHmRateCode in OCP table
BANCS.OUTPARAM.hm_to_acct_rate_code	HmToAcctRateCode in OCP table
BANCS.OUTPARAM.sttl_to_acct_rate_code	SttlToAcctRateCode in OCP table
BANCS.OUTPARAM.acct_sol_id	AcctSolId in OCP table
BANCS.OUTPARAM.pend_amt_in_inst_crncy	PendAmtInInstCrncy in OCP table

BANCS.OUTPARAM.td_ref_num	TdRefNum in OCP table
BANCS.OUTPARAM.set_rel_reg_stat	SetRelRegStat in OCP table

Return Value:

Return value is '0' when part transaction is found. If part transaction is not found, then user gets a fatal error.

The details of the part transaction stored in the OCP table is returned in the fields of BANCS repository and OUTPARAM class.

©Copyright Infosys Ltd.

Raise Exception for Clearing Transactions

This function raises an exception corresponding to the given exception code, during online lodging of outward clearing transactions. It returns the error description in case the exception code given was set as an error. It can be used in the context of Online clearing transaction Event (Event 'OWCLG_LODGE' of FinTran.scr script). This function checks if the given exception code is an error and if so, populates the error description in the OUTPARAM field with the appropriate error and returns '1'. Otherwise, it raises the exception just the same way the built-in exceptions are raised during lodging.

Syntax:

sv_a = urhk_OWCLG_RaiseException (Variable)

The variable can be a scratch pad variable (sv_b) or a string('482').

Input Arguments:

The input to this function through the INPARAM field is 'AcctNum'. The input through the function argument is an exception code.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value is '1', if exception was found to be an error. Else '0'.

This function returns the corresponding error description into the OUTPARAM class of BANCS repository if the given exception was an

error.

Field name in BANCS repository

BANCS.OUTPUTPARAM. ErrorDesc

For Example:

Say during lodging, an exception (code 482) needs to be raised if the part transaction amount is not multiple of 1000 in SBFIX scheme code. This is under the 'OWCLG_LODGE' event of the FinTran.scr script.

```
<-- start
```

```
BANCS.OUTPUT.successOrFailure = "S"
```

```
BANCS.OUTPUT.ErrorDesc = ""
```

```
BANCS.OUTPUT.AddtlDetails = ""
```

```
BANCS.OUTPUT.DeleteUAD = "Y"
```

```
if (BANCS.INPUT.Event == "OWCLG_LODGE") then
```

```
# If the amount is not multiple of 1000's in SBFIX scheme  
code, then raise
```

```
# an exception
```

```
if (BANCS.INPUT.SchemeCode == "SBFIX") then
```

```
sv_a = BANCS.INPUT.TranAmt
```

```
sv_b = strlen(sv_a)
```

```
sv_c = mid$(sv_a, sv_b - 6, sv_b)
```

```
if (sv_c != "000.00") then
```

```
# fill the INPARAM
```

```
BANCS.INPARAM.AcctNum = BANCS.INPUT.AcctNum
```

```
sv_d = urhk_OWCLG_RaiseException("482")
```

```
if (sv_d == 1) then
```

```
# It is an error.
```



```
BANCS.OUTPUT.successOrFailure = "F"  
BANCS.OUTPUT.ErrorDesc = BANCS.OUTPARAM.ErrorDesc  
exitscript  
endif  
endif  
endif  
exitscript  
<-- end
```

©Copyright Infosys Ltd.

Process ACH Nonfinancial Message

This new userhook takes AED entry_srl_num as Input and logic will be customized.

Syntax:

```
sv_b = urhk_ processACHNonFinMsg("")
```

sv_b: scratch pad variable to store the return value.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.ENTRYSRNUM	AED ENTRY SRL NUM (Mandatory)
BANCS.INPARAM.PAYSYSID	PAYSYSID which is of NACHA type

Output Repository Fields:

Field Name	Description
BANCS.INPARAM.successOrFailure	successOrFailure
BANCS.INPARAM.error	error

Userhook Output:

By default, product will update the necessary tables. If any additional logic or updates are required, the same can be done through this script.

©Copyright Infosys Ltd.

Check Duplicate Field

The Check Duplicate Field userhook is used to check if duplicate field entered during ACH upload already exists in AHD table.

Syntax:

```
sv_b = urhk_checkDuplicateField ("")
```

sv_b: scratch pad variable to store the return value.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM. DuplicateField	Duplicate Field
BANCS.INPARAM. UploadedDate	Uploaded Date
BANCS.INPARAM. DupChkDays	Duplicate Check Days
BANCS.INPARAM.BatchId	Batch Id entered by user in input file during ACH upload

Output Repository Field:

Field Name	Description
BANCS.OUTPARAM. DuplicateCount	Count indicating no. of records existing for above inputs

For Example:

To check duplicate field entered in upload file already exists for ACH_Input.dat|DEP0054599:

```
sv_a= "ACH_Input.dat|DEP0054599"
```

```
sv_b = 0
```

```
BANCS.OUTPUT.UploadedDate=BANCS.INPUT.BODDate
```

```
BANCS.OUTPUT.BatchId=" DEP0054599"
```

```
BANCS.INPARAM.DuplicateField=sv_a
BANCS.INPARAM.UploadedDate=BANCS.OUTPUT.UploadedDate
BANCS.INPARAM.DupChkDays=sv_b
BANCS.INPARAM.BatchId=BANCS.OUTPUT.BatchId
sv_c = urhk_checkDuplicateField("")
if(BANCS.OUTPARAM.DuplicateCount != "0") then
sv_e = urhk_getMSGCodeErrDesc("MSG0308766")
if(sv_e == 0) then
BANCS.OUTPUT.ErrorDesc = BANCS.OUTPARAM.errDesc
endif
endif
```

©Copyright Infosys Ltd.

Currency Calendar Userhooks

This section discusses:

[Calculate Next Currency Working Date](#)

[Input Date Coincides with Currency Holiday](#)

©Copyright Infosys Ltd.

Calculate Next Currency Working Date

The userhook derives the next working date from the input value date, currency code and direction (signifying Next or Previous).

Syntax:

```
sv_a = urhk_getNextCrncyWorkingDate("")
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The output is the common working day for the SOL and the currency. It is available in BANCS.OUTPARAM.WorkingDate.

For Example:

Suppose the Indian currency (INR) is on a holiday on 15th August of the year and the home currency code for the bank is British pound (GBP). The next working date is arrived at in the this manner:

```
BANCS.INPARAM.value_date = "15-08-2005"
```

```
BANCS.INPARAM.crncy_code = "INR"
```

```
BANCS.INPARAM. direction = "Y" (Y Next, N Previous)
```

sv_a = urhk_getNextCrncyWorkingDate("")

if (sv_a != 0) then

sv_b = BANCS.OUTPARAM. WorkingDate (now sv_b contains
the next working date)

Else

<error>

©Copyright Infosys Ltd.

Input Date Coincides with Currency Holiday

This userhook checks if the input value date is a specified holiday for the input currency code in the Currency Calendar. If either the currency code or date is invalid, a fatal error is displayed.

Syntax:

```
sv_a = urhk_isCrncyHoliday("")
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Returns SUCCESS or FAILURE.

For Example:

Suppose the Indian currency (INR) is on a holiday on 15th August of the year and the home currency code for the bank is British pound (GBP). The currency holiday check is done in the this manner:

```
BANCS.INPARAM.value_date = "15-08-2005"
```

```
BANCS.INPARAM.crncy_code = "INR"
```

```
sv_a = urhk_isCrncyHoliday("")
```


if (sv_a != 0) then

<not a holiday>

Else

<Holiday>

©Copyright Infosys Ltd.

Custom Batch Job Userhooks

This section discusses:

[Get Field Value for Service ID and Field Name](#)

[Put Field Value with Service ID and Field Name](#)

[Pre/Post Process Scripts for Batches](#)

©Copyright Infosys Ltd.

Get Field Value for Service ID and Field Name

This userhook is provided to get the field value for the passed service ID and field name.

Syntax:

```
sv_r      =      GetFldValueWithSrvIdAndFldName("<service_Id>|<field_name>")
```

Input Arguments:

Service ID and field name in a pipe separated format.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

BANCS.OUTPARAM.fieldValue - The field value for the passed for service ID and field name for the current record.

For Example:

```
start
```

```
trace on
```

```
sv_a = ONS.CUSTOM.currServiceId
```

```
if(sv_a == "AddBank") then
```

```
sv_r      =  
urhk_GetFldValueWithSrvIdAndFldName("AddBank|bankDet.bankCode"
```

```
if(sv_r == 0) then  
sv_b = BANCS.OUTPARAM.fieldValue  
endif  
exitscript  
end
```

©Copyright Infosys Ltd.

Put Field Value with Service ID and Field Name

This userhook is provided to put the field value for the passed service Id and field name.

Syntax:

```
sv_r = urhk_PutFldValueWithSrvIdAndFldName(" <service_Id>|<fld_name>|<fld_value>")
```

Input Arguments:

Service ID, field name and field value in a pipe separated format.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

None

For Example:

Start

trace on

sv_a = ONS.CUSTOM.currServiceId

if(sv_a == "AddBank") then

```
sv_r =  
urhk_PutFldValueWithSrvIdAndFldName("AddBank|bankDet.bankWkly
```

```
if(sv_r == 0) then
sv_b = print("Bank Weekly Off Modified to 1")
endif
exitscript
end
```

©Copyright Infosys Ltd.

Pre/Post Process Scripts for Batches

Two optional scripthooks have been provided for all batches written in batch driver framework.

■<exeName>_pre_process.scr : called before processing of each record

■<exeName>_post_process.scr : called after processing of each record

To access the value of the fields of the process structure for the specific record inside the above mentioned scripts, user hook `urhk_Batch_GetVal` can be used.

Syntax:

```
sv_b = urhk_Batch_GetVal("<fieldName>")
```

sv_b: scratch pad variable to store the return value.

Input Arguments:

The field name of the process Structure as defined in `gspc/bspc` has to be provided as argument to the userhook whose value is required.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.batchValue	The value of field can be accessed using this variable.
BANCS.OUTPARAM. successOrFailure	This variable's value is set to 'S' if the fetch is successful and set to 'F' if the fetch fails.

Returns SUCCESS if the field value is fetched successfully.

Returns FAILURE if the fetch fails.

For Example:

```
sv_b = urhk_Batch_GetVal("solId")
```

```
sv_d = BANCS.OUTPARAM.batchValue
```

©Copyright Infosys Ltd.

Customer Related Userhooks

This section discusses:

[Get All Address Details of a Customer](#)

[Get Customer Details](#)

[Get Document Details of a Customer](#)

[Customer Details in Bancs Repository](#)

[Get CRM Address of a Customer](#)

©Copyright Infosys Ltd.

Get All Address Details of a Customer

This userhook is used to fetch all address records of a customer.

Syntax:

```
sv_b = urhk_ getAllAddressDtIsOfCustUserHooks ("")
```

Input Arguments:

Cif ID

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.addrType_<number>	addrType_ <number>
BANCS.OUTPARAM.address1_<number>	address1_ <number>
BANCS.OUTPARAM.address2_<number>	address2_ <number>
BANCS.OUTPARAM.address3_<number>	address3_ <number>
BANCS.OUTPARAM.startDate_<number>	startDate_ <number>
BANCS.OUTPARAM.endDate_<number>	endDate_ <number>
BANCS.OUTPARAM.cityCode_<number>	cityCode_ <number>
BANCS.OUTPARAM.stateCode_<number>	stateCode_ <number>

BANCS.OUTPARAM.cntryCode_<number>	cntryCode_ <number>
BANCS.OUTPARAM.pinCode_<number>	pinCode_ <number>
BANCS.OUTPARAM.prefrdFormat_<number>	prefrdFormat_ <number>
BANCS.OUTPARAM.holdMailFlg_<number>	holdMailFlg_ <number>
BANCS.OUTPARAM.holdMailInitd_<number>	holdMailInitd_ <number>
BANCS.OUTPARAM.isAddProofRcvd_<number>	isAddProofRcvd_ <number>

Return Value:

Address records of a customer.

For Example:

```

<--start
trace on
sv_q      =      urhk_      getAllAddressDtIsOfCustUserHooks
("NEW10CIF1")
sv_m = BANCS.OUTPARAM.addressCount
sv_r = sv_m
sv_r=CINT(sv_r)
sv_r=sv_r+1
while(sv_r!=1)
sv_r=sv_r-1
sv_x=FORMAT$(sv_r,"%d")
sv_t=("BANCS").("OUTPARAM").("addrType_"+sv_x)
print(sv_t)

```

```
sv_t=("BANCS").("OUTPARAM").("address1_"+sv_x)
print(sv_t)
sv_t=("BANCS").("OUTPARAM").("address2_"+sv_x)
print(sv_t)
do
trace off
end-->
```

©Copyright Infosys Ltd.

Get Customer Details

This userhook is used to get various details of the customer like name, address details, TDS and charge details, Customer Exchange Rate Preferential code and so on.

```
sv_b = urhk_GetCustomerDetails (“”)
```

Input Arguments:

Valid Customer ID

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.cust_id	cust_id

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.Cifld	Cifld
BANCS.OUTPARAM.Custld	Custld
BANCS.OUTPARAM.CustShortName	CustShortName
BANCS.OUTPARAM.CustTitleCode	CustTitleCode
BANCS.OUTPARAM.CustName	CustName
BANCS.OUTPARAM.CustPermAddr1	CustPermAddr1
BANCS.OUTPARAM.CustPermAddr2	CustPermAddr2
BANCS.OUTPARAM.CustPermCityCode	CustPermCityCode
BANCS.OUTPARAM.CustPermStateCode	CustPermStateCode
BANCS.OUTPARAM.CustPermPinCode	CustPermPinCode
BANCS.OUTPARAM.CustPermCntryCode	CustPermCntryCode
BANCS.OUTPARAM.CustPermPhoneNum	CustPermPhoneNum

BANCS.OUTPARAM.CustPermTelexNum	CustPermTelexNum
BANCS.OUTPARAM.CustSex	CustSex
BANCS.OUTPARAM.CustGroup	CustGroup
BANCS.OUTPARAM.CustComuAddr1	CustComuAddr1
BANCS.OUTPARAM.CustComuAddr2	CustComuAddr2
BANCS.OUTPARAM.CustComuCityCode	CustComuCityCode
BANCS.OUTPARAM.CustComuStateCode	CustComuStateCode
BANCS.OUTPARAM.CustComuPinCode	CustComuPinCode
BANCS.OUTPARAM.CustComuCntryCode	CustComuCntryCode
BANCS.OUTPARAM.CustComuPhoneNum1	CustComuPhoneNum1
BANCS.OUTPARAM.CustComuPhoneNum2	CustComuPhoneNum2
BANCS.OUTPARAM.CustComuTelexNum	CustComuTelexNum
BANCS.OUTPARAM.CustOccpCode	CustOccpCode
BANCS.OUTPARAM.CustSectCode	CustSectCode
BANCS.OUTPARAM.CustSubSectCode	CustSubSectCode
BANCS.OUTPARAM.CustFirstAcctDate	CustFirstAcctDate
BANCS.OUTPARAM.CustRatingCode	CustRatingCode
BANCS.OUTPARAM.CustRatingDate	CustRatingDate
BANCS.OUTPARAM.CustMgrOpin	CustMgrOpin
BANCS.OUTPARAM.CustPassWord	CustPassWord
BANCS.OUTPARAM.TotTodAlwdTimes	TotTodAlwdTimes
BANCS.OUTPARAM.CustIntrodCustId	CustIntrodCustId
BANCS.OUTPARAM.CustIntrodCifId	CustIntrodCifId
BANCS.OUTPARAM.IntrodTitleCode	IntrodTitleCode
BANCS.OUTPARAM.CustIntrodName	CustIntrodName
BANCS.OUTPARAM.CustIntrodStatCode	CustIntrodStatCode
BANCS.OUTPARAM.CustTypeCode	CustTypeCode
BANCS.OUTPARAM.CustEmpId	CustEmpId

BANCS.OUTPARAM.CustStatCode	CustStatCode
BANCS.OUTPARAM.CustOthrBankCode	CustOthrBankCode
BANCS.OUTPARAM.CustStatChgDate	CustStatChgDate
BANCS.OUTPARAM.CustBusinessAssets	CustBusinessAssets
BANCS.OUTPARAM.CustPropAssets	CustPropAssets
BANCS.OUTPARAM.CustInvestments	CustInvestments
BANCS.OUTPARAM.CustNetWorth	CustNetWorth
BANCS.OUTPARAM.CustDeplnOthrBank	CustDeplnOthrBank
BANCS.OUTPARAM.CustAssetsAsOnDate	CustAssetsAsOnDate
BANCS.OUTPARAM.CustTotFundBase	CustTotFundBase
BANCS.OUTPARAM.CustTotNonFundBase	CustTotNonFundBase
BANCS.OUTPARAM.CustAdvnAsOnDate	CustAdvnAsOnDate
BANCS.OUTPARAM.CustOthrLim	CustOthrLim
BANCS.OUTPARAM.CustConst	CustConst
BANCS.OUTPARAM.CustFinYearEndMnth	CustFinYearEndMnth
BANCS.OUTPARAM.CustMinorFlg	CustMinorFlg
BANCS.OUTPARAM.CustNreFlg	CustNreFlg
BANCS.OUTPARAM.CustHlthCode	CustHlthCode
BANCS.OUTPARAM.CustCardHoldFlg	CustCardHoldFlg
BANCS.OUTPARAM.CustFreeText	CustFreeText
BANCS.OUTPARAM.CustAssetClass	CustAssetClass
BANCS.OUTPARAM.CustAssetClassDate	CustAssetClassDate
BANCS.OUTPARAM.CustCasteCode	CustCasteCode
BANCS.OUTPARAM.TdsTblCode	TdsTblCode
BANCS.OUTPARAM.TdsRemarks	TdsRemarks
BANCS.OUTPARAM.TdsExmptRefNo	TdsExmptRefNo
BANCS.OUTPARAM.TdsExmptEndDate	TdsExmptEndDate
BANCS.OUTPARAM.TdsSubmDate	TdsSubmDate
BANCS.OUTPARAM.TdsCustId	TdsCustId

BANCS.OUTPARAM.TdsCifId	TdsCifId
BANCS.OUTPARAM.NumOfAccounts	NumOfAccounts
BANCS.OUTPARAM.ShareHolderFlg	ShareHolderFlg
BANCS.OUTPARAM.ValueShareLead	ValueShareLead
BANCS.OUTPARAM.TotalSharesHeld	TotalSharesHeld
BANCS.OUTPARAM.NatIdCardNum	NatIdCardNum
BANCS.OUTPARAM.PsFreqType	PsFreqType
BANCS.OUTPARAM. PsFreqWeekNum	PsFreqWeekNum
BANCS.OUTPARAM.PsFreqWeekDay	PsFreqWeekDay
BANCS.OUTPARAM.PsFreqStartDd	PsFreqStartDd
BANCS.OUTPARAM.PsFreqHldyStat	PsFreqHldyStat
BANCS.OUTPARAM.DateOfBirth	DateOfBirth
BANCS.OUTPARAM.PanGirNum	PanGirNum
BANCS.OUTPARAM.PsprtNum	PsprtNum
BANCS.OUTPARAM.PsprtIssuDate	PsprtIssuDate
BANCS.OUTPARAM.PsprtDet	PsprtDet
BANCS.OUTPARAM.PsprtExpDate	PsprtExpDate
BANCS.OUTPARAM.CustMaritalStatus	CustMaritalStatus
BANCS.OUTPARAM.SegmentationClass	SegmentationClass
BANCS.OUTPARAM.CustagerNo	CustagerNo
BANCS.OUTPARAM.CustFaxNo	CustFaxNo
BANCS.OUTPARAM.PurgeAllowedFlg	PurgeAllowedFlg
BANCS.OUTPARAM.PurgeText	PurgeText
BANCS.OUTPARAM.Dsald	Dsald
BANCS.OUTPARAM. AllowSweepsFlg	AllowSweepsFlg
BANCS.OUTPARAM.CustPrefTillDate	CustPrefTillDate
BANCS.OUTPARAM.AcctMgrUserId	AcctMgrUserId
BANCS.OUTPARAM.CustSwiftCode	CustSwiftCode
BANCS.OUTPARAM.ChrgLevelCode	ChrgLevelCode

BANCS.OUTPARAM.ChrgDrForacid	ChrgDrForacid
BANCS.OUTPARAM.ChrgDrSolld	ChrgDrSolld
BANCS.OUTPARAM.CustChrgHistoryFlg	CustChrgHistoryFlg
BANCS.OUTPARAM.CustXchgRatePrefCode	CustXchgRatePrefCode
BANCS.OUTPARAM.CombinedStatementRequired	CombinedStatementRequired
BANCS.OUTPARAM.LoansStatementType	LoansStatementType
BANCS.OUTPARAM.TdStatementType	TdStatementType
BANCS.OUTPARAM.CombinedStatementChargeCode	CombinedStatementChargeCode
BANCS.OUTPARAM.StatementSolld	StatementSolld
BANCS.OUTPARAM.DespatchMode	DespatchMode
BANCS.OUTPARAM.CsLastPrintedDate	CsLastPrintedDate
BANCS.OUTPARAM.AddressType	AddressType
BANCS.OUTPARAM.CsNextDueDate	CsNextDueDate
BANCS.OUTPARAM.AllowSweeps	AllowSweeps
BANCS.OUTPARAM.CustomerEmpAddress1	CustomerEmpAddress1
BANCS.OUTPARAM.CustomerEmpAddress2	CustomerEmpAddress2
BANCS.OUTPARAM.CustomerEmpCityCode	CustomerEmpCityCode
BANCS.OUTPARAM.CustomerEmpStateCode	CustomerEmpStateCode
BANCS.OUTPARAM.CustomerEmpPinCode	CustomerEmpPinCode
BANCS.OUTPARAM.CustomerEmpCountryCode	CustomerEmpCountryCode
BANCS.OUTPARAM.CustomerEmpPhoneNumber1	CustomerEmpPhoneNumber1
BANCS.OUTPARAM.CustomerEmpPhoneNumber2	CustomerEmpPhoneNumber2
BANCS.OUTPARAM.CustomerEmpTelexNumber	CustomerEmpTelexNumber
BANCS.OUTPARAM.CustomerEmpEmailId	CustomerEmpEmailId
BANCS.OUTPARAM.CustomerPermEmailId	CustomerPermEmailId
BANCS.OUTPARAM.CustomerFreeCode%d	CustomerFreeCode%d
BANCS.OUTPARAM.CustomerFreeCode1	CustomerFreeCode1
BANCS.OUTPARAM.CustomerFreeCode2	CustomerFreeCode2

BANCS.OUTPARAM.CustomerFreeCode3	CustomerFreeCode3
BANCS.OUTPARAM.CustomerFreeCode4	CustomerFreeCode4
BANCS.OUTPARAM.CustomerFreeCode5	CustomerFreeCode5
BANCS.OUTPARAM.CustomerFreeCode6	CustomerFreeCode6
BANCS.OUTPARAM.CustomerFreeCode7	CustomerFreeCode7
BANCS.OUTPARAM.CustomerFreeCode8	CustomerFreeCode8
BANCS.OUTPARAM.CustomerFreeCode9	CustomerFreeCode9
BANCS.OUTPARAM.CustomerFreeCode10	CustomerFreeCode10
BANCS.OUTPARAM.FreeText1	FreeText1
BANCS.OUTPARAM.FreeText2	FreeText2
BANCS.OUTPARAM.FreeText3	FreeText3
BANCS.OUTPARAM.FreeText4	FreeText4
BANCS.OUTPARAM.FreeText5	FreeText5
BANCS.OUTPARAM.FreeText6	FreeText6
BANCS.OUTPARAM.FreeText7	FreeText7
BANCS.OUTPARAM.FreeText8	FreeText8
BANCS.OUTPARAM.FreeText9	FreeText9
BANCS.OUTPARAM.FreeText10	FreeText10
BANCS.OUTPARAM.FreeText11	FreeText11
BANCS.OUTPARAM.FreeText12	FreeText12
BANCS.OUTPARAM.FreeText13	FreeText13
BANCS.OUTPARAM.FreeText14	FreeText14
BANCS.OUTPARAM.FreeText15	FreeText15
BANCS.OUTPARAM.PhoneType	PhoneType
BANCS.OUTPARAM.EmailType	EmailType
BANCS.OUTPARAM.CustPrefIntDr	CustPrefIntDr
BANCS.OUTPARAM.CustPrefIntCr	CustPrefIntCr
BANCS.OUTPARAM.Nationality	Nationality
BANCS.OUTPARAM.SegmentType	SegmentType

BANCS.OUTPARAM.Blacklisted	Blacklisted
BANCS.OUTPARAM.Negated	Negated
BANCS.OUTPARAM.strfield14	strfield14
BANCS.OUTPARAM.strfield15	strfield15
BANCS.OUTPARAM.blacklistCount	blacklistCount
BANCS.OUTPARAM.negateCount	negateCount
BANCS.OUTPARAM.blacklistcode_<number>	blacklistcode_<number>
BANCS.OUTPARAM.negatecode_<number>	negatecode_<number>
BANCS.OUTPARAM.CustPermAddr3	CustPermAddr3
BANCS.OUTPARAM.CustComuAddr3	CustComuAddr3
BANCS.OUTPARAM.CustomerEmpAddress3	CustomerEmpAddress3
BANCS.OUTPARAM.RetailOrCorp	RetailOrCorp
BANCS.OUTPARAM.Staffflag	Staffflag
BANCS.OUTPARAM.CustomerDummy	CustomerDummy
BANCS.OUTPARAM.PrimaryRM	PrimaryRM
BANCS.OUTPARAM.Secondary RM	Secondary RM
BANCS.OUTPARAM.Tertiary RM	Tertiary RM
BANCS.OUTPARAM.RiskProfileScore	RiskProfileScore
BANCS.OUTPARAM.RiskProfileScoreExpiryDate	RiskProfileScoreExpiryDate
BANCS.OUTPARAM.ShortNameNative	ShortNameNative
BANCS.OUTPARAM.blacklistReasonDesc__<number>	blacklistReasonDesc__<number>
BANCS.OUTPARAM.negateReasonDesc__<number>	negateReasonDesc__<number>
BANCS.OUTPARAM.suspendCode__<number>	suspendCode__<number>
BANCS.OUTPARAM.suspendExpiryDate__<number>	suspendExpiryDate__<number>
BANCS.OUTPARAM.suspendReasonDesc__<number>	suspendReasonDesc__<number>

Return Value:

If the specified Customer is found to be valid return value will be '0'.

Else return value will be '1'.

For Example:

```
<--start
trace on
sv_q = urhk_getCustomerDetails("NEW10CIF1")
sv_a = BANCS.OUTPARAM. CustPermAddr1
sv_b = BANCS.OUTPARAM. CustPermAddr2
sv_c = BANCS.OUTPARAM. CustPermAddr3
print("CustPermAddr1*****"
"+sv_a)
print("CustPermAddr2*****"
"+sv_b)
print("CustPermAddr3***** "+sv_c)
trace off
end-->
```

This prints the address line 1,2 and 3 for the permanent address of the customer NEW10CIF1.

Previously only the address line 1 and 2 were available, henceforth the address line 3 would also be available.

©Copyright Infosys Ltd.

Get Document Details of a Customer

This userhook is used to fetch all entity document records of a customer.

Syntax:

```
sv_b = urhk_ getAllDocumentDtlsOfCustUserHooks ("")
```

Input Arguments:

Cif ID

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Cif_id	Cif_id

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.docType_ <number>	docType_ <number>
BANCS.OUTPARAM.docCode_ <number>	docCode_ <number>
BANCS.OUTPARAM.uniqueNo_ <number>	uniqueNo_ <number>
BANCS.OUTPARAM.placeOfIssue_ <number>	placeOfIssue_ <number>
BANCS.OUTPARAM.cntryOfIssue_ <number>	cntryOfIssue_ <number>
BANCS.OUTPARAM.issueDate_ <number>	issueDate_ <number>
BANCS.OUTPARAM.expiryDate_ <number>	expiryDate_ <number>

Return Value:

The number of document records of a customer.

For Example:

```
<--start
trace on
sv_q = urhk_ getDocumentDetailsOfCustomer ("NEW10CIF1")
sv_m = BANCS.OUTPARAM.docCount
sv_r = sv_m
sv_r=CINT(sv_r)
sv_r=sv_r+1
while(sv_r!=1)
sv_r=sv_r-1
sv_x=FORMAT$(sv_r,"%d")
sv_t=("BANCS").("OUTPARAM").("docType_"+sv_x)
print(sv_t)
sv_t=("BANCS").("OUTPARAM").("docCode_"+sv_x)
print(sv_t)
sv_t=("BANCS").("OUTPARAM").("uniqueNo_"+sv_x)
print(sv_t)
do
trace off
end-->
```

Customer Details in Bancs Repository

This userhook return customer details specified through the HCUMM menu option for a given customer. This userhook can be called from any script event. It checks for validity of a customer ID. Deleted and unverified status of customer ID is not considered.

For a valid customer ID, the customer details from CMG table are populated into repository fields.

This userhook is further modified to dump the Customer Exchange Rate Preferential code value as an output from the userhook.

Syntax:

```
sv_a = urhk_getCustomerDetails(Variable)
```

sv_a: scratch pad variable to store the return value.

The variable can be a scratch pad variable (sv_b) or a string ("CUST12").

Input Arguments:

✓ valid Customer ID

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.CustShortName	cust_short_name in CMG table
BANCS.OUTPARAM.CustTitleCode	cust_title_code in CMG table
BANCS.OUTPARAM.CustName	cust_name in CMG table

BANCS.OUTPARAM.CustPermAddr1	cust_perm_addr1 in CMG table
BANCS.OUTPARAM.CustPermAddr2	cust_perm_addr2 in CMG table
BANCS.OUTPARAM.CustPermCityCode	cust_perm_city_code in CMG table
BANCS.OUTPARAM.CustPermStateCode	cust_perm_state_code in CMG table
BANCS.OUTPARAM.CustPermPinCode	cust_perm_pin_code in CMG table
BANCS.OUTPARAM.CustPermCntryCode	cust_perm_cntry_code in CMG table
BANCS.OUTPARAM.CustPermPhoneNum	cust_perm_phone_num in CMG table
BANCS.OUTPARAM.CustPermTelexNum	cust_perm_telex_num in CMG table
BANCS.OUTPARAM.CustSex	cust_sex in CMG table
BANCS.OUTPARAM.CustGroup	cust_grp in CMG table
BANCS.OUTPARAM.CustComuAddr1	cust_comu_addr1 in CMG table
BANCS.OUTPARAM.CustComuAddr2	cust_comu_addr2 in CMG table
BANCS.OUTPARAM.CustComuCityCode	cust_comu_city_code in CMG table
BANCS.OUTPARAM.CustComuStateCode	cust_comu_state_code in CMG table
BANCS.OUTPARAM.CustComuPinCode	cust_comu_pin_code in CMG table
BANCS.OUTPARAM.CustComuCntryCode	cust_comu_cntry_code in CMG table
BANCS.OUTPARAM.CustComuPhoneNum1	cust_comu_phone_num_1 in CMG table
BANCS.OUTPARAM.CustComuPhoneNum2	cust_comu_phone_num_2 in CMG table
BANCS.OUTPARAM.CustComuTelexNum	cust_comu_telex_num in CMG table
BANCS.OUTPARAM.CustOccpCode	cust_occ_p_code in CMG table
BANCS.OUTPARAM.CustComuCode	cust_commu_code in CMG table
BANCS.OUTPARAM.CustSectCode	cust_sector_code in CMG table
BANCS.OUTPARAM.CustSubSectCode	cust_sub_sector_code in CMG table
BANCS.OUTPARAM.CustFirstAcctDate	cust_first_acct_date in CMG table
BANCS.OUTPARAM.CustRatingCode	cust_rating_code in CMG table
BANCS.OUTPARAM.CustRatingDate	cust_rating_date in CMG table
BANCS.OUTPARAM.CustMgrOpin	cust_mgr_opin in CMG table
BANCS.OUTPARAM.CustPassWord	cust_pw in CMG table
BANCS.OUTPARAM.TotTodAlwdTimes	tot_tod_alwd_times in CMG table

BANCS.OUTPARAM.CustIntrodCustId	cust_introd_cust_id in CMG table
BANCS.OUTPARAM.IntrodTitleCode	introd_title_code in CMG table
BANCS.OUTPARAM.CustIntrodName	cust_introd_name in CMG table
BANCS.OUTPARAM.CustIntrodStatCode	cust_introd_stat_code in CMG table
BANCS.OUTPARAM.CustTypeCode	cust_type_code in CMG table
BANCS.OUTPARAM.CustEmpId	cust_emp_id in CMG table
BANCS.OUTPARAM.CustStatCode	cust_stat_code in CMG table
BANCS.OUTPARAM.CustOthrBankCode	cust_othr_bank_code in CMG table
BANCS.OUTPARAM.CustStatChgDate	cust_stat_chg_date in CMG table
BANCS.OUTPARAM.CustBusinessAssets	cust_business_assets in CMG table
BANCS.OUTPARAM.CustPropAssets	cust_prop_assets in CMG table
BANCS.OUTPARAM.CustInvestments	cust_investmnts in CMG table
BANCS.OUTPARAM.CustNetWorth	cust_net_worth in CMG table
BANCS.OUTPARAM.CustDeplnOthrBank	cust_dep_in_othr_bank in CMG table
BANCS.OUTPARAM.CustAssetsAsOnDate	cust_assets_as_on_date in CMG table
BANCS.OUTPARAM.CustTotFundBase	cust_tot_fund_base in CMG table
BANCS.OUTPARAM.CustTotNonFundBase	cust_tot_non_fund_base in CMG table
BANCS.OUTPARAM.CustAdvnAsOnDate	cust_advn_as_on_date in CMG table
BANCS.OUTPARAM.CustOthrLim	cust_othr_lim in CMG table
BANCS.OUTPARAM.CustConst	cust_const in CMG table
BANCS.OUTPARAM.CustFinYearEndMnth	cust_fin_year_end_mnth in CMG table
BANCS.OUTPARAM.CustMinorFlg	cust_minor_flg in CMG table
BANCS.OUTPARAM.CustNreFlg	cust_nre_flg in CMG table
BANCS.OUTPARAM.CustHlthCode	cust_hlth_code in CMG table
BANCS.OUTPARAM.CustCardHoldFlg	cust_card_hold_flg in CMG table
BANCS.OUTPARAM.CustFreeText	cust_free_text in CMG table
BANCS.OUTPARAM.CustChoreMbrFlg	cust_chore_mbr_flg in CMG table
BANCS.OUTPARAM.TotModTimes	tot_mod_times in CMG table
BANCS.OUTPARAM.CustAssetClass	cust_asset_class in CMG table

BANCS.OUTPARAM.CustAssetClassDate	cust_asset_class_date in CMG table
BANCS.OUTPARAM.CustCasteCode	cust_caste_code in CMG table
BANCS.OUTPARAM.TdsTblCode	tds_tbl_code in CMG table
BANCS.OUTPARAM.TdsExempFlg	tds_exemp_flg in CMG table
BANCS.OUTPARAM.ResetExempFlg	reset_exemp_flg in CMG table
BANCS.OUTPARAM.TdsCustId	tds_cust_id in CMG table
BANCS.OUTPARAM.NumOfAccounts	num_of_accounts in CMG table
BANCS.OUTPARAM.ShareHolderFlg	share_holder_flg in CMG table
BANCS.OUTPARAM.MemberType	member_type in CMG table
BANCS.OUTPARAM.CustMembershipDate	cust_membership_date in CMG table
BANCS.OUTPARAM.ValueShareLead	value_share_held in CMG table
BANCS.OUTPARAM.TotalSharesHeld	total_shares_held in CMG table
BANCS.OUTPARAM.NatIdCardNum	nat_id_card_num in CMG table
BANCS.OUTPARAM.PsFreqType	ps_freq_type in CMG table
BANCS.OUTPARAM.PsFreqWeekNum	ps_freq_week_num in CMG table
BANCS.OUTPARAM.PsFreqWeekDay	ps_freq_week_day in CMG table
BANCS.OUTPARAM.PsFreqStartDd	ps_freq_start_dd in CMG table
BANCS.OUTPARAM.PsFreqHldyStat	ps_freq_hldy_stat in CMG table
BANCS.OUTPARAM.DateOfBirth	date_of_birth in CMG table
BANCS.OUTPARAM.PanGirNum	pan_gir_num in CMG table
BANCS.OUTPARAM.PsprtNum	psprt_num in CMG table
BANCS.OUTPARAM.PsprtIssuDate	psprt_issu_date in CMG table
BANCS.OUTPARAM.PsprtDet	psprt_det in CMG table
BANCS.OUTPARAM.PsprtExpDate	psprt_exp_date in CMG table
BANCS.OUTPARAM.CustMaritalStatus	cust_marital_status in CMG table
BANCS.OUTPARAM.CustPagerNo	cust_pager_no in CMG table
BANCS.OUTPARAM.CustFaxNo	cust_fax_no in CMG table
BANCS.OUTPARAM.FreeText1	free_text_1 in CMG table

BANCS.OUTPARAM.FreeText2	free_text_2 in CMG table
BANCS.OUTPARAM.FreeText3	free_text_3 in CMG table
BANCS.OUTPARAM.FreeText4	free_text_4 in CMG table
BANCS.OUTPARAM.FreeText5	free_text_5 in CMG table
BANCS.OUTPARAM.FreeText6	free_text_6 in CMG table
BANCS.OUTPARAM.FreeText7	free_text_7 in CMG table
BANCS.OUTPARAM.FreeText8	free_text_8 in CMG table
BANCS.OUTPARAM.FreeText9	free_text_9 in CMG table
BANCS.OUTPARAM.FreeText10	free_text_10 in CMG table
BANCS.OUTPARAM.FreeText11	free_text_11 in CMG table
BANCS.OUTPARAM.FreeText12	free_text_12 in CMG table
BANCS.OUTPARAM.FreeText13	free_text_13 in CMG table
BANCS.OUTPARAM.FreeText14	free_text_14 in CMG table
BANCS.OUTPARAM.FreeText15	free_text_15 in CMG table
BANCS.OUTPARAM.PurgeAllowedFlg	purge_allowed_flg in CMG table
BANCS.OUTPARAM.PurgeText	purge_text in CMG table
BANCS.OUTPARAM.CustPrefTillDate	cust_pref_till_date in CMG table
BANCS.OUTPARAM.AcctMgrUserId	acct_mgr_user_id in CMG table
BANCS.OUTPARAM.CustSwiftCode	cust_swift_code in CMG table
BANCS.OUTPARAM.IsSwiftCodeOfBank	is_swift_code_of_bank in CMG table
BANCS.OUTPARAM.ChrgLevelCode	chrg_level_code in CMG table
BANCS.OUTPARAM.ChrgDrForacid	chrg_dr_foracid in CMG table
BANCS.OUTPARAM.ChrgDrSolId	chrg_dr_sol_id in CMG table
BANCS.OUTPARAM.CustChrgHistoryFlg	cust_chrg_history_flg in CMG table
BANCS.OUTPARAM.CustXchgRatePrefCode	Cust_Xchg_Rate_Pref_Code in CMG table

Return Value:

If the specified customer is found to be valid, then return value is '0'.
Else return value is '1'.

The details is populated into class OUTPARAM of BANCS repository. Individual repository field names are specified against the field name.

For Example:

Customer Net worth may be required during generation of past due reports. The same is obtained as:

```
<--start
sv_a = BANCS.INPUT.CustId
# sv_a (Customer Id) is the input to user hook
sv_b = urhk_getCustomerDetails(sv_a)
# urhk_getCustomerDetails will return failure if customer Id
# passed is invalid.
# If return value is '1' then populate error message and exit
# from script.
if( sv_b == 1 ) then
exitscript
endif
sv_i = BANCS.OUTPARAM.CustName
sv_j = cint(BANCS.OUTPARAM.NumOfAccounts)
sv_k = cdouble(BANCS.OUTPARAM.CustNetWorth)
end
```

Get CRM Address of a Customer

The Get CRM Address of a Customer userhook is written to find the CRM address type based on given address ID and the cif ID as input parameters to the userhook. It determines whether the CIF is a corporate CIF.

For CUSTCOMMADD, it returns Mailing.

For CUSTPERMADD, for Corporate CIF, it returns Registered. Else, it returns Home.

For CUSTEMPADD, for Corporate CIF, it returns Head Office. Else, it returns Work.

Syntax:

```
sv_a = urhk_getCRMAddressTypeForCIF("")
```

sv_a: scratch pad variable to store the return value.

Input Arguments:

BANCS.INPARAM.cifId

BANCS.INPARAM.addrId

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.cifId	Valid cifId
BANCS.INPARAM.addrId	Valid addrId

Output Repository Field:

Field Name	Description
BANCS.OUTPARAM. outAddrId	Address of the Customer.

©Copyright Infosys Ltd.

Finacle Copyright

Date Related Userhooks

This section discusses:

[Adding Days and Months in a Given Date](#)

[Compute Next Execution Date](#)

[Convert CalBase Date to Gregorian Date](#)

[Convert Gregorian Date to CalBase Date](#)

[Date Add](#)

[Difference between Two Dates](#)

[Get Call Map Parameters](#)

[Get Currency Working Date for Given Number of Days](#)

[Get Date from Mnemonic](#)

[Get Four Digit Year](#)

[Get Local Calendar General Information](#)

[Get Next Frequency Date](#)

[Get Next Working Date](#)

[Given a Date get Date Difference from BOD](#)

[Register for Date Shift Operation](#)

[De-register for Date Shift Operation](#)

[Getting Holiday Status for Given Date](#)

[Get the Last Successful DC](#)

[Check Whether it is a DC Holiday](#)

[Difference between two dates in Months and Days](#)

[Get Difference between Two Frequencies](#)

[Get the Time Zone Description](#)

©Copyright Infosys Ltd.

Adding Days and Months in a Given Date

This userhook gives the new date after adding the month and date in old date.

Syntax:

```
Sv_a = ("old_date" + "|" + "no_of_months" + "|" + "no_of_days")
```

```
Sv_b = urhk_B2k_Add_MonthsDays_To_date(sv_a)
```

Input Arguments:

Input is a char string.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The userhook returns '0' on success.

BANCS.OUTPARAM.MODIFIED_DATE contains the modified date.

For Example:

```
<--start
```

```
#trace on
```

```
sv_a = "10"
```

```
sv_b = "12"
```

```
sv_d = "10-10-1998"
sv_f = sv_d + "|" + sv_a + "|" + sv_b
sv_e = urhk_B2k_Add_MonthsDays_To_date(sv_f)
if ( sv_e == 1 ) then
print("Date is invalid")
else
print(BANCS.OUTPARAM.MODIFIED_DATE)
#trace off
end-->
```

©Copyright Infosys Ltd.

Compute Next Execution Date

This userhook computes the next execution date for the given frequency and date information.

Syntax:

```
sv_a = urhk_computeNextExecDate("")
```

Input Arguments:

- Frequency Type (FreqType)
- Frequency Week Number (FreqWeekNum)
- Frequency Week Day (FreqWeekDay)
- Frequency Start day (FreqStartDd)
- Frequency Holiday Status (FreqHldyStat)
- From execution date (FromExecDate)
- To execution date (ToExecDate)
- Next Execution date (NextExecDate)

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Output for this userhook is the Next execution date (OutputDate).

For Example:

```
<--start
#trace on
#Computing the value for next_exec_date
BANCS.INPARAM.FreqType = sv_a
BANCS.INPARAM.FreqWeekNum = sv_b
BANCS.INPARAM.FreqWeekDay = sv_c
BANCS.INPARAM.FreqStartDd = sv_d
BANCS.INPARAM.FreqHldyStat = sv_e
BANCS.INPARAM.FromExecDate = sv_f
BANCS.INPARAM.ToExecDate = sv_g
BANCS.INPARAM.NextExecDate = sv_h
sv_a = urhk_computeNextExecDate("")
if (fieldexists(BANCS.OUTPARAM.OutputDate)) then
sv_q = BANCS.OUTPARAM.OutputDate
endif
#trace off
end-->
```

Convert CalBase Date to Gregorian Date

This userhook is used to convert the passed Gregorian date to Local date, as per the calendar base passed.

Syntax:

```
sv_b = urhk_ConvertCalBaseDateToGregorianDate(“”)
```

Input Arguments:

CalBase

Local Date

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.inputLocalDate	inputLocalDate
BANCS.INPARAM.calBase	calBase

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.outputGregDate	outputGregDate

Return Value:

EquivalentGregDate

For Example:

BANCS.INPARAM.inputLocalDate="20-03-1426 00:00:00"

BANCS.INPARAM.calBase="01"

```
sv_s=urhk_ConvertCalBaseDateToGregorianDate("")  
sv_t=BANCS.OUTPARAM.outputGregDate  
print (sv_t)
```

©Copyright Infosys Ltd.

Convert Gregorian Date to CalBase Date

This userhook is used to convert the passed Gregorian date to Local date, as per the calendar base passed.

Syntax:

```
sv_b = urhk_ConvertGregorianDateToCalBaseDate ("")
```

Input Arguments:

CalBase

Gregorian Date

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.CalBase	CalBase
BANCS.INPARAM.GregorianDate	GregorianDate

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.outputLocalDate	outputLocalDate

Return Value:

EquivalentLocalDate

For Example:

```
BANCS.INPARAM.inputGregDate="28-04-2005 00:00:00"
```

```
BANCS.INPARAM.calBase="01"
```

```
sv_u=urhk_ConvertGregorianDateToCalBaseDate("")  
sv_v=BANCS.OUTPARAM.outputLocalDate  
print (sv_v)
```

©Copyright Infosys Ltd.

Date Add

This function is used to add days to a given date. It enables adding of days based on the calendar base. This function enables handling of input dates in local format.

Syntax:

```
sv_b = urhk_B2k_date_add(var)
```

Input arguments:

- Date
- Days to add
- calBase

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Date	Date
BANCS.INPARAM.days to add	days to add
BANCS.INPARAM.calBase	calBase

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.dateadd	dateadd
BANCS.OUTPARAM.locdateadd	locdateadd

Return Value:

dateadd, locdateadd

For Example:

Sample code which adds days to acc_opn_date.

```
BANCS.INPARAM.calBase="01"
```

```
sv_e=BANCS.OUTPARAM.acct_opn_date
```

```
sv_q = BANCS.OUTPARAM.days
```

```
sv_v = LEFT$(sv_e,10) + "|" + FORMAT$(CINT(sv_q), "%d")
```

```
sv_m = urhk_B2k_date_add(sv_v)
```

```
sv_n = LEFT$(BANCS.OUTPARAM.dateadd,10)
```

```
sv_o = LEFT$(BANCS.OUTPARAM.locdateadd,10)
```

```
print sv_n
```

```
print sv_o
```

```
sv_s = urhk_B2k_dateDiff("")
```

```
sv_s = BANCS.OUTPARAM.dateDiff
```

©Copyright Infosys Ltd.

Difference between Two Dates

This function enables calculation of date difference even if the date passed is in the local format (based on cal base).

Syntax:

```
sv_b = urhk_B2k_dateDiff (“”)
```

Input Arguments:

Date1

Date2

calBase

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.date1	date1
BANCS.INPARAM.date2	date2
BANCS.INPARAM.calBase	calBase

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.dateDiff	dateDiff

Return Value:

dateDiff

For Example:

Example code to add days and months to given date, for calendar base Hijri.

```
sv_a = urhk_VAL_GetVal("loc_start_date")
```

```
sv_a = BANCS.OUTPARAM.fldValue
```

```
sv_s = BANCS.STDIN.BODDate
```

```
BANCS.INPARAM.date1 = sv_a
```

```
BANCS.INPARAM.date2 = sv_s
```

```
BANCS.INPARAM.calBase = "01"
```

```
sv_s = urhk_B2k_dateDiff("")
```

```
sv_s = BANCS.OUTPARAM.dateDiff
```

©Copyright Infosys Ltd.

Get Call Map Parameters

This userhook is used to get the calendar parameter mapping for a given calendar year and calendar type. Thus by passing the calendar base and the calendar year details such as Gregorian start date for the given year and days in each month of the year can be fetched.

Syntax:

```
sv_b = urhk_GetCalMapParameters("")
```

Input Arguments:

CalBase

LocalCalYear

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.CalBase	CalBase
BANCS.INPARAM.LocalCalYear	LocalCalYear

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.GregStartDate	GregStartDate
BANCS.OUTPARAM.DaysInMonths1	DaysInMonths1
BANCS.OUTPARAM.DaysInMonths2	DaysInMonths2
BANCS.OUTPARAM.DaysInMonths3	DaysInMonths3
BANCS.OUTPARAM.DaysInMonths4	DaysInMonths4
BANCS.OUTPARAM.DaysInMonths5	DaysInMonths5
BANCS.OUTPARAM.DaysInMonths6	DaysInMonths6

BANCS.OUTPUTPARAM.DaysInMonths7	DaysInMonths7
BANCS.OUTPUTPARAM.DaysInMonths8	DaysInMonths8
BANCS.OUTPUTPARAM.DaysInMonths9	DaysInMonths9
BANCS.OUTPUTPARAM.DaysInMonths10	DaysInMonths10
BANCS.OUTPUTPARAM.DaysInMonths11	DaysInMonths11
BANCS.OUTPUTPARAM.DaysInMonths12	DaysInMonths12
BANCS.OUTPUTPARAM.DaysInMonths13	DaysInMonths13

Return Value:

GregStartDate, DaysInM1, DaysInM2, DaysInM3, DaysInM4, DaysInM5, DaysInM6, DaysInM7, DaysInM8, DaysInM9, DaysInM10, DaysInM11, DaysInM12 and DaysInM13.

For Example:

Sample code to fetch and print calendar details for a particular year.

```
BANCS.INPUTPARAM.inputLocalYear="1426"
```

```
BANCS.INPUTPARAM.calBase="01"
```

```
sv_a=urhk_GetCalMapParameters("")
```

```
print(sv_a)
```

```
print(BANCS.OUTPUTPARAM.GregStartDate)
```

```
print(BANCS.OUTPUTPARAM.DaysInMonths1)
```

```
print(BANCS.OUTPUTPARAM.DaysInMonths2)
```

```
print(BANCS.OUTPUTPARAM.DaysInMonths3)
```

```
print(BANCS.OUTPUTPARAM.DaysInMonths4)
```

```
print(BANCS.OUTPUTPARAM.DaysInMonths5)
```

```
print(BANCS.OUTPUTPARAM.DaysInMonths6)
```

```
print(BANCS.OUTPUTPARAM.DaysInMonths7)
```

```
print(BANCS.OUTPUTPARAM.DaysInMonths8)
print(BANCS.OUTPUTPARAM.DaysInMonths9)
print(BANCS.OUTPUTPARAM.DaysInMonths10)
print(BANCS.OUTPUTPARAM.DaysInMonths11)
print(BANCS.OUTPUTPARAM.DaysInMonths12)
print(BANCS.OUTPUTPARAM.DaysInMonths13)
```

©Copyright Infosys Ltd.

Get Currency Working Date for Given Number of Days

This userhook is added to get the date which is alternate currency working day for the given parameters.

Syntax:

```
sv_b = urhk_getCrncyWorkingDateForGivenNumOfDays (<crncy code to be validated> + "|" + <entity type > + "|" + <facility id> + "|" + <tranche id> + "|" + <draw down notice ref> + "|" + <event from which this is being called>+"|" +<Date to be validated>+"|" +<No of days>+"|" + <currency holiday treatment flag>)
```

Input Arguments:

- Currency code to be validated
- Entity type
- Facility ID
- Tranche ID
- Draw down notice reference
- Event from which this userhook is being called
- Reference date
- No. of days
- Currency holiday treatment flag

All fields are mandatory.

Input Repository Fields:

--	--

Field Name	Description
BANCS.INPARAM.Currency code	Currency code
BANCS.INPARAM.entity type	entity type
BANCS.INPARAM.facility id	facility id
BANCS.INPARAM.tranche id	tranche id
BANCS.INPARAM.draw down notice reference	draw down notice reference
BANCS.INPARAM.reference date	reference date
BANCS.INPARAM.no. of days	no. of days
BANCS.INPARAM.currency holiday treatment flag	currency holiday treatment flag

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.AlternateWorkingDate	AlternateWorkingDate

Return value:

Alternate Working Date

©Copyright Infosys Ltd.

Get Date from Mnemonic

This function is used to accept an input date and return the equivalent date if the input date is mnemonic. It also enables to process local dates.

Syntax:

```
sv_b = urhk_GetDateFromMnemonic("")
```

Input Arguments:

Date

calBase

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Date	Date
BANCS.INPARAM.calBase	calBase

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.actualDate	actualDate
BANCS.OUTPARAM.locActualDate	locActualDate

Return Value:

actualDate, locActualDate

For Example:

Example code to fetch actual date for given mnemonic date, for

calendar base Hijri.

sv_a = urhk_VAL_GetVal("loc_start_date")

sv_a = BANCS.OUTPUTPARAM.fldValue

BANCS.INPARAM.inDate = sv_a

BANCS.INPARAM.calBase = '01'

sv_z = urhk_GetDateFromMnemonic("")

sv_m = BANCS.OUTPUTPARAM.actualDate

sv_s = BANCS.OUTPUTPARAM.locActualDate

©Copyright Infosys Ltd.

Get Four Digit Year

This function returns a four digit year, taking a two digit year as input. This function applies the B2K_slidingWindow on the two digit year and returns a four digit year.

Syntax:

The syntax for calling the function is `sv_r = urhk_B2k_getYear (2digitYear)`

The variable can be a scratch pad variable (`sv_b`) or a string("00").

Input Arguments:

2 digit year string

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The output is available in the repository variable "year" , which can be accessed as `BANCS.OUTPARAM year`.

For Example:

```
<--start
```

```
sv_r = urhk_B2k_getYear (2digitYear)
```

```
# if B2k_getYear function returns failure exit out of script.
```

```
if( sv_r != 0 ) then  
  exitscript  
endif  
sv_y = BANCS.OUTPARAM.year  
end-->
```

©Copyright Infosys Ltd.

Get Local Calendar General Information

This userhook is used to fetch general details about the local calendar in the system.

Syntax:

```
sv_b = urhk_getLocalCalGeneralInformation("")
```

Input Arguments:

None

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.processing_cal_base	processing_cal_base
BANCS.OUTPARAM.additional_cal_base	additional_cal_base
BANCS.OUTPARAM.last_local_cal_update	last_local_cal_update
BANCS.OUTPARAM.dso_in_progress	dso_in_progress
BANCS.OUTPARAM.min_loc_cal_yyyymm_modified	min_loc_cal_yyyymm_modified
BANCS.OUTPARAM.min_local_year	min_local_year
BANCS.OUTPARAM.max_local_year	max_local_year
BANCS.OUTPARAM.min_greg_date_mapped	min_greg_date_mapped
BANCS.OUTPARAM.max_greg_date_mapped	max_greg_date_mapped
BANCS.OUTPARAM.mapping_exist	mapping_exist

Return Value:

processing_cal_base, additional_cal_base, last_local_cal_update, dso_in_progress, min_loc_cal_yyyymm_modified, min_local_year, max_local_year, min_greg_date_mapped, max_greg_date_mapped and mapping_exist.

For Example:

The piece of code shows how the general details are fetched and how the mapping_exist filed is used to display error, if mapping does not exist.

```
#Getting the calendar information
```

```
sv_a = urhk_getLocalCalGeneralInformation("")
```

```
sv_x = BANCS.OUTPARAM.min_greg_date_mapped
```

```
sv_y = BANCS.OUTPARAM.max_greg_date_mapped
```

```
sv_t = BANCS.OUTPARAM.mapping_exist
```

```
if(sv_t == "N") then
```

```
sv_r = urhk_SRV_SetErr("Local calendar mapping is not  
available ")
```

```
exitscript
```

```
endif
```

©Copyright Infosys Ltd.

Get Next Frequency Date

Gets the next date as per the given frequency. This function enables calendar based calculation. This function now returns the next date based on the calendar base of the frequency passed. The function gives the next date in Gregorian and local formats. This function also enables handling of input dates in local format.

Syntax:

sv_b = urhk_Fin_FreqNextDate(“”)

Input Arguments:

- freq_type
- freq_week_num
- freq_week_day
- freq_start_dd
- freq_hldy_stat
- freq_cal_base
- input date

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.FreqType	FreqType
BANCS.INPARAM.FreqWeekNum	FreqWeekNum
BANCS.INPARAM.FreqWeekDay	FreqWeekDay
BANCS.INPARAM.FreqStartDd	FreqStartDd
BANCS.INPARAM.FreqHldyStat	FreqHldyStat

BANCS.INPARAM.FreqCalBase	FreqCalBase
BANCS.INPARAM.InputDate	InputDate

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.OutputDate	OutputDate
BANCS.OUTPARAM.locOutputDate	locOutputDate

Return Value:

output date, locOutputDate

For Example:

BANCS.INPARAM.FreqType = BANCS.INPUT.salFreqType

BANCS.INPARAM.FreqWeekNum =
BANCS.INPUT.salFreqWeekNum

BANCS.INPARAM.FreqWeekDay =
BANCS.INPUT.salFreqWeekDay

BANCS.INPARAM.FreqStartDd = BANCS.INPUT.salFreqStartDd

BANCS.INPARAM.FreqHldyStat = BANCS.INPUT.salFreqHldyStat

BANCS.INPARAM.FreqCalBase = BANCS.INPUT.salFreqCalBase

BANCS.INPARAM.InputDate = sv_i

sv_r = urhk_Fin_FreqNextDate("")

sv_i = BANCS.OUTPARAM.OutputDate

sv_j = BANCS.OUTPARAM.locOutputDate

Get Next Working Date

This userhook returns '0' on success and puts the next and previous working dates into the repository. To calculate the next or previous working date it considers the HOL (holiday) table to exclude the holidays. This returns the dates corresponding to Sol_id and datacentre.

Syntax:

```
Sv_e = "Sol_id" + "|" + "Date"
```

```
sv_a = urhk_GetNextWorkingDate(sv_e)
```

Input Arguments:

Input to userhook is a string which contains the sol_id and date.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPUTPARAM.NextWorkingDate	NextWorkingDate
BANCS.OUTPUTPARAM.PrevWorkingDate	PrevWorkingDate
BANCS.OUTPUTPARAM.NextDataCentreWorkingDate	NextDataCentreWorkingDate
BANCS.OUTPUTPARAM.PrevDataCentreWorkingDate	PrevDataCentreWorkingDate

Return Value:

This userhook returns '0' on success.

The next and previous dates are stored in repository.

For Example:

```
<--start
#trace on
sv_a = "SCGL" + "|" + "19-12-1998"
sv_e = urhk_GetNextWorkingDate(sv_a)
print(sv_e)
print(BANCS.OUTPARAM.NextWorkingDate)
print(BANCS.OUTPARAM.PrevWorkingDate)
print(BANCS.OUTPARAM.NextDataCentreWorkingDate)
print(BANCS.OUTPARAM.PrevDataCentreWorkingDate)
#trace off
end-->
```

©Copyright Infosys Ltd.

Given a Date get Date Difference from BOD

This userhook takes the date in dd-mm-yyyy format and returns back the date difference from BOD.

Syntax:

```
sv_a=urhk_B2k_getDateDiffFromBODDate(date) [Date in format:dd-mm-yyyy]
```

Input Arguments:

Date for which the date difference from BOD is needed. If the date is not in proper format, the userhook returns a failure.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return value:

BANCS.OUTPARAM.DateDiff

This contains the date difference from BOD for a given date.

©Copyright Infosys Ltd.

Register for Date Shift Operation

This userhook is used to register a particular date for the date shift operation. This ensures that, this date is automatically updated when there is a change in mapping and the particular date is effected. On call of this userhook, the passed date along with all the database details, is stored in the DSH table. All entries in this table is processed by a batch job, once a calendar change has occurred and the job is run.

Syntax:

```
sv_b = urhk_registerForDateShiftOperation("")
```

Input Arguments:

calBase

tgtTblName

tgtTblKeyClause

tgtDateColumnName

inputGregorianDate

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.calBase	calBase
BANCS.INPARAM.tgtTblName	tgtTblName
BANCS.INPARAM.tgtDateColumnName	tgtDateColumnName
BANCS.INPARAM.tgtTblKeyClause	tgtTblKeyClause
BANCS.INPARAM.inputGregorianDate	inputGregorianDate

Output Repository Fields:

Output not present.

Return Value:

None

For Example:

The piece of code shows how the GSP table's start_date is registered for Date Shift Operation.

```
BANCS.INPARAM.calBase = "01"
```

```
BANCS.INPARAM.tgtTblName = "GSP"
```

```
BANCS.INPARAM.tgtDateColumnName = "start_date"
```

```
sv_a = urhk_SRV_GetVal("genDtl.schm.schmCode")
```

```
sv_c = BANCS.OUTPARAM.srvValue
```

```
sv_c = "schm_code=" + sv_c + "|bank_id=" +  
BANCS.STDIN.contextBankId
```

```
BANCS.INPARAM.tgtTblKeyClause= sv_c
```

```
sv_a = urhk_SRV_GetVal("genDtl.effectiveDt")
```

```
sv_d = BANCS.OUTPARAM.srvValue
```

```
BANCS.INPARAM.inputGregorianDate = sv_d
```

```
sv_e = urhk_registerForDateShiftOperation("")
```

©Copyright Infosys Ltd.

De-register for Date Shift Operation

This userhook is used to de-register a particular date from the date shift operation. This is used if the date has already been registered for the operation. This function causes the particular entry if found, to be deleted from the DSH table. This ensures that this date is NOT be automatically updated, when there is a change in mapping.

Syntax:

```
sv_b = urhk_deRegisterForDateShiftOperation("")
```

Input Arguments:

calBase

tgtTblName

tgtTblKeyClause

tgtDateColumnName

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.calBase	calBase
BANCS.INPARAM.tgtTblName	tgtTblName
BANCS.INPARAM.tgtTblKeyClause	tgtTblKeyClause
BANCS.INPARAM.tgtDateColumnName	tgtDateColumnName

Output Repository Fields:

Output not present.

Return value:

None

For Example:

The piece of code shows how the GSP table's start_date is de-registered for Date Shift Operation.

```
sv_a = urhk_SRV_GetVal("genDtl.addtnlCalBase")
sv_b = BANCS.OUTPARAM.srvValue
BANCS.INPARAM.calBase = sv_b
BANCS.INPARAM.tgtTblName = "GSP"
BANCS.INPARAM.tgtDateColumnName = "start_date"
sv_a = urhk_SRV_GetVal("genDtl.schm.schmCode")
sv_c = BANCS.OUTPARAM.srvValue
sv_c = "schm_code=" + sv_c + "|bank_id=" +
BANCS.STDIN.contextBankId
BANCS.INPARAM.tgtTblKeyClause= sv_c
sv_e = urhk_deRegisterForDateShiftOperation("")
```

©Copyright Infosys Ltd.

Getting Holiday Status for Given Date

Using this userhook, we can get the holiday status of the date passed. The system checks the local calender (with the SOLID and DATE passed) and gives back the holiday status.

The output is available in BANCS.OUTPARAM.holidayStatus variable. The valid values for the field are:

- N - The date passed is not a holiday
- Y - The date passed is a holiday
- X - Record not found for the date passed in the calender

Syntax:

```
sv_r = urhk_GetHolidayStatus(sv_a)
```

Where sv_a is an input of the format "SOL_ID | date in DD-MM-YYYY format".

Input Arguments:

Input in the format SOLID| date in DD-MM-YYYY.

Input Repository Fields:

Input not present

Output Repository Fields:

Output not present

Return Value:

The output is available in BANCS.OUTPARAM.holidayStatus variable.
The valid values are N, Y and X.

For Example:

This is for checking whether a particular day is a holiday or not.

```
CUST.OPDATA.randomDate      = "19-07-2010"
```

```
CUST.OPDATA.solId           = "PA000001"
```

```
sv_a  = CUST.OPDATA.solId  + "|" + CUST.OPDATA.  
randomDate
```

```
sv_u = urhk_GetHolidayStatus(sv_a)
```

```
CUST.OPDATA.holidayStatus      =  
BANCS.OUTPARAM.holidayStatus
```

```
print(CUST.OPDATA.holidayStatus)
```

©Copyright Infosys Ltd.

Get the Last Successful DC

This userhook is used to get the last successful DC closure date

Syntax:

```
sv_b = urhk_sclastsuccDcClsDate ("")
```

Input Arguments:

Input arguments not present

Input Repository Fields:

Input Repository not present

Output Repository Fields:

BANCS.OUTPARAM. LastSuccDcClsDate

Return Value:

This user hook will return 0 on successful completion and on failure it will return 1 and error mesg will be stored in BANCS.OUTPARAM. retCode

For Example:

```
sv_z = urhk_sclastsuccDcClsDate ("")
print(sv_z)
if(sv_z != 0 ) then
print(BANCS.OUTPARAM.ErrMsg)
else
sv_v=BANCS.OUTPARAM. LastSuccDcClsDate
```

```
print(sv_v)
endif
```

©Copyright Infosys Ltd.

Check Whether it is a DC Holiday

This user hook finds out whether the given date is a DC Holiday.

Syntax:

```
sv_a = urhk_scrIsDcHoliday(<EffectiveDate>)
```

Input:

Valid Date.

Output Repository Fields:

The following output field is available for customizing this user hook

Output Field	Description
BANCS.OUTPUTPARAM.isDcHoliday	'Y' if it's a DC holiday

Output:

Returns Y if the date is not a datacentre working day.

For Example:

```
<--start
if(BANCS.INPARAM.EffectiveDate != "") then
    sv_h = urhk_scrIsDcHoliday(BANCS.INPARAM.EffectiveDate)
    sv_t = BANCS.OUTPUTPARAM.IsDcHoliday
    if(sv_t == "Y") then
        BANCS.OUTPUT.successOrFailure = "F"
        #MSG0319088 = "Effective Date is not DC Working
Day"
```

```
sv_f = urhk_getMSGCodeErrDesc("MSG0319088")
if(sv_f == 0) then
    BANCS.OUTPUT.errormsg =
BANCS.OUTPARAM.errDesc
endif
endif
endif
end - ->
```

©Copyright Infosys Ltd.

Check whether it is a ACH Holiday or Working Day

This userhook checks if the passed date is an ACH holiday. This userhook can be called from any script event and validates the input date. Returns failure if date format is not correct.

If inputs are valid, then the Holiday status is populated into Finacle repository.

Syntax:

```
sv_a = urhk_checkACHHoliday ("" )
```

Input:

Valid Date

Paysys ID

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.SettlementDate	Valid Date (in DD-MM-YYYY format).
BANCS.INPARAM.PaysysId	Paysys ID for which the Holiday status is required.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.holidayornot	Holiday Status for the given date and the Paysys ID.

Output:

If the input values are invalid then the return value is 1.

Holiday Status based on input date and Paysys ID is populated into the repository.

The Holiday Status also provides details if is a Working Day or a Holiday or if the record in the table is deleted.

For Example:

```
<--start
# Populate BOD date and Paysys ID
BANCS.INPARAM.PaysysId = BANCS.INPUT.paysysId
BANCS.INPARAM.SettlementDate = BANCS.STDIN.BODDate
sv_a = urhk_checkACHHoliday ("" )
if ( sv_a == 1 ) then
exitscript
endif
sv_i = BANCS.OUTPARAM.holidayornot
end - ->
```

©Copyright Infosys Ltd.

Get the Next ACH Working Date

This userhook returns the next ACH Working day for the given date and Paysys ID. This userhook can be called from any script event. Returns failure if date format is not correct.

If inputs are valid, then the Next ACH Working Date is populated into Finacle repository.

Syntax:

```
sv_a = urhk_nextACHWorkingdate("")
```

Input:

Valid Date

Paysys ID

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.SettlementDate	Valid Date (in DD-MM-YYYY format)
BANCS.INPARAM.PaysysId	Paysys ID for which the ACH working day is required.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.NextWorkingDate	Next ACH Working Date.

Output:

Next ACH Working Date for the given input values is populated into the repository.

For Example:

```
<--start
# Populate BOD date and Paysys ID
BANCS.INPARAM.PaysysId = BANCS.INPUT.paysysId
BANCS.INPARAM.SettlementDate = BANCS.STDIN.BODDate
sv_a = urhk_nextACHWorkingdate ("" )
if ( sv_a == 1 ) then
exitscript
endif
sv_i = BANCS.OUTPARAM.NextWorkingDate
end - ->
```

©Copyright Infosys Ltd.

Difference Between Two Dates in Months and Days

This user hook calculates the difference between two given dates in terms of calendar days and months depending on the flag `like_flag`. This flag determines whether the actual number of days in a calendar month should be used to calculate the difference between two given dates or a uniform number of days per month are to be used.

This `like_flag` should be set to YES, if the same number of days should be returned irrespective of the number of days in months (30 days). Otherwise, the `like_flag` should be set to NO, if actual number of days should be returned as per the calendar (for example, Feb – 28, Sep – 30, etc.)

Syntax:

```
sv_d = ("29-03-2002" + "|" + "28-02-2002" + "|" + "NO")  
sv_b = urhk_B2k_getMonthDayDiff(sv_d)
```

Input Arguments:

The system passes the following fields as input:

StartDate(dd-mm-yyyy)

EndDate(dd-mm-yyyy)

`like_flag`(NO or YES)

Input Repository Fields:

Input not present

Output Repository Fields:

The output is available in the BANCS.OUTPARAM.NumOfMonths and BANCS.OUTPARAM.NumOfDays fields, if the record is successfully inserted. In case the user hook fails to insert the record, the error message is found in the BANCS.OUTPARAM.errorMesg field.

Example:

```
< --start
trace on
sv_d = ("29-03-2002" + "|" + "28-02-2002" + "|" + "NO")
sv_b = urhk_B2k_getMonthDayDiff(sv_d)
if ( sv_b == 1) then
print ( BANCS.OUTPARAM.errorMesg)
else
print ( BANCS.OUTPARAM.NumOfMonths )
print ( BANCS.OUTPARAM.NumOfDays )
endif
exitscript
end-- >
```

©Copyright Infosys Ltd.

Get Difference between Two Frequencies

This userhook gets the difference between the given two frequencies and populates the ratio in terms of days in BANCS.OUTPARAM.FREQUENCY_RATIO.

Input Arguments:

Two frequencies.

For Example:

W| | | N|D| | | N|

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Ratio in terms of days between the two frequencies.

©Copyright Infosys Ltd.

Get the Time Zone Description

Get the Time Zone Description userhook is used to fetch the time zone description for the time zone passed.

Syntax:

```
sv_b = urhk_getTimeZoneDesc (“”)
```

sv_b: scratch pad variable to store the return value.

Input Repository Field:

Field Name	Description
BANCS.INPARAM.timeZoneCode (Mandatory)	Time Zone Code

Output Repository Field:

Field Name	Description
BANCS.OUTPARAM.Description	Description of the time zone passed.

©Copyright Infosys Ltd.

Demand Draft Userhooks

Remittance of funds is one of the services offered by banks. Remittances are effected by banks, using Demand Drafts (For remittances across cities). As the issue (credit) and payment (debit) of Demand Drafts happen at different branches, reconciliation of these debit and credit entries is essential. Apart from issuing and paying the drafts, banks are also required to modify, rectify, reverse, revalidate, cancel the drafts. They may also have to issue duplicate drafts if the original is lost.

Finacle provides userhooks which help us in retrieving DD related data from the database. Also a function is available which enables the change of DD Status.

This section discusses:

[Change DD Status](#)

[Get DD Credit Details](#)

[Get Scheme Details for DD Scheme Code and Currency](#)

©Copyright Infosys Ltd.

Change DD Status

This userhook changes the status of DD from Issued->Paid, Paid -> Issued and Lost -> Paid.

Syntax:

```
sv_b = urhk_changeDDStatus ("" )
```

Input Arguments:

- DD Number
- Issue Bank Code
- Issue Branch Code
- DD Issue Date
- DD Currency
- DD Scheme

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

DD Status

For Example:

```
<--start
```



```
trace on
if (repexists("BANCS") == 0) then
#{
createrep("BANCS")
#}
endif
BANCS.INPARAM.issBrCode = "0001"
BANCS.INPARAM.issBankCode = "024"
BANCS.INPARAM.ddNum = " 10014"
BANCS.INPARAM.issDate = "04-10-2002"
BANCS.INPARAM.crncy = "USD"
BANCS.INPARAM.schm_code = "DD1"
BANCS.INPARAM.ot_type = "!"
BANCS.INPARAM.final_status = "I"
BANCS.INPARAM.ref_num = ""
BANCS.INPARAM.solId = "000000"
sv_s = urhk_changeDDStatus("")
print(sv_s)
if (sv_s != 0) then
#{
sv_f = BANCS.OUTPARAM.successOrFailure
print(sv_f)
sv_e = BANCS.OUTPARAM.errorMesg
print(sv_e)
BANCS.OUTPUT.successOrFailure = "F"
BANCS.OUTPUT.scriptError = "urhk_changeDDStatus returns
```

```
FAILURE"
#}
else
#{
sv_s = BANCS.OUTPUTPARAM.dstStatus
print(sv_s)
sv_s = BANCS.OUTPUTPARAM.dstCautStatus
print(sv_s)
sv_s = BANCS.OUTPUTPARAM.ddcStatus
print(sv_s)
sv_r = BANCS.OUTPUTPARAM.adtRefNum
print(sv_r)
#}
endif
ENDOFSCRIPT:
trace off
end-->
```

©Copyright Infosys Ltd.

Get DD Credit Details

This userhook is used to retrieve credit details of a DD.

Syntax:

```
Sv_e = "tran_id" + "|" + "part_tran_srl_num" + "|" + "tran_date"
```

```
sv_a = urhk_getDDCDetails(sv_e)
```

Input Arguments:

The input is the DD credit key which is composed of:

- Transaction ID
- Part transaction serial number
- Transaction date

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.solid	solid
BANCS.OUTPARAM.ddDate	ddDate
BANCS.OUTPARAM.ddNum	ddNum
BANCS.OUTPARAM.ddCrncyCode	ddCrncyCode
BANCS.OUTPARAM.brCode	brCode
BANCS.OUTPARAM.ddRate	ddRate
BANCS.OUTPARAM.ddAmt	ddAmt
BANCS.OUTPARAM.commPostCrncyCode	commPostCrncyCode

BANCS.OUTPARAM.commPostsCrncyRate	commPostsCrncyRate
BANCS.OUTPARAM.commAmtCalc	commAmtCalc
BANCS.OUTPARAM.commAmtActual	commAmtActual
BANCS.OUTPARAM.commPtSrlNum	commPtSrlNum
BANCS.OUTPARAM.postAgeAmt	postAgeAmt
BANCS.OUTPARAM.postAgePtSrlNum	postAgePtSrlNum
BANCS.OUTPARAM.instrmntType	instrmntType
BANCS.OUTPARAM.instrmntAlpha	instrmntAlpha
BANCS.OUTPARAM.instrmntNum	instrmntNum
BANCS.OUTPARAM.purName	purName
BANCS.OUTPARAM.payeeName	payeeName
BANCS.OUTPARAM.purAcid	purAcid
BANCS.OUTPARAM.purCrncyCode	purCrncyCode
BANCS.OUTPARAM.purRate	purRate
BANCS.OUTPARAM.purAmt	purAmt
BANCS.OUTPARAM.contraPtSrlNum	contraPtSrlNum
BANCS.OUTPARAM.type	type
BANCS.OUTPARAM.stat	stat
BANCS.OUTPARAM.duplssuCnt	duplssuCnt
BANCS.OUTPARAM.duplssuDate	duplssuDate
BANCS.OUTPARAM.scCrncyCode	scCrncyCode
BANCS.OUTPARAM.scRate	scRate
BANCS.OUTPARAM.scCalc	scCalc
BANCS.OUTPARAM.scActual	scActual
BANCS.OUTPARAM.scTranId	scTranId
BANCS.OUTPARAM.scPtSrlNum	scPtSrlNum
BANCS.OUTPARAM.cancelPymtDate	cancelPymtDate
BANCS.OUTPARAM.prntOptn	prntOptn

BANCS.OUTPARAM.prntFlg	prntFlg
BANCS.OUTPARAM.prntCnt	prntCnt
BANCS.OUTPARAM.rectifiedCnt	rectifiedCnt
BANCS.OUTPARAM.ddCommDebtAcid	ddCommDebtAcid
BANCS.OUTPARAM.routingBankCode	routingBankCode
BANCS.OUTPARAM.bankCode	bankCode
BANCS.OUTPARAM.commCrRate	commCrRate
BANCS.OUTPARAM.partTranType	partTranType
BANCS.OUTPARAM.originalTranDate	originalTranDate
BANCS.OUTPARAM.originalTranId	originalTranId
BANCS.OUTPARAM.originalPartTranSrlNum	originalPartTranSrlNum

Return Value:

The output is populated in the repositories in the format BANCS.OUTPARAM.<field name>

For Example:

```

<--start
#trace on
sv_e=" AA23" + "|" + " 1" + "|" + "19-06-1998"
sv_r = urhk_getDDCDetails(sv_e)
print(BANCS.OUTPARAM.solid)
print(BANCS.OUTPARAM.ddDate)
print(BANCS.OUTPARAM.ddNum)
print(BANCS.OUTPARAM.ddCrncyCode)
print(BANCS.OUTPARAM.brCode)
print(BANCS.OUTPARAM.ddRate)
print(BANCS.OUTPARAM.ddAmt)

```

```
print(BANCS.OUTPARAM.commPostCrncyCode)
print(BANCS.OUTPARAM.commPostsCrncyRate)
print(BANCS.OUTPARAM.commAmtCalc)
print(BANCS.OUTPARAM.commAmtActual)
print(BANCS.OUTPARAM.commPtSrlNum)
print(BANCS.OUTPARAM.postAgeAmt)
print(BANCS.OUTPARAM.postAgePtSrlNum)
print(BANCS.OUTPARAM.instrmntType)
print(BANCS.OUTPARAM.instrmntAlpha)
print(BANCS.OUTPARAM.instrmntNum)
print(BANCS.OUTPARAM.purName)
print(BANCS.OUTPARAM.payeeName)
print(BANCS.OUTPARAM.purAcid)
print(BANCS.OUTPARAM.purCrncyCode)
print(BANCS.OUTPARAM.purRate)
print(BANCS.OUTPARAM.purAmt)
print(BANCS.OUTPARAM.contraPtSrlNum)
print(BANCS.OUTPARAM.type)
print(BANCS.OUTPARAM.stat)
print(BANCS.OUTPARAM.dupIssuCnt)
print(BANCS.OUTPARAM.dupIssuDate)
print(BANCS.OUTPARAM.scCrncyCode)
print(BANCS.OUTPARAM.scRate)
print(BANCS.OUTPARAM.scCalc)
print(BANCS.OUTPARAM.scActual)
print(BANCS.OUTPARAM.scTranId)
```

```
print(BANCS.OUTPARAM.scPtSrlNum)
print(BANCS.OUTPARAM.cancelPymtDate)
print(BANCS.OUTPARAM.prntOptn)
print(BANCS.OUTPARAM.prntFlg)
print(BANCS.OUTPARAM.prntCnt)
print(BANCS.OUTPARAM.rectifiedCnt)
print(BANCS.OUTPARAM.routingBrCode)
print(BANCS.OUTPARAM.ddCommDebtAcid)
print(BANCS.OUTPARAM.routingBankCode)
print(BANCS.OUTPARAM.bankCode)
print(BANCS.OUTPARAM.commCrRate)
print(BANCS.OUTPARAM.partTranType)
print(BANCS.OUTPARAM.originalTranDate)
print(BANCS.OUTPARAM.originalTranId)
print(BANCS.OUTPARAM.originalPartTranSrlNum)
#trace off
end-->
```

©Copyright Infosys Ltd.

Get Scheme Details for DD Scheme Code and Currency

This userhook takes scheme code of DDA scheme type and currency, and furnish details of that scheme. The details returned by userhook are as per the output.

Syntax:

```
sv_a=urhk_getDDSchemeCrncyDetails(schm_code|currency)
```

Input Arguments:

Scheme code belonging to DDA scheme type and currency for which the details are needed. The input must be a valid DDA Scheme type only else the output is a fatal error.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.schmCode	.
BANCS.OUTPARAM.crncyCode	.
BANCS.OUTPARAM.ddnUsedFlg	.
BANCS.OUTPARAM.ddaUsedFlg	.
BANCS.OUTPARAM.invtSerUsedFlg	.
BANCS.OUTPARAM.invtMicrUsedFlg	.
BANCS.OUTPARAM.rptLostUsedFlg	.
BANCS.OUTPARAM.ddXferBacid	.
BANCS.OUTPARAM.dfitInvtLocnClass	.

BANCS.OUTPARAM.dfltInvtLocnCode	.
BANCS.OUTPARAM.invtClass	.
BANCS.OUTPARAM.dfltInvtType	.
BANCS.OUTPARAM.dfltMicrInvtType	.
BANCS.OUTPARAM.otInvtType	.
BANCS.OUTPARAM.ttInvtType	.
BANCS.OUTPARAM.olInvtType	.
BANCS.OUTPARAM.tlInvtType	.
BANCS.OUTPARAM.otMicrInvtType	.
BANCS.OUTPARAM.ttMicrInvtType	.
BANCS.OUTPARAM.olMicrInvtType	.
BANCS.OUTPARAM.tlMicrInvtType	.
BANCS.OUTPARAM.prntSrlExistFlg	.
BANCS.OUTPARAM.ddnWarnLevelExcpCode	.
BANCS.OUTPARAM.ddCommChangedExcpCode	.
BANCS.OUTPARAM.ddPaidExcpCode	.
BANCS.OUTPARAM.ddRptLostExcpCode	.
BANCS.OUTPARAM.ddNonPymtAdvExcpCode	.
BANCS.OUTPARAM.ddDuplIssuedExcpCode	.
BANCS.OUTPARAM.ddCancScChngdExcpCode	.
BANCS.OUTPARAM.ddOrgTranNaExcpCode	.
BANCS.OUTPARAM.ddAcctMismatchExcpCode	.
BANCS.OUTPARAM.ddModAfterEodExcpCode	.
BANCS.OUTPARAM.ddPrntAfterEodExcpCode	.
BANCS.OUTPARAM.ddReprntAftEodExcpCode	.
BANCS.OUTPARAM.improperAdvExcpCode	.
BANCS.OUTPARAM.improperStatusExcpCode	.
BANCS.OUTPARAM.ddTemplateFileName	.
BANCS.OUTPARAM.defaultBankCode	.

BANCS.OUTPUTPARAM.defaultBrCode	.
BANCS.OUTPUTPARAM.bankersChequeFlg	.
BANCS.OUTPUTPARAM.sendlbrFlg	.
BANCS.OUTPUTPARAM.commCrncyHcFc	.
BANCS.OUTPUTPARAM.ddSellRateCode	.
BANCS.OUTPUTPARAM.ddBuyRateCode	.
BANCS.OUTPUTPARAM consolidatePartTranFlg	.
BANCS.OUTPUTPARAM consolidatePartTranBacid	.
BANCS.OUTPUTPARAM.hotItemExcpCode	.
BANCS.OUTPUTPARAM.bankersChequeExcpCode	.
BANCS.OUTPUTPARAM.ddNotThrolbrExcpCode	.
BANCS.OUTPUTPARAM.pymtOfDupDdExcpCode	.
BANCS.OUTPUTPARAM.pymtOfLostDdExcpCode	.
BANCS.OUTPUTPARAM.pymtOfStopDdExcpCode	.
BANCS.OUTPUTPARAM.localEventId	.
BANCS.OUTPUTPARAM.otherEventId	.

Return Value:

The values related to the scheme code, currency combination is available as output.

©Copyright Infosys Ltd.

DSA Related Userhooks

This section discusses:

[Get DSA Effective Available Limit](#)

[Get DSA Master Details](#)

[Get DSA Parameter Details](#)

[Get DSA Product Details](#)

[Get DSA Turnover Details](#)

[Get DSA Turnover Details for Scheme Type](#)

[Get Past Due Cycles](#)

[Update DSA Turnover Details](#)

©Copyright Infosys Ltd.

Get DSA Effective Available Limit

This userhook is provided to compute the Effective Available Limit and also fetches the Account Details such as Foracid, Account Short Name, Account SOL ID and Account Currency Code.

Syntax:

```
sv_a = urhk_getDSAEffAvailLimit()
```

Input Arguments:

- Account ID (acid)
- DSA ID (dsa_id)
- Select Record Flag (sel_flg)
- Increment Decrement Flag (inc_dec_flg)
- DSA Document Transaction Utilization Limit (util_dsa_doc_txn_limit)

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return value:

Account ID (foracid), Account Short Name (acctShortName), Account Currency Code (acctCrncyCode), Account SOL ID (acctSolId), Effective Available Limit Amount (effAvailLimit) and Effective Available Limit Currency (effAvailLimitCrncy).

For Example:

```
<--start
#trace on
BANCS.INPARAM.acid = sv_a
BANCS.INPARAM.dsa_id = sv_b
BANCS.INPARAM.sel_flg = sv_c
BANCS.INPARAM.inc_dec_flg = sv_d
BANCS.INPARAM.util_dsa_doc_txn_limit = sv_e
sv_f = urhk_getDSAEffAvailLimit()
if (sv_f == 0) then
if (fieldexists(BANCS.OUTPARAM.effAvailLimit)) then
BANCS.OUTPUT.eff_avail_limit =
BANCS.OUTPARAM.effAvailLimit
endif
if (fieldexists(BANCS.OUTPARAM.effAvailLimitCrncy)) then
BANCS.OUTPUT.eff_avail_limit_crncy =
BANCS.OUTPARAM.effAvailLimitCrncy
endif
endif
#trace off
end-->
```

Get DSA Master Details

This userhook gives the DSA master details for the given DSA ID.

Syntax:

```
sv_a = urhk_getDSAMasterDetails ("dsald")
```

Input Arguments:

DSA ID

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.DSAMInd	DSAMInd
BANCS.OUTPARAM.dsald	dsald
BANCS.OUTPARAM.delFlg	delFlg
BANCS.OUTPARAM.dsaSolId	dsaSolId
BANCS.OUTPARAM.dsaName	dsaName
BANCS.OUTPARAM.dsaAddr1	dsaAddr1
BANCS.OUTPARAM.dsaAddr2	dsaAddr2
BANCS.OUTPARAM.dsaCityCode	dsaCityCode
BANCS.OUTPARAM.dsaRegionCode	dsaRegionCode
BANCS.OUTPARAM.dsaCntryCode	dsaCntryCode
BANCS.OUTPARAM.dsaPinCode	dsaPinCode
BANCS.OUTPARAM.dsaPhoneNum	dsaPhoneNum
BANCS.OUTPARAM.dsaFaxNum	dsaFaxNum

BANCS.OUTPUTPARAM.dsaMobileNum	dsaMobileNum
BANCS.OUTPUTPARAM.dsaEmailId	dsaEmailId
BANCS.OUTPUTPARAM.dsaLicenseNum	dsaLicenseNum
BANCS.OUTPUTPARAM.remarks	remarks
BANCS.OUTPUTPARAM.sysDsaGrade	sysDsaGrade
BANCS.OUTPUTPARAM.userDsaGrade	userDsaGrade
BANCS.OUTPUTPARAM.dsaType	dsaType
BANCS.OUTPUTPARAM.dsaLevel	dsaLevel
BANCS.OUTPUTPARAM.parentDsald	parentDsald
BANCS.OUTPUTPARAM.commPaymentMode	commPaymentMode
BANCS.OUTPUTPARAM.commConsol	commConsol
BANCS.OUTPUTPARAM.commConsolDsald	commConsolDsald
BANCS.OUTPUTPARAM.commCreditAcct	commCreditAcct
BANCS.OUTPUTPARAM.taxEventId	taxEventId
BANCS.OUTPUTPARAM.dsaBlackListed	dsaBlackListed
BANCS.OUTPUTPARAM.dsaDocTxnLimit	dsaDocTxnLimit
BANCS.OUTPUTPARAM.dsaDocTxnLimitCrncy	dsaDocTxnLimitCrncy
BANCS.OUTPUTPARAM.dsaDocTxnUtilLimit	dsaDocTxnUtilLimit
BANCS.OUTPUTPARAM.freeCode1	freeCode1
BANCS.OUTPUTPARAM.freeCode2	freeCode2
BANCS.OUTPUTPARAM.freeCode3	freeCode3
BANCS.OUTPUTPARAM.freeCode4	freeCode4
BANCS.OUTPUTPARAM.dsaTorFreqType	dsaTorFreqType
BANCS.OUTPUTPARAM.dsaTorFreqWeekNum	dsaTorFreqWeekNum
BANCS.OUTPUTPARAM.dsaTorFreqWeekDay	dsaTorFreqWeekDay
BANCS.OUTPUTPARAM.dsaTorFreqStartDD	dsaTorFreqStartDD
BANCS.OUTPUTPARAM.dsaTorFreqHldyStat	dsaTorFreqHldyStat
BANCS.OUTPUTPARAM.dsaNextTurnoverDate	dsaNextTurnoverDate
BANCS.OUTPUTPARAM.dsaOpenedDate	dsaOpenedDate

BANCS.OUTPUTPARAM.cosDD	cosDD
BANCS.OUTPUTPARAM.cosDDCrncyCode	cosDDCrncyCode
BANCS.OUTPUTPARAM.cosDDBenefName	cosDDBenefName
BANCS.OUTPUTPARAM.cosDDPayableAt	cosDDPayableAt
BANCS.OUTPUTPARAM.cosDDBankCode	cosDDBankCode
BANCS.OUTPUTPARAM.cosDDBrCode	cosDDBrCode
BANCS.OUTPUTPARAM.cosDDDeductComm	cosDDDeductComm

Return Value:

Return value not available.

©Copyright Infosys Ltd.

Get DSA Parameter Details

This userhook gives the DSA parameter details into the repository.

Syntax:

```
sv_a = urhk_getDSAParamDetails ("")
```

Input Arguments:

None

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPUTPARAM.DSAPInd	DSAPInd
BANCS.OUTPUTPARAM.sysGenDsald	sysGenDsald
BANCS.OUTPUTPARAM.dsaParmId	dsaParmId
BANCS.OUTPUTPARAM.nextDsaNumCode	nextDsaNumCode
BANCS.OUTPUTPARAM.cosAcctBacid	cosAcctBacid
BANCS.OUTPUTPARAM.ddAcctBacid	ddAcctBacid
BANCS.OUTPUTPARAM.ddChargeEventId	ddChargeEventId

Return value:

Return value not available.

Get DSA Product Details

This userhook populates the DSA product details into the repository.

Syntax:

```
sv_a = urhk_getDSAProductDetail ("" )
```

Input Arguments:

DSA ID

Product Code

Currency Code

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.DSPRInd	DSPRInd
BANCS.OUTPARAM.dsald	dsald
BANCS.OUTPARAM.productCode	productCode
BANCS.OUTPARAM.crncyCode	crncyCode
BANCS.OUTPARAM.delFlg	delFlg
BANCS.OUTPARAM.commissionAlwd	commissionAlwd
BANCS.OUTPARAM.endDate_S	endDate_S
BANCS.OUTPARAM.dsaCrPrefIntPcnt	dsaCrPrefIntPcnt
BANCS.OUTPARAM.dsaDrPrefIntPcnt	dsaDrPrefIntPcnt
BANCS.OUTPARAM.fixedCommEventId	fixedCommEventId

BANCS.OUTPARAM.fixedFreqType	fixedFreqType
BANCS.OUTPARAM.fixedFreqWeekNum	fixedFreqWeekNum
BANCS.OUTPARAM.fixedFreqWeekDay	fixedFreqWeekDay
BANCS.OUTPARAM.fixedFreqStartDD	fixedFreqStartDD
BANCS.OUTPARAM.fixedFreqHldyStat	fixedFreqHldyStat
BANCS.OUTPARAM.nextFixedCommCalcDate	nextFixedCommCalcDate
BANCS.OUTPARAM.freeCode	freeCode
BANCS.OUTPARAM.perdCommEventId_1	perdCommEventId_1
BANCS.OUTPARAM.perdFreqType_1	perdFreqType_1
BANCS.OUTPARAM.perdFreqWeekDay_1	perdFreqWeekDay_1
BANCS.OUTPARAM.perdFreqStartDD_1	perdFreqStartDD_1
BANCS.OUTPARAM.perdFreqHldyStat_1	perdFreqHldyStat_1
BANCS.OUTPARAM.targetAmt_1	targetAmt_1
BANCS.OUTPARAM.targetDateForm_1	targetDateForm_1
BANCS.OUTPARAM.targetDateTo_1	targetDateTo_1
BANCS.OUTPARAM.penalCommEventId_1	penalCommEventId_1
BANCS.OUTPARAM.nextCommCalcDate_1	nextCommCalcDate_1
BANCS.OUTPARAM.perdCommEventId_2	perdCommEventId_2
BANCS.OUTPARAM.perdFreqType_2	perdFreqType_2
BANCS.OUTPARAM.perdFreqWeekNum_2	perdFreqWeekNum_2
BANCS.OUTPARAM.perdFreqWeekDay_2	perdFreqWeekDay_2
BANCS.OUTPARAM.perdFreqStartDD_2	perdFreqStartDD_2
BANCS.OUTPARAM.perdFreqHldyStat_2	perdFreqHldyStat_2
BANCS.OUTPARAM.targetAmt_2	targetAmt_2
BANCS.OUTPARAM.targetDateForm_2	targetDateForm_2
BANCS.OUTPARAM.targetDateTo_2	targetDateTo_2
BANCS.OUTPARAM.penalCommEventId_2	penalCommEventId_2
BANCS.OUTPARAM.nextCommCalcDate_2	nextCommCalcDate_2
BANCS.OUTPARAM.perdCommEventId_3	perdCommEventId_3

BANCS.OUTPARAM.perdFreqType_3	perdFreqType_3
BANCS.OUTPARAM.perdFreqWeekNum_3	perdFreqWeekNum_3
BANCS.OUTPARAM.perdFreqWeekDay_3	perdFreqWeekDay_3
BANCS.OUTPARAM.perdFreqStartDD_3	perdFreqStartDD_3
BANCS.OUTPARAM.perdFreqHldyStat_3	perdFreqHldyStat_3
BANCS.OUTPARAM.targetAmt_3	targetAmt_3
BANCS.OUTPARAM.targetDateForm_3	targetDateForm_3
BANCS.OUTPARAM.targetDateTo_3	targetDateTo_3
BANCS.OUTPARAM.penalCommEventId_3	penalCommEventId_3
BANCS.OUTPARAM.nextCommCalcDate_3	nextCommCalcDate_3
BANCS.OUTPARAM.perdCommEventId_4	perdCommEventId_4
BANCS.OUTPARAM.perdFreqType_4	perdFreqType_4
BANCS.OUTPARAM.perdFreqWeekNum_4	perdFreqWeekNum_4
BANCS.OUTPARAM.perdFreqWeekDay_4	perdFreqWeekDay_4
BANCS.OUTPARAM.perdFreqStartDD_4	perdFreqStartDD_4
BANCS.OUTPARAM.perdFreqHldyStat_4	perdFreqHldyStat_4
BANCS.OUTPARAM.targetAmt_4	targetAmt_4
BANCS.OUTPARAM.targetDateForm_4	targetDateForm_4
BANCS.OUTPARAM.targetDateTo_4	targetDateTo_4
BANCS.OUTPARAM.penalCommEventId_4	penalCommEventId_4
BANCS.OUTPARAM.nextCommCalcDate_4	nextCommCalcDate_4
BANCS.OUTPARAM.perdCommEventId_5	perdCommEventId_5
BANCS.OUTPARAM.perdFreqType_5	perdFreqType_5
BANCS.OUTPARAM.perdFreqWeekNum_5	perdFreqWeekNum_5
BANCS.OUTPARAM.perdFreqWeekDay_5	perdFreqWeekDay_5
BANCS.OUTPARAM.perdFreqStartDD_5	perdFreqStartDD_5
BANCS.OUTPARAM.perdFreqHldyStat_5	perdFreqHldyStat_5
BANCS.OUTPARAM.targetAmt_5	targetAmt_5

BANCS.OUTPARAM.targetDateForm_5	targetDateForm_5
BANCS.OUTPARAM.targetDateTo_5	targetDateTo_5
BANCS.OUTPARAM.penalCommEventId_5	penalCommEventId_5
BANCS.OUTPARAM.nextCommCalcDate_5	nextCommCalcDate_5

Return value:

Return value not available.

©Copyright Infosys Ltd.

Get DSA Turnover Details

This userhook gives the DSA turnover details for the given DSA ID, product code, currency code and As on date. The inputs are to be separated by a delimiter. This userhook populates DSA turnover details into repository.

Syntax:

```
sv_a = urhk_getDSATurnoverDetails ("" )
```

Input Arguments:

DSA ID
Product Code
Currency Code
As on Date

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.DSATOInd_S	DSATOInd_S
BANCS.OUTPARAM.dsald_S	dsald_S

Return Value:

Return value not available.

Get DSA Turnover Details for Scheme Type

This userhook gives the DSA turnover details for the given DSA ID, Scheme Type, Currency Code and As on Date. The inputs are to be separated by a delimiter.

Syntax:

```
sv_a = urhk_getDSATODetailsForSchemeType ()
```

Input Arguments:

- DSA ID
- Scheme type
- Currency code
- As on Date

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.DSATOInd_S	DSATOInd_S
BANCS.OUTPARAM.dsald_S	dsald_S
BANCS.OUTPARAM.schemeType_S	schemeType_S
BANCS.OUTPARAM.crncyCode_S	crncyCode_S
BANCS.OUTPARAM.turnoverInd_S	turnoverInd_S
BANCS.OUTPARAM.endDate_S	endDate_S
BANCS.OUTPARAM.numOfDrTrans_S	numOfDrTrans_S

BANCS.OUTPARAM.numOfCrTrans_S	numOfCrTrans_S
BANCS.OUTPARAM.totDrBalAmt_S	totDrBalAmt_S
BANCS.OUTPARAM.totCrBalAmt_S	totCrBalAmt_S
BANCS.OUTPARAM.highDrBalAmt_S	highDrBalAmt_S
BANCS.OUTPARAM.highCrBalAmt_S	highCrBalAmt_S
BANCS.OUTPARAM.lowDrBalAmt_S	lowDrBalAmt_S
BANCS.OUTPARAM.lowCrBalAmt_S	lowCrBalAmt_S
BANCS.OUTPARAM.cumDrTotAmt_S	cumDrTotAmt_S
BANCS.OUTPARAM.cumCrTotAmt_S	cumCrTotAmt_S
BANCS.OUTPARAM.lastRunCumBal_S	lastRunCumBal_S
BANCS.OUTPARAM.lastRunAvgBal_S	lastRunAvgBal_S
BANCS.OUTPARAM.cumCurBal_S	cumCurBal_S
BANCS.OUTPARAM.cumOpenBal_S	cumOpenBal_S
BANCS.OUTPARAM.cumCloseBal_S	cumCloseBal_S
BANCS.OUTPARAM.totSanctionLim_S	totSanctionLim_S
BANCS.OUTPARAM.cumTotSanctionLim_S	cumTotSanctionLim_S
BANCS.OUTPARAM.totDrawingPower_S	totDrawingPower_S
BANCS.OUTPARAM.cumTotDrawingPower_S	cumTotDrawingPower_S
BANCS.OUTPARAM.numOfAcctsOpened_S	numOfAcctsOpened_S
BANCS.OUTPARAM.totNumOfAcctsOpened_S	totNumOfAcctsOpened_S
BANCS.OUTPARAM.numOfAcctsClosed_S	numOfAcctsClosed_S
BANCS.OUTPARAM.totNumOfAcctsClosed_S	totNumOfAcctsClosed_S
BANCS.OUTPARAM.clsAcctsElgForPenal_S	clsAcctsElgForPenal_S
BANCS.OUTPARAM.totClsAcctsElgForPenal_S	totClsAcctsElgForPenal_S
BANCS.OUTPARAM.DSATOInd_O	DSATOInd_O
BANCS.OUTPARAM.dsald_O	dsald_O
BANCS.OUTPARAM.schemeType_O	schemeType_O
BANCS.OUTPARAM.crncyCode_O	crncyCode_O
BANCS.OUTPARAM.turnoverInd_O	turnoverInd_O

BANCS.OUTPARAM.endDate_O	endDate_O
BANCS.OUTPARAM.numOfDrTrans_O	numOfDrTrans_O
BANCS.OUTPARAM.numOfCrTrans_O	numOfCrTrans_O
BANCS.OUTPARAM.totDrBalAmt_O	totDrBalAmt_O
BANCS.OUTPARAM.totCrBalAmt_O	totCrBalAmt_O
BANCS.OUTPARAM.highDrBalAmt_O	highDrBalAmt_O
BANCS.OUTPARAM.highCrBalAmt_O	highCrBalAmt_O
BANCS.OUTPARAM.lowDrBalAmt_O	lowDrBalAmt_O
BANCS.OUTPARAM.lowCrBalAmt_O	lowCrBalAmt_O
BANCS.OUTPARAM.cumDrTotAmt_O	cumDrTotAmt_O
BANCS.OUTPARAM.cumCrTotAmt_O	cumCrTotAmt_O
BANCS.OUTPARAM.lastRunCumBal_O	lastRunCumBal_O
BANCS.OUTPARAM.lastRunAvgBal_O	lastRunAvgBal_O
BANCS.OUTPARAM.cumCurBal_O	cumCurBal_O
BANCS.OUTPARAM.cumOpenBal_O	cumOpenBal_O
BANCS.OUTPARAM.cumCloseBal_O	cumCloseBal_O
BANCS.OUTPARAM.totSanctionLim_O	totSanctionLim_O
BANCS.OUTPARAM.cumTotSanctionLim_O	cumTotSanctionLim_O
BANCS.OUTPARAM.totDrawingPower_O	totDrawingPower_O
BANCS.OUTPARAM.cumTotDrawingPower_O	cumTotDrawingPower_O
BANCS.OUTPARAM.numOfAcctsOpened_O	numOfAcctsOpened_O
BANCS.OUTPARAM.totNumOfAcctsOpened_O	totNumOfAcctsOpened_O
BANCS.OUTPARAM.numOfAcctsClosed_O	numOfAcctsClosed_O
BANCS.OUTPARAM.totNumOfAcctsClosed_O	totNumOfAcctsClosed_O
BANCS.OUTPARAM.clsAcctsElgForPenal_O	clsAcctsElgForPenal_O
BANCS.OUTPARAM.totClsAcctsElgForPenal_O	totClsAcctsElgForPenal_O

Return Value:

DSA Turnover details: Self and Others.

The Self-Turnover details is available only if DSATOInd_S is set to YES. Otherwise other fields must not be accessed.

The Others-Turnover details is available only if DSATOInd_O is set to YES. Otherwise other fields must not be accessed.

For Example:

```
# Get the inputs from repository
sv_a = BANCS.INPUT.dsaId
sv_b = BANCS.INPUT.schemeType
sv_c = BANCS.INPUT.crncyCode
sv_d = BANCS.INPUT.asOnDate
sv_e = sv_a + "|" + sv_b + "|" + sv_c + "|" + sv_d
#call the user hook for getting DSA turnover details
sv_n = urhk_ getDSATODetailsForSchemeType (sv_e)
IF (sv_n != 0) THEN
BANCS.OUTPUT.successOrFailure="F"
BANCS.OUTPUT.error= "DSA Turnover details not found"
ENDIF
```

©Copyright Infosys Ltd.

Get Past Due Cycles

This userhook gets the 15 past due cycles for a given account. This userhook retrieves ten past due cycles.

Syntax:

```
sv_a = urhk_getPastDueCycles("A/c Number")
```

Where, sv_a is a scratch pad variable to store the return value.

Input arguments:

The input for this userhook is the Account ID.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.Cycle1	.
BANCS.OUTPARAM.Cycle2	.
BANCS.OUTPARAM.Cycle3	.
BANCS.OUTPARAM.Cycle4	.
BANCS.OUTPARAM.Cycle5	.
BANCS.OUTPARAM.Cycle6	.
BANCS.OUTPARAM.Cycle7	.
BANCS.OUTPARAM.Cycle8	.
BANCS.OUTPARAM.Cycle9	.
BANCS.OUTPARAM.Cycle10	.
BANCS.OUTPARAM.Cycle11	.

BANCS.OUTPARAM.Cycle12	.
BANCS.OUTPARAM.Cycle13	.
BANCS.OUTPARAM.Cycle14	.
BANCS.OUTPARAM.Cycle15	.
BANCS.OUTPARAM.ErrMsg	.

Return value:

Return value not available.

For Example:

```
#call the user hook for getting Past Due Cycles
```

```
<--start
```

```
trace on
```

```
sv_a = urhk_getPastDueCycles("LA1")
```

```
print(sv_a)
```

```
if (sv_a == 0) then
```

```
print(BANCS.OUTPARAM.Cycle1)
```

```
print(BANCS.OUTPARAM.Cycle2)
```

```
print(BANCS.OUTPARAM.Cycle3)
```

```
print(BANCS.OUTPARAM.Cycle4)
```

```
print(BANCS.OUTPARAM.Cycle5)
```

```
print(BANCS.OUTPARAM.Cycle6)
```

```
print(BANCS.OUTPARAM.Cycle7)
```

```
print(BANCS.OUTPARAM.Cycle8)
```

```
print(BANCS.OUTPARAM.Cycle9)
```

```
print(BANCS.OUTPARAM.Cycle10)
```

```
print(BANCS.OUTPARAM.Cycle11)
```

```
print(BANCS.OUTPARAM.Cycle12)
print(BANCS.OUTPARAM.Cycle13)
print(BANCS.OUTPARAM.Cycle14)
print(BANCS.OUTPARAM.Cycle15)
else
print(BANCS.OUTPARAM.ErrMsg)
endif
exitscript
end-->
```

©Copyright Infosys Ltd.

Update DSA Turnover Details

An account may be sourced or disbursed through DSA (direct selling agent). Every DSA has a turnover history maintained, that is, how many accounts opened, how many accounts closed during a particular duration.

This DSA turnover is updated as a batch job depending on the frequency. So, if, say in January, 23 accounts were opened and 20 accounts were closed, the same is stored in the turnover table on the day the job runs, say 1st February.

However, if a value dated account opening is done on 10th February with account open date as 11th January, it does not figure in the turnover table.

For this purpose at the time of account opening this userhook is provided. The purpose of this hook is to update the turnover at the time of account opening itself when the value dated account opening is done and the turnover has been already inserted by the batch program. The same is the case for closure of accounts.

If the opening is value dated but the turnover job for the day is not yet run, the hook will do nothing, as this opening will be picked up when the job runs next.

Syntax:

```
sv_a = urhk_acopDsatoUpdate("")
```

Input Arguments:

```
usrhk_getval(TBAFATAL, "inpSourceDealCode",  
usrhk_SourceDealCode, sizeof(dsaldType))
```

```
usrhk_getval(TBAFATAL, "inpSchmCode", usrhk_SchmCode,  
sizeof(schemeCodeType))
```

```
usrhk_getval(TBAFATAL, "inpAcctOpnDate", usrhk_AcctOpnDate,  
sizeof(tbaDateType))
```

```
usrhk_getval(TBAFATAL, "inpAcctCrncyCode", usrhk_AcctCrncyCode, sizeof(currencyCodeType))
```

```
usrhk_getval(TBAFATAL, "inpSanctLim", usrhk_SanctLimStr, sizeof(tbaAmtStrType))
```

```
usrhk_getval(TBAFATAL, "inpDrwngPower", usrhk_DrwngPowerStr, sizeof(tbaAmtStrType))
```

```
usrhk_getval(TBAFATAL, "inpIncOrDecFlag", SingleCharStr, 2)
```

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Output parameters are:

- errMesg - can be non null if successOrFailure is "F"

- successOrFailure - can be "S" or "F"

For Example:

Please check the example B2K_AcopDsatoUpdate.sscr in sample/scripts directory.

Fetch Data from DB

This section discusses:

[DB Cursor Close](#)

[DB Cursor Fetch](#)

[DB Cursor Open](#)

[DB Cursor Open with bind variables](#)

[Get Details from CTC Table for Transaction Code](#)

[Selection of Fields from Database](#)

[Selection of Fields from Database using Bind Variables](#)

[Selection of Fields from Database using Bind Variables for Larger Column Length](#)

[Using SQL Statement in Script](#)

[Using SQL Statement with bind variables in script](#)

[Get Interpool Amounts for a Financial Year](#)

[Get Details of RPC Disbursements](#)

[Get List of Accounts Based on Instrument Number](#)

[Get Past Due Cycles](#)

[Get Pool Available Amount](#)

[Get Tranche Outstanding](#)

[Get User Additional Details](#)

[Getting Scheme Details into Bancs Repository](#)

[Fetch the Event Type Description](#)

[Get Conversion Rate](#)

[Get Tax Rate](#)

[Get Destination Bank ID](#)

©Copyright Infosys Ltd.

DB Cursor Close

This userhook closes the cursor for the given cursor_number. This userhook is called after the records have been fetched from the cursor through the dbCursorFetch userhook.

Syntax:

```
Sv_m = BANCS.OUTPARAM.DB_CURSOR_NUMBER  
sv_a = urhk_dbCursorClose(sv_m)
```

Input Arguments:

Input is a DB_CURSOR_NUMBER returned by dbCursorOpen.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

This userhook returns '0' on successful closing the cursor.

For Example:

```
<--start  
trace on  
sv_p = ""  
sv_q = ""  
sv_b = "acid,foracid | select acid,foracid from gam where  
cust_id = ' ACME' "
```

```
sv_e = urhk_dbCursorOpen(sv_b)
if ( sv_e == 0 ) then
sv_m = BANCS.OUTPARAM.DB_CURSOR_NUMBER
sv_r = urhk_dbCursorFetch(sv_m)
while(sv_r == 0)
sv_p = sv_p + BANCS.OUTPARAM.acid
sv_q = sv_q + BANCS.OUTPARAM.foracid
sv_r = urhk_dbCursorFetch(sv_m)
do
sv_x = urhk_dbCursorClose(sv_m)
endif
print(sv_p)
print(sv_q)
trace off
end-->
```

©Copyright Infosys Ltd.

DB Cursor Fetch

This userhook fetches the records from open cursor for the given cursor_number. For fetching all the records from the cursor this userhook is used in the loop.

Syntax:

```
Sv_m = BANCS.OUTPARAM.DB_CURSOR_NUMBER  
sv_a = urhk_dbCursorFetch(sv_m)
```

Input Arguments:

Input is a DB_CURSOR_NUMBER returned by dbCursorOpen.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.acid	.
BANCS.OUTPARAM.foracid	.

Return Value:

The userhook returns '0' on successful fetching the record. The fetched record is stored in the OUTPARAM repository.

DB Cursor Open

This userhook opens the cursor for the given query and puts the db_cursor_number in the repository.

Syntax:

```
Sv_m = "acid,foracid | select acid,foracid from gam where cust_id =  
"XYZ"
```

```
sv_a = urhk_dbCursorOpen(sv_m)
```

Input Arguments:

Input is a string that is the query corresponding which cursor is opened.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The userhook returns '0' on successful open. The opening cursor number is stored in BANCS.OUTPARAM.DB_CURSOR_NUMBER.

For Example:

```
<--start  
trace on  
sv_p = ""  
sv_q = ""
```

```
sv_b = "acid,foracid | select acid,foracid from gam where  
cust_id = ' ACME' "  
sv_e = urhk_dbCursorOpen(sv_b)  
if ( sv_e == 0 ) then  
sv_m = BANCS.OUTPARAM.DB_CURSOR_NUMBER  
sv_r = urhk_dbCursorFetch(sv_m)  
while(sv_r == 0)  
sv_p = sv_p + BANCS.OUTPARAM.acid  
sv_q = sv_q + BANCS.OUTPARAM.foracid  
sv_r = urhk_dbCursorFetch(sv_m)  
do  
sv_x = urhk_dbCursorClose(sv_m)  
endif  
print(sv_p)  
print(sv_q)  
trace off  
end-->
```

©Copyright Infosys Ltd.

DB Cursor Open with Bind Variables

This userhook opens the cursor for the given query and puts the db_cursor_number in the repository.

Syntax:

```
lv_r = urhk_dbCursorOpenWithBind(lv_s)
```

Input Arguments:

SQL Statement

Input Repository Fields:

BANCS.INPARAM.BINDVARS

The inputs should be passed to the SQL query through BANCS.INPARAM.BINDVARS with delimiter as "|". In the query the following placeholders are supported:

- ❏SVAR: String bind values (including VARCHAR, CHAR, DATE data types)
- ❏NVAR: Numeric bind values (for NUMBER data type)

Output Repository Fields:

Output not present.

Return Value:

The output fields are stored in BANCS.OUTPARAM.<title field>. An error free condition is represented by 0 value of BANC.OUTPARAM.DB_ERRCODE.

For Example:

```
BANCS.INPARAM.BINDVARS = lv_a + "|" + FORMAT$(lv_b, "%f") + "|" + FORMAT$(lv_c, "%f")
```

```
lv_s = "acid, foracid, clrBalAmt | "
```

```
lv_s = lv_s + "SELECT acid, foracid, clr_bal_amt "
```

```
lv_s = lv_s + "FROM gam "
```

```
lv_s = lv_s + "WHERE acct_crncy_code = ?SVAR "
```

```
lv_s = lv_s + "AND clr_bal_amt between ?NVAR AND ?NVAR "
```

```
lv_s = lv_s + "AND entity_cre_flg = 'Y' "
```

```
lv_s = lv_s + "AND del_flg = 'N' "
```

```
lv_r = urhk_dbCursorOpenWithBind(lv_s)
```

Note:

The other user-hooks required for CURSORS remain same: urhk_dbCursorFetch() and urhk_dbCursorClose().

©Copyright Infosys Ltd.

Get Details from CTC Table for Transaction Code

This function gets all the attributes of a 'Clearing transaction code' (as printed on the instrument) that is maintained in the CTC table of Finacle. Clearing transaction codes are used in India to distinguish between various types of clearing.

Syntax:

The syntax for calling the function is:

```
sv_r = urhk_GetCTCDetailsFromTranCode(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("10")

Input Arguments:

The input to the function is Clearing transaction code

Example - "10"

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Returns '1' if the transaction code does not exists in the CTC table, else returns 0.

All the attributes of the transaction code stored in the CTC table is returned in the fields of BANCS repository and OUTPARAM class mentioned here:

Field Name in CTC table

Field name in BANCS repository

sys_arrived_acct_flg BANCS.OUTPARAM.SysArrivedAcctFlg

'Y' or 'N' Automatically arrive at the account number using cheque issue records.

dr_bacid BANCS.OUTPARAM.DrBacid

If system arrived flag is 'N', then account place holder that is to be used.

instrmnt_type BANCS.OUTPARAM.InstrmntType

Default instrument type for the transaction code.

media_load_flg BANCS.OUTPARAM.MediaLoadFlg

Can this record be loaded electronically.

applicability_ind BANCS.OUTPARAM.ApplicabilityInd

I'nward clearing, O'utward Clearing or B'oth

consolidation_flg BANCS.OUTPARAM.ConsolidationFlg

default_upld_acct_prefix BANCS.OUTPARAM.

DefaultUpldAcctPrefix

Not being used now.

search_schm_type BANCS.OUTPARAM.SearchSchmType

System will look at these scheme types for faster access based on cheque book issue accounts.

Selection of Fields from Database

This userhook allows script writer to select data from database tables. The user specifies the title of each selected field separated by comma before pipe symbol and specifies the complete query after that. It stores the output error code of the query in `BANCS.OUTPARAM.DB_ERRCODE` and message in `BANCS.OUTPARAM.DB_ERRMSG`. In case the query returns more than single row, it gives a fatal error.

Syntax:

```
lv_r = urhk_dbSelectWithBind(lv_s)
```

The variable can be a scratch pad variable (`sv_a`) or a string (" today | SELECT sysdate FROM dual").

Note:

The variable `BANCS.INPARAM.BINDVARS` should be initialized as null before using the user-hook in any static query.

Input Arguments:

SQL Statement

Input Repository Fields:

The inputs should be passed to the SQL query through `BANCS.INPARAM.BINDVARS` with delimiter as "|". And in the query the following placeholders are supported.

- `?SVAR:` String bind values (including VARCHAR, CHAR, DATE data types)

- **?NVAR:** Numeric bind values (for NUMBER data type)

Output Repository Fields:

Output not present.

Return Value:

The output fields are stored in BANCS.OUTPARAM.<title field>. An error free condition is represented by 0 value of BANC.OUTPARAM.DB_ERRCODE.

For Example:

```
BANCS.INPARAM.BINDVARS=lv_d
print(BANCS.INPARAM.BINDVARS)
lv_r="solTenor,solId|select user_sol_tenor, sol_id from
tbaadm.upr where user_id=?SVAR"
print(lv_r)
lv_s=urhk_dbSelectWithBind(lv_r)
print(BANCS.OUTPARAM.solTenor)
print(BANCS.OUTPARAM.solId)
```

Note:

1. The repository variable *BANCS.INPARAM.BINDVARS* should be used to create a string with the bind values as pipe ("|") separated.
2. The place holders (*?SVAR* or *?NVAR*) in the SQL query are not enclosed with quotes.
3. The order of using place holders should be same as the literals defined in the repository variables.
4. The total count of literals should exactly match with the number of

place holders used in the query.

5. *The repository variable BANCS.INPARAM.BINDVARS can be reinitialized and reused if a script contains more than one query. Once the user-hook is executed, this variable (BANCS.INPARAM.BINDVARS) is no longer required.*
6. *For queries with LIKE condition, the variable value has to be prefixed/suffixed with "%" while creating the string BANCS.INPARAM.BINDVARS itself. In the query, "LIKE" has to be used instead of "=". "%" should not be used with place holders.*
7. *For number type bind variables (?NVAR), the FORMAT\$ function should be used in BANCS.INPARAM.BINDVARS. Otherwise, a variable of numeric data type cannot be concatenated in a repository variable of string data type.*

Advantage

Using bind variables in the customization scripts improves the performance. The SQL queries are not required to be hard-parsed if bind variables are used. This reduces the CPU usage, and shortens the time required for executing the queries.

Writing the SQL queries are even simpler. This would help writing clean reusable queries in customization scripts.

Limitation

In bind variables, pipes ("|") are used as field delimiter. So bind variables cannot be used if the record contains any pipe ("|").

Selection of Fields from Database Using Bind Variables

This userhook allows script writer to select data from database tables. The user specifies the title of each selected field separated by comma before pipe symbol and specifies the complete query after that. It stores the output error code of the query in `BANCS.OUTPARAM.DB_ERRCODE` and message in `BANCS.OUTPARAM.DB_ERRMSG`. In case the query returns more than single row, it gives a fatal error.

Syntax:

```
lv_r = urhk_dbSelectWithBind(lv_s)
```

The variable can be a scratch pad variable (`sv_a`) or a string ("today | SELECT sysdate FROM dual").

Note:

The variable `BANCS.INPARAM.BINDVARS` should be initialized as null before using the user-hook in any static query.

Input Arguments:

SQL Statement

Input Repository Fields:

The inputs should be passed to the SQL query through `BANCS.INPARAM.BINDVARS` with delimiter as "|". In the query the following placeholders are supported:

- ❏SVAR: String bind values (including VARCHAR, CHAR, DATE data types)
- ❏NVAR: Numeric bind values (for NUMBER data type)

Output Repository Fields:

Output not present.

Return Value:

The output fields are stored in BANCS.OUTPARAM.<title field>. An error free condition is represented by 0 value of BANC.OUTPARAM.DB_ERRCODE.

For Example:

```
BANCS.INPARAM.BINDVARS=lv_d
print(BANCS.INPARAM.BINDVARS)
lv_r="solTenor,solId|select user_sol_tenor, sol_id from
tbaadm.upr where user_id=?SVAR"
print(lv_r)
lv_s=urhk_dbSelectWithBind(lv_r)
print(BANCS.OUTPARAM.solTenor)
print(BANCS.OUTPARAM.solId)
```

Note:

- 1.The repository variable *BANCS.INPARAM.BINDVARS* should be used to create a string with the bind values as pipe ("|") separated.
- 2.The place holders (?SVAR or ?NVAR) in the SQL query are not enclosed with quotes.
- 3.The order of using place holders should be same as the literals defined in the repository variables.
- 4.The total count of literals should exactly match with the number of place holders used in the query.
- 5.The repository variable *BANCS.INPARAM.BINDVARS* can be reinitialized and reused if a script contains more than one query. Once the user-hook is executed, this variable (*BANCS.INPARAM.BINDVARS*) is no longer required.
- 6.For queries with LIKE condition, the variable value has to be prefixed/suffixed with "%" while creating the string

BANCS.INPARAM.BINDVARS itself. In the query, "LIKE" has to be used instead of "=". "%" should not be used with place holders.

7. For number type bind variables (?NVAR), the FORMAT\$ function should be used in BANCS.INPARAM.BINDVARS. Otherwise, a variable of numeric data type cannot be concatenated in a repository variable of string data type.

Advantage

Using bind variables in the customization scripts improves the performance. The SQL queries are not required to be hard-parsed if bind variables are used. This reduces the CPU usage, and shortens the time required for executing the queries.

Writing the SQL queries are even simpler. This would help writing clean reusable queries in customization scripts.

Limitation

In bind variables, pipes ("|") are used as field delimiter. So bind variables cannot be used if the record contains any pipe ("|").

©Copyright Infosys Ltd.

Selection of Fields from Database Using Bind Variables for Larger Column Length

This userhook allows script writer to select data from database tables. The user specifies the title of each selected field separated by comma before pipe symbol and specifies the complete query after that. It stores the output error code of the query in `BANCS.OUTPARAM.DB_ERRCODE` and message in `BANCS.OUTPARAM.DB_ERRMSG`. In case the query returns more than single row, it gives a fatal error.

This userhook is same as `urhk_dbSelectWithBind()`, but here the column length can be even larger (4000 characters).

Syntax:

```
lv_r = urhk_dbSelectLongWithBind(lv_s)
```

The variable can be a scratch pad variable (`sv_a`) or a string ("today | SELECT sysdate FROM dual").

Note:

The variable `BANCS.INPARAM.BINDVARS` should be initialized as null before using the user-hook in any static query.

Input Arguments:

SQL Statement

Input Repository Fields:

The inputs should be passed to the SQL query through `BANCS.INPARAM.BINDVARS` with delimiter as "|". In the query the following placeholders are supported:

❏SVAR: String bind values (including VARCHAR, CHAR, DATE

data types)

¶NVAR: Numeric bind values (for NUMBER data type)

Output Repository Fields:

Output not present.

Return Value:

The output fields are stored in BANCS.OUTPUTPARAM.<title field>. An error free condition is represented by 0 value of BANC.OUTPUTPARAM.DB_ERRCODE.

For Example:

```
BANCS.INPARAM.BINDVARS=lv_d
print(BANCS.INPARAM.BINDVARS)
lv_r="solTenor,solId|select    user_sol_tenor,    sol_id    from
tbaadm.upr  where user_id=?SVAR"
print(lv_r)
lv_s=urhk_dbSelectLongWithBind(lv_r)
print(BANCS.OUTPUTPARAM.solTenor)
print(BANCS.OUTPUTPARAM.solId)
```

©Copyright Infosys Ltd.

Using SQL Statement in Script

This userhook allows script writer to perform DML Statements on the database tables. Any updation in the database with this userhook makes an entry in the SDA table.

Syntax:

sv_r = urhk_dbSQL(variable)

The variable can be a scratch pad variable (sv_a) or a string ("update Upr Set User_logged_on_flg = 'N' where session_id = 'xxxx'").

Input Arguments:

SQL Statement

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The output fields are stored in BANCS.OUTPARAM.<title field>. An error free condition is represented by 0 value of BANC.OUTPARAM.DB_ERRCODE.

©Copyright Infosys Ltd.

Using SQL Statement with Bind Variables in Script

This userhook allows script writer to perform DML Statements on the database tables using bind variables.

Syntax:

```
lv_r = urhk_dbSqlWithBind(lv_s)
```

Input Arguments:

SQL Statement

Input Repository Fields:

The inputs should be passed to the SQL query through BANCS.INPARAM.BINDVARS with delimiter as "|". In the query the following placeholders are supported.

❏SVAR: String bind values (including VARCHAR, CHAR, DATE data types)

❏NVAR: Numeric bind values (for NUMBER data type)

Output Repository Fields:

Output not present.

Return Value:

The output fields are stored in BANCS.OUTPARAM.<title field>. An error free condition is represented by 0 value of BANC.OUTPARAM.DB_ERRCODE.

For Example:

```
lv_b='C'
```

lv_c="101"

lv_d="58818B"

BANCS.INPARAM.BINDVARS=lv_b+"|"+lv_c+"|"+lv_d

lv_r="update tbaadm.upr set user_sol_tenor=?SVAR, sol_id=?
SVAR where user_id=?SVAR"

lv_s=urhk_dbSQLWithBind(lv_r)

©Copyright Infosys Ltd.

Get Interpool Amounts for a Financial Year

This userhook fetches Interpool Investment, Interpool Borrowing, Direct Borrowing amounts, Interpool Profit Received, Interpool Profit Paid and Pool Indicative Profit Rate from corresponding tables for a financial year.

Syntax:

```
sv_b = urhk_getAmtsForIOMPINQ ("")
```

Input Arguments:

Pool Code

Currency

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Pool Code	Pool Code
BANCS.INPARAM.currency	currency

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.interpoolInvAmt	interpoolInvAmt
BANCS.OUTPARAM.interpoolBorAmt	interpoolBorAmt
BANCS.OUTPARAM.directBorAmt	directBorAmt
BANCS.OUTPARAM.interpoolProfRcvd	interpoolProfRcvd
BANCS.OUTPARAM.interpoolProfPaid	interpoolProfPaid
BANCS.OUTPARAM.poolIndictProfRate	poolIndictProfRate

Return Value:

interpoolInvAmt, interpoolBorAmt, directBorAmt, interpoolProfRcvd,
interpoolProfPaid, poolIndictProfRate

©Copyright Infosys Ltd.

Get Details of RPC Disbursements

Syntax:

```
sv_a = urhk_B2k_getRPCDisbDetails("DISB_ID")
```

Input Arguments:

■ The input passed is the disb_id.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.Disbld	Disbld
BANCS.OUTPARAM.RpcAcct	RpcAcct
BANCS.OUTPARAM.ECGCAppFlag	ECGCAppFlag
BANCS.OUTPARAM.LastECGCCalcDate	LastECGCCalcDate
BANCS.OUTPARAM.DueDate	DueDate
BANCS.OUTPARAM.ExtendedDueDate	ExtendedDueDate
BANCS.OUTPARAM.DisbursementDate	DisbursementDate
BANCS.OUTPARAM.DisbursementAmount	DisbursementAmount
BANCS.OUTPARAM.Currency	Currency
BANCS.OUTPARAM.OutstandingAmount	OutstandingAmount
BANCS.OUTPARAM.IntCalcUptoDateFlag	IntCalcUptoDateFlag

Return Value:

Return value not available.

For Example:

```
<--start
#trace on
sv_e = urhk_B2k_getRPCDisbDetails("AA105")
print(BANCS.OUTPUTPARAM.DisbId)
print(BANCS.OUTPUTPARAM.RpcAcct)
print(BANCS.OUTPUTPARAM.ECGCAppFlag)
print(BANCS.OUTPUTPARAM.LastECGCCalcDate)
print(BANCS.OUTPUTPARAM.DueDate)
print(BANCS.OUTPUTPARAM.ExtendedDueDate)
print(BANCS.OUTPUTPARAM.DisbursementDate)
print(BANCS.OUTPUTPARAM.DisbursementAmount)
print(BANCS.OUTPUTPARAM.Currency)
print(BANCS.OUTPUTPARAM.OutstandingAmount)
print(BANCS.OUTPUTPARAM.IntCalcUptoDateFlag)
#trace off
end-->
```

©Copyright Infosys Ltd.

Get List of Accounts Based on Instrument Number

Given an instrument, this userhook gives all the accounts for which the instrument number has been issued and are usable (that is, the status for which is not PASSED, DESTROYED or PAID CHEQUE RETURNED). Given a cheque number, this function lists all the accounts for which a cheque having that number has been issued.

Syntax:

The syntax for calling the function is:

```
sv_r = urhk B2k_GetAccountsForInstrument("")
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.InstrumentAlpha	The alpha portion of the cheque number (if passed as "" then for all instrument alpha's) (numeric portion of the cheque number)
BANCS.INPARAM.SchemeType	Match accounts which belong to this scheme type (if passed as "" then records for all schemes are considered) (numeric portion of the cheque number)
BANCS.INPARAM.SolId	Match accounts which belong to this Sol. (if passed as "" then records for all sols are selected) (numeric portion of the cheque number)
BANCS.INPARAM.MinIssueDate	Match cheque books which have been issued beyond this date (format of date in DD-MM-YYYY HH24:MM:SS) (if passed as "" then for records are selected for all dates).

	(numeric portion of the cheque number)
--	--

Output Repository Fields:

Output not present.

Return Value:

Always returns as '0'.

All output variables is populated in the OUTPARAM class of the BANCS repository. The variables are (all STRINGS) :

▲AcctIdList - concatenated string of 16 character account numbers left justified, filled by trailing spaces

For Example:

("ACCOUNT1 ACCOUNT2 ") in case of two accounts

▲AcctIdCount - The number of accounts in the list

For Example:

Get the list of saving bank accounts for all sol's to which 56005678 instrument has been issued after 12th January and it is usable.

BANCS.INPARAM.SolId = ""

BANCS.INPARAM.SchemeType = "SBA"

BANCS.INPARAM.MinIssueDate = "12-01-1999 00:00:00"

BANCS.INPARAM.InstrumentAlpha = ""

BANCS.INPARAM.InstrumentNumber = "56005678"

sv_r = urhk_B2k_GetAccountsForInstrument(sv_s)

if (sv_r == 0) then

```
sv_l = CINT(BANCS.OUTPUTPARAM.AcctIdCount)
if (sv_l == 1) then
    #single Account
    sv_a = rtrim(BANCS.OUTPUTPARAM.AcctIdList)
    print(sv_a)
else
    # Multipl Accounts
    sv_i = 0
    while (sv_i < sv_l)
        sv_d = sv_i * 8
        sv_a = mid$(BANCS.OUTPUTPARAM.AcctIdList, sv_d, 8)
        rtrim(sv_a)
        print(sv_a)
        sv_i = sv_i + 1
    do
    endif
endif
```

©Copyright Infosys Ltd.

Get Past Due Cycles

This userhook retrieves ten past due cycles.

Syntax:

```
sv_a = urhk_getPastDueCycles("A/c Number")
```

Where sv_a is a scratch pad variable to store the return value.

Input Arguments:

The input for this userhook is the Account ID.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.Cycle1	.
BANCS.OUTPARAM.Cycle2	.
BANCS.OUTPARAM.Cycle3	.
BANCS.OUTPARAM.Cycle4	.
BANCS.OUTPARAM.Cycle5	.
BANCS.OUTPARAM.Cycle6	.
BANCS.OUTPARAM.Cycle7	.
BANCS.OUTPARAM.Cycle8	.
BANCS.OUTPARAM.Cycle9	.
BANCS.OUTPARAM.Cycle10	.
BANCS.OUTPARAM.Cycle11	.
BANCS.OUTPARAM.Cycle12	.

BANCS.OUTPARAM.Cycle13	.
BANCS.OUTPARAM.Cycle14	.
BANCS.OUTPARAM.Cycle15	.
BANCS.OUTPARAM.ErrMsg	.

Return Value:

Return value not available.

For Example:

```
#call the user hook for getting Past Due Cycles
```

```
<--start
```

```
trace on
```

```
sv_a = urhk_getPastDueCycles("LA1")
```

```
print(sv_a)
```

```
if (sv_a == 0) then
```

```
print(BANCS.OUTPARAM.Cycle1)
```

```
print(BANCS.OUTPARAM.Cycle2)
```

```
print(BANCS.OUTPARAM.Cycle3)
```

```
print(BANCS.OUTPARAM.Cycle4)
```

```
print(BANCS.OUTPARAM.Cycle5)
```

```
print(BANCS.OUTPARAM.Cycle6)
```

```
print(BANCS.OUTPARAM.Cycle7)
```

```
print(BANCS.OUTPARAM.Cycle8)
```

```
print(BANCS.OUTPARAM.Cycle9)
```

```
print(BANCS.OUTPARAM.Cycle10)
```

```
else
```

```
print(BANCS.OUTPARAM.ErrMsg)
```

```
endif  
exitscript  
end-->
```

©Copyright Infosys Ltd.

Get Pool Available Amount

Syntax:

```
sv_a = urhk_getPoolEffAvailAmt(sv_c).
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.EffAvailableAmt	EffAvailableAmt
BANCS.OUTPARAM.FullEffAvailableAmt	FullEffAvailableAmt
BANCS.OUTPARAM.FxEffAvailableAmt	FxEffAvailableAmt
BANCS.OUTPARAM.FxFullEffAvailableAmt	FxFullEffAvailableAmt
BANCS.OUTPARAM.PoolEffAvailableAmt	PoolEffAvailableAmt
BANCS.OUTPARAM.errorMesg	errorMesg

Return Value:

The output is returned in the fields of BANCS repository and OUTPARAM class.

For Example:

```
< --start
```

```
BANCS.INPARAM.InBuff ="BMLAA"
```

```
sv_c = BANCS.INPARAM.InBuff
```



```
sv_a = urhk_getPoolEffAvailAmt(sv_c)
if (sv_a == 1) then
sv_b = BANCS.OUTPUTPARAM.errorMesg
print(sv_b)
else
sv_b = BANCS.OUTPUTPARAM.EffAvailableAmt
print(sv_b)
sv_c = BANCS.OUTPUTPARAM.FullEffAvailableAmt
print(sv_c)
sv_d = BANCS.OUTPUTPARAM.FxEffAvailableAmt
print(sv_d)
sv_e = BANCS.OUTPUTPARAM.FxFullEffAvailableAmt
print(sv_e)
sv_f = BANCS.OUTPUTPARAM.PoolEffAvailableAmt
print(sv_f)
endif
exitscript
end -->
```

©Copyright Infosys Ltd.

Get Tranche Outstanding

This userhook provides the outstanding of a given tranche ID.

Syntax:

```
sv_b = urhk_getTrancheOutstanding (<tranche id>)
```

Input Arguments:

Tranche ID

The input is a mandatory parameter.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Tranche Id	Tranche Id

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.tranche_outstanding	tranche_outstanding

Return Value:

tranche_outstanding - is the output field.

©Copyright Infosys Ltd.

Get User Additional Details

This function gets user additional detail for an entity say bill or forward contract.

Syntax:

```
sv_r=urhk_B2k_GetUserAdditionalDetails("")
```

Input Arguments:

The inputs are Module ID and Module Key through BANCS.INPARAM class.

Valid values of Module ID are BIL, FEX, BG, FWC, and DC.

Module Key is a field to identify unique entity

For Example:

For a bill module ID can be Sol ID and Bill ID concatenated with a slash '/' character.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

This function returns the user additional detail into AdditionalDetails field of OUTPARAM class of BANCS repository. Field can comprise of many values concatenated by pipe characters.

For Example:

Let us say that the bank has captured two remarks for each bill. The script in BM menu option through which the bank can get the value of these two fields is:

```
BANCS.INPARAM.ModuleId='BIL'
BANCS.INPARAM.ModuleKey='SOL1' + '/' + 'BILL125'
sv_r=urhk_B2k_GetUserAdditionalDetails("")
if (sv_r == 0) then
sv_a=BANCS.OUTPARAM.AdditionalDetails
sv_l = getposition(sv_a, "|")
BANCS.FRMVALUE.Remarks1 = mid$(sv_a, 0, sv_l-1)
sv_b = strlen(sv_a)
sv_a = mid$(sv_a, sv_l, sv_b - sv_l)
BANCS.FRMVALUE.Remarks2=sv_a
else
BANCS.FRMVALUE.Remarks1 = ""
BANCS.FRMVALUE.Remarks2 = ""
endif
```

See Lodge Bills in the Inland Bills module for BM menu option details.

©Copyright Infosys Ltd.

Getting Scheme Details into Bancs Repository

This function returns scheme details information in the FINACLE database. It can be used in the context of any Script Event or Workflow. This function returns some attributes of the given scheme in the repository fields if the scheme is found. If the scheme does not exist, then a fatal error occurs. Hence the scheme code passed must be a valid scheme code.

Syntax:

sv_a = urhk_getSchemeDetailsInRepository (Variable)

The variable can be a scratch pad variable (sv_b) or a string("SBGEN").

Input Arguments:

The input to this function is a valid (existing) SCHEME CODE.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.SchemeDescription	schm_desc in GSP Table
BANCS.OUTPARAM.InterestPaidPlaceHolder	int_paid_bacid in GSP table
BANCS.OUTPARAM.InterestCollPlaceHolder	int_coll_bacid in GSP table
BANCS.OUTPARAM.ServiceChargePlaceHolder	serv_chrg_bacid in GSP table
BANCS.OUTPARAM.SchmChequeAllowed	chq_alwd_flg in GSP table
BANCS.OUTPARAM.SchemeSupervisorUserId	schm_supr_user_id in GSP table
BANCS.OUTPARAM.NewAccountDuration	new_acct_duration in GSP table

BANCS.OUTPARAM.DormantAccountCriteria	dorm_acct_criteria in GSP table
BANCS.OUTPARAM.MinPostingWorkClass	min_posting_workclass in GSP table
BANCS.OUTPARAM.SchemeType	schm_type in GSP table
BANCS.OUTPARAM.InterestPaidFlg	int_paid_flg in GSP table
BANCS.OUTPARAM. InterestCollectedFlg	int_coll_flg in GSP table
BANCS.OUTPARAM.StaffSchemeFlg	staff_schm_flg in GSP table
BANCS.OUTPARAM. NRE_SchemeFlg	nre_schm_flg in GSP table
BANCS.OUTPARAM.FD_CreditPlaceHolder	fd_cr_bacid in GSP table
BANCS.OUTPARAM.FD_DebitPlaceHolder	fd_dr_bacid in GSP table
BANCS.OUTPARAM.StopCrDrInd	stp_cr_dr_ind in GSP table
BANCS.OUTPARAM. IsFCNR	fcnr_flg in GSP table
BANCS.OUTPARAM.Int_P_and_L_PlaceHolder	int_pandl_bacid in GSP table
BANCS.OUTPARAM.ParkingPlaceHolder	Parking_bacid in GSP table
BANCS.OUTPARAM.AdvanceIntPlaceHolder	Advance_int_bacid in GSP table
BANCS.OUTPARAM.TDS_ParkingPlaceHolder	tds_parking_bacid in GSP table
BANCS.OUTPARAM.Penal_P_and_L_PlaceHolder	penal_pandl_bacid in GSP table
BANCS.OUTPARAM.DefaultClearingTranCode	dflt_clg_tran_code in GSP table
BANCS.OUTPARAM. DefaultInstrumentType	dflt_inst_type in GSP table
BANCS.OUTPARAM.Int_P_and_L_CrPlaceHolder	int_pandl_bacid_cr in GSP table
BANCS.OUTPARAM.Int_P_and_L_DrPlaceHolder	int_pandl_bacid_dr in GSP table
BANCS.OUTPARAM. CombStmtSchmLevelMess1	cs_schm_mesg1 in GSP table
BANCS.OUTPARAM.CombStmtSchmLevelMess2	cs_schm_mesg2 in GSP table
BANCS.OUTPARAM.PSFreqType	ps_freq_type in GSP table
BANCS.OUTPARAM.PSFreqWeekNum	ps_freq_week_num in GSP table
BANCS.OUTPARAM.PSFreqWeekDay	ps_freq_week_day in GSP table
BANCS.OUTPARAM.PSFreqStartDate	ps_freq_start_dd in GSP table
BANCS.OUTPARAM.PSFreqHolStat	ps_freq_hldy_stat in GSP table

Return Value:

Return value is always '0'. If scheme not found then Fatal Error occurs.

This function returns the following Scheme details into OUTPARAM class of BANCS repository if the scheme exists in the data center database.

For Example:

Let us say that banker wants to change a certain scheme which is FCNR to change later to Non FCNR so the FCNR flg field must be unprotected and if it is Non FCNR the field must be protected.

```
<--start
```

```
sv_a = BANCS.INPUT. SchmCode
```

```
# -----  
---
```

```
# sv_a (SchmCode) is the input to user hook  
getSchemeDetailsInRepository
```

```
# Call user hook getSchemeDetailsInRepository to put the  
Scheme details
```

```
# into BANCS repository.
```

```
# -----  
---
```

```
sv_b = urhk_getSchemeDetailsInRepository (sv_a)
```

```
# FCNR flag is "Y" for fcnr type of schemes.
```

```
# -----  
---
```

```
# If the FCNR flag of scheme code class is equal to "Y"
```

```
# then protect dispblk1.fcnr_flg field.
```

```
# -----  
---
```

```
if (BANCS.OUTPARAM. IsFCNR == "Y") then
```

```
sv_z = "dispblk1.fcnr_flg|U"  
sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)  
else  
sv_z = "dispblk1. fcnr_flg|P"  
sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)  
endif  
exitscript  
end-->
```

Note:

Refer to urhk_TBAF_ChangeFieldAttrib given separately for its usage.

©Copyright Infosys Ltd.

Fetch the Event Type Description

getEventType:

This user hook will fetch the event type description. Event type is passed as input and event description is the output.

Syntax:

```
sv_b = urhk_getEventType(sv_a)
```

Input:

BANCS.INPPARAM.eventType

Output:

BANCS.OUTPARAM.eventDesc

For Example:

NA

©Copyright Infosys Ltd.

Get Conversion Rate

This userhook is built to accept the customer ID as the input parameter to the userhook . If the customer ID is specified then the Preferential Exchange Rate will be derived , else the Card Exchange Rate is derived .

Syntax:

```
sv_b = urhk_GetConversionRate ("")
```

Input Arguments:

● Cust ID (The input is not a mandatory parameter)

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.CustId	CustId

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.conv_rate	conv_rate

Return Value:

conv_rate (This is the customer preferential exchange rate).

©Copyright Infosys Ltd.

Get Tax Rate

This userhook is built to accept the customer ID as the input parameter to the userhook . If the customer ID is specified then the Preferential Exchange Rate is derived, else the Card Exchange Rate is derived.

Syntax:

```
sv_b = urhk_GetTaxRate (“”)
```

Input Arguments:

Cust ID

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.CustId	CustId

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.taxRate	taxRate
BANCS.OUTPARAM.taxRateCode	taxRateCode

Return Value:

taxRate, taxRateCode

Get Destination Bank ID

This user hook is used to derive the destination bank when receiver and sender bank details are provided.

Syntax:

```
sv_r = urhk_getDestinationBankId("")
```

Input:

- Receiver Bank Code
- Receiver Branch Code
- Sender Bank Code
- Branch Code

Input Repository Fields:

The following input fields are available for customizing this user hook

Input Field	Description
BANCS.INPARAM.recvBankCode	Receiver's Bank Code
BANCS.INPARAM.recvBranchCode	Receiver's Branch Code
BANCS.INPARAM.sendrBankCode	Sender's Bank Code
BANCS.INPARAM.sendrBranchCode	Sender's Branch Code

Output Repository Fields:

The following output fields are available for customizing this user hook

Output Field	Description
	Success or Failure depending on whether

BANCS.OUTPARAM.successOrFailure	userhook is executed successfully or not
BANCS.OUTPARAM.otherBankId	Destination Bank ID

Output:

Destination Bank ID

For Example:

```
<--start
  if (PAYSYS.ACH.InterEntityFlg == "Y") then
    BANCS.INPARAM.recvBankCode=rtrim(PAYSYS.ACH.BankCode)
    BANCS.INPARAM.recvBranchCode=rtrim(PAYSYS.ACH.BranchCod
    BANCS.INPARAM.sendrBankCode=BANCS.STDIN.myBankCode
    BANCS.INPARAM.sendrBranchCode=BANCS.STDIN.myBrCode
    sv_f=urhk_getDestinationBankId("")
    if ( BANCS.OUTPARAM.successOrFailure == "S") then
      BANCS.OUTPUT.SenderBankId=BANCS.OUTPARAM.otherBa
      PAYSYS.ACH.SenderBankId=BANCS.OUTPARAM.otherBankI
      BANCS.OUTPUT.ReceiverBankId=""
    endif
  endif
endif
end-->
```

Get Fee Transaction Details during Commercial Loan Payment

This userhook returns fee transaction details like Charge credit foracid, event ID, event type, collection amount etc. based on loan account ID. The script hook is used during loan payment transaction. If the inputs are valid, the values are populated in the Finacle repository.

Syntax

```
sv_a = urhk_getCLPayFeeTranDetails("")
```

Input

- Valid Loan Account Number

Input Repositories

Field Name	Description
BANCS.INPARAM.LoanAccount	Valid Loan Account number in GAM table.

Output Repository Fields

Field Name	Description
BANCS.OUTPARAM.ChargeCollAcct	Charge Collection account, that is, credit account for fee collection
BANCS.OUTPARAM.EventId	Valid Event ID
BANCS.OUTPARAM.EventType	Valid Event type
BANCS.OUTPARAM.CompB2kId	CompB2kId which contains acid, Charge type and Serial number
BANCS.OUTPARAM.FeeCalcAmt	Fee System Calculated amount

BANCS.OUTPARAM.FeeCollectionAmt	Fee Current Collection amount
BANCS.OUTPARAM.FeeTranParticular	Fee Tran Particulars
BANCS.OUTPARAM.FeeRateCode	Valid Rate code
BANCS.OUTPARAM.FeeRate	Valid Rate for the rate code

Output

If the input values are invalid then the return value is 1

©Copyright Infosys Ltd.

Get Loan Scheme Details Repository

This user hook returns scheme details based on scheme code passed. The script hook is used during reversal of loan payment transaction.

Syntax:

```
sv_a = urhk_getLoanSchmDetailsInRepository("")
```

Input:

Loan Account ID

Scheme Code

Currency Code

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.LoanAcctId	Valid Loan Account Number in GAM table.
BANCS.INPARAM.schmCode	Valid Scheme code in LSP table.
BANCS.INPARAM.ToCurrency	Valid currency code in LSP table.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.DicgcFeeFlowId	DICGC fee flow ID
BANCS.OUTPARAM.DefltCollFlowId	Default collection flow ID
BANCS.OUTPARAM.IdmFlowId	Interest demand flow ID
BANCS.OUTPARAM.PidmdFlowId	Penal interest demand flow ID
BANCS.OUTPARAM.DfltEiFlowId	Default Equated Instalment flow ID
BANCS.OUTPARAM.DfltPrincipalFlowId	Default Principal demand flow ID

BANCS.OUTPUTPARAM.DfltDisbmtFlowId Default disbursement flow ID

Output:

Returns 1 if the input values are invalid.

For Example:

```
BANCS.INPUTPARAM.schmCode = BANCS.OUTPUTPARAM.schmCode
                                BANCS.INPUTPARAM.ToCurrency =
BANCS.OUTPUTPARAM.acctCrncyCode
                                BANCS.INPUTPARAM.LoanAcctId =
MTT.LAAddnlDetails.LoanAcctId
    sv_l = urhk_getLoanSchmDetailsInRepository("")
    IF (sv_l != 0) THEN
        MSG0312071="Could not get Loan Account
scheme details for "
        sv_e=urhk_getMSGCodeErrDesc("MSG0312071")
        if(sv_e == 0) then
                                MTT.Error.Description =
BANCS.OUTPUTPARAM.errDesc + " [" +
MTT.LAAddnlDetails.LoanAcctId + "]"
                                endif
        sv_r = urhk_MTTTS_ReportErrorCondition("")
        BANCS.OUTPUT.successOrFailure="F"
    ENDIF
    MTT.LAAddnlDetails.DemandFlowId=BANCS.OUTPUTPARAM.De
```

Get Fee Transaction Details during Loan Payment

This user hook returns fee transaction details like Charge credit account, event ID, event type, collection amount etc. based on loan account ID and reversal transaction ID. The script hook is during reversal of loan payment transaction.

Syntax:

```
sv_a = urhk_getLaPaymentFeeTranDetails("")
```

Input:

Valid Loan Account Number

Reversal Transaction ID

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.LoanAccount	Valid Loan Account Number in GAM table.
BANCS.INPARAM.RevTranId	Reversal tran ID in CXL table.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.ChargeCollAcct	Charge Collection account, that is, credit account for fee collection specified in PTTM.
BANCS.OUTPARAM.EventId	Valid Event ID.
BANCS.OUTPARAM.EventType	Valid Event type.
BANCS.OUTPARAM.FeeCalcAmt	Fee System Calculated amount.
BANCS.OUTPARAM.FeeCollectionAmt	Fee Current Collection amount.
BANCS.OUTPARAM.FeeRateCode	Valid Rate code.
BANCS.OUTPARAM.FeeRate	Valid Rate for the rate code.

Output:

Returns 1 if the input values are invalid.

©Copyright Infosys Ltd.

Get the Bank Identifier for a given Paysys ID/Bank Code/Branch Code Combination

This userhook returns the Bank Identifier and other details for a Paysys ID/Bank Code/Branch Code combination. This userhook can be called from any script event. Returns failure if either Paysys ID/Bank Code/Branch Code is Null Or if there is no Bank Identifier for the given input values.

If inputs are valid, then the Bank Identifier is populated into Finacle repository.

Syntax:

```
sv_a = urhk_GetBICForBankBranch("")
```

Input:

- Paysys ID
- Bank Code
- Branch Code

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.bankCode	Valid Bank Code.
BANCS.INPARAM.branchCode	Valid Branch Code.
BANCS.INPARAM.paysysID	Valid Paysys ID.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.bankIdentifier	Bank Identifier for the above input Combination

BANCS.OUTPUTPARAM.bkeArrangement	Flag to identify if the Bilateral Key Exchange arrangement exists.
BANCS.OUTPUTPARAM.includeCharge	Flag to indicate whether the charges are to be sent along with the remittance amount.
BANCS.OUTPUTPARAM.indirectParticipant	Flag to indicate whether the bank/branch is a direct participant to this paysys or not.
BANCS.OUTPUTPARAM.repBankCode	If indirect participant, this field will have the representing bank code for the paysys ID.
BANCS.OUTPUTPARAM.repBranchCode	If indirect participant, this field will have the representing branch code for the paysys ID.
BANCS.OUTPUTPARAM.repBankIdentifier	If indirect participant, this field will have the representing Bank Identifier for the paysys ID.

Output:

Bank Identifier for the entered Paysys ID/Bank Code/Branch Code combination is populated into the repository.

For Example:

```

<--start
# Populate Paysys ID, Branch Code, Bank code
BANCS.INPUTPARAM.paysysID = BANCS.INPUT.paysysId
BANCS.INPUTPARAM.bankCode = BANCS.INPUT.bankCode
BANCS.INPUTPARAM.branchCode = BANCS.INPUT.branchCode
sv_a = urhk_GetBICForBankBranch ("" )
if ( sv_a == 1 ) then
  exitscript
endif
sv_i = BANCS.OUTPUTPARAM.bankIdentifier
end - ->

```

Get the Bank Code/Branch Code for the given Paysys ID/Bank Identifier Combination

This userhook returns the Bank Code/Branch Code values for the given Paysys ID and Bank Identifier combination. This userhook can be called from any script event. Returns failure if either Paysys ID/ Bank Identifier is Null Or if there is no Bank/Branch Code for the given input values.

If inputs are valid, then the Bank Code and Branch are populated into Finacle repository.

Syntax:

```
sv_a = urhk_GetBRBICDtls ("" )
```

Input:

■ Paysys ID

■ Bank Identifier

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.bankIdentifierCode	Valid Bank Identifier
BANCS.INPARAM.paysysID	Valid Paysys ID

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.bankCode	Bank Code for the above input Combination.
BANCS.OUTPARAM.branchCode	Branch Code for the above input Combination.
BANCS.OUTPARAM.bkeArrangement	Flag to identify if the Bilateral Key Exchange arrangement exists.

BANCS.OUTPUTPARAM.includeCharge	Flag to indicate whether the charges are to be sent along with the remittance amount.
BANCS.OUTPUTPARAM.indirectParticipant	Flag to indicate whether the bank/branch is a direct participant to this paysys or not.
BANCS.OUTPUTPARAM.repBankCode	If indirect participant, this field will have the representing bank code for the paysys ID.
BANCS.OUTPUTPARAM.repBranchCode	If indirect participant, this field will have the representing branch code for the paysys ID.
BANCS.OUTPUTPARAM.numOfRec	Number of records present in DB for the given Paysys ID and Bank Identifier values

Output:

Bank Code and Branch Code for the entered Paysys ID and Bank Identifier combination are populated into the repository.

For Example:

```

<--start
# Populate Paysys ID, Bank Identifier
BANCS.INPUTPARAM.paysysID = BANCS.INPUT.paysysId
BANCS.INPUTPARAM.bankIdentifierCode =
BANCS.INPUT.bankIdentifierCode
sv_a = urhk_GetBRBICDtls ("")
if ( sv_a == 1 ) then
  exitscript
endif
SV_I = BANCS.OUTPUTPARAM.BANKCODE
SV_J = BANCS.OUTPUTPARAM.BRANCHCODE
end - ->

```

Get the Nostro and the Vostro Account Details for the Bank Code/Branch Code/Currency Code Combination

This userhook returns the Nostro and Vostro Account details for the given Bank/Branch/Currency code combination. This userhook can be called from any script event . Returns failure if either Bank/Branch/Currency code is Null

If inputs are valid, then the Nostro and Vostro Account Details are populated into Finacle repository.

Syntax:

```
sv_a = urhk_GetCorrBankAcctDetails ("" )
```

Input:

Bank Code

Branch Code

Currency Code

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.bankCode	Valid Bank Code.
BANCS.INPARAM.branchCode	Valid Branch Code.
BANCS.INPARAM.crncyCode	Valid Currency Code.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.nostroAcctID	Nostro Account ID for the above input

Combination.

BANCS.OUTPUTPARAM.nostroMirrorAcctPH	Nostro Mirror Account ID for the above input Combination.
BANCS.OUTPUTPARAM.nostroDefaultFlag	Nostro Default Flag for the above input Combination.
BANCS.OUTPUTPARAM.nostroOperationFlag	Nostro Operational Flag for the above input Combination.
BANCS.OUTPUTPARAM.vostroForacid	Vostro Account ID for the above input Combination.
BANCS.OUTPUTPARAM.vostroDefaultFlg	Vostro Default Flag for the above input Combination.

Output:

Nostro and Vostro Account Details for the given input values are populated into the repository.

For Example:

```
<--start
# Populate Bank,Branch and Currency Code values
BANCS.INPUTPARAM.bankCode = BANCS.INPUT.bankCode
BANCS.INPUTPARAM.branchCode = BANCS.INPUT.branchCode
BANCS.INPUTPARAM.crncyCode= BANCS.INPUT.crncyCode
sv_a = urhk_GetCorrBankAcctDetails ("")
if ( sv_a == 1 ) then
  exitscript
endif
sv_i = BANCS.OUTPUTPARAM.nostroAcctID
sv_j = BANCS.OUTPUTPARAM.vostroForacid
end - ->
```

©Copyright Infosys Ltd.

Finacle Copyright

General Scripting Userhooks

This section discusses:

[Call Batch Executable from Script](#)

[Check for Pipe Character in the Input String](#)

[Execute Service for Template](#)

[Execute Template SRV](#)

[Execute a Service Function](#)

[FXBILL_GETTRAN](#)

[FXBILL_INITTRAN](#)

[Generate Past Due Report](#)

[Get Amount Calculated Based on Setup in Amount Code Table](#)

[Accept the Amount String](#)

[Get Cash Account Foracid](#)

[Get Interest between Dates](#)

[Get Module Information](#)

[Get Orchestration Transaction ID Sequence Number](#)

[Get Past Due Display or Print String](#)

[Get Syn Agent Upfront Fee](#)

[Get Total Profit and Loss Amount for Pool](#)

[Get Total Sanction Limit of All Accounts Linked to DSA](#)

[IBR Userhooks](#)

[Initiate Audit for Logical Unit of Work](#)

[Interest Rate for an Account as on Given Date in Bancs Repository](#)

[Next Number Generation](#)

[Populate Collateral Information](#)

[Populate Reason Code for ACH](#)

[Put Environment Variable](#)

[Print Repository Fields](#)

[Raise Exception for Transaction Created through HTM](#)

[Raising Exceptions](#)

[Round Off Amount](#)

[Save User Additional Details](#)

[Update BJM](#)

[User Defined TOD](#)

[Userhook - GetFldValueWithSrvIdAndFldName](#)

[Userhook - PutFldValueWithSrvIdAndFldName](#)

[Check Pending Verification for Security](#)

[Execute Web Based External Application in Web Instance for Finacle](#)

[Populate Pool Details for a Specified Cycle](#)

[Print Repository](#)

[Mask Input](#)

[Enabling Debug Logs](#)

[Discretionary Limit Set for the User](#)

[Get additional details for HLPSP](#)

[Amount in LAKH/MILLION format using commas](#)

[Get Limit B2kId](#)

[Get the Cust ID](#)

[Get B2k ID for the Given Event ID](#)

[Userhook for the mrbx Report Function](#)

[Get Error Description for the Error Code](#)

[Put Custom Exception](#)

[Put Exceptions into Repository](#)

©Copyright Infosys Ltd.

Populate Pool Details for a Specified Cycle

This user hook is used to fetch the data for Pool Provision Amt, Pool Suspended Amt, Pool Write Off Amt and Profit from Asset Amt from AHT table for a specified cycle.

Syntax:

```
sv_q = urhk_PopulateAmtField("")
```

Input Arguments:

NA

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.profitDistFromDate	Profit Distribution From Date
BANCS.INPARAM.profitDistToDate	Profit Distribution To Date
BANCS.INPARAM. poolCode	Pool Code
BANCS.INPARAM.crncyCode	Currency Code

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.asset_Profit_Amt	Asset Profit Amount
BANCS.OUTPARAM.pool_Suspend_Amt	Pool Suspend Amount
BANCS.OUTPARAM.pool_WriteOff_Amt	Pool WriteOff Amount
BANCS.OUTPARAM.pool_Prov_Amt	Pool Provision Amount

Return Value:

Example:

```
BANCS.INPARAM.profitDistFromDate = sv_a  
BANCS.INPARAM.profitDistToDate = sv_b  
BANCS.INPARAM.poolCode = sv_c  
BANCS.INPARAM.crncyCode = sv_d  
sv_q = urhk_PopulateAmtField("")  
sv_e = BANCS.OUTPARAM.asset_Profit_Amt  
sv_f = BANCS.OUTPARAM.pool_Suspend_Amt  
sv_i = BANCS.OUTPARAM.pool_WriteOff_Amt  
sv_j = BANCS.OUTPARAM.pool_Prov_Amt
```

©Copyright Infosys Ltd.

Call Batch Executable from Script

This userhook calls batch exe (executable) or COM script whose name is passed to the script hook.

Syntax:

```
sv_c = "batch_exe_name"  
sv_a = urhk_TBAF_CallBatchExe(sv_c)
```

Input Arguments:

Input is a character string.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The userhook returns the value '0' or '1' depending on the success or failure of the userhook to execute the exe. If the executable is not found then it returns the value '1'; otherwise, it returns the value '0'.

For Example:

```
<--start  
#trace on  
sv_c = "batch_exe_name"  
sv_m = urhk_TBAF_CallBatchExe(sv_c)
```



```
# return value can be checked in sv_m  
print(sv_m)  
exitscript  
end-->
```

©Copyright Infosys Ltd.

Check for Pipe Character in the Input String

This userhook searches for pipe char ("|") in the input string. If the input string contains the pipe character, the userhook returns failure.

Syntax:

```
sv_a = urhk_B2k_checkForPipeChar ("inputstring")
```

Input Arguments:

Userhook accepts "inputstring" as the input. The maximum size of the string must be 1024 characters only.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

This function returns '0' if there are no pipe characters in the input string. otherwise it returns non zero value.

©Copyright Infosys Ltd.

Execute Service for Template

This userhook is used to execute the service that creates actual transaction for that template type (for example ONS menu name HPORDM), given as input for the given template ID and template type combination.

Syntax:

```
sv_b = urhk_ExecSrvcForTemplate("<Template_id>| <Tempalte Type>")
```

Input Arguments:

- Template ID
- Template Type

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Template Id	Template Id
BANCS.INPARAM.Template Type	Template Type

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.transaction_id	transaction_id

Return value:

transaction_id

For Example:

```
<--start
trace on
sv_a = urhk_SetUrhkInp("pordHeaderDtls.totalAmt|0|USD")
sv_a = urhk_SetUrhkInp("pordHeaderDtls.drExecutionDate|26-06-2002 00:00:00")
sv_a = urhk_SetUrhkInp("pordHeaderDtls.drAcct.foracid|KGK-SB1")
sv_a = urhk_SetUrhkInp("pordHeaderDtls.drAcct.crncyCode|USD")
sv_a = urhk_SetUrhkInp("pordHeaderDtls.drAcct.solId|105")
sv_a = urhk_SetUrhkInp("pordHeaderDtls.drAcct.acctName|SINGH")
sv_a = urhk_SetUrhkInp("pordHeaderDtls.drAcct.acctShortName|NEELA")
sv_a = urhk_SetUrhkInp("pordHeaderDtls.exchRate|1.0000")
sv_a = urhk_SetUrhkInp("pordHeaderDtls.reqExecDate|29-06-2002 00:00:00")
sv_a = urhk_SetUrhkInp("pordPaymentDtls.crValueDate|26-06-2003 00:00:00")
sv_a = urhk_SetUrhkInp("pordPaymentDtls.remitAmt|10.00|USD")
sv_a = urhk_SetUrhkInp("pordPaymentDtls.remitCrncy|USD")
sv_a = urhk_SetUrhkInp("pordPaymentDtls.crExecutionDate|26-06-2002 00:00:00")
sv_a = urhk_SetUrhkInp("pordPaymentDtls.awiBankBranch.brCode|111")
sv_a = urhk_SetUrhkInp("pordPaymentDtls.awiBankBranch.bankCode|001")
```

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.awiBankBranch.brName|BANK
OF SING")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.awiBic|SWIFUS01111")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.benefPartyBic|SWIFUS01111")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.ourCorrespBankBranch.brCode|101")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.ourCorrespBankBranch.bankCode|101")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.ourCorrespBankBranch.brName|C
101")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.ourCorrespBic|SWISUSBS101")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.recvrBankBranch.brCode|101")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.recvrBankBranch.bankCode|DBS')

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.recvrBic|SWISUSBS101")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.routedPaysysId.refCode|SWIFT")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.routedPaysysId.refDesc|SWIFT")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.outwardMappingEqn|RR|OC|OC|C

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.instructedAmt|10.00|USD")

sv_a =
urhk_SetUrhkInp("pordPaymentDtls.instructedCrncy|USD")

sv_a =

```

urhk_SetUrhkInp("pordPaymentDtls.benefPartyAddrTypeInd|B")
sv_a =
urhk_SetUrhkInp("pordPaymentDtls.benefPartyBankBranch.brCode|11")
sv_a =
urhk_SetUrhkInp("pordPaymentDtls.benefPartyBankBranch.bankCode")
sv_a =
urhk_SetUrhkInp("pordPaymentDtls.benefPartyBankBranch.brName|B
OF SING")
sv_a =
urhk_SetUrhkInp("pordPaymentDtls.awiAddrTypeInd|B")
sv_a =
urhk_SetUrhkInp("pordPaymentDtls.chrgDrAcct.foracid|KKGK-
SB1")
sv_a =
urhk_SetUrhkInp("pordPaymentDtls.chrgDrAcct.crncyCode|USD")
sv_a =
urhk_SetUrhkInp("pordPaymentDtls.chrgDrAcct.solId|105")
sv_a =
urhk_SetUrhkInp("pordPaymentDtls.chrgDrAcct.acctName|SINGH")
sv_a =
urhk_SetUrhkInp("pordPaymentDtls.chrgDrAcct.acctShortName|NEEL")
sv_a = urhk_SetUrhkInp("mappedPODetails.serialCoverFlg|S")
sv_a =
urhk_SetUrhkInp("mappedPODetails.outwardMappingEqn|RR|OC|OC|")
sv_a =
urhk_SetUrhkInp("mappedPODetails.mappedOCBankBranch.brCode|1")
sv_a =
urhk_SetUrhkInp("mappedPODetails.mappedOCBankBranch.bankCode")
sv_a =
urhk_SetUrhkInp("mappedPODetails.mappedOCBic|SWISUSBS101")
sv_a =
urhk_SetUrhkInp("mappedPODetails.mappedRecvrBankBranch.brCode")

```

```

sv_a =
urhk_SetUrhkInp("mappedPODetails.mappedRecvrBankBranch.bankC

sv_a =
urhk_SetUrhkInp("mappedPODetails.mappedRecvrBic|SWISUSBS101'

sv_a =
urhk_SetUrhkInp("mappedPODetails.mappedAwiBankBranch.brCode|1

sv_a =
urhk_SetUrhkInp("mappedPODetails.mappedAwiBankBranch.bankCod

sv_a =
urhk_SetUrhkInp("mappedPODetails.mappedAwiBic|SWIFUS01111")

sv_a = urhk_SetUrhkInp("calcCreditSideChargesFlg|N")

sv_a =
urhk_SetUrhkOut("pordHeaderDtls.pymtRefNum|srlnum")

sv_c=urhk_ExecSrvcForTemplate("HPORDM|POUHK2")

if (sv_c != 0) then
#{
sv_d = urhk_SRV_SetErr("Error occured in processing")
print(sv_d)
#}
endif
#trace off
end-->

```

Execute Template SRV

The template related services from Finacle script to invoke are:

- 6RV_ModifyGenTemplate
- 6RV_VerifyGenTemplate
- 6RV_CancelGenTemplate
- 6RV_DelGenTemplate
- 6RV_UnDelGenTemplate
- 6RV_FetchGenTemplateForVerify
- 6RV_FetchGenTemplateForCancel
- 6RV_FetchGenTemplateForInquiry
- 6RV_FetchGenTemplateForModify
- 6RV_FetchGenTemplateForDelete
- 6RV_FetchGenTemplateForUnDelete
- 6RV_FetchGenTemplateForInstantiation

Syntax:

```
sv_a = urhk_ExecTemplateSrv("<template id>|<entity type>|  
<message name>|<service name>|same_user_verify =  
Y|retain_all_output = Y")
```

Input Arguments:

Template ID

Entity type

Message name

Service name

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

No output.

For Example:

Refer to fetch_template.sscr and verify_template.sscr in sample scripts (samscripts) folder.

©Copyright Infosys Ltd.

Execute a Service Function

This user hook is created to execute a service function. This function is like the ExecSrv function but does not commit the changes done by the SRV call. It leaves the commit discretion to the script hook caller. The rollback also will only rollback the changes done by this function and not rollback the previous changes before this user hook is called. This function expects the ONS_Init call to have happened before this userhook is called.

Syntax:

```
sv_a = urhk_ExecSRVNoCommit("<<SRVName>>")
```

Input

When SRV for Post is used , the input is :- SRV name

When SRV for Add is used , the inputs are :- SRV name, retain_all_output flag(Y/N)

When SRV for Fetch is called , the inputs are:- SRV name, same_user_verify flag(Y/N), retain_all_output flag(Y/N)

Output

This user hook will return 0 on successful completion and 1 on failure. The error message will be stored in BANCS.OUTPARAM. retCode

For Example:

```
<--start
trace on
sv_a = urhk_CopyOutToIn("")
sv_s=urhk_ExecSrvNoCommit("SRV_AddCashDepTran|retain_all_outp
```

```
sv_a = urhk_CopyOutToIn("")
sv_a=
  urhk_ExecSrvNoCommit("SRV_FetchCashDepTranForPost|same_user
sv_a = urhk_CopyOutToIn("")
sv_a=urhk_ExecSrvNoCommit("SRV_PostCashDepTran")
if(sv_a != 0 ) then
print(BANCS.OUTPARAM.ErrMsg)
endif
trace off
end-->
```

©Copyright Infosys Ltd.

FXBILL_GETTRAN

This assumes that there is a linklist available globally containing the part transaction details such as glb_fxb_main_ptr->tran_ll. If it does not exist then it returns fatal error. We can call this userhook in a loop so that it fetches one record at a time and puts the details into OUTPARAM class into the fields mentioned here:

MTT_PTRAN_ACCOUNT
MTT_PTRAN_REFAMOUNT
MTT_PTRAN_AMOUNT
MTT_PTRAN_REFCURRENCY
MTT_PTRAN_CURRENCY
MTT_PTRAN_BUSINESSTYPE
MTT_PTRAN_REFPTRANSRLNUM
MTT_PTRAN_PROTECT_VALUE
MTT_PTRAN_RATECODE
MTT_PTRAN_CONVRATE
MTT_PTRAN_PARTICULARS
MTT_PTRAN_REMARKS
MTT_PTRAN_INST_TYPE
MTT_PTRAN_INST_DATE
MTT_PTRAN_INST_NUM
MTT_PTRAN_VALDATE
MTT_PTRAN_REPORTCODE

MTT_PTRAN_REFNUM

These part transaction details can be used to post the part transaction.

Syntax:

```
sv_a = urhk_FxBill_GetTran("")
```

Input Arguments:

None

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

None

©Copyright Infosys Ltd.

FXBILL_INITTRAN

This userhook sets fxUsrHkPtr to NULL which is used by FxBill_GetTran.

Syntax:

```
sv_a = urhk_FxBill_InitTran("")
```

Input Arguments:

None

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

None

©Copyright Infosys Ltd.

Get Amount Calculated Based on Setup in Amount Code Table

Given an input amount, currency code and Amount Table code, this function returns the resultant percentage amount. This function applies the same logic as Finacle in determining the commission percentage based on the amount slab setup.

Syntax:

The syntax for calling the function is:

```
sv_r = urhk_B2k_GetASTMAmount ("" )
```

Input Arguments:

All the input variables must be populated in the INPARAM class of the BANCS repository. The variables are (all STRINGS):

- InputAmount - The amount that determines the slab
- CurrencyCode - The currency code of the amount
- AmountTableCode - The code to be used to access the Amount Slab table

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Function returns '0' or '1'. If '1', then there was failure in determining

the percentage. If '0', all output variables is populated in the OUTPARAM class of the BANCS repository. The variables are (all STRINGS):

●OutputAmount - The percentage amount

For Example:

```
<--Start
sv_p = BANCS.INPUT.DDAmount
BANCS.INPARAM.InputAmount = sv_p
BANCS.INPARAM.CurrencyCode = "INR"
BANCS.INPARAM.AmountTableCode = "DDCOM"
Sv_r = urhk_B2k_GetASTMAmount ("")
If (sv_r ==1)
# Error processing
endif
sv_q = BANCS.OUTPARAM.OutputAmount
# sv_q contains the charge amount
End-->
```

©Copyright Infosys Ltd.

Accept the Amount String

This user hook is created to accept the amount string in user format and convert it to standard format.

Syntax:

```
sv_b = urhk_convertAmountToStdFormat ("")
```

Input

Amount string in user format is the input for this userhook.

Output

BANCS.OUTPARAM. stdFormatAmt

This user hook will return 0 on successful completion and on failure it will return 1 and error mesg will be stored in BANCS.OUTPARAM. retCode

For Example:

```
<--start
trace on
BANCS.INPARAM.userFormatAmt = "100,00,000.00"
sv_z=urhk_convertAmountToStdFormat("")
print(sv_z)
if(sv_z!=0) then
print(BANCS.OUTPARAM.ErrMsg)
else
sv_v=BANCS.OUTPARAM.stdFormatAmt
print(sv_v)
```

```
endif  
exitscript  
trace off  
stop-->
```

©Copyright Infosys Ltd.

Get Cash Account Foracid

This userhook is created to accept the currency code from the user and return the cash Foracid.

Syntax:

```
sv_b = urhk_getCashAccountForacid("")
```

Input Arguments:

Input currency code

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The Cash Foracid is returned based on the currency code that was passed as an input parameter.

For Example:

```
<--start
#trace on
BANCS.INPARAM.CurrencyCode
sv_z = urhk_getCashAccountForacid("")
BANCS.OUTPARAM.Foracid
#trace off
```

end-->

©Copyright Infosys Ltd.

Finacle Copyright

Get Interest between Dates

Interest collected in the given period for a loan or OD account.

Syntax:

```
sv_b = urhk_getInterestBetweenDates (“”)
```

Input Arguments:

BANCS.INPARAM.acctNum= Foracid

BANCS.INPARAM.fromDate= Starting Date

BANCS.INPARAM.toDate= Ending Date

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctNum	acctNum
BANCS.INPARAM.fromDate	fromDate
BANCS.INPARAM.toDate	toDate

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.intAmt	intAmt
BANCS.OUTPARAM.CrOrDrInd	CrOrDrInd
BANCS.OUTPARAM.crncyCode	crncyCode
BANCS.OUTPARAM.errorMesg	errorMesg

Return Value:

If SUCCESS:

BANCS.OUTPARAM.intAmt

BANCS.OUTPARAM.CrOrDrInd

BANCS.OUTPARAM.crncyCode

If FAILURE:

BANCS.OUTPARAM.errorMesg

For Example:

```
<--start
```

```
trace on
```

```
BANCS.INPARAM.acctNum="BDSB001"
```

```
BANCS.INPARAM.fromDate="01-05-2002 00:00:00"
```

```
BANCS.INPARAM.toDate="30-09-2002 00:00:00"
```

```
sv_a=urhk_getInterestBetweenDates("")
```

```
print(sv_a)
```

```
if (sv_a==0) then
```

```
sv_b = BANCS.OUTPARAM.intAmt
```

```
sv_c = BANCS.OUTPARAM.CrOrDrInd
```

```
else
```

```
sv_b = BANCS.OUTPARAM.errorMesg
```

```
endif
```

```
print(sv_b)
```

```
print(sv_c)
```

```
trace off
```

```
end-->
```

©Copyright Infosys Ltd.

Finacle Copyright

Get Module Information

Syntax:

```
sv_a = urhk_B2k_getModuleInfo("Module")
```

Where sv_a is a scratch pad variable to store the return value.

Input Arguments:

The input passed is one of the legal modules of Finacle Core Banking Solution specified in b2k_modules.h file.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.mod_osUserIdmod_osUserId	osUserId
BANCS.OUTPARAM.mod_appUserId	appUserId
BANCS.OUTPARAM.mod_workDir	workDir
BANCS.OUTPARAM.mod_version	version

Return Value:

The user hook returns the value '0', if the string generates module information; otherwise, it returns the value '1'.

For Example:

```
<--start
#trace on
sv_e = urhk_B2k_getModuleInfo("TDL")
```



```
print(BANCS.OUTPARAM.mod_osUserId)
print(BANCS.OUTPARAM.mod_appUserId)
print(BANCS.OUTPARAM.mod_workDir)
print(BANCS.OUTPARAM.mod_version)
#trace off
end-->
```

©Copyright Infosys Ltd.

Get Orchestration Transaction ID Sequence Number

This userhook is added to get the Orchestration Transaction ID sequence for making Finacle Core Outbound Transaction.

Syntax:

```
sv_b = urhk_ GetOrchTranIdSeqNo ("")
```

Input Arguments:

None

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.successOrFailure	successOrFailure
BANCS.OUTPARAM.AddlMessage	AddlMessage
BANCS.OUTPARAM.ErrorMesg	ErrorMesg
BANCS.OUTPARAM.ErrorCode	ErrorCode

Return Value:

An output parameter BANCS.OUTPARAM.orchtranId is populated, which is the orchestration Transaction ID and is used for Finacle Core Outbound Transaction.

Get Syn Agent Upfront Fee

This userhook is added to dump the upfront fee for a given tranche and draw down amount.

Syntax:

```
sv_b = urhk_getSynAgentUpfrontFee (<tranche Id> + "|" + <draw down amt>)
```

Input Arguments:

Tranche ID and draw down amount are the input parameters, both are mandatory.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.tranche Id	tranche Id
BANCS.INPARAM.draw down amt	draw down amt

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.totalUpfrontFee	totalUpfrontFee

Return Value:

totalUpfrontFee is the output variable, it holds the total upfront fee to be collected for the given parameters.

Get Total Profit and Loss Amount for Pool

This userhook fetches Interpool Indicative Profit Amount, Interpool Adjustment Balance Amount and Asset Investment Amount.

Syntax:

```
sv_b = urhk_getTotProfAndLossAmtsForPool("")
```

Input Arguments:

Pool code

Currency Code

Profit Distribution Date

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Pool code	Pool code
BANCS.INPARAM.Crncy Code	Crncy Code
BANCS.INPARAM.Profit Distribution Date	Profit Distribution Date

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.InterPool Profit/Loss Amount	InterPool Profit/Loss Amount
BANCS.OUTPARAM.Total Profit/Loss amount	Total Profit/Loss amount

Return Value:

InterPool Profit/Loss Amount, Total Profit/Loss amount

©Copyright Infosys Ltd.

Finacle Copyright

IBR Userhooks

Add User Defined Message into IBR for Reconciliation. This userhook adds a record into the 'Responding table' of Finacle Core Banking Solution that is retrieved by the IBR program for transmission to the IBR center. To match all transactions, a corresponding entry has to be sent by a remote application to the IBR center. Typically, this function is used in the process of responding to an online request by a remote application.

Note:

This function does not Commit to the database. Commit needs to be done by the script using this hook and by calling the DBSql userhook.

Syntax:

```
sv_a = urhk_PutUDTXN_into_IBR ("" )
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.SolID	SolID
BANCS.INPARAM.Tran_date	Transaction Date
BANCS.INPARAM.SerialNum	Transaction Serial Num
BANCS.INPARAM.Application_ID	Application ID
BANCS.INPARAM.Tran_amt	Transaction Amount
BANCS.INPARAM.Tran_Crnncy	Transaction Currency
BANCS.INPARAM.CustID	Cust ID
BANCS.INPARAM.IBRCenter	IBR Center
BANCS.INPARAM.Particulars_1	Particulars 1

BANCS.INPARAM.Particulars_2	Particulars 2
-----------------------------	---------------

Output Repository Fields:

Output not present.

Return Value:

This userhook returns the value '0' if the record creation is successful; otherwise, it returns the value '1'.

©Copyright Infosys Ltd.

Initiate Audit for Logical Unit of Work

The function initializes the internal audit structure so that it can be used for the current LUW. After each call to this hook the audit reference number for the audit records changes.

Syntax:

```
sv_a = urhk_InitAudit("")
```

Input Arguments:

Input is a null value.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.SuccessOrFailure	SuccessOrFailure
BANCS.OUTPARAM.errorMesg	errorMesg

Return Value:

The ErrorMesg field is populated only if the SuccessOrFailure is F. If the SuccessOrFailure field is S, the operation is successful.

For Example:

```
< --start
```

```
sv_d = urhk_InitAudit("")
```

```
print(sv_d)
```

```
print(BANCS.OUTPARAM.SuccessOrFailure)
```



```
print(BANCS.OUTPARAM.errorMesg)
```

```
exitscript
```

```
end -->
```

©Copyright Infosys Ltd.

Get Total Sanction Limit of All Accounts Linked to DSA

This userhook gives the total sanction limit of all accounts linked to the DSA which are either closed or charged off or paid off for the given DSA ID, product code, currency code, start date and end date. The inputs are to be separated by a delimiter |.

Syntax:

```
sv_a = urhk_getCPCAcctSancLimitForDSA ()
```

Input Arguments:

DSA ID

Product code

Currency code

Start date

End Date

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The output is available in BANCS.OUTPARAM. totSancLimit.

For Example:

```
# Get the inputs from repository
sv_a = BANCS.INPUT.dsaId
sv_b = BANCS.INPUT.productCode
sv_c = BANCS.INPUT.crncyCode
sv_d = BANCS.INPUT.startDate
sv_e = BANCS.INPUT.endDate
sv_f = sv_a + '|' + sv_b + '|' + sv_c + '|' + sv_d + '|' + sv_e
#call the user hook for getting total Sanction Limit
sv_n = urhk_getCPCAcctSanclimitForDSA (sv_f)
Output is in this field: BANCS.OUTPARAM.totSanclimit
```

©Copyright Infosys Ltd.

Interest Rate for an Account as on Given Date in Bancs Repository

This userhook returns absolute value of interest rate for a given account number and a given date and specified amount. This userhook can be called from any script event. It validates the account number, as on date and amount.

If inputs are valid, it populates interest rate into the Finacle repositories.

Syntax:

```
sv_a = urhk_getInterestRateForAccount("")
```

sv_a: scratch pad variable to store the return value.

Input Arguments:

Valid account number

As on Date

Balance as on date

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctNum	Valid Account Number
BANCS.INPARAM.asOnDate	As on Date(in DD-MM-YYYY format)
BANCS.INPARAM.balance	Balance

Output Repository Fields:

Output not present.

Return Value:

If the input values are invalid, then return value is 1. Also if the interest rate could not be arrived at due to improper interest version set up or non-maintenance of interest version as on the specified date, then return value is 1.

This userhook obtains the normal absolute value of interest rate as of BOD date. If the balance amount specified is positive, then the interest rates mentioned against credit balances in IVSM or TVSM is looked into. Similarly the debit balance interest rate entries is looked into for negative balances.

Interest rate is populated into the repository Interest Rate BANCS.OUTPARAM.interestRate.

For Example:

To obtain interest rate as on BOD date for an account, the code to be implemented is:

```
<--start

# Get account balance as on BOD date.
BANCS.INPARAM.acctId = BANCS.INPUT.acctId
BANCS.INPARAM.asOnDate = BANCS.STDIN.BODDate
sv_a = urhk_getAcctBalanceAsOnDate("")
# Populate the input repositories for getting interest rate
BANCS.INPARAM.balance = BANCS.OUTPARAM.acctBal
BANCS.INPARAM.acctNum = BANCS.INPUT.acctId
BANCS.INPARAM.asOnDate = BANCS.STDIN.BODDate
sv_a = urhk_getInterestRateForAccount("")
if ( sv_a == 1 ) then
```

exitscript

endif

sv_i = cdouble(BANCS.OUTPARAM.interestRate)

end-->

©Copyright Infosys Ltd.

Next Number Generation

This userhook is used to generate the next number using check digit.

Syntax:

```
sv_a = urhk_GetNextNumber("")
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

- BANCS.INPARAM.numType
- BANCS.INPARAM.nextNum
- BANCS.INPARAM.checkDigitFlag
- BANCS.INPARAM.prefixString
- BANCS.INPARAM.suffixString
- BANCS.INPARAM.applyString
- BANCS.INPARAM.CheckDigitFlag

Output Repository Fields:

- BANCS.OUTPARAM.errorString
- BANCS.OUTPARAM.successOrFailure
- BANCS.OUTPARAM.applyString

■BANCS.OUTPUTPARAM.nextNum

Return Value:

Return value not available.

For Example:

```
<--start
BANCS.OUTPUT.errorString = ""
BANCS.OUTPUT.checkDigit = ""
BANCS.OUTPUT.nextNum = BANCS.INPUT.nextNum
BANCS.OUTPUT.applyString = BANCS.INPUT.applyString
BANCS.OUTPUT.successOrFailure = "S"
# check for num type other than account-"A"
IF (BANCS.INPUT.numType != "A") THEN
BANCS.OUTPUT.errorString = "No logic specified in the script
for num type other than account
"
BANCS.OUTPUT.successOrFailure = "F"
exitscript
ENDIF
sv_a = CLASSEXISTS("BANCS", "CHKDGT")
if (sv_a == 0) then
CREATECLASS("BANCS", "CHKDGT", 5)
endif
BANCS.CHKDGT.applyString = BANCS.INPUT.applyString
BANCS.CHKDGT.nextNum = BANCS.INPUT.nextNum
# MODULUS 11 LOGIC STARTS
```



```
# sv_r is the remainder counter set to 1 for outer while loop.
sv_r = 1

# sv_d is the counter in outer while loop.if sv_r equals 1, calls
the user hook.
sv_d = 0

# sv_l stores the length of apply string.
sv_l = 0

# Find the remainder of the apply string which is not equal to 1.
while (sv_r == 1)
sv_b = 2
sv_s = 0
sv_l = STRLEN(BANCS.CHKDGT.applyString)
# sv_c is the counter for the inner while loop.
sv_c = 1
IF (sv_d > 0) THEN
BANCS.INPARAM.numType = BANCS.INPUT.numType
BANCS.INPARAM.nextNum = BANCS.INPUT.nextNum
BANCS.INPARAM.checkDigitFlag = BANCS.INPUT.checkDigitFlag
BANCS.INPARAM.prefixString = BANCS.INPUT.prefixString
BANCS.INPARAM.suffixString = BANCS.INPUT.suffixString
# Caution: Always use this user hook to get the next number
sv_u = urhk_GetNextNumber("")
if (sv_u != 0) then
BANCS.OUTPUT.errorString = "could not fetch next number"
BANCS.OUTPUT.successOrFailure = "F"
exitscript
```

```

endif
BANCS.CHKDGT.applyString = BANCS.OUTPARAM.applyString
BANCS.CHKDGT.nextNum = BANCS.OUTPARAM.nextNum
ENDIF
sv_l = STRLEN(BANCS.CHKDGT.applyString)
GOSUB getModuloRemainder
sv_x = sv_s / 11
sv_r = sv_s - (sv_x * 11)
sv_d = sv_d + 1
# sub routine is outside the if-endif condition
getModuloRemainder:
while (sv_c <= sv_l)
    sv_i
    CINT(GETSTRING(CHARAT(BANCS.CHKDGT.applyString,sv_l -
    sv_c)))
    sv_s = sv_s + (sv_b * sv_i)
    sv_b = sv_b + 1
    IF (sv_b > 7) THEN
        sv_b = 2
    ENDIF
    sv_c = sv_c + 1
do
return
do
# sv_z is checkDigit
# sv_r on exit from while loop.
IF (sv_r > 0) THEN

```

sv_z = 11 - sv_r

ELSE

sv_z = 0

ENDIF

BANCS.OUTPUT.successOrFailure = "S"

BANCS.OUTPUT.nextNum = BANCS.CHKDGT.nextNum

BANCS.OUTPUT.checkDigit = format\$(sv_z, "%d")

BANCS.OUTPUT.applyString = BANCS.CHKDGT.applyString

exitscript

end-->

©Copyright Infosys Ltd.

Populate Collateral Information

This userhook is used to get the various collateral properties such as total primary accounts, total primary nodes, total collateral nodes, total collateral accounts, and so on for the given security identified by the security code.

Syntax:

```
sv_a = urhk_Populate_ColateralInfo("")
```

Input Arguments:

Security code (security_code).

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The list of output values are:

- Total primary accounts (tot_prime_accts)
- Total value of all the primary accounts (tot_prime_accts_amt)
- Total primary nodes (tot_prime_nodes)
- Total value of all the primary nodes (tot_prime_nodes_amt)
- Total collateral accounts (tot_colla_accts)

■ Total value of all the collateral accounts (tot_colla_accts_amt)

■ Total collateral nodes (tot_colla_nodes)

■ Total value of all the collateral nodes (tot_colla_nodes_amt)

For Example:

```
<<--start
#trace on
sv_b = "TEST"
BANCS.INPARAM.secu_code = sv_b
sv_a = urhk_Populate_ColateralInfo("")
if (fieldexists(BANCS.OUTPARAM.tot_prime_accts)) then
sv_o = BANCS.OUTPARAM.tot_prime_accts
endif
sv_a = urhk_VAL_SetVal("tot_prime_accts|" + sv_o)
#trace off
end-->
```

©Copyright Infosys Ltd.

Put Environment Variable

Syntax:

```
sv_a = urhk_putEnv("env_variable|value")
```

Where sv_a is a scratch pad variable to store the return value.

Input Arguments:

The input is the environment variable and the value of the variable.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value not available.

For Example:

```
<--start
trace on
sv_a = BANCS.INPUT.ContextSolId
print(sv_a)
sv_b = BANCS.INPUT.Filename
print(sv_b)
sv_c = sv_a + "|" + sv_b
print(sv_c)
```

```
sv_n = urhk_putEnv(sv_c)
```

```
exitscript
```

```
end-->
```

©Copyright Infosys Ltd.

Print Repository Fields

This userhook is used to print the structure of a repository. The input to this user hook is the repository name.

Syntax:

```
sv_e = urhk_B2k_PrintRepos("BANCS")
```

Input Arguments:

It requires to export GLOBAL_TRACE=ON/OFF in start script of finlistval bin folder.

For Example:

```
export GLOBAL_TRACE=ON;
```

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

If GLOBAL_TRACE variable is not exported or set to value as **ON** in the start script of finlistval bin folder then trc file is generated else if GLOBAL_TRACE variable set to value as **OFF** then trc file is not generated.

For Example:

```
<--start
```


trace on

sv_r=urhk_B2k_PrintRepos("BANCS")

end-->

©Copyright Infosys Ltd.

Raise Exception for Transaction Created through HTM

This function raises a transaction exception, given the exception code. In case the exception is set as an error, the error description is returned as an outparam field. It can be used in the context of any Online transaction Event or Workflow.

Syntax:

sv_a = urhk_FTS_RaiseException (Variable)

The variable can be a scratch pad variable (sv_b) or a string('481').

Input Arguments:

The inputs to this function (through the INPARAM fields) are ExceptionIgnoreFlg, DrCrInd, ValMode.

The input through function argument is exception code.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value is '1' if exception is an error, else '0'.

This function returns the corresponding error description into the OUTPARAM class of BANCS repository if the exception code passed has been set as an error.

Field name in BANCS repository

BANCS.OUTPUTPARAM.ErrorDesc

For Example:

In the TM_ENTRY event of the FinTran.scr script, the two checks needed to be done for an account of SBFIX scheme are:

- If the transaction amount is greater than 1,00,000.00 then display an error.
- If the transaction amount is not a multiple of 1000, then raise an exception code, say 432.

The relevant part of the script will look like this:

```
<--start
BANCS.OUTPUT.successOrFailure = "S"
BANCS.OUTPUT.ErrorDesc = ""
BANCS.OUTPUT.AddtlDetails = ""
BANCS.OUTPUT.DeleteUAD = "Y"
if (BANCS.INPUT.Event == "TM_ENTRY") then
# If the scheme is SBFIX
if (BANCS.INPUT.SchemeCode == "SBFIX") then
# If the amount is > 1 Lakh, throw error.
sv_a = cdouble(BANCS.INPUT.TranAmt)
if (sv_a > 100000) then
BANCS.OUTPUT.successOrFailure = "F"
BANCS.OUTPUT.ErrorDesc = "Amt exceeds 1 Lakh in SBFIX
Scheme!"
exitscript
endif
```

```

# If the amount is not multiple of 1000, raise an exception 432.
sv_a = BANCS.INPUT.TranAmt
sv_b = strlen(sv_a)
sv_c = mid$(sv_a, sv_b - 6, sv_b)
if (sv_c != "000.00") then
    BANCS.INPARAM.ValMode = BANCS.INPUT.ValMode
    BANCS.INPARAM.DrCrInd = BANCS.INPUT.DrCrInd
    BANCS.INPARAM.ExceptionIgnoreFlg =
    BANCS.INPUT.ExceptionIgnoreFlg
    sv_d = urhk_FTS_RaiseException("432")
    # If the return value is 1 then it is an error. Otherwise the
    # exception has been raised successfully.
    if (sv_d == 1) then
        # The exception is an error.
        BANCS.OUTPUT.successOrFailure = "F"
        BANCS.OUTPUT.ErrorDesc = BANCS.OUTPARAM.ErrorDesc
        exitscript
    endif
endif
endif
endif

```

Raising Exceptions

Using this function exceptions can be raised at verify time in the menu options BM, FBM, IRM, ORM, IDCM, ODCM, MNTFWC, CNCLFWC, and EXTFWC.

See Lodge Bills in the Inland Bills module for BM menu option details.

See Lodge a Foreign Bill in the Foreign Bills module for FBM menu option details.

See Lodge and Realize Inward Telegraphic Transfer in the Remittances module for IRM menu option details.

See Issue an Outward Telegraphic Transfer in the Remittances module for ORM menu option details.

See Issue a Documentary Credit in the Documentary Credits module for ODCM menu option details.

See Advise Inward Documentary Credit in the Documentary Credits module for IDCM menu option details.

See Book Forward Contract in the Forward Contracts module for MNTFWC menu option details.

See Cancel Forward Contract in the Forward Contracts module for CNCLFWC menu option details.

See Extend Forward Contract in the Forward Contracts module for EXTFWC menu option details.

Syntax:

```
sv_r=urhk_B2k_PutUserException (ExceptionCode)
```

Input Arguments:

The input is exception code. Exception code must be a valid exception code setup using HEXCDM.

See Define Exception Codes in the Exception Handling module for HEXCDM menu option details.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

None

For Example:

In case bank wish to raise an exception whenever forward contract is booked for USD (as from currency)

```
<--start
```

```
sv_a = urhk_TBAF_InquireFieldValue("datblk1.from_crncy_code")
```

```
sv_l = B2KTEMP.TEMPSTD.TBAFRESULT
```

```
if (sv_l == "USD") then
```

```
sv_a = urhk_B2k_PutUserException("FWU")
```

```
endif
```

```
exitscript
```

```
end-->
```

©Copyright Infosys Ltd.

Round Off Amount

Given an input amount, **round off to** amount, and round off flag, this function gives the rounded off amount. This function applies the same logic as Finacle in roundingoff the amount.

Syntax:

The syntax for calling the function is `sv_r = urhk_B2k_RoundOff ("" )`

Input Arguments:

All the input variables must be populated in the INPARAM class of the BANCS repository. The variables are (all STRINGS):

InputAmount - The amount that needs to be converted

RoundOffAmt - The amount to be rounded off to

For Example:

If the amount is in Rupees

100 - 100 Rupees

.01 - 1 Paise

RoundOffFlag - Valid values are:

H - roundoff to the next higher amount

L - roundoff to the previous lower amount

N - roundoff to the nearest amount

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Function returns '0' or '1'. If '1', then there was failure in roundingoff. If 0, all output variables is populated in the OUTPARAM class of the BANCS repository. The variables are (all STRINGS) :

OutputAmount - The roundedoff amount

For Example:

```
<--start
trace on
sv_a = 10986792.2358
#Input Amount
sv_b = 100
# Round off to 100 Rupees
sv_c = "L"
# Lower amount
BANCS.INPARAM.InputAmount = sv_a
BANCS.INPARAM.RoundOffAmt = sv_b
BANCS.INPARAM.RoundOffFlag = sv_c
sv_r = urhk_B2k_RoundOff ("")
if (sv_r == 1) then
# Error processing
endif
sv_q = BANCS.OUTPARAM.OutputAmount
```


sv_q contains the rounded amount. This should be
10986700.0000

end-->

©Copyright Infosys Ltd.

Save User Additional Details

This function adds or modifies user additional detail for an entity say bill or forward contract. Currently this function is enabled for the menu options BM, FBM, IRM, ORM, IDCM, ODCM, MNTFWC, CNCLFWC, and EXTFWC.

See Lodge Bills in the Inland Bills module for BM menu option details.

See Lodge a Foreign Bill in the Foreign Bills module for FBM menu option details.

See Lodge and Realize Inward Telegraphic Transfer in the Remittances module for IRM menu option details.

See Issue an Outward Telegraphic Transfer in the Remittances module for ORM menu option details.

See Issue a Documentary Credit in the Documentary Credits module for ODCM menu option details.

See Advise Inward Documentary Credit in the Documentary Credits module for IDCM menu option details.

See Book Forward Contract in the Forward Contracts module for MNTFWC menu option details.

See Cancel Forward Contract in the Forward Contracts module for CNCLFWC menu option details.

See Extend Forward Contract in the Forward Contracts module for EXTFWC menu option details.

Syntax:

```
sv_r=urhk_B2k_PutUserAdditionalDetails("")
```

Input Arguments:

The inputs are Module ID, ModuleKey and AdditionalDetails through BANCS.INPARAM class.

Valid values of Module ID are BIL, FEX, BG, FWC, and DC.

Module Key is a field to identify unique entity e.g. for a bill module ID can be Sol ID and Bill ID concatenated with a slash '/' character

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

None

For Example:

Let us say that the bank wish to put two remarks for each bill. Bank can put the value of these two fields through following script in BM menu option.

```
BANCS.INPARAM.ModuleId='BIL'
```

```
BANCS.INPARAM.ModuleKey='SOL1' + '/' + 'BILL125'
```

```
BANCS.INPARAM.      AdditionalDetails      ='REAMRKS1'      +  
'REMARKS2'
```

```
sv_a=urhk_B2k_PutUserAdditionalDetails("")
```

©Copyright Infosys Ltd.

Update BJM

This userhook is used to update job status in BJM table. It is a wrapper over the function updateBJMJobStatus. JobStatus by default is 'N' which is Not yet invoked. It could be changed to 'S' Success or 'F' Failure or 'I' Invoked. Return value of updateBJMJobStatus is either '0' or '1'.

Syntax:

```
sv_b = urhk_updateBJM("")
```

Input Arguments:

sol_id

job_id

job_exec_date

run_for_date

run_srl_num

bank_id

job_status

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.sol_id	sol_id
BANCS.INPARAM.job_id	job_id
BANCS.INPARAM.job_exec_date	job_exec_date
BANCS.INPARAM.run_for_date	run_for_date
BANCS.INPARAM.run_srl_num	run_srl_num

BANCS.INPARAM.bank_id	bank_id
BANCS.INPARAM.job_status	job_status

Output Repository Fields:

Output not present.

Return Value:

None

For Example:

The piece of code that shows how the job status is updated in BJM table through the userhook updateBJM.

```
sv_c = "106"
```

```
sv_f = "AM1"
```

```
sv_a = "13-JAN-2001"
```

```
sv_b = "13-JAN-2001"
```

```
sv_d = "001"
```

```
sv_m = "01"
```

```
sv_k = "F"
```

```
BANCS.INPARAM.sol_id = sv_c
```

```
BANCS.INPARAM.job_id = sv_f
```

```
BANCS.INPARAM.job_exec_date = sv_a
```

```
BANCS.INPARAM.run_for_date = sv_b
```

```
BANCS.INPARAM.run_srl_num = sv_d
```

```
BANCS.INPARAM.bank_id = sv_m
```

```
BANCS.INPARAM.job_status = sv_k
```

```
sv_q = urhk_updateBJM("")
```

©Copyright Infosys Ltd.

Finacle Copyright

User Defined TOD

With this userhook we can give user defined TOD, if transaction amount is greater than the effective available amount. This userhook is used depending upon the condition defined in the script. This can be called through the sample script CDPreProc.sscr.

Syntax:

```
sv_a = urhk_createUserDefndTOD(sv_m)
```

Where `sv_m = Format$(sv_x,"19.6f")`

Where `sv_x = BANCS.INPUT.TranAmt - BANCS.OUTPARAM.FullEffAvailableAmt`

If `sv_x >= 0`, then only TOD will be granted.

Input Arguments:

Input arguments not available.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value not available.

For Example:

```
sv_f = CDOUBLE(BANCS.INPUT.TranAmt)
```

```
sv_y = sv_f/100
sv_g = BANCS.INPUT.TrnCrcncy
sv_c = BANCS.INPUT.Command
sv_h = urhk_getAcctDetailsInRepository(sv_d)
if (sv_h != 0) then
MTT.Error.Description="Unable To Get GAM record"
sv_r = urhk_MTTTS_ReportErrorCondition("")
endif
sv_a = CDOUBLE(BANCS.OUTPARAM.FullEffAvailableAmt)
sv_x = sv_y - sv_a
sv_m = FORMAT$(sv_x, "%19.6f")
if (sv_e == "EBA") then
if (sv_c == "CWDR" ) then
if (sv_x >= 0) then
sv_z = urhk_createUserDefndTOD(sv_m)
if(sv_z != 0) then
BANCS.OUTPUT.successOrFailure="S"
exitscript
endif
endif
endif
else
BANCS.OUTPUT.successOrFailure="S"
exitscript
endif
BANCS.OUTPUT.successOrFailure="S"
```


end-->

©Copyright Infosys Ltd.

Finacle Copyright

Userhook - GetFldValueWithSrvIdAndFldName

This userhook is provided to get the field value for the passed service ID and field name. This userhook must be used only in the pre/post service invoke scripts of custom batch job.

Syntax:

```
sv_r      =      GetFldValueWithSrvIdAndFldName("<service_Id>|<field_name>")
```

Input Arguments:

Service ID and field name in a pipe separated format.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

BANCS.OUTPARAM.fieldValue - The field value for the passed for service ID and field name for the current record.

For Example:

```
<--start
```

```
trace on
```

```
sv_a = ONS.CUSTOM.currServiceId
```

```
if(sv_a == "AddBank") then
```

```
sv_r
```

```
=
```

```
urhk_GetFldValueWithSrvIdAndFldName("AddBank|bankDet.bankCode  
if(sv_r == 0) then  
sv_b = BANCS.OUTPARAM.fieldValue  
endif  
exitscript  
end-->
```

©Copyright Infosys Ltd.

Userhook - PutFldValueWithSrvIdAndFldName

This userhook is provided to put the field value for the passed service ID and field name. This userhook must be used only in the pre or post service invoke scripts of custom batch job.

Syntax:

```
sv_r      =      urhk_PutFldValueWithSrvIdAndFldName("<service_Id>|<fld_name>|<fld_value>")
```

Input Arguments:

Service ID, field name and field value in a pipe separated format.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

None

For Example:

```
<--start
trace on
sv_a = ONS.CUSTOM.currServiceId
if(sv_a == "AddBank") then

sv_r                                     =
urhk_PutFldValueWithSrvIdAndFldName("AddBank|bankDet.bankWkly
```

```
if(sv_r == 0) then  
sv_b = print("Bank Weekly Off Modified to 1")  
endif  
exitscript  
end-->
```

©Copyright Infosys Ltd.

Check Pending Verification for Security

This userhook is provided to check whether any verification is pending for the given security identified by the security code.

Syntax:

```
sv_a = urhk_Check_PendVerifcationForSecurity("")
```

Input Arguments:

Input for this userhook is the security code (secu_code).

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Output is a return code (retCode) whose value is either '1' or '2'. If any verification is pending at the limit level then the return code is 1. If any verification is pending at the account level of the given security code then the return code is '2'.

For Example:

```
<--start
#trace on
sv_b = "TEST"
BANCS.INPARAM.secu_code = sv_b
```

```
sv_a = urhk_Check_PendVerifcationForSecurity("")  
if (fieldexists(BANCS.OUTPARAM.retCode)) then  
sv_z = BANCS.OUTPARAM.retCode  
endif  
#trace off  
end-->
```

©Copyright Infosys Ltd.

Execute Web Based External Application in Web Instance for Finacle

This userhook TBAF_ShowHTML is used to execute a Web-based external application in the Web Interface for Finacle Core Banking Solution environment. When this userhook is called, the Core Banking Solution applet displays the URL in the specified window and waits for an indication to know that it has finished. The invoked external application must call the "sendHTMLResp" method of the Core Banking Solution applet to indicate that it is complete. The applet function can be called through a JavaScript.

Syntax for "sendHTMLResp" method: void sendHTMLResp(String sResultStr) sResultStr contains a result string in standard HTML form data format. For example, (Name=Value&). Name indicates a repository class and field name in Class.Field format. Value indicates the value.

The repository information passed is used to recreate the repository for the calling script. The following repository fields must be populated before calling the userhook.

Syntax:

```
sv_a = urhk_TBAF_ShowHTML("repository_name")
```

where sv_a stores the return value and repository_name is the repository through which the userhook communicates with the script.

It must be of string type and have two classes - STDIN and STDOUT

Input Arguments:

The userhook takes inputs from STDIN class.

Input Repository Fields:

Field Name	Description
BANCS.STDIN.URL	Contains the URL of the Web application to be executed.
BANCS.STDIN.FRAME	Contains the name of the frame or window in which the URL is displayed.

Output Repository Fields:

Field Name	Description
BANCS.STDOUT.ErrorCode	The code of the error that has occurred. This must be set to '0'.
BANCS.STDOUT.ErrorDesc	The description of the error code. This must be set to an empty string "".

Return Value:

The userhook populates the outputs into the STDOUT class. The script checks the repository for the return code from the application on return from the userhook.

For Example:

Error Codes

Error communication with user machine. The external application was not invoked.

Error understanding response message. The external application was invoked. But the response string had an error.

Invalid message from user machine. The external application was invoked. But it returned an unrecognized message.

Cancelled. The external application was invoked but before it could respond back, the user cancelled the operation using the cancel button.

Mask Input

This user hook is added to mask a given alphanumeric input (max of 50 chars) with a user provided masking character (optional) or by the default masking character “ * “ when user has not provided the it as input.

Syntax:

sv_b = urhk_ maskInput (“”) (With no optional input)

sv_b = urhk_ maskInput (“#”) (With optional input)

Input Parameters:

Alphanumeric input (max 50 chars), Display digit, masking char (optional)

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.maskOutput	Output obtained after masking

Example

1. With default masking character

Repository Inputs:

- Foracid = “TEST123”
- displayDigit = 2

Repository Output:

- MaskOutput = “*****23”

2. With User given masking character

Repository Inputs:

- Foracid = “TEST123”

- displayDigit = 3
- masking character = #

Repository Output:

- MaskOutput = "####123"
-

©Copyright Infosys Ltd.

Enabling Debug Logs

This user hook is used to enable or disable debug logs. It is used to update the dbg_enabled_flg to YES from LGI table if the user wants to enable the debug log feature. This is set to NO when the user disables the debug log feature.

Syntax:

```
sv_b = urhk_modifyDbgEnabled (“”)
```

Input Arguments:

It takes user_id,session_id and function code from the BANCS.INPARAM repository.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.user_id	User ID
BANCS.INPARAM.session_id	Session ID
BANCS.INPARAM.function code	Function code

Output Repository Fields:

Output not present

Return value:

It updates the dbg_enabled_flg of LGI table.

For Example:

```
BANCS.INPARAM.session_id = BANCS.INPUT.session_id
```

BANCS.INPARAM.user_id = BANCS.INPUT.user_id

BANCS.INPARAM.VAL_CALL_MODE
BANCS.INPUT.VAL_CALL_MODE =

sv_b = urhk_modifyDbgEnabled("")

if (sv_b != 0) then

exitscript

endif

©Copyright Infosys Ltd.

Discretionary Limit Set for the User

The Discretionary Limit Set for the User userhook returns the discretionary limit set up for a user.

If inputs are valid, then the discretionary amount is populates into Finacle repository.

Syntax:

```
sv_a = urhk_getUserEMDiscretionaryLimit ("")
```

sv_a: scratch pad variable to store the return value.

Input Argument:

■userId

The above input must be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM. userId	Valid user id setup in HUPR menu

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.userDiscretionaryLimAmt	The amount which is set as discretionary limit in HRPM menu cooresponding to the role id of the user id passed as input.
BANCS.OUTPARAM.amtCrncy	The currency for the discretionary limit amount.

Return Value:

If the input values are invalid then the return value is 1.

For Example:

To obtain the discretionary limit set for the user, the steps to be are:

```
<--start
```

```
sv_n = urhk_getUserEMDiscretionaryLimit("")
```

```
if(0 != sv_n)
```

```
#{
```

```
    #BANCS.OUTPUT.errorDesc = ""
```

```
    #BANCS.OUTPUT.xcpnDesc = ""
```

```
    #sv_c = BANCS.OUTPARAM.Xcpn_num
```

```
    #if (sv_c > 0) then
```

```
    #{
```

```
        #sv_d = 1
```

```
        #while (sv_d <= sv_c)
```

```
        #{
```

```
            #sv_f = FORMAT$(sv_d,"%d")
```

```
            #sv_e = ("BANCS").("OUTPARAM").  
("XcpnCode_"+sv_f)
```

```
            #BANCS.OUTPUT.xcpnDesc =  
BANCS.OUTPUT.xcpnDesc + sv_e
```

```
            #sv_e = ("BANCS").("OUTPARAM").  
("XcpnCode_"+sv_f)
```

```
            #sv_e = ("BANCS").("OUTPARAM").("Tag_"+sv_f)
```

```
            #sv_d = sv_d + 1
```

```
        #}
```

```
    do
```

```
#}  
endif  
#}  
#else  
#{  
    #Fetch the user discretionary limit and currency.  
    #sv_n = BANCS.OUTPARAM.userDiscretionaryLimAmt  
    #sv_c = BANCS.OUTPARAM.amtCrncy  
    #BANCS.OUTPUT.errorDesc = ""  
#}  
#endif  
end-->
```

©Copyright Infosys Ltd.

Get Additional Details for HLAPSP

B2K_LAPSPRecAdditional_Dtls

Overview

Input:

The defined input is available in a repository called BANCS, in a class called INPUT with the field names mentioned.

You can access the input by referring to the fields in the following manner:

BANCS.INPUT.<fieldName>

Field Name	Description
BANCS.INPUT.record_type	record type
BANCS.INPUT.acct_num	account number
BANCS.INPUT.charge_type	charge type
BANCS.INPUT.deferment_type	deferment type
BANCS.INPUT.topup_indOutput	topup indOutput

Output:

The output of the script is available in the following repository fields:

BANCS.OUTPUT.<fieldName>

Field Name	Description
BANCS.OUTPUT.lapsp_custom_output	lapsp_custom_output

Default logic in case script is not present:

Typical usage scenario(s):

Example:

```
#####  
# Description : This is the sample script to dump additional information  
at #  
# the end of each and every line of spool file generated by LAPSP.#  
# #  
#          Input          :          record_type,          acct_num,  
charge_type,deferment_type,topup_ind#  
# Output : psp_custom_output #  
# Logic : if we want to populate field1 and field2 at the end of each #  
# line the we need to populate psp_custom_output #  
# as "|field1|field2" #  
#####  
  
<--start  
#trace on  
print("----- LAPSP REPORT -----")  
sv_a = BANCS.INPUT.record_type  
print(sv_a)  
sv_b = BANCS.INPUT.acct_num  
print(sv_b)  
sv_d = BANCS.INPUT.charge_type  
print(sv_d)  
sv_e = BANCS.INPUT.deferment_type
```

```
print(sv_e)
```

©Copyright Infosys Ltd.

Finacle Copyright

Amount in LAKH/MILLION Format Using Commas

This user hook is provided to get the amount in the LAKH/MILLION format using commas.

Syntax:

```
sv_d = ("29-03-2002" + "|" + "28-02-2002" + "|" + "NO")  
sv_b = urhk_B2k_getMonthDayDiff(sv_d)
```

Input Arguments:

No input arguments.

Input Repository Fields:

All the Input variables should be populated in the INPARAM class of the BANCS repository.

BANCS.INPARAM.InputAmount : Amount to be converted

BANCS.INPARAM.FromCurrency: Amount currency

BANCS.INPARAM.AmountWidth: Amount width.

Output Repository Fields:

All the output variables should be populated in the OUTPARAM class of the BANCS repository.

For Example:

This is the part of a script where the user hook is called for getting the amount in LAKH/MILLION format using commas. This script EnteredTran.sscr is present in TBA_SAMPLE directory.

```
if(sv_y <= 0) then
```

```
HOTKEY.ACCTDTLS.PoolEffAvailableAmt='0'  
    BANCS.INPARAM.InputAmount=HOTKEY.ACCTDTLS.PoolEffAvaila  
    BANCS.INPARAM.FromCurrency=HOTKEY.ACCTDTLS.acctCrncycor  
  
BANCS.INPARAM.AmountWidth="17"  
  
sv_z = urhk_B2k_floatWithComma("")  
  
HOTKEY.ACCTDTLS.PoolEffAvailableAmt=  
    BANCS.OUTPARAM.OutputAmount  
  
Endif
```

©Copyright Infosys Ltd.

Get Limit B2kld

This user hook is created to get limit B2kld for a given limit prefix & limit suffix pair.

Syntax:

```
sv_b = urhk_GET_LimitB2kldfromLimPfxSfx ("")
```

Input

Inputs for this user hook are limitPrefix and limitSuffix.

Output

BANCS.OUTPARAM.limitB2kld

This user hook will return 0 on successful completion and on failure 1 on failure. The error message will be stored in BANCS.OUTPARAM.retCode.

For Example:

```
<--start
trace on
BANCS.INPARAM. limitPrefix = BANCS.INPUT.limitPrefix
BANCS.INPARAM. limitSuffix = BANCS.INPUT.limitSuffix
sv_z = urhk_GET_LimitB2kldfromLimPfxSfx ("")
print(sv_z)
if(sv_z != 0 ) then
    andprint(BANCS.OUTPARAM.ErrMsg)
else
    sv_v=BANCS.OUTPARAM. limitB2kld
```

```
endif  
exitscript  
trace off  
end-->
```

©Copyright Infosys Ltd.

Get The Cust ID

This function is used to get the CustId corresponding to the CifId entered in the front end from the table.

Syntax:

```
sv_q = urhk_B2k_CifIdToCustId ("" )
```

Input

CifId

Output

Return value is 0, if CustId is returned, else 1.

For Example:

```
<--start
trace on
sv_q=urhk_B2k_CifIdToCustId("S1001")
print(BANCS.OUTPARAM.CustId)
trace off
end-->
```

©Copyright Infosys Ltd.

Get B2k ID for the Given Event ID

This user hook is used for getting the b2kid for an object ID based on the event type. The valid event types are ACCNT, LOANS, DOCCR, FWCIBILL, FBILL and BNKGR only.

Syntax:

```
sv_a = urhk_getB2kIdForEventId(<object_id>)
```

Input Arguments:

A valid event ID is the input and is to be passed as a parameter.

Output Arguments:

Return Value is 0, if b2kId is returned, else 1.

Output Repository Fields:

The following output field is available for customizing this user hook

Output Field	Description
BANCS.OUTPARAM.eventB2kId	B2k ID corresponding to the event

For Example:

```
<--start
```

```
sv_q = urhk_getB2kIdForEventId(sv_h)
```

```
IF (sv_q != 0) THEN
```

```
    MTT.Error.Description="Invalid Event Type/Id"
```

```
    sv_r = urhk_MTTs_ReportErrorCondition("")
```

```
    BANCS.OUTPUT.successOrFailure="F"
```

```
        goto ENDOFSCRIPT
    ENDIF
    sv_t = BANCS.OUTPUTPARAM.eventB2kId
ENDIF
end-->
```

©Copyright Infosys Ltd.

Userhook for the mrbx Report Function

This user hook is used for mrbx report generation. It internally calls genRpt4004 to generate reports.

Syntax:

```
sv_r = urhk_B2k_GenRpt4004(ptemplateFile| dataFile | reportFile |  
reposFile| finrptBuff).
```

Input:

TemplateFile

dataFile

reportFile

reposFile

finrptBuff

Output Repository Fields:

The following output fields are available for customizing this user hook

Output Field	Description
BANCS.OUTPARAM.reportFileName	Report file name is obtained.
BANCS.OUTPARAM.templateFileName	Template file name

Output:

Success or failure depending on whether the report is generated successfully or not.

Get Error Description for the Error Code

This user hook is used to get the error description for the error code.

Syntax:

```
sv_a = urhk_getMSGCodeErrDesc(<MSG code>)
```

Input:

Valid Message ID.

Output Repository Fields:

The following output field is available for customizing this user hook

Output Field	Description
BANCS.OUTPUT.errormsg	Error Message Description

Output:

0 on success.

For Example:

```
<--start
    #MSG0311552 = "The Delinquency cycle cannot more
    than the max. Delinquency cycle."
    sv_a = urhk_getMSGCodeErrDesc("MSG0311552")
    IF(sv_a == 0) THEN
        BANCS.OUTPUT.errormsg =
BANCS.OUTPARAM.errDesc
    ENDIF
end-->
```

©Copyright Infosys Ltd.

Finacle Copyright

Put Custom Exception

This user hook puts the user exceptions in exception LL. This is for both financial and non-financial transactions. While passing the transaction key, if there are 4 different keys, then just concatenate the same separated by '-' and pass it to the user hook.

Syntax:

```
sv_a                                     =  
urhk_putCustomException(excp_code\FinancialTransactionFlag\Trans
```

Input:

- Exception Code
- Financial Transaction Flag
- Transaction Details
- XcpnCondFileLineNumInd
- contextInfo
- TransactionKey

Output:

Failure (1) if exception could not be put or Success (0) if the exception could be put.

For Example:

```
<--start  
putCustomException(ABC|Y|0000MyForacid|INR200|bauu9090.cxx73  
txnDate-txnId)  
end-->
```

Explanation:

ABC -> Exception Code ,

Y->Finanacial Txn (N for Non Financial),
refAcctExcpDesc,XcpnCondFileLineNumInd,
contextInfo,

txnKey-txnDate-txnId --> Transaction Key info which is used
for audit.

©Copyright Infosys Ltd.

Put Exceptions into Repository

This user hook puts exceptions into repository.

Syntax:

```
sv_a = urhk_putExceptionsIntoRepository(“”)
```

Input:

Exception LL to be put in repository.

Output Repository Fields:

The following output fields are available for customizing this user hook

Output Field	Description
BANCS.OUTPUTPARAM.ErrMsg	Error Message
BANCS.OUTPUTPARAM.NumOfExcpsEncountered	Number of exceptions encountered
EXCP.INPUT_<Rec#>.excpCode	Exception Code
EXCP.INPUT_<Rec#>.codeType	Type of Exception Code
EXCP.INPUT_<Rec#>.minWorkClass	Minimum work Class for exception
EXCP.INPUT_<Rec#>.excpWorkClass	Exception Work Class, who can override the exception
EXCP.INPUT_<Rec#>.ignoreXcpnFlg	Ignore Exception Flag

Output:

Puts values in the fields after selecting from the EXT table.

For Example:

```
<--start
```



```

sv_r = urhk_putExceptionsIntoRepository("")
print(sv_r)
sv_e = BANCS.OUTPARAM.ErrMsg
print(sv_e)
sv_f = BANCS.OUTPARAM.NumOfExcpsEncountered
print(sv_f)
if (sv_f == 0) then
    exitscript
endif
#Counter
sv_n = 1
while (sv_n <= sv_f)
    BANCS.INPUT.COUNT = sv_n
    sv_v = ("EXCP").("INPUT_" + BANCS.INPUT.COUNT).
("excpCode")
    print(sv_v)
    sv_v = ("EXCP").("INPUT_" + BANCS.INPUT.COUNT).
("codeType")
    print(sv_v)
    sv_v = ("EXCP").("INPUT_" + BANCS.INPUT.COUNT).
("minWorkClass")
    print(sv_v)
    sv_v = ("EXCP").("INPUT_" + BANCS.INPUT.COUNT).
("excpWorkClass")
    print(sv_v)
    sv_v = ("EXCP").("INPUT_" + BANCS.INPUT.COUNT).
("ignoreXcpnFlg")
    print(sv_v)

```

```
sv_n = sv_n + 1  
print(sv_n)  
do  
end - ->
```

©Copyright Infosys Ltd.

Insert Lien Details in Repository

This userhook creates and populates mfLienDtIsLL, which is further used to dump the details in the repository. This userhook can be called from any script event and validates the inputs.

If inputs are valid, then a Lien details LL is created, failure is returned otherwise.

Syntax:

```
sv_a = urhk_insertLienDtIs(“”)
```

Input:

Valid Transaction ID

Operative Account

Currency Code

Part Tran Number

Lien Amount

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.mfTranId	Valid Transaction ID.
BANCS.INPARAM.acctForacid	Valid Operative account.
BANCS.INPARAM.crcnyCode	Valid Currency code.
BANCS.INPARAM.partTranSrlNum	Part Tran Serial Number generated given as input.
BANCS.INPARAM.lienAmtStr	Lien amount generated.

Output Repository Fields:

Field Name	Description
BANCS.OUTPUT.successOrFailure	This userhook returns 0 on successful completion else 1 on Failure, its result is stored in BANCS.OUTPUT.successOrFailure as below: 'S'- Success 'F'- Failure

Output:

If inputs to the script are invalid/ NULL, results in below script error:

“The execution of the user hook urhk_insertLienDtIs failed.”

If inputs are valid, script calls insertIntoLienDtIsLL and populates the Lien Details LL(mfLienDtIsLL).

For Example:

```
<--start
    sv_l = BANCS.OUTPUT.partTranSrlNum
    sv_t = CDOUBLE(MTT.InputDetails.TranAmt)
    BANCS.INPARAM.mfTranId =
MTT.InputDetails.MFTranId
    BANCS.INPARAM.partTranSrlNum = sv_l
    BANCS.INPARAM.acctForacid =
MTT.InputDetails.TranForacid
    BANCS.INPARAM.crcnyCode =
MTT.InputDetails.TranCrcny
    BANCS.INPARAM.lienAmtStr = sv_t
    sv_y = urhk_insertLienDtIs("")
    if(sv_y != 0) then
        BANCS.OUTPUT.successOrFailure = "F"
        #MSG0318994 = "urhk_insertLienDtIs returns
FAILURE"
```

```
sv_a = urhk_getMSGCodeErrDesc("MSG0318994")
if(sv_a == 0) then
    BANCS.OUTPUT.scriptError =
BANCS.OUTPARAM.errDesc
endif
endif
end - ->
```

©Copyright Infosys Ltd.

Write into a File

This userhook is used to write data into a File.

Syntax

```
sv_a = urhk_WriteIntoFile("Data to be dumped")
```

Here, sv_a is a scratch pad variable which is used to store the return value.

Input

Data to be written to the file.

Input Repositories

Field Name	Description
BANCS.INPARAM.dumpFileName	File to which the data to be written.

Output Repository Fields

NA

Return Value

SUCCESS

Inserts a Record in the Negative TD LL

This userhook inserts a record in the negative TD LL if the td interest is negative.

Syntax

```
sv_a = urhk_fillNegativeTDLL ()
```

sv_a: scratch pad variable to store the return value.

Input Arguments

NA

Input Repositories

Field Name	Description
BANCS.INPARAM.totalInterest	Interest amount.
BANCS.INPARAM.entityId	Entity ID.
BANCS.INPARAM.entityType	Entity Type
BANCS.INPARAM.ActualEntity	Entity type description
BANCS.INPARAM.endDate	Date

Output Repository Field

NA

Return Value

SUCCESS

©Copyright Infosys Ltd.

Finacle Copyright

Insert Records into PQT Table

This user hook is created to insert the received records into PQT table.

Syntax

```
sv_b = urhk_B2k_Insert_PQT ("")
```

Inputs

Inputs for this user hook are file_name, report_name, report_to and no_of_pages.

Output

This user hook returns 0 on successful completion and 1 on failure. The error message is stored in BANCS.OUTPARAM. retCode

For Example:

```
<--start
#trace on
BANCS.INPARAM. file_name = "xxx1.dat"
BANCS.INPARAM. report_name= "test1.mrt"
BANCS.INPARAM. report_to= "user1"
BANCS.INPARAM. no_of_pages= "100"
sv_z = urhk_B2k_Insert_PQT ("")
print(sv_z)
if(sv_z != 0 ) then
    andprint(BANCS.OUTPARAM.ErrMsg)
endif
```

```
exitscript  
#trace off  
end-->
```

©Copyright Infosys Ltd.

Generate Past Due Details Inquiry and Report

This section discusses:

[Generate Past Due Report](#)

[Get Past Due Display or Print String](#)

[Get Past Due Record](#)

©Copyright Infosys Ltd.

Generate Past Due Report

This userhook past due generates report using the report template specified. This userhook is specific to PDADI menu in print mode.

This userhook may be used to generate additional reports in formats specified. This userhook is applicable only in print mode of PDADI menu option.

Before calling this function a few strings need to be passed to system through userhook `urhk_putPastDueDispPrintString`. These strings is dumped into a spool file and report generation routine is called. Report generated by this userhook is in addition to default report generated in any case. The report generated is put into print queue.

Syntax:

```
sv_a = urhk_generatePastDueReport(Variable)
```

sv_a: scratch pad variable to store the return value

Variable: Scratch pad variable containing formatted string or maha report template name.

Input Arguments:

Maha report template

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value is success ('0') if reported is generated successfully.

For Example:

Example to build and pass string for report generation

```
sv_b = urhk_getPastDueRecord("")
while (sv_b == 0)
# Pad string with spaces
sv_r = sv_r + 1
sv_a = BANCS.OUTPARAM.setId
sv_c = BANCS.OUTPARAM.acctId
sv_d = BANCS.OUTPARAM.solId
sv_e = BANCS.OUTPARAM.schmCode
sv_f = BANCS.OUTPARAM.glSubHeadCode
sv_h = BANCS.OUTPARAM.acctCrncyCode
sv_i = BANCS.OUTPARAM.acctMgrId
sv_j = BANCS.OUTPARAM.custId
sv_l = urhk_getAcctDetailsInRepository(sv_c)
sv_m = BANCS.OUTPARAM.acctName
sv_p = BANCS.OUTPARAM.ClearBalance
sv_l = urhk_getAdvanceDetailsForAccount(sv_c)
if (sv_l == 0) then
sv_n = BANCS.OUTPARAM.PastDueXferDate
else
sv_n = ""
endif
# Build the final string.
```

```
sv_t = "|"
sv_g = sv_a + sv_t + sv_c + sv_t + sv_d + sv_t + sv_e sv_t +
sv_m + sv_t + sv_n
sv_z = urhk_putPastDueDispPrintString(sv_g)
sv_b = urhk_getPastDueRecord("")
do
sv_r = urhk_generatePastDueReport("pastDue.mrt")
# If urhk_generatePastDueReport function returns failure,
# populate error message and exit out of script.
if ( sv_r == 1 ) then
BANCS.OUTPUT. errorMessage="Could not print Report"
exitscript
endif
end
```

©Copyright Infosys Ltd.

Get Past Due Display or Print String

This userhook accepts a string, which is either displayed through PDADI menu option or dumped for report generation. This userhook is specific to PDADI menu.

A formatted string is accepted and the same is displayed on multiple record screen or dump for report generation based on 'Display or Report' flag specified in PDADI menu option. This userhook may be called any number of times. The maximum length of the string can be 2500 characters.

The string passed for display is cut to 150 characters and display the string in two lines of 75 characters each. The string passed for display should be a formatted one.

For generating report, string passed must contain individual fields separated by a '|' character. The number of fields in string must match with the record layout of the Maha Report Template being used for report generation.

Syntax:

```
sv_a = urhk_putPastDueDispPrintString (Variable)
```

sv_a: scratch pad variable to store the return value

Variable: Scratch pad variable containing formatted string.

Input Arguments:

Formatted string

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value is always be success ('0').

For Example:

Example to build and pass a display string

#Get past due record and populate all required values into

#scratch pad variables.

```
sv_b = urhk_getPastDueRecord("")
```

```
while (sv_b == 0)
```

```
# Pad string with spaces
```

```
sv_r = sv_r + 1
```

```
sv_a = BANCS.OUTPARAM.setId
```

```
sv_a = rpad(sv_a, 8)
```

```
sv_c = BANCS.OUTPARAM.acctId
```

```
sv_d = BANCS.OUTPARAM.solId
```

```
sv_d = rpad(sv_d, 8)
```

```
sv_e = BANCS.OUTPARAM.schmCode
```

```
sv_e = rpad(sv_e, 5)
```

```
sv_f = BANCS.OUTPARAM.glSubHeadCode
```

```
sv_f = rpad(sv_f, 5)
```

```
sv_h = BANCS.OUTPARAM.acctCrncyCode
```

```
sv_h = rpad(sv_h, 3)
```

```
sv_i = BANCS.OUTPARAM.acctMgrId
```

```
sv_i = rpad(sv_i, 15)
```

```
sv_j = BANCS.OUTPARAM.custId
```



```

sv_j = rpad(sv_j, 9)
sv_k = rpad(sv_c, 16)
sv_l = urhk_getAcctDetailsInRepository(sv_c)
sv_m = BANCS.OUTPUTPARAM.acctName
if (strlen(sv_m) >= 10) then
sv_m = left$(sv_m, 10)
else
sv_m = rpad(sv_m, 10)
endif
sv_p = BANCS.OUTPUTPARAM.ClearBalance
sv_p = CDOUBLE(sv_p)
sv_x = sv_x + sv_p
sv_p = format$(sv_p, "%17.02f")
sv_l = urhk_getAdvanceDetailsForAccount(sv_c)
if (sv_l == 0) then
sv_n = BANCS.OUTPUTPARAM.PastDueXferDate
else
sv_n = ""
endif
if (strlen(sv_n) >= 10) then
sv_n = left$(sv_n, 10)
else
sv_n = rpad(sv_n, 10)
endif
# Build the final string.
sv_t = " "

```

```
sv_g = sv_k + sv_t + sv_d + sv_t + sv_m + sv_t + sv_n
```

```
sv_z = urhk_putPastDueDispPrintString(sv_g)
```

```
sv_b = urhk_getPastDueRecord("")
```

```
do
```

```
Example to build and pass string for report generation
```

```
sv_b = urhk_getPastDueRecord("")
```

```
while (sv_b == 0)
```

```
# Pad string with spaces
```

```
sv_r = sv_r + 1
```

```
sv_a = BANCS.OUTPARAM.setId
```

```
sv_c = BANCS.OUTPARAM.acctId
```

```
sv_d = BANCS.OUTPARAM.solId
```

```
sv_e = BANCS.OUTPARAM.schmCode
```

```
sv_f = BANCS.OUTPARAM.glSubHeadCode
```

```
sv_h = BANCS.OUTPARAM.acctCrcncyCode
```

```
sv_i = BANCS.OUTPARAM.acctMgrId
```

```
sv_j = BANCS.OUTPARAM.custId
```

```
sv_l = urhk_getAcctDetailsInRepository(sv_c)
```

```
sv_m = BANCS.OUTPARAM.acctName
```

```
sv_p = BANCS.OUTPARAM.ClearBalance
```

```
sv_l = urhk_getAdvanceDetailsForAccount(sv_c)
```

```
if (sv_l == 0) then
```

```
sv_n = BANCS.OUTPARAM.PastDueXferDate
```

```
else
```

```
sv_n = ""
```

```
endif
```

Build the final string.

sv_t = "|"

sv_g = sv_a + sv_t + sv_c + sv_t + sv_d + sv_t + sv_e sv_t +
sv_m + sv_t + sv_n

sv_z = urhk_putPastDueDispPrintString(sv_g)

sv_b = urhk_getPastDueRecord("")

do

end

©Copyright Infosys Ltd.

Get Past Due Record

This userhook returns selected records based on selection criteria specified in PDADI menu option. This userhook is specific to menu option PDADI. Records selected based on Finacle specific selection criteria specified in PDADI menu option is populated into Finacle repository fields. On first call to this userhook, the first selected record is passed. On subsequent calls, the next selected record is passed through repositories. A call after fetching all records returns failure (return value 1).

Syntax:

```
sv_a = urhk_getPastDueRecord ("" )
```

sv_a: scratch pad variable to store the return value.

Input Arguments:

Input arguments not available.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.setId	Sol Set Id
BANCS.OUTPARAM.acctId	Account number
BANCS.OUTPARAM.acctCrncyCode	Account Currency code
BANCS.OUTPARAM.custId	Customer Id
BANCS.OUTPARAM.schmCode	Account scheme Code
BANCS.OUTPARAM.glSubHeadCode	GI Sub Head Code

BANCS.OUTPARAM.solId	Account sol Id
BANCS.OUTPARAM.acctMgrId	Account manager user id

Return Value:

6ol Set ID

Account number

Account Currency code

Customer ID

Account scheme Code

GI Sub Head Code

Account sol ID

Account manager user ID

For Example:

```

<--start
sv_b = urhk_getPastDueRecord("")
while (sv_b == 0)
sv_a = BANCS.OUTPARAM.acctId
sv_b = BANCS.OUTPARAM.custId
sv_c = BANCS.OUTPARAM.acctName
sv_j = "|"
sv_l = sv_a + sv_j + sv_b + sv_c
sv_r = urhk_putPastDueDispPrintString (sv_l)
sv_b = urhk_getPastDueRecord("")
do

```

```
sv_r = urhk_generatePastDueReport (sv_a)
# If urhk_generatePastDueReport function returns failure,
# populate error message and exit out of script.
if ( sv_r == 1 ) then
BANCS.OUTPUT. errorMessage="Could not print Report"
exitscript
endif
end-->
```

©Copyright Infosys Ltd.

IBAN Userhooks

This section discusses:

[Fetch Account Number from IBAN](#)

[Fetch IBAN from Account Number](#)

[Generate the Check Digits](#)

[Validation of IBAN](#)

©Copyright Infosys Ltd.

Fetch Account Number from IBAN

This user hook can be used to derive the foracid for the given Currency Code and MCY IBAN Number combination.

Syntax:

```
sv_b = urhk_getForacidFromIBANnumberForMCY (“”)
```

Input Arguments:

iban_number, crncy_code

Input Repository Fields:

Field Name	Description
BANCS.INPUT.iban_number	iban_number
BANCS.INPUT.crncy_code	crncy_code

Output Repository Fields:

Field Name	Description
BANCS.OUTPUT.foracid	foracid

Return Value:

foracid.

Example

Sample code to derive foracid for the given IBAN Number and Currency Code combinations.

BANCS.INPARAM. iban_number ="US86DBS102DHEERM CY1"

BANCS.INPARAM crncy_code ="USD"

sv_u=urhk_getForacidFromIBANnumberForMCY("")

sv_v=BANCS.OUTPARAM. foracid

print (sv_v)

©Copyright Infosys Ltd.

Fetch IBAN from Account Number

The userhook getIBANFromForacid is provided to get IBAN from account number.

Syntax:

```
sv_u = urhk_getIBANFromForacid("")
```

Input Arguments:

The input is a single field, Foracid which is the Account Number.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

This userhook returns the IBAN (The International Bank Account Number of the account) if the Foracid is valid

©Copyright Infosys Ltd.

Generate the Check Digits

This user hook is created to generate the checkdigit for the input string using modulo97 algorithm as per ECBS standard. The input string will be an IBAN number with temporary check digits "00".

Syntax:

```
sv_b = urhk_getCheckDigitForIBAN ("")
```

Input

Link list is the input to the user hook.

Output

BANCS.OUTPARAM. check_digit

This user hook will return 0 on successful completion and on failure it will return 1 and error message will be stored in BANCS.OUTPARAM. retCode

For Example:

```
<--start
trace on
BANCS.INPARAM.inputString="BE00510007547061"
sv_z=urhk_getCheckDigitForIBAN("")
print(sv_z)
if(sv_z!=0)then
print(BANCS.OUTPARAM.ErrMsg)
trace off
end-->
```

©Copyright Infosys Ltd.

Finacle Copyright

Validation of IBAN

The sample script validateIBAN.scr is provided to validate the IBAN number. IBAN is validated for checkdigit correctness. Check digit validation is done by the userhook validateIBAN. The userhook returns the remainder after calculating the modulus 97. The userhook uses modulo97 algorithm to calculate the checkdigit as per ECBS standards. For the check digit to be correct, the remainder must be 1.

Script:

The script validateIBAN.sscr is present in the \$TBA_SAMPLE/samscripts directory.

Syntax:

```
sv_u = urhk_validateIBAN("")
```

Input

The defined inputs are available in the BANCS repository, INPUT class and with the field names mentioned below. One can access the inputs by referring to the fields in the following manner:

BANCS.INPUT.<FieldName>

Field	Description
IBAN	The International Bank Account Number of the account.

Output

The output is accessed in the script in the following manner:

BANCS.OUTPUT. <Fieldname>

Field	Description

Success or Failure	Indicates if IBAN is valid.
--------------------	-----------------------------

For Example:

```
<--start
trace on
BANCS.OUTPUT.successOrFailure = "S"
sv_a = BANCS.INPUT.IBANNumber
#sv_a = "FR7311689007040088484027995"
  print(sv_a)
  BANCS.INPARAM.IBANNumber = sv_a
  sv_u = urhk_validateIBAN("")
  if (sv_u != 0) then
    BANCS.OUTPUT.successOrFailure = "F"
    exitscript
  endif
  sv_g = BANCS.OUTPARAM.IBAN_modulo
  print(sv_g)
  if (sv_g == 1) then
    print("valid IBAN")
    BANCS.OUTPUT.successOrFailure = "S"
  else
    print("invalid IBAN")
    BANCS.OUTPUT.successOrFailure = "F"
  endif
exitscript
trace off
```

end-->

©Copyright Infosys Ltd.

Finacle Copyright

Insert Record into Tables

This section discusses:

[Create DCTI Record](#)

[Create Entry in BJM](#)

[Create FEL Entry](#)

[Insert Audit Records for Custom Table](#)

[Insert into CFATbls](#)

[UpdateAADFrmPORA](#)

[Insert into GPC Table](#)

[Insert Notification Data into NOTFN Table](#)

[Update Notification Data to NOTFN Table](#)

©Copyright Infosys Ltd.

Create DCTI Record

This userhook create or insert record into DCTI table for Delivery channel transaction Inquiry. Before inserting the record into the table it validates the Acct_id passed. If there is any error in inserting the record into the table the error message is stored in outparam repository.

Syntax:

```
sv_a = urhk_ CreateDCTIRecord("")
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.ReferenceNo	ReferenceNo
BANCS.INPARAM.DelChannelId	DelChannelId
BANCS.INPARAM.DCDeviceId	DCDeviceId
BANCS.INPARAM.DCLocation	DCLocation
BANCS.INPARAM.DCMesgId	DCMesgId
BANCS.INPARAM.TranDate	TranDate
BANCS.INPARAM.DCUserId	DCUserId
BANCS.INPARAM.CustId	CustId
BANCS.INPARAM.SolId	SolId
BANCS.INPARAM.AcctNum	AcctNum
BANCS.INPARAM.TranType	TranType
BANCS.INPARAM.TranAmt	TranAmt
BANCS.INPARAM.Currency	Currency

BANCS.INPARAM.TranStatus	TranStatus
BANCS.INPARAM.TranRmks	TranRmks
BANCS.INPARAM.FreeCode1	FreeCode1
BANCS.INPARAM.FreeCode2	FreeCode2
BANCS.INPARAM.FreeText1	FreeText1
BANCS.INPARAM.FreeText2	FreeText2
BANCS.INPARAM.FreeText3	FreeText3

Output Repository Fields:

Output not present.

Return Value:

This userhook returns '0' on successful completion and on failure it returns '1' and error message is stored in BANCS.OUTPARAM.ErrMsg.

For Example:

```

start
trace on
BANCS.INPARAM.ReferenceNo="ATM12345545403200166556"
BANCS.INPARAM.DelChannelId="BTM"
BANCS.INPARAM.DCDeviceId="DD556"
BANCS.INPARAM.DCLocation="BANGALORE"
BANCS.INPARAM.DCMesgId="MSG6"
BANCS.INPARAM.TranDate="10-12-1998"
BANCS.INPARAM.DCUserId="USER1236"
BANCS.INPARAM.CustId="ACME"
BANCS.INPARAM.SolId="SCGL"

```

```
BANCS.INPARAM.AcctNum="SB4"  
BANCS.INPARAM.TranType="A6"  
BANCS.INPARAM.TranAmt="10000"  
BANCS.INPARAM.Currency="INR"  
BANCS.INPARAM.TranStatus="FINE"  
BANCS.INPARAM.TranRmks="EXCELLENT"  
BANCS.INPARAM.FreeCode1="FREE1"  
BANCS.INPARAM.FreeCode2="FREE2"  
BANCS.INPARAM.FreeText1="FREETEXT1"  
BANCS.INPARAM.FreeText2="FREETEXT2"  
BANCS.INPARAM.FreeText3="FREETEXT3"  
sv_e = urhk_CreateDCTIRecord("")  
print(sv_e)  
if(sv_e != 0 ) then  
print(BANCS.OUTPARAM.ErrMsg)  
trace off  
end-->
```

©Copyright Infosys Ltd.

Create Entry in BJM

This userhook is used to insert a record in BJM table. It is a wrapper over the function createBJMEntry. Here job_exec_date is same as the BOD date. Run_for_date is the date if there was a holiday on a particular day and the batch job is run for the previous day. Run_Srl_Num is the number of a batch job which gets incremented if all the keys are same to avoid any confusion to know which batch job was run and when. Exec_db_stat_code is invoking process, that is, either it can be User Invoked, Sol level, DC level process. Exec_start_time is the system when the user executes a batch job. Return value of createBJMEntry is either '0' or '1'.

Syntax:

```
sv_b = urhk_createEntryInBJM(“”)
```

Input Arguments:

job_exec_date

run_for_date

sol_id

run_srl_num

job_group

job_id

bank_id

exec_db_stat_code

job_desc

exec_start_time

exec_end_time

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.job_exec_date	job_exec_date
BANCS.INPARAM.run_for_date	run_for_date
BANCS.INPARAM.sol_id	sol_id
BANCS.INPARAM.run_srl_num	run_srl_num
BANCS.INPARAM.job_group	job_group
BANCS.INPARAM.job_id	job_id
BANCS.INPARAM.bank_id	bank_id
BANCS.INPARAM.exec_db_stat_code	exec_db_stat_code
BANCS.INPARAM.job_desc	job_desc
BANCS.INPARAM.exec_start_time	exec_start_time

Output Repository Fields:

Output not present.

Return Value:

None

For Example:

The piece of code shows how the record is inserted into BJM table through the userhook createBJMEntry.

```
sv_a = "13-JAN-2001"
```

```
sv_b = "13-JAN-2001"
```

```
sv_c = "106"
```

```
sv_d = "001"
```

```
sv_e = "001"
sv_f = "AM1"
sv_g = "U"
sv_h = "TEST"
sv_i = "13-JAN-2000"
sv_j = "14-JAN-2000"
sv_k = "N"
sv_m = "01"
BANCS.INPARAM.job_exec_date = sv_a
BANCS.INPARAM.run_for_date = sv_b
BANCS.INPARAM.sol_id = sv_c
BANCS.INPARAM.run_srl_num = sv_d
BANCS.INPARAM.job_group = sv_e
BANCS.INPARAM.job_id = sv_f
BANCS.INPARAM.exec_db_stat_code = sv_g
BANCS.INPARAM.job_desc = sv_h
BANCS.INPARAM.exec_start_time = sv_i
BANCS.INPARAM.exec_end_time = sv_j
BANCS.INPARAM.job_status = sv_k
BANCS.INPARAM.bank_id = sv_m
sv_q = urhk_createEntryInBJM("")
```

Create FEL Entry

Userhook to put the posting failure reason during SI execution into FEL table.

Syntax:

```
sv_b = urhk_createFelEntry (“”)
```

Input Arguments:

BANCS.INPARAM.acid = (This is the account id. Dumped into the script as BANCS.INPUT.debitAcctIds)

BANCS.INPARAM.entity_id = (This is the SI Serial No. This is dumped into the script as BANCS.INPUT.siSrINum)

BANCS.INPARAM.failure_reason = (This is the actual failure reason dumped into the script as BANCS.INPUT.failure_reason)

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acid	acid
BANCS.INPARAM.entity_id	entity_id
BANCS.INPARAM.failure_reason	failure_reason

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.successOrFailure	successOrFailure

Return Value:

SUCCESS or FAILURE depending on the status of insert in FEL table.

For Example:

B2K_SIFailureReasonCapture.scr

<--start

trace on

print(BANCS.INPUT.SiSrINo)

print(BANCS.INPUT.debitAcctIds)

print(BANCS.INPUT.errorCodes)

print(BANCS.INPUT.errorMessages)

sv_c = BANCS.INPUT.errorMessages

sv_b = getposition(sv_c, "|")

print(sv_b)

sv_a = MID\$(BANCS.INPUT.errorMessages,0,sv_b-1)

print(sv_a)

BANCS.INPARAM.acid = BANCS.INPUT.debitAcctIds

BANCS.INPARAM.entity_id = BANCS.INPUT.SiSrINo

BANCS.INPARAM.failure_reason = sv_a

sv_e = urhk_createFelEntry("")

print(sv_e)

if (sv_e == 0) then

BANCS.OUTPUT.successOrFailure = "S"

else

BANCS.OUTPUT.successOrFailure = "F"

endif

trace off

end-->

©Copyright Infosys Ltd.

Finacle Copyright

Insert Audit Records for Custom Table

The userhook creates the audit records for custom tables based on the input parameters.

Syntax:

```
sv_a = urhk_Audit("")
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.TABLENAME	The name of the custom table. The length of the table name can be up to 7 characters.
BANCS.INPARAM.AUDIT.KEY	The key to the audit table. If the key is a single field, the input is a single field. If the audit key is composite, it is made up of the various fields in the table. The input for a single field audit key is as follows: • AUDIT.KEY.yourkeyfieldname="whatever" • If the key is composite, the input will be: • AUDIT.KEY.yourkeyfield1="first field" • AUDIT.KEY.yourkeyfield2="second key" • AUDIT.KEY.yourlastkeyfield="lastkeyvalule" The hook parses the key appropriately.
BANCS.INPARAM.AUDITSOLID	The SOL ID to be used for the audit. This field is used only if the ACCTNUM is not specified.
BANCS.INPARAM.GLSUBHEAD CODE	The GL sub head code used for the audit.
BANCS.INPARAM.OPERCODE	The code of the function performed. Valid Values:

	A - Add C - Copy M - Modify U - Undelete D - Delete P - Physical delete V - Verify X - Cancel
BANCS.INPARAM.OLD.*	The old value of the fields.
BANCS.INPARAM.NEW.*	The new value of the fields.
BANCS.INPUT.ACCTNUM	The formatted account number to be audited.
BANCS.INPARAM.CALLING_FUNC_NAME	The SRV or Code Block that is creating the Audit Record
BANCS.INPARAM.AUDIT_ENTITY_ID	The Business Entity for which the Audit record is being created

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.SuccessOrFailure	SuccessOrFailure
BANCS.OUTPARAM.errorMesg	errorMesg
BANCS.OUTPARAM.CALLING_FUNC_NAME	Updates the calling_func_name field in the ADT table for the Audit record
BANCS.OUTPARAM.AUDIT_ENTITY_ID	Updates the entity_id field in the ADT table for the Audit record

Return Value:

Return value not available.

Example:

```

<--start
trace on
sv_d = REPEXISTS("AUDIT")

```

```
if (sv_d == 0) then
CREATEREP("AUDIT")
endif
sv_d = CLASSEXISTS("AUDIT","NEW")
if (sv_d == 0) then
CREATECLASS("AUDIT","NEW",5)
endif
sv_d = CLASSEXISTS("AUDIT","OLD")
if (sv_d == 0) then
CREATECLASS("AUDIT","OLD",5)
endif
sv_d = CLASSEXISTS("AUDIT","KEY")
if (sv_d == 0) then
CREATECLASS("AUDIT","KEY",5)
endif
sv_d = REPEXISTS("BANCS")
if (sv_d == 0) then
CREATEREP("BANCS")
endif
sv_d = CLASSEXISTS("BANCS","INPARAM")
if (sv_d == 0) then
CREATECLASS("BANCS","INPARAM",5)
endif
sv_d = urhk_InitAudit("")
print(sv_d)
print(BANCS.OUTPARAM.successOrFailure)
```

```
print(BANCS.OUTPUTPARAM.errorMesg)
AUDIT.OLD.field1="Abcd"
AUDIT.NEW.field1="zyxw"
AUDIT.OLD.field2="oldvalue"
AUDIT.NEW.field2="newvalue"
AUDIT.KEY.field1="first"
AUDIT.KEY.field2="second"
AUDIT.KEY.field3="third"
BANCS.INPUTPARAM.TABLENAME="CUSTOM"
BANCS.INPUTPARAM.ACCTNUM="ALOK-SBA"
BANCS.INPUTPARAM.GLSUBHEADCODE="ALGEN"
BANCS.INPUTPARAM.OPERCODE="X"
sv_d = urhk_Audit("")
print(sv_d)
print(BANCS.OUTPUTPARAM.successOrFailure)
print(BANCS.OUTPUTPARAM.errorMesg)
sv_d = urhk_dbSQL("COMMIT")
print(sv_d)
# Suppose an audit record for an account id SBA10001 is to be
# inserted, the following statements would be required in the
# script:
BANCS.INPUTPARAM.AUDITSOLID="027"
BANCS.INPUTPARAM.TABLENAME="CUSTOM"
BANCS.INPUTPARAM.ACCTNUM="SBA10001"
BANCS.INPUTPARAM.GLSUBHEADCODE="SBGEN"
BANCS.INPUTPARAM.OPERCODE="X"
BANCS.INPUTPARAM.REFNUM="D11189692"
```

BANCS.INPARAM.AUTOVERIFY="N"

BANCS.INPARAM. CALLING_FUNC_NAME="SRV_openSBAcct"

BANCS.INPARAM. AUDIT_ENTITY_ID="SBA10001"

sv_d = urhk_Audit("")

exitscript

end-->

©Copyright Infosys Ltd.

Update Part Transaction Details in Repository

This userhook updates the part tran details for MF transaction. The userhook can be called from any script event and validates transaction ID. A validation for account type is carried out, if the scheme account is found out to be invalid then the script returns SUCCESS without processing further.

If inputs are valid, userhook updates the part transaction details in the MFTDT / MFDHT table based on the tran ID.

Syntax:

```
sv_a = urhk_updatePartTransNum(“”)
```

Input:

- Valid Transaction ID
- MF Transaction Serial Number
- Record date
- Valid Fund ID
- Valid Investment ID
- Payment type
- Valid Folio Number
- Operative Account
- Part Tran Serial Number

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.mfTranId	Valid Transaction ID.

BANCS.INPARAM.mfTranSrlNum	Transaction serial number.
BANCS.INPARAM.recordDate	Record date.
BANCS.INPARAM.fundId	Valid Fund ID from MFFMT.
BANCS.INPARAM.investId	Investment ID from.
BANCS.INPARAM.paymentType	Payment type.
BANCS.INPARAM.folioNum	Folio Number.
BANCS.INPARAM.operAcct	Operative Account
BANCS.INPARAM.partTranSrlNum	Part Tran Serial Number.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.successOrFailure	Success or Failure depending on whether part tran is updated properly in MFTDT or MFHDT table.

Output:

If account is invalid then the script returns SUCCESS without processing/ updating the table.

If the table updation fails in any case then the script returns FAILURE with the failure message as below:

‘The execution of the user hook urhk_updatePartTransNum failed’.

For Example:

```

<--start
    sv_l = BANCS.OUTPARAM.partTranSrlNum
    # For updating MFTDT
        BANCS.INPARAM.mfTranId =
MTT.InputDetails.MFTranId
        BANCS.INPARAM.mfTranSrlNum =
MTT.InputDetails.MFTranSrlNum

```



```

# For updating MFDHT - for Dividend Settlement

        BANCS.INPARAM.recordDate      =
MTT.InputDetails.RecordDate

        BANCS.INPARAM.fundId           =
MTT.InputDetails.FundId

        BANCS.INPARAM.investId         =
MTT.InputDetails.InvestId

        BANCS.INPARAM.paymentType      =
MTT.InputDetails.PaymentType

        BANCS.INPARAM.folioNum         =
MTT.InputDetails.FolioNum

        BANCS.INPARAM.operAcct         =
MTT.InputDetails.TranForacid

        BANCS.INPARAM.partTranSrlNum = sv_l
        sv_y = urhk_updatePartTransNum("")
        if(sv_y != 0) then
            BANCS.OUTPUT.successOrFailure = "F"
            #MSG0318993 = "urhk_updatePartTransNum
returns FAILURE"
            sv_a = urhk_getMSGCodeErrDesc("MSG0318993")
            if(sv_a == 0) then
                BANCS.OUTPUT.scriptError =
BANCS.OUTPARAM.errDesc
            endif
            goto ENDOFSCRIPT
        endif
    end - ->

```

Insert into CFATbls

This functions of the developed **Insert into CFATbls** userhook are:

- Insert records in OCASHT and OCASDT table, with the details related to schedule which is passed to this userhook.

- The table gets updated on the basis of update Indicator(updateInd) passed from script

- If the updateInd is Yes, then both the tables gets updated with the passed values

- If the updateInd is OCASHT_UPDATE('H') or OCASDT_UPDATE('T'), depending on the flag, which is passed from script, then only the requested table gets updated

- Depending on the action passed, the modification or insertion into the table happens for the records

- In the current scenario, default logic of bucket calculation is applied. You can put the required logic in the script and call the update of tables accordingly.

Syntax:

```
sv_b = urhk_insertIntoCFATbls(“”)
```

sv_b: scratch pad variable to store the return value.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.ruleId	Rule Id
BANCS.INPARAM.zoneCode	Zone Code
BANCS.INPARAM.solId	Zone Sol Id
BANCS.INPARAM.zoneDate	Zone Date

BANCS.INPARAM.setNum	Set Number
BANCS.INPARAM.endInstr	Endorsed Instrument
BANCS.INPARAM.redInstr	Re deposited Instrument
BANCS.INPARAM.doubtInCol	Doubt in Collectability
BANCS.INPARAM.emrgncyCond	Emergency Condition
BANCS.INPARAM.tranCode	Tran Code
BANCS.INPARAM.ociCrncyCode	OCI Currency Code
BANCS.INPARAM.instrmntAmt	Instrument Amount
BANCS.INPARAM.ociRcreTime	OCI Recreation Time
BANCS.INPARAM.valueDate	Value Date
BANCS.INPARAM.instrSrlNum	Instrument Serial Number
BANCS.INPARAM.instrId	Instrument ID
BANCS.INPARAM.instrSchdlSrlNum	Instrument schedule serial Number
BANCS.INPARAM.instrLocType	Instrument Location type
BANCS.INPARAM.depLocType	Deposit Location type
BANCS.INPARAM.bankCode	Bank Code
BANCS.INPARAM.brCode	Branch Code
BANCS.INPARAM.tranValueDate	Transaction Value Date
BANCS.INPARAM.shdBalCode	Schedule Balance Date
BANCS.INPARAM.repCode	Repository Code
BANCS.INPARAM.sortCode	Sort Code
BANCS.INPARAM.acctSolId	Account Sol Id
BANCS.INPARAM.payingAcctId	Paying Account ID
BANCS.INPARAM.futCrFlagAtTranPost	Future Credit Flag at Transaction Posting
BANCS.INPARAM.issExtnCntr	Issuing extension Centre
BANCS.INPARAM.issBankCode	Issuing Bank Code
BANCS.INPARAM.issBrCode	Issuing Branch Code
BANCS.INPARAM.extnCode	Extension Code
BANCS.INPARAM.extnNumOfDays	Extended Number of days
BANCS.INPARAM.acid	Acid

BANCS.INPARAM.ocpCrncyCode	Ocp Currency Code
BANCS.INPARAM.partTranAmt	Part tran Amount
BANCS.INPARAM.acctCrncyCode	Account Currency Code
BANCS.INPARAM.ocpRcreTime	Ocp recreation Time
BANCS.INPARAM.partTranSrlNum	Part Tran Serial Number
BANCS.INPARAM.MRHTMatchRuleCond	MRHT Match Rule Condition
BANCS.INPARAM.GAMSchmCode	GAM scheme code
BANCS.INPARAM.GAMAcctOpnDate	GAM Account Open Date
BANCS.INPARAM.ocpRecCount	Ocp Record Count
BANCS.INPARAM.ociRecCount	Oci Record Count
BANCS.INPARAM.updOCTbl	Updated OC Table
BANCS.INPARAM.action	Action

Userhook Output:

Returns Success if all records inserted successfully.

©Copyright Infosys Ltd.

updateAADFrmPORA

The functionalities of **updateAADFrmPORA** userhook are:

- Move records from PORA to ADD.
- This happens for a given PO number and EntrySrlNum of AED table.
- Input values for this script is Paysys Id, PO Number and AED Entry Srl Num.
- The Output is Success or Failure.

Syntax:

```
sv_b = urhk_updateAADFrmPORA("")
```

sv_b: scratch pad variable to store the return value.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.PaysysId	Paysys Id (Mandatory)
BANCS.INPARAM.RoutedPaysysRefNum	Entry Srl Num of AED Table (Mandatory)
BANCS.INPARAM.PymtRefNum	PO Num of PORA Table

Userhook Output:

Returns Success, if input record gets added successfully else, returns Failure.

Insert into GPC Table

This functionalities of the developed insert into GPC Table userhook are:

- Insert 2 records in GPC based on MPTC and HPRCM, one with common group code and all the other with group code similar to reason_id / payment_code.

- For example, if MPTC menu option is used to create payment code ACH1, then on verification, GPC will have to do entities, both with payment_code as ACH1 but with group_id as ACH1 and ALL respectively.

Syntax:

```
sv_b = urhk_insertIntoGPC("")
```

sv_b: scratch pad variable to store the return value.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.payment_code	paymentCode (Mandatory)
BANCS.INPARAM.entity_type	entityType (Mandatory)
BANCS.INPARAM.bank_id	bankId (Mandatory)
BANCS.INPARAM.group_code	groupCode (Mandatory)
BANCS.INPARAM.group_code_desc	groupCodeDesc (Mandatory)

Userhook Output:

Returns Success if buffman insert into GPC succeeds, else failure.

Insert Notification Data into NOTFN Table

Insert Notification Data into NOTFN Table userhook is called from the scripTHOOK mentioned above. This userhook takes all the data passed from the script and insert the same in the notification table. It generates the serial number for the notification request and it sends it as output. The output has a success or failure flag also.

If inputs are valid, then the end of day Notification data is populated into Finacle repository.

Syntax:

```
sv_a = urhk_insertRecIntoNOTFN ("" )
```

sv_a: scratch pad variable to store the return value.

Input Argument:

◆Notification Data

The above input must be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.	CIF id of customer to whom notification has to be sent.
BANCS.INPARAM.notificationAcct	Account of customer for which notification has to be sent
BANCS.INPARAM.notifnRefId	The entity on which event is taking place. ACH refNum, PO RefNum etc.
BANCS.INPARAM.notificationType	Action specification like PO, ACH ...
BANCS.INPARAM.notifnSubAction1	Sub Action for the action if event is ACH then e.g., 'Inward', 'Outward'
BANCS.INPARAM.notifnSubAction2	Sub Action for the action if event is ACH

	outward then e.g., 'Credit', 'Debit'
BANCS.INPARAM.notificationInitiateDate	Date on which notification has to be sent
BANCS.INPARAM.notificationTransactionType	Whether it is financial or non financial transaction.
BANCS.INPARAM.notificationDataFields	This will have the data which will be passed as pipe separated. This will have the generic data which can be useful for capturing and would vary from case to case basis.
BANCS.INPARAM.notificationAmount	Amount of notification if applicable
BANCS.INPARAM.notificationCurrency	Currency for the notification if applicable
BANCS.INPARAM.Notification Type	Type of notification which needs to be sent whether Mail/Advice/SMS etc.
BANCS.INPARAM.notificationCustomData	Custom data which needs to be sent.
BANCS.INPARAM.notificationFreeText1	Will be used for Customization
BANCS.INPARAM.notificationFreeText2	Will be used for Customization
BANCS.INPARAM.Notification Custom Flag	Will be used for Customization
BANCS.INPARAM.notificationCustomDate	Custom date for customization

Output Repository Fields:

Field Name	Description
BANCS.OUTPUT.entrySrlNum	Entry serial Number for the NOTFN table
BANCS.OUTPUT.successOrFailure	Success or Failure indicator

Return Value:

If the input values are invalid then the return value is 1.

End of day Notification details are populated into the repository.

For Example:

To insert the NOTFN table with the Notification data:

```
<--start
```

```
# Populate Notification data into Inparam
```



```
BANCS.INPARAM.cifId = sv_c
sv_b = urhk_SRV_GetVal ("achcodCrit.achTranRefNum")
BANCS.INPARAM.notifnRefId = BANCS.OUTPARAM.srvValue
BANCS.INPARAM.notificationType = "Advice"
BANCS.INPARAM.notifnAction = "A"
sv_c = "D"
BANCS.INPARAM.notifnSubAction1 = sv_c
sv_d = "O"
BANCS.INPARAM.notifnSubAction2 = sv_d
sv_a = urhk_insertRecIntoNOTFN ("")
end-->
```

©Copyright Infosys Ltd.

Update Notification Data to NOTFN Table

Update Notification Data to NOTFN Table userhook is called from the script hook mentioned above. This userhook is used for future enhancement or for customization. This script is used in modification scenario and is used for updating the new values in notification table. It takes the serial number in NOTFN table as key field and other given details as input to update the table.

If inputs are valid, then the end of day Notification data is populated into Finacle repository.

Syntax:

```
sv_a = urhk_updateRecIntoNOTFN ("" )
```

sv_a: scratch pad variable to store the return value.

Input Argument:

◆Notification Data

The above input must be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.	CIF ID of customer to whom notification has to be sent.
BANCS.INPARAM.notificationAcct	Account of customer for which notification has to be sent
BANCS.INPARAM.notifnRefId	The entity on which event is taking place. ACH refNum,PO RefNum etc.
BANCS.INPARAM.notificationType	Action specification like PO, ACH ...
BANCS.INPARAM.notifnSubAction1	Sub Action for the action if event is ACH then e.g., 'Inward', 'Outward'

BANCS.INPARAM.notifnSubAction2	Sub Action for the action if event is ACH outward then e.g., 'Credit', 'Debit'
BANCS.INPARAM.notificationInitiateDate	Date on which notification has to be sent
BANCS.INPARAM.notificationTransactionType	Whether it is financial or non financial transaction.
BANCS.INPARAM.notificationDataFields	This will have the data which will be passed as pipe separated. This will have the generic data which can be useful for capturing and would vary from case to case basis.
BANCS.INPARAM.notificationAmount	Amount of notification if applicable
BANCS.INPARAM.notificationCurrency	Currency for the notification if applicable
BANCS.INPARAM.Notification Type	Type of notification which needs to be sent whether Mail/Advice/SMS etc.
BANCS.INPARAM.notificationCustomData	Custom data which needs to be sent.
BANCS.INPARAM.notificationFreeText1	Will be used for Customization
BANCS.INPARAM.notificationFreeText2	Will be used for Customization
BANCS.INPARAM.Notification Custom Flag	Will be used for Customization
BANCS.INPARAM.notificationCustomDate	Custom date for customization

Output Repository Fields:

Field Name	Description
BANCS.OUTPUT.entrySrlNum	Entry serial Number for the NOTFN table
BANCS.OUTPUT.successOrFailure	Success or Failure indicator

Return Value:

If the input values are invalid then the return value is 1.

End of day Notification details are populated into the repository.

For Example:

To update the NOTFN table with the Notification data:

```
<--start
```

```
# Populate Notification data into Inparam
BANCS.INPARAM.cifId = sv_c
sv_b = urhk_SRV_GetVal ("achcodCrit.achTranRefNum")
BANCS.INPARAM.notifnRefId = BANCS.OUTPARAM.srvValue
BANCS.INPARAM.notificationType = "Advice"
BANCS.INPARAM.notifnAction = "A"
sv_c = "D"
BANCS.INPARAM.notifnSubAction1 = sv_c
sv_d = "O"
BANCS.INPARAM.notifnSubAction2 = sv_d
sv_a = urhk_updateRecIntoNOTFN ("")
end-->
```

©Copyright Infosys Ltd.

Interest Related Userhooks

This section discusses:

[Get Derived Interest Amount](#)

©Copyright Infosys Ltd.

Get Derived Interest Amount

This user hook is used to get the total interest applicable on an account based on the inputs such as the rate of interest, the time period, the type of interest, and so on.

Syntax

```
sv_b=urhk_getDerivedInteresAmt(“”)
```

Input Arguments:

Valid Account Number, mandatory

Interest Rate, mandatory

Interest Type

Comp Frequency

Term, conditionally mandatory

Start Date(mandatory if term not given)

End Date (mandatory if term not given)

CurrencyCode

Scheme Code

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.foracid	The account for which the interest calculation is to be done.
BANCS.INPARAM.intRate	This field will specify the interest rate using which the interest needs to be calculated. Note: <i>This rate would be compared to the <nrml_pcmt_cr> field in the ITC table and if the rate entered in the input is not the latest then the rate of the ITC table would be used.</i>

BANCS.INPARAM.intType	Interest Type. Note: <i>If this field does not contain any value, then based on the type of account the value of this field is set. In case the account is an operative type of account, the intType is set to <Simple Daily> and in case the account is of TDA type, the <int_method> field from TID table is used as the intType for processing.</i>
BANCS.INPARAM.compFreq	Comp Frequency. Note: <i>In case this field does not contain any value, then the value of the <int_freq_type_cr> field of GSP table is used by default.</i>
BANCS.INPARAM.term	This field specifies the period for which the amount is to be calculated. Note: <i>In case the start and end date are not specified the period beings from BOD and ends at BOD+term.</i>
BANCS.INPARAM.startDate	Start Date Note: <i>In case the BANCS.INPARAM.term field is null, this field is needed to signify the starting date of the period for which the interest is to be calculated.</i>
BANCS.INPARAM.endDate	End Date Note: <i>In case the BANCS.INPARAM.term field is null, this field is needed to signify the end date of the period for which the interest is to be calculated.</i>
BANCS.INPARAM.crncyCode	Currency Code
BANCS.INPARAM.schmCode	Scheme Code

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.totalIntAmt.	The total interest amount based on the input parameters.

©Copyright Infosys Ltd.

Finacle Copyright

MTT Userhooks

This section discusses:

[Check if Customer Instructions Exists](#)

[Define Part Transaction](#)

[Define Transaction](#)

[Distribute Amount as per Customer Instruction](#)

[End Definition of a Transaction or Part Transaction](#)

[Get Additional Input Details in MTT](#)

[MTTS Define Charge Part Transaction](#)

[MTTS_Get PTT Records](#)

[MTTS_Initialise_PTT](#)

[MTTS_HandleExceptions](#)

[Populate Input Details](#)

[Report Error Condition](#)

[Initialize MTTS](#)

[Close MTTS](#)

[Get CIF ID](#)

[Get Document Details of a Customer](#)

©Copyright Infosys Ltd.

Check if Customer Instructions Exists

This script hook is used to set customer instructions for various flow codes. For each flow code, the user can set up a payment system whereby the amount calculated for the various flow codes can be distributed as per the payment system. If any amount remains after distribution, it is credited to another account as per the payment system. If no credit account is mentioned, the amount is credited to the same account in case of interest run. In case of account closure, the amount is credited to an office account when the credit account is not mentioned at account level. This hook checks for customer instructions for a given account and instruction type. The hook returns '0' if customer instruction exists.

Syntax:

```
sv_a = urhk_MTTS_CheckIfCustInstExists("<LABEL>")
```

Where LABEL is not used within the userhook.

Input Arguments:

The defined input is available in a repository called MTT.PayCollDetails.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value not available.

©Copyright Infosys Ltd.

Finacle Copyright

DEFINEPARTTRAN

This script hook accepts all data required to define a complete part transaction. This function allows the script to initiate a financial transaction by defining some of its attributes. The attributes are first made available in the MTT.PartTranDetails class. A call is made to define the part transaction after that. The class variables are identified in the input repository fields section.

Syntax:

```
sv_a = urhk_MTTT_DefinePartTran (Variable)
```

Where the variable can be a scratch pad variable (sv_b) or a string like XYZ.

Input Arguments:

The input string contains the identification of the part-transaction. This is used to consolidate multiple part-transactions into one part-transaction for all part-transactions where consolidate flag is set.

Example - "XYZ"

Input Repository Fields:

It fields accepted from the PartTranDetails class are:

Field Name	Description
MTT.PartTranDetails.Account	The account number (foracid) of the account to be debited or credited.
MTT.PartTranDetails.RefAmount	The amount to be debited or credited. Positive amount indicates credit, while a negative amount indicates a debit.
MTT.PartTranDetails.ValueDateDiff	The value date of the part-transaction in terms of number of days from BOD date (+ or -).

MTT.PartTranDetails.Refnum	Corresponds to ref_num in DTD table. Max 20 characters.
MTT.PartTranDetails.RefCurrency	The currency that the amount is in.
MTT.PartTranDetails.RateCode	Rate code to be used in case the account currency is not them same as the amount currency.
MTT.PartTranDetails.Rate	The rate of conversion can be specified instead of a rate code. If both are specified, then the rate is used.
MTT.PartTranDetails.ReversalDateDiff	The date in terms of number of days past BOD date.
MTT.PartTranDetails.Remarks	Remarks for the part-transaction. Max of 30 characters.
MTT.PartTranDetails.Particulars	Particulars for the part-transaction. Max of 50 characters.
MTT.PartTranDetails.CustomerId	Customer ID of the part-transaction if other than customer who owns the account.
MTT.PartTranDetails.ReportCode	Valid report code for the part-transaction.
MTT.PartTranDetails.PrintAdviceInd	Flag indicating whether advice to customer needs to be printed or not.
MTT.PartTranDetails.PtranBusinessType	This is specific to the MTT event. Each event describes the additional processing that happens for special values of this field, in screvent.doc.
MTT.PartTranDetails.Consolidate	Flag indicating if this part-transaction has to be consolidated with other part-transactions which have the same part-transaction identifier.
MTT.PartTranDetails.ProxyPstgInd	Flag indicating whether this part-transaction is allowed to be proxy-posted, if required.
MTT.PartTranDetails.TODRefType	Reference type of the TOD (which is set up at scheme level) that must be used in case of insufficient balance in the account. If this field is set, automatic TOD is generated in case of shortfall.
MTT.PartTranDetails.TODAmount	The amount of TOD to be granted. The TOD amount can be calculated by applying user logic in the script and populated into

	this field so that the TOD is granted for the exact amount populated into this field. To grant TOD it is not only sufficient to populate TODAmount, TODRefType should also be populated. If the TOD amount is not populated in the script but TODRefType is alone populated then system calculates the TOD amount (this amount is nothing but insufficient balance required to post the current part transaction) that is to be granted.
MTT.PartTranDetails.CarveAmtFlg	If set to "Y", the system automatically carves the amount during transaction creation that is automatically removed when the part-transaction is posted. This is used to lock sufficient funds in the account, even if posting fails.
MTT.PartTranDetails.TranIdentifier	The identifier of the transaction to which this part-transaction belongs.
MTT.PartTranDetails.ProxyReversalDate	In case the transaction is being proxy posted and the reversal should happen on a future date (this is also taken as the difference between BOD date).
MTT.PartTranDetails.ProxyReversalValueDate	In case the transaction is being proxy posted and the reversal transaction must have a different value date.
MTT.PartTranDetails.ProxyEventType	The proxy event type for that part transaction if the part transaction is being proxy posted.
MTT.PartTranDetails.ProxyForacid	The proxy account on which the proxy transaction has to be created.
MTT.HOAddnlDetails.HOTranType	The transaction type of the HO transaction (Originating , responding).
MTT.HOAddnlDetails.OrgTranId	Original transaction ID (valid for HO reversal transactions).
MTT.HOAddnlDetails.OrgTranDate	Original transaction date (valid for HO reversal transactions).
MTT.HOAddnlDetails.OrgPTranSrINum	Valid for HO reversal transactions.
MTT.HOAddnlDetails.CategoryCode	Transaction category code.

MTT.HOAddnlDetails.ToFromBankCode	To from Bank code.
MTT.HOAddnlDetails.ToFromBranchCode	To from branch code.
MTT.HOAddnlDetails.AdvcExtnCntrCode	Extension counter code.
MTT.HOAddnlDetails.AdviceInd	Advice indicator of the HO part transaction.
MTT.HOAddnlDetails.AdviceNumber	Advice number of the HO part transaction.
MTT.HOAddnlDetails.BarAdviceDate	Bar or advice date of the HO part transaction.
MTT.HOAddnlDetails.BillNumber	Bill number of the HO part transaction.
MTT.HOAddnlDetails.HeaderTextCode	Header text code.
MTT.HOAddnlDetails.RecvAdvcDate	Received Advice date of the HO part transaction.
MTT.HOAddnlDetails.RecvAdvcNum	Received advice number
MTT.HOAddnlDetails.DuplicateAdvcFlg	Flag, whether the advice is duplicate or not.
MTT.HOAddnlDetails.Particular1	Particulars field of HO part transaction.
MTT.HOAddnlDetails.Particular2	Particulars field of HO part transaction.
MTT.HOAddnlDetails.Particular3	Particulars field of HO part transaction.
MTT.HOAddnlDetails.Particular4	Particulars field of HO part transaction.
MTT.HOAddnlDetails.Particular5	Particulars field of HO part transaction.
MTT.HOAddnlDetails.amtLine1	Amount.
MTT.HOAddnlDetails.amtLine2	Amount.
MTT.HOAddnlDetails.amtLine3	Amount.
MTT.HOAddnlDetails.amtLine4	Amount.
MTT.HOAddnlDetails.amtLine5	Amount.
MTT.HOAddnlDetails.BatRemarks	Remarks of the HO part transaction Additional details.
MTT.HOAddnlDetails.PayAcct	The account belonging to the other bank or branch.
MTT.HOAddnlDetails.HeaderFreeText	Header free text of the HO part transaction.
MTT.LAAddnlDetails.DemandFlowId	Valid loan demand flow ID. This flow ID must be of 'Collection" flow nature for credit part transactions. Basically this is the collection flow ID, the offset method and

	offset sequence associated with this flow ID is used to squaring of demand in particular order. For debit part transactions the flow ID must be of "Demand" flow nature and of "Interest"/"Bank Charges"/ "Other Charges" flow type. In this case a demand gets raised with the given flow ID.
MTT.LAAddnlDetails.TypeOfDemand	Valid Values : P - Principal Demands only, I - Interest Demands only, B - Bank Charge Demands Only, O - Other Charge Demands Only A - All demands This field is applicable only for credit part transactions. In addition to the offset method and offset sequence associated with collection flow ID, the value specified in this field is used to identify demands that are to be squared off. This field has significant with interest route flag of loan account. If interest route flag is "L" Keep interest liability in Loan Account, then this field can have any of the above given values. If interest route flag is "O" Keep interest liability in office a/c, then this field can have only "P" as valid value.
MTT.LAAddnlDetails.OriginOfTran	Valid Values : "INT", "LDS". This field is applicable only for credit part transactions. This field is also associated with interest route flag of Loan account. If interest route flag is "O" for loan account then any direct credits to loan account (through HTM) can be controlled by an exception. Moreover, if the credit transaction to loan account is credited by Interest calculation process or Loan Demand Satisfaction Process then this exception is not raised. So to avoid the exception either "INT", "LDS" can be populated into this field.
MTT.LAAddnlDetails.LoanAcctId	Loan account ID on behalf of which transaction is taking place to Loan Interest

	Account. During interest calculation process, interest is calculated for this loan account but interest debit is taking place to loan interest account and during loan demand satisfaction process interest collection is taking place to this loan account and actual credit transaction is taking place to loan interest account.
MTT.TDAddnlDetails.TDFlowCode	Valid deposit flow ID. The transaction that is taking place to deposit account is under this flow ID. Interest calculation process can have the valid values as: II - Interest Inflow IO - Interest Outflow CI - Consolidate Interest Inflow CO - Consolidate Interest Outflow

Output Repository Fields:

Output not present.

Return Value:

The return value is '0' in case the transaction definition is accepted. If not, it returns '1'.

For Example:

#The commission part transaction based on Bill Amount, Bill Currency and Commission Code is:

```
if (MTT.InputDetails.CommissionCode != "") then
```

```
BANCS.INPARAM.InputAmount=MTT.InputDetails.BillAmount
```

```
BANCS.INPARAM.CurrencyCode=MTT.InputDetails.HomeCurrency
```

```
BANCS.INPARAM.AmountTableCode=MTT.InputDetails.CommissionCo
```

```
sv_r = urhk_B2k_GetASTMAmount("")
```

```
sv_z = CDOUBLE(BANCS.OUTPARAM.OutputAmount)
```

```
MTT.PartTranDetails.RefAmount=CDOUBLE(BANCS.OUTPARAM.Output
```

MTT.PartTranDetails.Account=MTT.InputDetails.CommAcct
sv_z = MTT.InputDetails.CommAcct
MTT.PartTranDetails.RefCurrency=MTT.InputDetails.HomeCurrency
MTT.PartTranDetails.ValueDateDiff="0"
MTT.PartTranDetails.Consolidate="N"
MTT.PartTranDetails.TranIdentifier

©Copyright Infosys Ltd.

Define Transaction

You can use this script hook to initiate a transaction. This function allows the script to initiate a financial transaction by defining some of its attributes. A name is used while the transaction is being defined so that multiple transactions can be defined in the same script. Each transaction having a unique identifier string value.

Syntax:

`sv_a = urhk_MTTTS_DefineTransaction (Variable)`

The variable can be a scratch pad variable (sv_b) or a string like XYZ.

Input Arguments:

Input String contains the identification of the transaction. This identifier must be referred to while defining the part-transaction to link the part-transaction to the transaction.

Example - "XYZ"

Input Repository Fields:

It inputs accepted from the TranDetails class are:

Click	To
MTT.TranDetails.TranDateDiff	The date of the transaction. This has to be specified as the number of days relative to the BOD date. The value for this field will be 0 if the transaction is being created as of that day.
MTT.TranDetails.TranType	The tran type for the particular event. This field does not support back dated transactions.
MTT.TranDetails.TranSubType	The tran sub type for the particular event.
MTT.TranDetails.TranEventType	The event type of the particular transaction.
MTT.TranDetails.Remarks	Any remarks about the transaction.

MTT.TranDetails.IgnoreExcp	Flag which indicates if exceptions can be overridden or not.
MTT.TranDetails.Consolidate	Indicates that some of the part transactions under the particular transaction will be consolidated.
MTT.TranDetails.ReversalFlg	Flag which indicates if a reversal transaction is to be generated for the current transaction or not.
MTT.TranDetails.TranIdOnline	Flag which indicates whether creation of manual transaction ID's is enabled or not.

Output Repository Fields:

Output not present.

Return value:

The return value is '0' in case the transaction definition is accepted. If not, it is '1'.

Note:

1. Flexibility has been provided to the user to generate manual or system generated transaction at MTT level. This is made available with the help of the field TranIdOnline available in MTT.TranDetails. This can be set to Y in case the user wants to generate manual transaction. This can be set in the script as shown :

MTT.TranDetails.TranIdOnline = Y If this value is not set or set to N, the system will generate the transaction ids.

2. Unlike the use of **get_next_system_ft_tran_num** to generate system transaction ids, all the menus identified for manual generated transaction ids will call **get_next_online_ft_num**. If the event is invoked from BJS, then system transaction id will be generated. If it is invoked by a user, manual transaction id will be generated.

Distribute Amount as Per Customer Instruction

This script hook is used in places where customer instructions like interest payments are set up. This hook provides a facility to first check if the customer instructions are specified for a particular account and then distribute the amount as per the set up. The user must maintain a default account to which the remaining amount is credited or debited. This function allows the distribution of a given amount as per customer instructions. There is a facility to check if customer instructions have been specified for a given account and if so, distribute the amount as per the instructions. The amount remaining is assigned to a default account.

Syntax:

```
sv_a = urhk_MTTs_DefineAsPerCustomerInstruction ("" )
```

Input Arguments:

The defined input is available in a repository called MTT.PayCollDetails.

Input Repository Fields:

Field Name	Description
MTT.PayCollDetails.PayCollCrncy	Currency code of the amount that needs to be distributed as per customer instructions.
MTT.PayCollDetails.AccountNo	The account number for which the customer instructions are to be checked.
MTT.PayCollDetails.RemainAmtAccountNo	Default account number to which the remaining amount is to be put in after completion of customer instructions.
MTT.PayCollDetails.PayCollAmount	The amount which should be distributed as per the customer instructions.
MTT.PayCollDetails.TranIdentifier	Unique transaction name given (in the MTTs_DefineTransaction script hook) to the

	transaction to which all customer part transactions has to be added.
MTT.PayCollDetails.PrB2kType	The flow code for which the payments system records will be fetched will be fetched for the account for which interest run is being done.

Output Repository Fields:

Output not present.

Return Value:

The return value is '0' if customer instruction exists, else it returns '1'.

©Copyright Infosys Ltd.

End Definition of a Transaction or Part Transaction

This script hook is required if it has a loop for processing multiple entities. It signifies the end of the complete definition for a given entity and the start of the processing for the next entity. This script hook also requires no inputs. This function signals the end of definition for a part-transaction or transaction. It is needed only if the script has a loop for processing multiple entities. This hook ensures that a part transaction is created in internal storage variables within the application based on the MTT.PartTranDetails class variables populated.

Syntax:

```
sv_a = urhk_MTTS_EndDefinition ("" )
```

Input Arguments:

There is no input for this function.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The return value is '0' in case the processing is successful. If not, it returns '1'.

Get Additional Input Details in MTT

Using this function multiple additional inputs can be obtained in MTT script. Currently this is enabled in FBM/IRM/ORM to get conversion screen details. Using this function total credit amount to customer can be obtained.

Syntax:

```
sv_r = urhk_MTTs_PopulateSubInputDetails("")
```

Input Arguments:

None

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

RefAmount, RefCurrency, Amount, TranCurrency, PtranBusinessType, RefPtranSrlNum, ProtectValue, Rate, Ratecode, Particulars, Remarks, ValueDate (diff from bod date) and Refnum

All these fields gets populated in PartTranDetails Class of MTT repository in case of FBM/IRM/ORM menu option.

See Lodge a Foreign Bill in the Foreign Bills module for FBM menu option details.

See Lodge and Realize Inward Telegraphic Transfer in the Remittances module for IRM menu option details.

See Issue an Outward Telegraphic Transfer in the Remittances module for ORM menu option details.

For Example:

```
sv_r = urhk_MTTs_PopulateSubInputDetails("")
while(sv_r < 1)
print (MTT.PartTranDetails.RefAmount)
print(MTT.PartTranDetails.PTranBusinessType)
MTT.PartTranDetails.Consolidate="N"
MTT.PartTranDetails.TranIdentifier="FBCharges"
sv_r = urhk_MTTs_PopulateSubInputDetails("")
do
```

©Copyright Infosys Ltd.

MTTS Define Charge Part Transaction

You can use this script hook to initiate a transaction. This puts the relevant charge records also. This function allows the script to initiate a financial transaction by defining some of its attributes along with the Charge Record. The attributes are first made available in the MTT.PartTranDetails class and MTT.ChargeDetails class. If fields from the MTT.ChargeDetails class related to the Charge Record available then this userhook adds the data pertaining to the charge record into the database.

Syntax:

```
sv_a = urhk_MTTT_DefineChargePartTran(Variable)
```

The variable can be a scratch pad variable or a string. This variable is used as the Part Transaction identifier by MTT.

Input Arguments:

The input required for this userhook is the input required for urhk_MTTT_DefinePartTran. In addition to these the details required in the MTT.ChargeDetails is mentioned in the Input Repository Fields section.

Input Repository Fields:

Field Name	Description
MTT.ChargeDetails.EventType	The event type for the account for the charge that is being calculated. String(5)
MTT.ChargeDetails.EventId	The event ID for the account for the charge that is being calculated. String(25)
MTT.ChargeDetails.CollectionCrncy	The currency in which the charge is to be collected String(3)

MTT.ChargeDetails.CalculatedAmount	The charge amount as calculated by the system. String(35)
MTT.ChargeDetails.ActualAmount	The actual charge amount that is charged to the customer. String(35)
MTT.ChargeDetails.ServiceSolld	The solid ID for the transaction. String(8)
MTT.ChargeDetails.ChrgPrtcls	The charge particulars for the charge details. String(50)
MTT.ChargeDetails.ChrgRmks	The charge remarks for the charge details. String(30)
MTT.ChargeDetails.PtranBusType	This is specific to the MTT event. Each event describes the additional processing that happens for special values of this field. String5()
MTT.ChargeDetails.RateCode1	The rate code that is used in case the calculation and collection currency of the charge amounts are different. String(5)
MTT.ChargeDetails.ModifyFlg	This flag indicates whether the charges calculated by the system can be modified by the user or not. The valid values are Y and N. Char(1)
MTT.ChargeDetails.ConsolidatedDr	This flag indicates whether the charges calculated by the system that has to be debited to customer account is to be consolidated or not. The valid values are Y and N. Char(1)
MTT.ChargeDetails.ChargeCustld	The customer ID for whom the charge is being calculated. String(9)
MTT.ChargeDetails.Rate1	The rate to be used in case the calculation and collection currency of the charge amounts are different
MTT.ChargeDetails.ParentTranIdentifier	The Tran Identifier for the transaction String(20)
MTT.ChargeDetails.ParentPartTranIdentifier	The Part Tran identifier for the transaction. String(20)

Output Repository Fields:

Output not present.

Return Value:

The return value is '0' in case the transaction definition is accepted. If not, it returns a non-zero value.

For a sample script refer to the sample scripts directory in \$TBA_PROD_ROOT/sample/scripts/ bcacctmaintchrg.sscr.

©Copyright Infosys Ltd.

MTTS_Get PTT Records

This userhook is used to access the MCS records that would have been initialised by the userhook MTTS_InitPtt.

See Define Event ID in the Administrator module for MCS menu option details.

Each call to this userhook gets the next record from the PTT link list that was initialised by calling the userhook MTTS_InitPtt.

Syntax:

```
sv_a = urhk_MTTT_GetPttRect(Variable)
```

The variable can be a scratch pad variable or a string.

Input Arguments:

There are no inputs for this userhook. This must be called only after calling the userhook MTTS_InitPtt.

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
MTT.ChargeDetails.EventType	The event type for the account for the charge that is being calculated. String(5)
MTT.ChargeDetails.EventId	The event ID for the account for the charge that is being calculated. String(5)
MTT.ChargeDetails.CalculatedAmount	The charge amount as calculated by the system. String(35)
MTT.ChargeDetails.ActualAmount	The actual charge amount that is charged to the customer. String(35)

MTT.ChargeDetails.CollectionCrncy	The currency in which the charge is collected. String(3)
MTT.ChargeDetails.ServiceSolld	The solid ID for the transaction. String(8)
MTT.ChargeDetails.ChrgPrtcls	The charge particulars for the charge details.v String(50)
MTT.ChargeDetails.ChrgRmks	The charge remarks for the charge details. String(30)
MTT.ChargeDetails.PtranBusType	This is specific to the MTT event. Each event describes the additional processing that happens for special values of this field, in screvent.doc String(5)
MTT.ChargeDetails.RateCode1	The rate code that is used in case the calculation and collection currency of the charge amounts are different. String(5)
MTT.ChargeDetails.ModifyFlg	This flag indicates whether the charges calculated by the system can be modified by the user or not. The valid values are Y and N. Char(1)
MTT.ChargeDetails.ConsolidatedDr	This flag indicates whether the charges calculated by the system that has to be debited to customer account is to be consolidated or not. The valid values are Y and N. Char(1)
MTT.ChargeDetails.ChargeCustId	The customer ID for whom the charge is being calculated. String(9)
MTT.PartTranDetails.Account	The Account to which the charge amount is credited or debited as per the set up in MCS. String(16)
MTT.PartTranDetails.Amount	The Charge amount as calculated by MCS. String(35)
MTT.PartTranDetails.RefAmount	The Ref Amount for the transaction. String(35)
MTT.PartTranDetails.ValueDate	The Difference between the Value date from the transaction date
MTT.PartTranDetails.Refnum	The Ref Num for the part tran. String(20)
MTT.PartTranDetails.RefCurrency	The reference currency for the transaction. String(3)
MTT.PartTranDetails.Rate	The Rate that is used for the part tran. String(35)
MTT.PartTranDetails.ReportCode	The report code for the part tran. String(5)

MTT.PartTranDetails.Remarks	The remarks for the part tran. String(30)
MTT.PartTranDetails.Particulars	The particulars for the part tran. String(50)
MTT.PartTranDetails.PrintAdviceInd	Whether advice needs to be printed or not. Character(1)

Return Value:

The return value is '0' if the call to the userhook is successful. If the call is not successful the return value is '1'.

If the return value is '0', the fields returned by the userhook are as given in the class MTT.ChargeDetails:

The return value is '0' in case the initialisation is done. If not, it is '1'.

For Example:

The sample script can be accessed in sample scripts directory in \$TBA_PROD_ROOT/sample/scripts/ bcacctmaintchrg.sscr.

©Copyright Infosys Ltd.

MTTS_Initialise_PTT

In Finacle charges are setup through the menu option MCS. For a given Event ID and Event Type MCS menu option stores a set of records. This particular userhook initialises the repository fields related to charges class, for given inputs access the data as set up in MCS menu option and create a linklist of PTT records. This userhook should be called before calling the userhook MTTS_GetPttRec.

See Define Event ID in the Administrator module for MCS menu option details.

Syntax:

sv_a = urhk_MTTS_InitPtt(Variable)

Input Arguments:

The input required for this userhook is required in the class MTT.ChargeDetails.

Input Repository Fields:

Field Name	Description
MTT.ChargeDetails.EventType	The event type for the account for the charge that is being calculated.String(5)
MTT.ChargeDetails.EventId	The event ID for the account for the charge that is being calculated.String(25)
MTT.ChargeDetails.ChrgInpCrncy	The input currency is the currency of the account for which the charge is being calculated.String(3)
MTT.ChargeDetails.ChrgInpAmount	The input amount based on which the charge is calculated. This is useful in cases where the MCS set up calculates charge using the percentage method.String(35)
MTT.ChargeDetails.PartyAccount	The account for which the charge is being calculated.String(16)

Output Repository Fields:

Field Name	Description
MTT.ChargeDetails.MultipleCrncyChrg	If this field has the value of "Y", it means that the charges collected are in more than one currency. If the field has the value "N" it means that the charges are in a single currency. Character(1)

Return Value:

The return value is '0' in case the initialisation is done. If not, it is a non zero value. Also when the return value is 0, the field populated in the class MTT.ChargeDetails is mentioned in the Output Repository Fields section.

For Example:

The sample script can be accessed in sample scripts directory in \$TBA_PROD_ROOT/sample/scripts/ bcacctmaintchrg.sscr.

©Copyright Infosys Ltd.

MTTS_HandleExceptions

This user hook is used for handling exceptions related to MTT. User hook accepts the value for how to handle the exception. If the value is P (POST) or E(ENTRY) then function for completing the transaction is called. If this function returns the Failure then user hook returns the FAILURE.

Syntax:

```
sv_r = urhk_MTTT_HandleExceptions("")
```

Input

Input variable is accessed from MTTInputRec structure

Output

Success or failure

For Example:

This is the part of the script mttonlinexcpn.sscr where the user hook is called for handling the exceptions. This script mttonlinexcpn.sscr is present in TBA_SCRIPTS directory

```
MTT.PartTranDetails.RefAmount=42
```

```
MTT.PartTranDetails.Account="ALOK-SBA"
```

```
MTT.PartTranDetails.RefCurrency="INR"
```

```
MTT.PartTranDetails.ValueDate="0"
```

```
MTT.PartTranDetails.Consolidate="Y"
```

```
MTT.PartTranDetails.Particulars="DD Issued to SB1"
```

```
MTT.PartTranDetails.TranIdentifier="TestTransaction"
```

```
MTT.PartTranDetails.PTranBusinessType=""
```

```
sv_r = urhk_MTTTS_DefinePartTran("Customer2")
print(sv_r)
sv_r = urhk_MTTTS_EndDefinition("")
print(sv_r)
sv_r = urhk_MTTTS_HandleExceptions("POST")
print(sv_r)
sv_r = urhk_MTTTS_Close("")
print(sv_r)
end-->
```

©Copyright Infosys Ltd.

Define Loan Part Tran

This is the User hook to get the Part Tran related details from the script. This also calls the functions to get the additional details from the script according to Scheme Type of account. If the inputs are valid, the values are populated in the Finacle repository.

Syntax

```
sv_a = urhk_MTTT_DefineLoanPartTran("")
```

Input Repositories

Field Name	Description
BANCS.INPARAM.Account	Valid Loan Account number in GAM table.
BANCS.INPARAM.RefCurrency	Reference currency
BANCS.INPARAM.RefAmount	Reference amount
BANCS.INPARAM.ValueDate	Value date
BANCS.INPARAM.GIDate	GL date
BANCS.INPARAM.Refnum	Reference number
BANCS.INPARAM.RateCode	Valid rate code
BANCS.INPARAM.Rate	Valid rate
BANCS.INPARAM.Remarks	Remarks
BANCS.INPARAM.Particulars	Particulars
BANCS.INPARAM.Particulars2	Particulars 2
BANCS.INPARAM.ParticularsCode	Particulars code
BANCS.INPARAM.ScriptEventType	Fee Current Collection amount
BANCS.INPARAM.CifId	Fee Tran Particulars

BANCS.INPARAM.ReportCode	Report code
BANCS.INPARAM.PartyCode	Party code
BANCS.INPARAM.PrintAdviceInd	Print Advice indicator
BANCS.INPARAM.PTranBusinessType	Part transaction business type
BANCS.INPARAM.RefPtranSrlNum	Valid Rate code
BANCS.INPARAM.RefTranId	Referral Tran ID
BANCS.INPARAM.RefTranDate	Referral Transaction date
BANCS.INPARAM.RefInputKey	Referral Input key
BANCS.INPARAM.ProtectValue	Protect value
BANCS.INPARAM.Consolidate	Consolidate flag
BANCS.INPARAM.ProxyPstgInd	Proxy Posting indicator
BANCS.INPARAM.ProxyReversalDate	Proxy reversal date
BANCS.INPARAM.ProxyReversalValueDate	Proxy reversal value date
BANCS.INPARAM.ProxyForacid	Proxy foracid
BANCS.INPARAM.ProxyEventType	Proxy event type
BANCS.INPARAM.TODRefType	Temporary overdraft reference type
BANCS.INPARAM.TodEntityType	Temporary overdraft entity type
BANCS.INPARAM.TodLevIntFlg	Temporary overdraft level interest flag
BANCS.INPARAM.TodExpiryDate	Temporary overdraft expiry date
BANCS.INPARAM.TodPenalDate	Temporary overdraft penal date
BANCS.INPARAM.TodNrmlIntPcnt	Temporary overdraft normal interest percent
BANCS.INPARAM.TodPenalIntPcnt	Temporary Penal interest percent
BANCS.INPARAM.TodPermittedBy	Temporary permitted by

BANCS.INPARAM.TodRemarks	Temporary overdraft remarks
BANCS.INPARAM.PSTType	PST type
BANCS.INPARAM.PSTCurrency	PST currency
BANCS.INPARAM.PSTPurpose	PST purpose
BANCS.INPARAM.PSTRemarks	PST remarks
BANCS.INPARAM.PSTRateCode	PST rate code
BANCS.INPARAM.PSTRate	PST rate
BANCS.INPARAM.TODAmount	Temporary overdraft amount
BANCS.INPARAM.FxTranAmount	FX transaction amount
BANCS.INPARAM.LienAmount	Lien amount
BANCS.INPARAM.PSTPurchasedAmount	PST purchased amount
BANCS.INPARAM.PSTSoldAmount	PST sold amount
BANCS.INPARAM.CarveAmtFlg	Carve amount flag
BANCS.INPARAM.LienFlg	Lien flag
BANCS.INPARAM.LienForacid	Lien foracid (account number)
BANCS.INPARAM.LienType	Lien type
BANCS.INPARAM.LienRemarks	Lien Remarks
BANCS.INPARAM.LienArgB2kld	Lien Argument b2k ID
BANCS.INPARAM.ACPARTB2kld	Account Partition b2k ID
BANCS.INPARAM.ACPARTB2kType	Account Partition b2k type
BANCS.INPARAM.BreakConsolOn	Break Consolidation on
BANCS.INPARAM.TranIdentifier	Tran identifier
BANCS.INPARAM.CrncyHolChkFlg	Currency holiday Check flag
BANCS.INPARAM.InstrmntType	Instrument Type

BANCS.INPARAM.InstrmntDate	Instrument Date
BANCS.INPARAM.InstrmntAlpha	Instrument Alpha
BANCS.INPARAM.InstrmntNum	Instrument number
BANCS.INPARAM.TreaRate	Treasury rate
BANCS.INPARAM.TreaRefNum	Treasury Reference number
BANCS.INPARAM.referralId	Referral ID
BANCS.INPARAM.svsTranId	SVS Tran ID
BANCS.INPARAM.EntryUserId	Entry User ID
BANCS.INPARAM.EntryTime	Entry time
BANCS.INPARAM.DemandFlowId	Demand Flow ID
BANCS.INPARAM.TypeOfDemand	Type of Demand
BANCS.INPARAM.LoanAcctId	Loan Account ID
BANCS.INPARAM.OriginOfTran	Origin of Tran
BANCS.INPARAM.ReversalTranId	Reversal Tran ID
BANCS.INPARAM.ReversalTranDate	Reversal Tran date
BANCS.INPARAM.ReversalPtranSrINum	Reversal Part transaction serial number
BANCS.INPARAM.ReversalDmdSrINum	Reversal Demand Serial number
BANCS.INPARAM.ReversalShdINum	Reversal Schedule number
BANCS.INPARAM.PrincipalAmt	Principal amount
BANCS.INPARAM.InterestAmt	Interest amount
BANCS.INPARAM.compInterestAmt	Compound Interest amount
BANCS.INPARAM.penOnInterestAmt	Penal on Interest amount
BANCS.INPARAM.penOnPrincipalAmt	Penal on Principal amount
BANCS.INPARAM.OtherChrgAmt	Other charge amount

BANCS.INPARAM.BankChrgAmt	Bank charge amount
BANCS.INPARAM.DemandSatisfyFlg	Demand satisfy flag
BANCS.INPARAM.AdvancePaymentFlg	Advance payment flag
BANCS.INPARAM.raiseDemandFlg	Raise demand flag
BANCS.INPARAM.DemandAmt	Valid Demand amount
BANCS.INPARAM.oFlowAdjFlg	Overflow Adjustment flag
BANCS.INPARAM.prepayTypeFlg	Prepayment type flag
BANCS.INPARAM.intCollOnPrepayFlg	Interest Collection on Prepayment flag
BANCS.INPARAM.DeferredInterestAmt	Valid Deferred Interest Amount
BANCS.INPARAM.EMICapDefInterestAmt	Valid EMI Cap Deferred Interest amount
BANCS.INPARAM.NormalInterestAmt	Valid Normal Interest Amount
BANCS.INPARAM.PenalInterestAmt	Valid Penal Interest Amount
BANCS.INPARAM.OverdueInterestAmt	Valid Overdue Interest Amount
BANCS.INPARAM.InterestOverdueAmt	Valid Interest Overdue amount.
BANCS.INPARAM.HolidayInterestAmt	Valid Holiday Interest Amount
BANCS.INPARAM.AccruedPenalInterestAmt	Valid Accrued Penal Interest Amount
BANCS.INPARAM.DeferredInterestDemandAmt	Valid Deferred Interest demand amount
BANCS.INPARAM.NormalPrincipalAmt	Valid Normal Principal amount
BANCS.INPARAM.PrincipalOverdueAmt	Valid Principal overdue amount
BANCS.INPARAM.PlannedPrepayAmt	Valid Planned prepayment amount
BANCS.INPARAM.transferType	Transfer type
BANCS.INPARAM.TopUpDisbFlg	Top-up Disbursement flag.
BANCS.INPARAM.intRefundPrePymntFlg	

	Interest Refund of pre-payment flag
BANCS.INPARAM.marginMoneyAmt	Valid Margin money amount
BANCS.INPARAM.retentionAmt	Valid retention amount
BANCS.INPARAM.builderProfitAmt	Valid builder profit amount

Output Repository Fields

Field Name	Description
BANCS.OUTPARAM.ChargeCollAcct	Charge Collection account, that is, credit account for fee collection
BANCS.OUTPARAM.EventId	Valid Event ID
BANCS.OUTPARAM.EventType	Valid Event type
BANCS.OUTPARAM.CompB2kId	CompB2kId which contains acid, Charge type and Serial number
BANCS.OUTPARAM.FeeCalcAmt	Fee System Calculated amount
BANCS.OUTPARAM.FeeCollectionAmt	Fee Current Collection amount
BANCS.OUTPARAM.FeeTranParticular	Fee Tran Particulars
BANCS.OUTPARAM.FeeRateCode	Valid Rate code
BANCS.OUTPARAM.FeeRate	Valid Rate for the rate code

Output

If the input values are invalid then the return value is 1.

©Copyright Infosys Ltd.

Generate Charge Part Transaction

This userhook is used to generate the Charge Part Tran based on the inputs passed. It is used to define the Charge part tran if any for a transaction. It creates the entry in the CXL table for the charge part transactions.

Syntax:

```
sv_a      =      urhk_MTTTS_DefineEntityChargePartTran(<part      tran
Identifier>)
```

Input:

Part Tran Identifier

Input Repository Fields:

MTT Class fields

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.partTranSrlNum	Part Tran Serial number is returned.

Output:

Returns Success on successful creation of part tran.

For Example:

```
<--start
```

```
sv_m = CDOUBLE(MTT.InputDetails.FeeAmt)
```

```
MTT.PartTranDetails.RefCurrency=MTT.InputDetails.ChargeCurrenc
```

```

#MSG0309154= " fee debit"
sv_f = urhk_getMSGCodeErrDesc("MSG0309154")
if(sv_f == 0) then
    MTT.PartTranDetails.Particulars=MTT.InputDetails.chargeAccnt
+ BANCS.OUTPARAM.errDesc
endif
MTT.PartTranDetails.Consolidate="N"
MTT.PartTranDetails.TranIdentifier="BuybackAuth"
MTT.PartTranDetails.Account =
MTT.InputDetails.chargeAccnt
MTT.PartTranDetails.RefAmount= sv_m
MTT.PartTranDetails.PTranBusinessType = "BBAUTH"
MTT.PartTranDetails.sysPartTranCode ="PTRN02667"
MTT.ChargeDetails.EventType = MTT.InputDetails.eventType
MTT.ChargeDetails.EventId = MTT.InputDetails.eventId
sv_x = CDOUBLE(MTT.InputDetails.calcAmt)
MTT.ChargeDetails.CalculatedAmount = sv_x
MTT.ChargeDetails.ActualAmount = sv_m
MTT.ChargeDetails.CollectionCrncy =
MTT.InputDetails.ChargeCurrencyCode
MTT.ChargeDetails.ChargeCifId =
MTT.InputDetails.chargeCifId
MTT.ChargeDetails.TargetAcctId =
MTT.InputDetails.chargeAccnt
MTT.ChargeDetails.CompB2kId = MTT.InputDetails.entityId
+ "#" + MTT.InputDetails.eventType + "#" +
MTT.InputDetails.SrlNum
MTT.ChargeDetails.CompB2kIdType =
MTT.InputDetails.entityType

```

MTT.ChargeDetails.SrlNum = MTT.InputDetails.SrlNum

MTT.ChargeDetails.ChrgRmks = MTT.InputDetails.ChrgRmks

MTT.ChargeDetails.ServiceSolId = BANCS.STDIN.mySolId

#Ref Amount is passed as Positive because this fee is considered as Expense Fee,

#the user hook DefineEntityChargePartTran takes care of Debiting the charge tran

sv_z = urhk_MTTs_DefineEntityChargePartTran("FEE_DEBIT")

end - ->

©Copyright Infosys Ltd.

Populate Input Details

This script hook populates the event specific fields in the Input Details class for the next entity that needs processing. This is needed at the beginning of the script hook and in each loop, if multiple entities need to be processed at a time. This script hook requires no inputs. The return value of the hook must be checked for each of the items to be processed. This function gets all the details put out by the MTT event in repository variables. The details depend on the current event that is in process. This is equivalent to defining the transaction header details in some respects.

Syntax:

```
sv_a = urhk_MTTS_PopulateInputDetails ("")
```

Input Arguments:

There is no input to this function.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The return value is zero (0) in case the processing is successful. If not, it returns '1'. The actual fields depend on the event.

Report Error Condition

This script hook is used to check certain logical conditions that can be considered as an error and results in the transaction not being generated. Such errors are reported back to the application program using this hook. The error message describing the condition is passed in the script hook. This hook accepts the field Repository Name with the value MTT from the Error class. The event specific program handles the appropriate error condition.

For Example:

Any error reported from the script hook is printed in the error report as part of the Interest calculation event.

This function allows the script to report an error condition, based on which the system will abort the processing of the current transaction(s). The handling of this condition is event-specific.

For Example:

The error description is printed in the error report in the Interest Calculation event.

Syntax:

```
sv_a = urhk_MTTTS_ReportErrorCondition ("")
```

Input Arguments:

There is no input string to this function.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The return value is zero (0) in case the transaction definition is accepted. If not, it returns 1.

For Example:

```
#sv_k contains interest to be recovered  
# If there is no balance and there is no operative account  
specified,  
# raise an error condition  
if(sv_k>CDOUBLE(MTT.InputDetails.firstCustAcctAvailAmt))THEN  
if(MTT.InputDetails.drOperAcctIfNoBalFlg=="N")THEN  
MTT.Error.Description="Interest Skipped insuff.Bal"  
sv_r = urhk_MTTS_ReportErrorCondition("")  
ENDIF  
ENDIF
```

©Copyright Infosys Ltd.

Initialize MTTS

This user hook initializes static and global variables before calling the MTTS script.

Syntax:

```
sv_a = urhk_MTTs_Initialize("")
```

Where sv_a is a scratch pad variable to store the return value.

Input Arguments:

There is no input string to this function

Input Repository Fields:

The inputs are to be populated into the following repositories.

Field Name	Repository Name
PostTranFlg	BANCS.INPARAM.PostTranFlg
CommitFlg	BANCS.INPARAM.CommitFlg
InsertTranWithValidnFlg	BANCS.INPARAM.InsertTranWithValidnFlg
RestrictModifyInd	BANCS.INPARAM.RestrictModifyInd
ErrorCheckForLuw	BANCS.INPARAM.ErrorCheckForLuw

Output Repository Fields:

Output not present.

Return Value:

The return value is '0' in case the transaction definition is accepted. If not, it returns a non-zero value.

For a sample script refer to the sample scripts directory in
\$TBA_PROD_ROOT/sample/scripts/mttbatchxcpn.sscr

©Copyright Infosys Ltd.

Close MTTs

This user hook is used to close link lists and delete repositories that were opened when the MTTs was initialized

Syntax:

```
sv_a = urhk_MTTs_Close("")
```

Where sv_a is a scratch pad variable to store the return value.

Input Arguments:

There is no input string to this function

Input Repository Fields:

There is no input Repository string to this function

Output Repository Fields:

Output not present.

Return Value:

The return value is '0' in case the transaction definition is accepted. If not, it returns a non-zero value.

For a sample script refer to the sample scripts directory in \$TBA_PROD_ROOT/sample/scripts/mttbatchxcpn.sscr

©Copyright Infosys Ltd.

Get CIF ID

This userhook is used to fetch all entity document records of a customer.

Syntax:

```
sv_b = urhk_B2k_CustIdToCifId("")
```

Input Arguments:

There is no input string to this function

Input Repository Fields:

BANCS.INPARAM.CustId

Output Repository Fields:

BANCS.OUTPARAM.CifId

Return Value:

This user hook will return 0 on successful completion and 1 on failure. The error message will be stored in BANCS.OUTPARAM.retCode.

For Example

```
BANCS.INPARAM. CustId = "TEST100"  
sv_z = urhk_B2k_CustIdToCifId ("")  
print(sv_z)  
if(sv_z != 0 ) then  
andprint(BANCS.OUTPARAM.ErrMsg)  
else
```

sv_v=BANCS.OUTPARAM. CifId

endif

exitscript

©Copyright Infosys Ltd.

Payment System Userhooks

This section discusses:

- [ACH Userhooks](#)
- [SWIFT Userhooks](#)
- [Fetch Unique Bic For PaysysId](#)
- [Get Address Details For Bic Bank and Branch](#)
- [Fetch Value For Bic, PaysysId And Additional Details](#)
- [Fetch Directory Reference Number For Bic And PaysysId](#)
- [Fetch Additional Details For Directory Reference Number](#)
- [Remaining Value of Prepayment Permitted in a Year](#)
- [Updates Blacklist Status of a Payment](#)

©Copyright Infosys Ltd.

ACH Userhooks

ACH is 'Automatic Clearing House'. There are userhooks in Finacle which enables the handling of different operations like 'Inquire', 'Modify' and 'Delete' related to ACH. Also, default population of details like 'Reason Code' and Status update for ACH is possible with the use of scripting userhooks.

This section discusses:

[Handling Operations from AED Table from Script](#)

[Populate Reason Code for ACH](#)

[Fund Validation for ACH](#)

[Update Entry Status for ACH](#)

©Copyright Infosys Ltd.

Handling Operations from AED Table from Script

The userhook ACH_AED_Op handles operations for AED Table from the script aedOutwardTxnUpload.scr for the create functionality and the script aedInquiry.scr for the inquire, modify and delete functionalities

Syntax:

```
sv_u = urhk_ACH_AED_Op("")
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.PaysysId	The payment system ID.
BANCS.INPARAM.EntrySrlNum	The entry serial number.
BANCS.INPARAM.ReturnFileName	The return file name.
BANCS.INPARAM.ReturnDate	The return date.
BANCS.INPARAM.OrigAcct	The originator account.
BANCS.INPARAM.OrigId	The originator ID.
BANCS.INPARAM.OrigInfo1	Custom originator information.
BANCS.INPARAM.OrigInfo2	Custom originator information.
BANCS.INPARAM.OrigName	The originator's name.
BANCS.INPARAM.SrvCode	The service code.
BANCS.INPARAM.RecvAcct	The receiver account.
BANCS.INPARAM.RecvId	The receiver ID.
BANCS.INPARAM.RecvInfo1	Custom receiver information.
BANCS.INPARAM.RecvInfo2	Custom receiver information.

BANCS.INPARAM.RecvName	The receiver's name.
BANCS.INPARAM.RecvAcid	The receiver's ID.
BANCS.INPARAM.TranAmt	The transaction amount.
BANCS.INPARAM.TranCrncy	The transaction currency.
BANCS.INPARAM.CommissionAmt	The commission amount.
BANCS.INPARAM.TotalAmt	The total amount.
BANCS.INPARAM.OutwardCommission	The outward commission.
BANCS.INPARAM.RejectAmt	The return amount to be passed.
BANCS.INPARAM.OtherAmt1	Custom amount.
BANCS.INPARAM.OtherAmt2	Custom amount.
BANCS.INPARAM.EffectiveDate	The effective date.
BANCS.INPARAM.SettlementDate	The settlement date.
BANCS.INPARAM.DueDate	The due date.
BANCS.INPARAM.Date 1	Custom date.
BANCS.INPARAM.Date 2	Custom date.
BANCS.INPARAM.BankCode	The bank code.
BANCS.INPARAM.BranchCode	BranchCode
BANCS.INPARAM.SolId	The SOL ID of the bank.
BANCS.INPARAM.OrigCode	The originator code.
BANCS.INPARAM.RejCode	The reject code.
BANCS.INPARAM.RejDesc	The rejection description.
BANCS.INPARAM.EntryStatus	The entry status.
BANCS.INPARAM.ReturnStatus	The return status.
BANCS.INPARAM.TxnStatus	The transaction status.
BANCS.INPARAM.FailProcess	Indicates if transaction is automated or manual.
BANCS.INPARAM.status2	Custom status.
BANCS.INPARAM.ReturnInd	The return indicator.
BANCS.INPARAM.CrDrInd	The credit and debit indicator.
BANCS.INPARAM.InwOutwInd	The inward and outward transaction indicator.

BANCS.INPARAM.FreshRetEntry	The fresh return entry.
BANCS.INPARAM.TranCode	The transaction code.
BANCS.INPARAM.CustTranId	The customer transaction ID.
BANCS.INPARAM.CustTranDate	The customer transaction date.
BANCS.INPARAM.SettTranId	The Set transaction ID.
BANCS.INPARAM.SettTranDate	The Set transaction date.
BANCS.INPARAM.TraceNum1	Custom trace number.
BANCS.INPARAM.TraceNum2	Custom trace number.
BANCS.INPARAM.TraceNum3	Custom trace number.
BANCS.INPARAM.ApprovedBy	The name of the approver.
BANCS.INPARAM.ApproverCmnt	The approver's comment.
BANCS.INPARAM.FreeCode1	Custom data.
BANCS.INPARAM.FreeCode2	Custom data.
BANCS.INPARAM.FreeCode3	Custom data.
BANCS.INPARAM.FreeCode4	Custom data.
BANCS.INPARAM.FreeCode5	Custom data.
BANCS.INPARAM.FreeText1	Custom data.
BANCS.INPARAM.FreeText2	Custom data.
BANCS.INPARAM.FreeText3	Custom data.
BANCS.INPARAM.FreeText4	Custom data.
BANCS.INPARAM.FreeText5	Custom data.
BANCS.INPARAM.Source ID	The source ID.

Output Repository Fields:

Output not present.

Return Value:

The same fields as Input, with values modified as required.

©Copyright Infosys Ltd.

Finacle Copyright

Populate Reason Code for ACH

This userhook is created to Populate Reason Code and Reason ID for ACH instructions.

Syntax:

```
sv_b = urhk_PopulateReasonCodeForACH ("" )
```

Input Arguments:

entrySrlNum

paysysId

reason ID

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.entrySrlNum	Entry Serial Number
BANCS.INPARAM.paysysId	Payment System ID
BANCS.INPARAM.BeginChqAlpha	Alpha Portion of the Begin Cheque Number
BANCS.INPARAM.reasonId	Reason ID

Output Repository Fields:

Output not present.

Return Value:

No output from the userhook.

For Example:

```
<--start
```

```
#trace on
sv_a = "10486"
sv_b = "ACH9"
sv_c = "0009"
BANCS.INPARAM.entrySrINum=sv_a
BANCS.INPARAM.paysysId=sv_b
BANCS.INPARAM.reasonId=sv_c
sv_z = urhk_PopulateReasonCodeForACH("")
#trace off
end-->
```

©Copyright Infosys Ltd.

Fund Validation for ACH

This user hook does the fund validation for ACH.

Syntax:

```
sv_a = urhk_doExcessFundValidation (“”)
```

sv_a: scratch pad variable to store the return value.

Input Arguments:

NA

Input Repository Fields:

Field Name	Description
EXCESS.GENERAL.DrAcct	The debit account on which excess fund validation is to be done.
EXCESS.GENERAL.PaysysId	Paysys ID of the aed instruction. The instruction will or will_not come under excess based on this paysys ID.
EXCESS.GENERAL.EntrySrlNum	Enter serial number of the instruction.
EXCESS.GENERAL.LienSrlNum	Formed lienSrlnum for marking lien on the account.
EXCESS.GENERAL.Rate	Rate of the instruction. Significant in multi currency transaction.
EXCESS.GENERAL.RateCode	Rate code of the instruction. Significant in multi currency transaction.
EXCESS.GENERAL.DrAmount	The instruction's debit amount.
EXCESS.GENERAL.TranCrncyCode	The tran currency code of the AED instruction.
EXCESS.GENERAL.grantTODFlag	The flag to say whether TOD has to be granted for the account for the instruction.
EXCESS.GENERAL.ModuleId	This ID says from which module id the userhook is being called from. Example: HACHCP, HASP
EXCESS.GENERAL.InOutInd	The indicator to say whether the instruction is a inward or an outward one.

EXCESS.GENERAL.CrDrInd	The indicator to say whether the instruction is a credit or an debit one.
------------------------	---

Output Repository Fields:

Returns SUCCESS/FAILURE.

Field Name	Description
EXCESS.GENERAL.ErrorMessage	Contains the error message during excess fund validation.
EXCESS.GENERAL.SuccessOrFailure	Flag denoting the success or failure of the excess fund validation.

Return Value:

NA

©Copyright Infosys Ltd.

Update Entry Status for ACH

This userhook is defined to Update the Entry Status for ACH instructions.

Syntax:

```
sv_b = urhk_UpdateEntryStatusForACH ("" )
```

Input Arguments:

entrySrlNum

paysysId

fromEntryStatus

toEntryStatus

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.entrySrlNum	Entry Serial Number
BANCS.INPARAM.paysysId	Payment System ID
BANCS.INPARAM.fromEntryStatus	From Entry Status
BANCS.INPARAM.toEntryStatus	To Entry Status

Output Repository Fields:

Output not present.

Return Value:

No output from the userhook.

For Example:

```
<--start
#trace on
sv_a = "10486"
sv_b = "ACH9"
sv_c ='P'
sv_d ='R'
BANCS.INPARAM.entrySrINum=sv_a
BANCS.INPARAM.paysysId=sv_b
BANCS.INPARAM.fromEntryStatus=sv_c
BANCS.INPARAM.toEntryStatus=sv_d
sv_z = urhk_UpdateEntryStatusForACH("")
#trace off
end-->
```

©Copyright Infosys Ltd.

SWIFT Userhooks

This section discusses:

[Create SMH Record](#)

[Get SMH Record](#)

[Update SMH Record](#)

[Outward Swift Transaction Creation](#)

[Populate Amount Field](#)

©Copyright Infosys Ltd.

Create SMH Record

This userhook inserts an entry into the SMH table. You have to provide the values for creating the SMH record. These values are inserted in the table. The key of SMH swift serial number is returned, if the insertion is successful.

Syntax:

```
sv_u = urhk_createSMHRecord("")
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.EventId	The event ID of the record that has to be created in the SMH table.
BANCS.INPARAM.EventType	The event type of the record that has to be created in the SMH table.
BANCS.INPARAM.SolId	The Sol ID of the record that has to be created in the SMH table.
BANCS.INPARAM.MtNum	The message type number for the record that has to be created in the SMH table.
BANCS.INPARAM.RecieverSwiftCode	The receiver swift code for the record that has to be created in the SMH table
BANCS.INPARAM.SwiftMessage	The swift message value for the record that has to be created in the SMH table.
BANCS.INPARAM.Status	The status value for the record that has to be created in the SMH table. Valid Values: N - Message not ready R - Message ready

	T - Message transferred D - Message deleted S - Message to be resent
BANCS.INPARAM.BillFunc	The bill function value for the record that has to be created in the SMH table.
BANCS.INPARAM.EventNum	The event number for the record that has to be created in the SMH table.
BANCS.INPARAM.Remarks	The remarks for the record that has to be created in the SMH table.
BANCS.INPARAM.TranId	The transaction ID for the record that has to be created in the SMH table.
BANCS.INPARAM.TranDate	The transaction date for the record that has to be created in the SMH table.
BANCS.INPARAM.PtranSrlNum	The part transaction ID for the record that has to be created in the SMH table.
BANCS.INPARAM.EntityCreFlg	The entity creation flag for the record that has to be created in the SMH table.
BANCS.INPARAM.Currency	The currency of transaction for the record that has to be created in the SMH table.
BANCS.INPARAM.SysOrManual	The system or manual flag for the record that has to be created in the SMH table. Valid Values: S -System M - Manual
BANCS.INPARAM.LastTransmittedBy	The code of the person who had last transmitted the swift message for the record that is being created.
BANCS.INPARAM.LastTransmittedOn	The date on which the last transmission of the swift message has taken place for the record that has to be created in the SMH table.
BANCS.INPARAM.TypeOfMessage	The type of message that has to be updated. Valid Values: S - SWIFT M - MT999 T - Telex message

Output Repository Fields:

Output not present.

Return Value:

If the record is updated successfully, the output is available in BANCS.OUTPARAM.sw_srl_num. If the updating of the record is unsuccessful, an error message is available in BANCS.OUTPARAM.ErrMsg field.

For Example:

```
<--start
```

```
BANCS.INPARAM.EventId=BANCS.INPUT.EventId
```

```
BANCS.INPARAM.EventType=BANCS.INPUT.EventType
```

```
BANCS.INPARAM.SolId=BANCS.INPUT.SolId
```

```
BANCS.INPARAM.MtNum=BANCS.INPUT.MtNum
```

```
BANCS.INPARAM.RecoverSwiftCode=BANCS.INPUT.RecoverSwiftCode
```

```
BANCS.INPARAM.SwiftMessage=BANCS.INPUT.SwiftMessage
```

```
BANCS.INPARAM.Status=BANCS.INPUT.Status
```

```
BANCS.INPARAM.BillFunc=BANCS.INPUT.BillFunc
```

```
BANCS.INPARAM.EventNum=BANCS.INPUT.EventNum
```

```
BANCS.INPARAM.Remarks=BANCS.INPUT.Remarks
```

```
BANCS.INPARAM.TranId=BANCS.INPUT.TranId
```

```
BANCS.INPARAM.TranDate=BANCS.INPUT.TranDate
```

```
BANCS.INPARAM.PtranSrlNum= BANCS.INPUT.PtranSrlNum
```

```
BANCS.INPARAM.TranAmt=BANCS.INPUT.TranAmt
```

```
BANCS.INPARAM.Currency=BANCS.INPUT.Currency
```

```
BANCS.INPARAM.SysOrManual=BANCS.INPUT.SysOrManual
```

```
BANCS.INPARAM.LastTransmittedBy=BANCS.INPUT.LastTransmittedB
BANCS.INPARAM.LastTransmittedOn=BANCS.INPUT.LastTransmittedC
BANCS.INPARAM.TypeOfMessage=BANCS.INPUT.TypeOfMessage
sv_r = urhk_createSMHRecord("")
if ( sv_r == 1 ) then
sv_v=BANCS.OUTPARAM.ErrMsg
print(sv_v)
else
sv_v=BANCS.OUTPARAM.sw_srl_num
endif
exitscript
end-->
```

©Copyright Infosys Ltd.

Get SMH Record

Syntax:

```
sv_r = urhk_getSMHRecord("")
```

This userhook is used to get the fields from SMH table for the Swift serial number passed. In addition to that the swift message is parsed into different fields.

Input Arguments:

All the Input variables are populated in the INPARAM class of the BANCS repository.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.SwSrlNum	The record corresponding to the swift serial number in SMH table for which the fields have to be retrieved.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.EventId	The event ID corresponding to the record in the SMH table that is retrieved.
BANCS.OUTPARAM.EventType	The event type corresponding to the record in the SMH table that is retrieved.
BANCS.OUTPARAM.SolId	The Sol ID corresponding to the swift serial number in SMH table that is retrieved.
BANCS.OUTPARAM.MtNum	The message type number corresponding to the swift serial number in SMH table that is retrieved.
BANCS.OUTPARAM.ReceiverSwiftCode	The receiver swift code for the corresponding record in the SMH table that is retrieved.
BANCS.OUTPARAM.SwiftMessage	The swift message value for the record that

	is retrieved.
BANCS.OUTPARAM.Status	The status value for the corresponding record that is retrieved. Valid Values: N - Message not ready R - Message ready T - Message transferred D - Message deleted S - Message to be resent
BANCS.OUTPARAM.BillFunc	The bill function value for the record that is retrieved.
BANCS.OUTPARAM.EventNum	The event number value for the record that is retrieved.
BANCS.OUTPARAM.Remarks	The remarks value for the record that is retrieved.
BANCS.OUTPARAM.TranId	The transaction ID for the record that is retrieved.
BANCS.OUTPARAM.PtranSrlNum	The part transaction ID for the record that is retrieved.
BANCS.OUTPARAM.TranDate	The transaction date for the record that is retrieved.
BANCS.OUTPARAM.TranAmt	The transaction amount for the record that is retrieved.
BANCS.OUTPARAM.LastTransmittedBy	The code of the person who had last transmitted the swift message for the record that is retrieved.
BANCS.OUTPARAM.LastTransmittedOn	The date on which the last swift message was transmitted for the record that is retrieved.
BANCS.OUTPARAM.SysOrManual	The system or manual flag that has to be updated. Valid Values: S - System M- Manual
BANCS.OUTPARAM.EntityCreFlag	The entity create flag that has to be updated. Valid Values:

	N - Unverified Y - Verified
BANCS.OUTPUTPARAM.Currency	The currency of the transaction for the record that is retrieved.
BANCS.OUTPUTPARAM.TypeOfMessage	The type of messages that has to be updated. Valid Values: S - SWIFT M - MT999 T - Telex message
BANCS.OUTPUTPARAM.LchgUserId	The user ID of the person who had last changed the record that is retrieved.
BANCS.OUTPUTPARAM.LchgTime	The time at which the last change had taken place for the record that is retrieved.
BANCS.OUTPUTPARAM.InOutInd	The Inward/Outward indicator. Valid Values: I - Incoming message O - Outgoing message
BANCS.OUTPUTPARAM.ExecutionDate	The date on which the message has to be transmitted. Note: <i>It is used for sending a message for a future date.</i>
BANCS.OUTPUTPARAM.MessageKind	The kind of message. Valid Values: U - Urgent N - Normal
BANCS.OUTPUTPARAM.PaysysId	The payment system ID. Valid Values: SWIFT - Swift payment system Note: <i>It is the ID of the payment system which transports the message. This is maintained as PAYMENT_SYSTEM_TYPE through HRRCDM for reference type CK.</i> <i>See Maintain Reference Code in the Administrator module for HRRCDM menu option details.</i>

BANCS.OUTPUTPARAM.RelatedMtNum	The related message number. For Example: '202' is related message number of 103 if sent for cover message.
BANCS.OUTPUTPARAM.RelatedSwSrlNum	The swift serial number of the related message.
BANCS.OUTPUTPARAM.PDEFlag	Indicates whether duplicate messages are possible or not. Valid Values: Y-Yes N-No
BANCS.OUTPUTPARAM.STPFlag	Indicates Straight Through Processing (STP). In case of outward messages, the SWIFT message is generated and transmitted with the corresponding event in Core Banking Solution. When the event that triggers the SWIFT message is verified, the generated message is marked as ready for transmission. It is picked up by the Finacle process so that it can be transmitted to the SWIFT machine. In case of Inward messages, if a message is marked for STP, the application picks up the message for processing and creates the necessary event and financial transaction in the Core Banking Solution database. Valid Values: Y- Yes (Only STP messages) N - No (Only Non-STP messages) NULL- (Both STP and Non-STP)
BANCS.OUTPUTPARAM.ReceiverBIC	The BIC (Bank Identifier Code) of the message receiver maintained through BICM.
BANCS.OUTPUTPARAM.SenderBIC	The BIC (Bank Identifier Code) of the message sender maintained through BICM.
BANCS.OUTPUTPARAM.ValueDate	The value date for the message (Field 32A in

	swift messages).
BANCS.OUTPARAM.SenderRefNum	The sender's reference number (Field TAG20 in swift message.)
BANCS.OUTPARAM.ReasonforFailure	Indicates the reason for STP failure for incoming messages.
BANCS.OUTPARAM.InstructedAmt	The instructed amount for which the message is generated (Field 33B in swiftmessages).
BANCS.OUTPARAM.InstructedCrncy	Indicates the currency code of the instructed_amt (Field 33B in swift messages.)
BANCS.OUTPARAM.SenderCorrBankcode	The sender's correspondent bank. Note: <i>This field is used for Tag53 of MT110.</i>
BANCS.OUTPARAM.SenderCorrBrcode	The sender's correspondent branch. Note: <i>This field is used for Tag53 of MT110.</i>
BANCS.OUTPARAM.ReceiverCorrBankcode	The receiver's correspondent bank. Note: <i>This field is used for Tag54 of MT110</i>
BANCS.OUTPARAM.ReceiverCorrBrcode	The receiver's correspondent branch. Note: <i>This field is used for Tag54 of MT110</i>
BANCS.OUTPARAM.RoutingRefNum	The payment order (PORD) reference number for routing. Note: <i>This field is populated if the SWIFT message is created for forwarding a Payment Order instruction.</i>
BANCS.OUTPARAM. MsgPriorityNum	The message priority number
BANCS.OUTPARAM.Utr	Utr

Return Value:

The userhook returns the value '0' or '1'. If the return value is '1', then there was failure in getting the SMH record. In case of failure, the output variables are populated in the OUTPARAM class of the BANCS repository. The userhook also populates an error message in the field ErrMsg. The error message conveys the reason for the failure in retrieving the record. If the value is '0', the output variables are populated in the OUTPARAM class of the BANCS repository.

For Example:

```
<--Start
```

```
#####
```

```
# The Swift Serial Number of SMH Table record that has to be  
retrieved
```

```
#####
```

```
BANCS.INPARAM.SwSrINum = "12345"
```

```
sv_r = urhk_getSMHRecord("")
```

```
#####
```

```
# If the field ErrMsg exists then it means that retrieving of  
record from SMH Table
```

```
# has not happened.
```

```
# And the Error Message will be in the field ErrMsg.
```

```
# Else manipulate the fields retrieved.
```

```
#####
```

```
if (fieldexists(BANCS.OUTPARAM.ErrMsg)) then
```

```
print(BANCS.OUTPARAM.ErrMsg)
```

```
else
```

```
#####
```

Populating the values in the Repository SWIFT and Class MT100 created.

#####

SWIFT.MT100.Remarks = BANCS.OUTPUTPARAM.Remarks

SWIFT.MT110.senderSwiftCode = BANCS.INPUT.field1

SWIFT.MT110.mtNumber = BANCS.INPUT.field2

SWIFT.MT110.receiverSwiftCode = BANCS.INPUT.field3

SWIFT.MT110.sendersTRN = BANCS.INPUT.field4

endif

End-->

©Copyright Infosys Ltd.

Update SMH Record

This function is used to update the fields in the SMH table.

Syntax:

```
Sv_r = urhk_updateSMHRecord("")
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.SwSrlNum	The swift serial number for the record in the SMH table that has to be updated.
BANCS.INPARAM.ReceiverSwiftCode	The receiver swift code field that has to be updated.
BANCS.INPARAM.SwiftMessage	The swift message value that has to be updated.
BANCS.INPARAM.Status	The status value that has to be updated. Valid Values: - N - Message not ready - R - Message ready - T - Message transferred - D - Message deleted - S - Message to be resent
BANCS.INPARAM.BillFunc	The bill function value that has to be updated. It has to be specified only for foreign bills.
BANCS.INPARAM.EventNum	The event number value that has to be updated.
BANCS.INPARAM.Remarks	The remarks value that has to be updated.
BANCS.INPARAM.SysOrManual	The system or manual flag that has to be updated. Valid Values: - S - System - M - Manual

BANCS.INPARAM.TypeOfMessage	The type of message that has to be updated. Valid Values: S - SWIFT M - MT999 T - Telex message
BANCS.INPARAM.EntityCreFlag	The entity created flag that has to be updated. Valid Values: N - Unverified Y - Verified

Output Repository Fields:

Field Name	Description
BANCS.INPARAM.InOutInd	The Inward or Outward indicator. Valid Values: I - Incoming message O - Outgoing message
BANCS.INPARAM.ExecutionDate	The date on which the message to be transmitted. It is used for sending a message for a future date.
BANCS.INPARAM.MessageKind	The Kind of message. Valid Values: U - Urgent N - Normal
BANCS.INPARAM.PaysysId	The payment system ID. Valid Values: SWIFT - Swift payment system Note: <i>It is the ID of the payment system which transports the message. This is maintained as PAYMENT_SYSTEM_TYPE through HRRCDM menu option for reference type CK.</i> <i>See Maintain Reference Code in the Administrator module for HRRCDM menu option details.</i>
BANCS.INPARAM.RelatedMtNum	The related message number.

	For Example: '202' is related message number of 103 if sent along for cover message.
BANCS.INPARAM.RelatedSwSrlNum	The SWIFT serial number of the related message.
BANCS.INPARAM.PDEFlag	Indicates whether duplicate messages are possible or not. Valid values: Y-Yes N-No
BANCS.INPARAM.STPFlag	Indicates Straight Through Processing (STP). In the case of Outward messages, the SWIFT message is generated and transmitted with the corresponding event in Core Banking Solution. When the event that triggers the SWIFT message is verified, the generated message is marked as ready for transmission. It is picked up by the Finacle process so that it can be transmitted to the SWIFT machine. In case of Inward messages, if a message is marked for STP, the application picks up the message for processing and creates the necessary event and financial transaction in the Core Banking Solution database. Valid Values: Y- Yes (Only STP messages) N - No (Only Non-STP messages) Null (Both STP and Non-STP)
BANCS.INPARAM.ReceiverBIC	The BIC (Bank Identifier Code) of the message receiver maintained through BICM.
BANCS.INPARAM.SenderBIC	The BIC (Bank Identifier Code) of the message sender maintained through BICM.
BANCS.INPARAM.ValueDate	The value date for the message (Field 32A in swift messages).
BANCS.INPARAM.SenderRefNum	The sender's reference number (Field TAG20 in swift message).
BANCS.INPARAM.ReasonforFailure	Indicates the reason for STP failure for incoming messages.

BANCS.INPARAM.InstructedAmt	The instructed amount for which the message is generated (Field 33B in swift messages.)
BANCS.INPARAM.InstructedCrcy	Indicates the currency code of the instructed_amt (Field 33B in swift messages).
BANCS.INPARAM.SenderCorrBankcode	The sender's correspondent bank. Note: <i>This field is used for Tag53 of MT110.</i>
BANCS.INPARAM.SenderCorrBrcode	The sender's correspondent branch. Note: <i>This field is used for Tag53 of MT110.</i>
BANCS.INPARAM.ReceiverCorrBankcode	The receiver's correspondent bank. Note: <i>This field is used for Tag54 of MT110.</i>
BANCS.INPARAM.ReceiverCorrBrcode	The receiver's correspondent branch. Note: <i>This field is used for Tag54 of MT110.</i>
BANCS.INPARAM.RoutingRefNum	Indicates the payment order (PORD) reference number for routing. Note: <i>This field is populated if the SWIFT message is created for forwarding a payment order instruction.</i>

Return Value:

The userhook returns the value '0' or '1'. If the return value is '1', then the userhook has failed to update the SMH record. All output variables is populated in the OUTPARAM class of the BANCS repository. In case of failure, it populates the error message in the field ErrMsg. This error conveys the reason for failure in updating the record.

The output variables are strings.

For Example:

```
<--Start
```



```
#####
# The Swift Serial Number of SMH Table record that has to be
# updated.
# The Swift Message that has to be updated
# The Status is N (Not Ready to send)
# The Entity Create Flag as N ( Unverified)
#####
BANCS.INPARAM.SwSrlNum = "12345"
BANCS.INPARAM.ReceiverSwiftCode = "SBINFRPP"
BANCS.INPARAM.SwiftMessage =
":28C:1|:60F:C981210INR0|:61:981210C100,00"
BANCS.INPARAM.Status = "N"
BANCS.INPARAM.BillFunc= ""
BANCS.INPARAM.EventNum = ""
BANCS.INPARAM.Remarks = "Swift Message 768"
BANCS.INPARAM.SysOrManual = "S"
BANCS.INPARAM.TypeOfMessage = "S"
BANCS.INPARAM.EntityCreFlag = "N"
sv_r = urhk_updateSMHRecord("")
#####
# If the field ErrMsg exists then it means that update of SMH
# Table has not happened,
# And the Error Message will be in the field ErrMsg.
# Else do the COMMIT of the Updated Record in SMH Table.
#####
if (fieldexists(BANCS.OUTPARAM.ErrMsg)) then
print(BANCS.OUTPARAM.ErrMsg)
```

```
else  
sv_r = urhk_dbSql("COMMIT")  
endif  
End-->
```

©Copyright Infosys Ltd.

Outward Swift Transaction Creation

This userhook is used to create an outward swift transaction based on the given input parameters. The userhook also helps us in arriving at the final settlement account based on the payment system type. After derication the parameter finalSettlementAcct will be dumped as an output from the userhook.

Syntax:

```
sv_b = urhk_OutSwiftTranCreation (“”)
```

Input Arguments:

NA

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.reRoutePymtSysType	Re Route Payment System Type
BANCS.INPARAM.mesgGenerationReqd	Message Generation Required

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.finalSettlementAcct	Final Settlement Account

Return Value:

NA

Example:

NA

©Copyright Infosys Ltd.

Finacle Copyright

Populate Amount Field

This user hook is used to fetch the data for Pool Provision Amt, Pool Suspended Amt, Pool Write Off Amt and Profit from Asset Amt from AHT table for a specified cycle.

Syntax:

```
sv_q = urhk_PopulateAmtField("")
```

Input arguments:

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.profitDistFromDate	Profit Distribution From Date
BANCS.INPARAM.profitDistToDate	Profit Distribution To Date
BANCS.INPARAM. poolCode	Pool Code
BANCS.INPARAM.crncyCode	Currency Code

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.asset_Profit_Amt	Asset Profit Amount
BANCS.OUTPARAM.pool_Suspend_Amt	Pool Suspend Amount
BANCS.OUTPARAM.pool_WriteOff_Amt	Pool WriteOff Amount
BANCS.OUTPARAM.pool_Prov_Amt	Pool Provision Amount

Return Value:

NA

Example:

BANCS.INPARAM.profitDistFromDate = sv_a

BANCS.INPARAM.profitDistToDate = sv_b

BANCS.INPARAM.poolCode = sv_c

BANCS.INPARAM.crncyCode = sv_d

sv_q = urhk_PopulateAmtField("")

sv_e = BANCS.OUTPARAM.asset_Profit_Amt

sv_f = BANCS.OUTPARAM.pool_Suspend_Amt

sv_i = BANCS.OUTPARAM.pool_WriteOff_Amt

sv_j = BANCS.OUTPARAM.pool_Prov_Amt

©Copyright Infosys Ltd.

ECS Userhooks

This section discusses:

- Update the ECS Enabled Flag as Y at Account Level in the GAC Table

©Copyright Infosys Ltd.

Get Address Details for Bic Bank and Branch

Syntax

```
sv_b = urhk_GetAddressDtlsForBicBankBranch("")
```

Functionality

Takes the BIC or bank and branch combination from the user and fetches Address details

Input

BANCS.INPPARAM.bankIdentifierCode

BANCS.INPPARAM.bankCode

BANCS.INPPARAM.branchCode

Either bankIdentifierCode or bankCode and branchCode should be passed.

Output

Returns address details (BANCS.OUTPARAM.BranchName, BANCS.OUTPARAM.Addr1, BANCS.OUTPARAM.Addr2, BANCS.OUTPARAM.CityCode).

Returns FAILURE if the fetch fails.

Examples

None

Fetch Unique Bic for PaysysId

Syntax

```
sv_b = urhk_fetchUnqBicForPaysysId ("")
```

Functionality

The user hook is used to fetch the bic for a paysys Id, given a different paysys Id and its corresponding bic. The user hook will be able to derive the data only if linking is already done for these records through MND or LND menus.

Input

BANCS.INPPARAM.fromBic

BANCS.INPPARAM.fromPaysysId

BANCS.INPPARAM.toPaysysId

All inputs parameters are mandatory.

Output

Returns BANCS.OUTPARAM.toBic if the fetch is successful.

Returns FAILURE if the fetch fails

Examples

None

Fetch Value for Bic, PaysysId and Additional Details

Syntax

```
sv_b = urhk_fetchValueForBicPaysysIdAndAddnDtl("")
```

Functionality

Takes the BIC, Paysys Id and additional details column name from the user and returns the value of the column.

Input

BANCS.INPARAM.bic

BANCS.INPARAM.paysysId

BANCS.INPARAM.fieldName

All the input parameters are mandatory.

Output

Returns BANCS.OUTPARAM.fieldValue.

Returns failure if unable to derive output.

Examples

None

©Copyright Infosys Ltd.

Fetch Directory Reference Number for Bic and PaysysId

Syntax

```
sv_b = urhk_fetchDirRefNumForBicAndPaysysId ("")
```

Functionality

Takes the BIC and Paysys Id from the user and fetches the Directory Reference Number for that BIC and Paysys Id combination from NDT and the value will be returned back to the user.

Input

BANCS.INPARAM.bic

BANCS.INPARAM.paysysId

All the input parameters are mandatory.

Output

Returns BANCS.OUTPARAM.dirRefNum.

Returns failure if unable to derive output.

Examples

None

Fetch Additional Details for Directory Reference Number

Syntax

```
sv_b = urhk_fetchAddnDtIsForDirRefNum ("")
```

Functionality

Takes the Directory Reference Number and Additional Details field name from the user and returns the value of the Additional Details field.

Input

BANCS.INPARAM.dirRefNum

BANCS.INPARAM.fieldName

All the input parameters are mandatory.

Output

Returns BANCS.OUTPARAM.fieldValue.

Returns failure if unable to derive output.

Examples

None

©Copyright Infosys Ltd.

Remaining Value of Prepayment Permitted in a Year

This userhook is used to calculate Remaining Value of Prepayment Permitted in a Year..

Syntax:

```
sv_a = urhk_getAcctBalanceAsOnDate("")
```

sv_a: scratch pad variable to store the return value.

Input Arguments:

- acid

The above inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acid	Corresponding acid value of the account number in GAM table

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.prepaymentAmt	Calculated remaining prepayment amount

Example:

To calculate remaining prepayment amt for account Id PRELA1:

For Example:

```
sv_b = BANCS.INPUT.foracid (PRELA1)
```

```
sv_c = urhk_B2k_ForacidToAcid(sv_b)
if(sv_c == 0) then
    sv_b = BANCS.OUTPUTPARAM.Acid
endif
BANCS.INPUTPARAM.acid = sv_b
sv_a = urhk_getPermittedPrepaymentAmts("")
if (sv_a == 0) then
    if (fieldexists(BANCS.OUTPUTPARAM.prepaymentAmt)) then
        BANCS.OUTPUT.prepayment_amt =
BANCS.OUTPUTPARAM.prepaymentAmt
```

©Copyright Infosys Ltd.

Updates Blacklist Status of a Payment

This functions of developed **Updates Blacklist Status of a Payment** userhook are:

- Updates the blacklist status of a payment in BLVD table
- Depending on the status received from external body on the payment sent, the same is updated in the BLVD table
- The input for this userhook is Paysys Id, Paysys Type, Payment Entry Srl Number, Bank ID, Blacklist Status received, additional details on Status and Remarks
- All these input variables are passed by external body and in return this userhook return success or failure

Syntax:

```
sv_b = urhk_updateBlkListPymtValDetails(“”)
```

sv_b: scratch pad variable to store the return value.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.b2k_id	Entry_Srl_Num (Mandatory)
BANCS.INPARAM.b2k_type	Paysys Type (Mandatory)
BANCS.INPARAM.paysys_id	Paysys Id (Mandatory)
BANCS.INPARAM.bank_id	Bank Id (Mandatory)
BANCS.INPARAM.blklist_status	Status Received
BANCS.INPARAM.status_details	Additional details on Status Received
BANCS.INPARAM.remarks	Any remarks provided

Userhook Output:

Returns Success, if input record gets added successfully else returns Failure.

©Copyright Infosys Ltd.

Overdraft Userhooks

Overdraft Userhooks

- [Check If Tran Used COD](#)

©Copyright Infosys Ltd.

Check If Tran Used COD

This user hook is added to check if the debit transaction will utilize Courtesy Overdraft.

Syntax

sv_b = urhk_checkIfTranUsedCOD ("") (With no optional input)

Input Arguments:

Account Number (Foracid)

Transaction Amount (TranAmt)

Transaction Currency Code (CurrencyCode)

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Foracid	Account number
BANCS.INPARAM.TranAmt	Transaction Amount
BANCS.INPARAM.CurrencyCode	The currency code of amount

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.IsCODUsed	This field determines if COD is utilized.
BANCS.OUTPARAM.errorMesg	This field contains error message description in case of failure.

Example

```
<--start
```

```
trace on
```

```
BANCS.INPARAM.Foracid = "MRSB5_5"
```

```
BANCS.INPARAM.TranAmt = "2000.75"
BANCS.INPARAM.CurrencyCode = "INR"
sv_m=urhk_checkIfTranUsedCOD("")
if(sv_m==0) then
#{
print(BANCS.OUTPARAM.IsCODUsed)
#}
else
#{
print(.)
#}
endif
trace off
end-->
```

Repository Inputs:

- Foracid = "MRSB5_5"
- TranAmount = "2000.75"
- CurrencyCode = "INR"

Repository Output:

In case account has only 3000 INR running limit of CO Type.

- IsCODUsed = "Y"

Else, if account has clear balance of 3000 INR.

- IsCODUsed = "N"

Referral Userhooks

This section discusses:

[Raising an Amount Based Exception](#)

[Setting Referral Data](#)

©Copyright Infosys Ltd.

Raising an Amount Based Exception

Amount Based Exception in a custom menu can be raised through the userhook in the back-end script. The userhook (RaiseABEForReferral) raises an ABE for any custom menu. The inputs need to be set from the script with every input separated by a pipe (|). The values sent through the userhook must be limited to 1024 bytes.

Syntax:

```
sv_b = urhk_RaiseABEForReferral (amt + "|" + crncyCode + "|" + tranType + "|" + tranEvent + "|" + acctId + "|" + partTranSrlNum)
```

Input Arguments:

There are no inputs available to the script.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Once the userhook is called, the check for amount based exception is done based on the values sent. If the amount based exception is encountered then the exceptions is displayed to the user and it needs to be referred for approval.

If the amount based exception conditions are not met then it returns from there and nothing is done.

For Example:

Suppose the requirement is to raise an Amount based exception for a custom menu, then the above userhook is called. The amount, currency code and the event needs to be passed and the check is done by the code available in the product.

©Copyright Infosys Ltd.

Setting Referral Data

SetReferralData is a userhook where in Name-Value pairs is sent as an input to the userhook. The name and values are limited to 512 and 1024 bytes respectively.

This userhook compares the field name with that of each record of custom input LL (Received from front-end) and performs the following:

- ✚ exists, updates the value in the custom input LL for that field name with the new value set in the userhook.
- ✚ not found, appends the record at the end of the custom input LL with the name-value details.

The userhook that has been defined to set the custLL with the updated or additional value.

This userhook is used during adding a new referral to populate custom data.

Syntax:

```
sv_a = urhk_SetReferralData("NAME|"+Value)
```

Input Arguments:

There are no inputs available to the script.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Name value pair data to be inserted into referral tables and then submitted along with referral submission.

For Example:

Suppose the requirement is to append a dc-alias at the end of the customer ID specified in the custom menu,

Hence in the specified script appending the dc-alias (viz., RF),

```
IF ((BANCS.INPUT.srvcallmode == "A")) THEN
```

```
sv_a          =          urhk_SetReferralData("custid|RF"+  
BANCS.INPUT.custid)
```

```
ENDIF
```

©Copyright Infosys Ltd.

Loan Related Userhooks

This section discusses:

[Get Loans Demand Satisfaction Fee Transactions Details](#)

[Get Loans Pay Fee Transaction Details](#)

[Get Loans Pay Fee Transaction Details](#)

[Get Loans Demand Satisfaction Fee Transactions Details](#)

[Get Loan Account Outstandings](#)

©Copyright Infosys Ltd.

Get Loans Demand Satisfaction Fee Transactions Details

Syntax:

```
sv_r = urhk_getLadspFeeTranDetails(MTT.InputDetails.LoanAcct)
```

Input Arguments:

The input is the formatted account ID of the loan account

Input Repository Fields:

Field Name	Description
MTT.InputDetails.ChargeCollAcct	ChargeCollAcct
MTT.InputDetails.EventId	EventId
MTT.InputDetails.EventType	EventType
MTT.InputDetails.FeeCalcAmt	FeeCalcAmt
MTT.InputDetails.FeeCalcCrncy	FeeCalcCrncy
MTT.InputDetails.FeeCollectionAmt	FeeCollectionAmt
MTT.InputDetails.FeeRateCode	FeeRateCode
MTT.InputDetails.FeeRate	FeeRate
MTT.InputDetails.ConsolAcct	ConsolAcct
MTT.InputDetails.FeeTranParticular	FeeTranParticular

Output Repository Fields:

Output not present.

Return Value:

Return value not available.

©Copyright Infosys Ltd.

Finacle Copyright

Get Loans Pay Fee Transaction Details

The user or system can select different charge types to be off-set for every account. Each charge may have part transactions that credit more than one office account, based on the MCS setup. This userhook is called in order to get individual part transaction amounts and the corresponding credit account for the credit leg of charge off-setting part transactions. It is called in a loop to retrieve all charge part transaction records. It is necessary to put part transactions for each record retrieved to balance the transaction and ensure that the posting does not fail. The contra amount for all fee transactions is made available as the input to the script, LaPayTranCreation.scr. The userhook returns the value 0, after retrieving all records.

See Define Event ID in the Administrator module for MCS menu option details.

Note:

*This userhook is applicable only for LAPAY menu option. For details on the LAPAY menu option, refer to the chapter, **Loan Account Maintenance and Inquiry**, in the Term Loans manual.*

Syntax:

```
sv_r = urhk_getLaPayFeeTranDetails(MTT.InputDetails.LoanAccount)
```

Input Arguments:

The input is the formatted account ID of the Loan Account

Input Repository Fields:

Field Name	Description
MTT.InputDetails.ChargeCollAcct	ChargeCollAcct
MTT.InputDetails.EventId	EventId
MTT.InputDetails.EventType	EventType

MTT.InputDetails.FeeCalcAmt	FeeCalcAmt
MTT.InputDetails.FeeCalcCrncy	FeeCalcCrncy
MTT.InputDetails.FeeCollectionAmt	FeeCollectionAmt
MTT.InputDetails.FeeRateCode	FeeRateCode
MTT.InputDetails.FeeRate	FeeRate
MTT.InputDetails.ConsolAcct	ConsolAcct
MTT.InputDetails.FeeTranParticular	FeeTranParticular

Output Repository Fields:

Output not present.

Return value:

Return value not available.

©Copyright Infosys Ltd.

Get Loans Pay Fee Transaction Details

The user or system can select different charge types to be off-set for every account. Each charge may have part transactions that credit more than one office account, based on the MCS setup. This userhook is called in order to get individual part transaction amounts and the corresponding credit account for the credit leg of charge off-setting part transactions. It is called in a loop to retrieve all charge part transaction records. It is necessary to put part transactions for each record retrieved to balance the transaction and ensure that the posting does not fail. The contra amount for all fee transactions is made available as the input to the script, LaPayTranCreation.scr. The userhook returns the value 0, after retrieving all records.

See Define Event ID in the Administrator module for MCS menu option details.

Note:

This userhook is applicable only for LAPAY menu option.

Syntax:

```
sv_r = urhk_getLaPayFeeTranDetails(MTT.InputDetails.LoanAccount)
```

Input Arguments:

The input is the formatted account ID of the Loan Account.

Input Repository Fields:

Input not present.

Output Repository Fields:

■MTT.InputDetails.ChargeCollAcct

■MTT.InputDetails.EventId
■MTT.InputDetails.EventType
■MTT.InputDetails.FeeCalcAmt
■MTT.InputDetails.FeeCalcCrncy
■MTT.InputDetails.FeeCollectionAmt
■MTT.InputDetails.FeeRateCode
■MTT.InputDetails.FeeRate
■MTT.InputDetails.ConsolAcct
■MTT.InputDetails.FeeTranParticular
■MTT.InputDetails.currentCollAmt
■MTT.InputDetails.chargeRateCode

Return value:

Return value not available.

©Copyright Infosys Ltd.

Get Loans Demand Satisfaction Fee Transactions Details

Syntax:

```
sv_r = urhk_getLadspFeeTranDetails(MTT.InputDetails.LoanAcct)
```

Input Arguments:

The input is the formatted account ID of the loan account.

Input Repository Fields:

Input not present.

Output Repository Fields:

- MTT.InputDetails.ChargeCollAcct
- MTT.InputDetails.EventId
- MTT.InputDetails.EventType
- MTT.InputDetails.FeeCalcAmt
- MTT.InputDetails.FeeCalcCrncy
- MTT.InputDetails.FeeCollectionAmt
- MTT.InputDetails.FeeRateCode
- MTT.InputDetails.FeeRate
- MTT.InputDetails.ConsolAcct
- MTT.InputDetails.FeeTranParticular

- MTT.InputDetails.chargeForacid
- MTT.InputDetails.currentCollAmt
- MTT.InputDetails.chargeRateCode

Return Value:

Return value not available.

©Copyright Infosys Ltd.

Get Loan Account Outstandings

Syntax:

```
sv_b = urhk_getLoanAcctOutstandings("")
```

Functionality:

This user hook accepts account id (foracid) as the input parameter and validates the entered foracid and returns the outstanding amount for the given load account.

Input:

BANCS.INPPARAM.foracid

The input is a mandatory parameter

Output:

Returns outstanding amount for given loan account if valid account id is given.

Returns FAILURE and error message in case of failure.

Example:

NA

©Copyright Infosys Ltd.

Lien Related Userhooks

This section discusses:

Update Lien

©Copyright Infosys Ltd.

Trade Finance Related Userhooks

This section discusses:

[FXBILL_GETTRAN](#)

[FXBILL_INITTRAN](#)

[Get Additional Input Details in MTT](#)

[Get Details of RPC Disbursements](#)

[Get User Additional Details](#)

[Raising Exceptions](#)

[Save User Additional Details](#)

©Copyright Infosys Ltd.

FXBILL_GETTRAN

This assumes that there is a linklist available globally containing the part transaction details as `glb_fxb_main_ptr->tran_ll`, if it does not exist then it returns a fatal error. We can call this userhook in a loop so that it fetches one record at a time and puts the details into OUTPARAM class into the mentioned fields:

MTT_PTRAN_ACCOUNT
MTT_PTRAN_REFAMOUNT
MTT_PTRAN_AMOUNT
MTT_PTRAN_REFCURRENCY
MTT_PTRAN_CURRENCY
MTT_PTRAN_BUSINESSTYPE
MTT_PTRAN_REFPTRANSRLNUM
MTT_PTRAN_PROTECT_VALUE
MTT_PTRAN_RATECODE
MTT_PTRAN_CONVRATE
MTT_PTRAN_PARTICULARS
MTT_PTRAN_REMARKS
MTT_PTRAN_INST_TYPE
MTT_PTRAN_INST_DATE
MTT_PTRAN_INST_NUM
MTT_PTRAN_VALDATE
MTT_PTRAN_REPORTCODE

MTT_PTRAN_REFNUM

These part transaction details can be used to post the part transaction.

Syntax:

```
sv_a = urhk_FxBill_GetTran(“”)
```

Input Arguments:

None

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.MTT_PTRAN_ACCOUNT	MTT_PTRAN_ACCOUNT
BANCS.OUTPARAM.MTT_PTRAN_REFAMOUNT	MTT_PTRAN_REFAMOUNT
BANCS.OUTPARAM.MTT_PTRAN_AMOUNT	MTT_PTRAN_AMOUNT
BANCS.OUTPARAM.MTT_PTRAN_REFCURRENCY	MTT_PTRAN_REFCURRENCY
BANCS.OUTPARAM.MTT_PTRAN_CURRENCY	MTT_PTRAN_CURRENCY
BANCS.OUTPARAM.MTT_PTRAN_BUSINESSTYPE	MTT_PTRAN_BUSINESSTYPE
BANCS.OUTPARAM.MTT_PTRAN_REFPTRANSRLNUM	MTT_PTRAN_REFPTRANSRLN
BANCS.OUTPARAM.MTT_PTRAN_PROTECT_VALUE	MTT_PTRAN_PROTECT_VALU
BANCS.OUTPARAM.MTT_PTRAN_CONVRATE	MTT_PTRAN_CONVRATE
BANCS.OUTPARAM.MTT_PTRAN_PARTICULARS	MTT_PTRAN_PARTICULARS
BANCS.OUTPARAM.MTT_PTRAN_REMARKS	MTT_PTRAN_REMARKS
BANCS.OUTPARAM.MTT_PTRAN_INST_TYPE	MTT_PTRAN_INST_TYPE
BANCS.OUTPARAM.MTT_PTRAN_INST_DATE	MTT_PTRAN_INST_DATE

BANCS.OUTPARAM.MTT_PTRAN_INST_NUM	MTT_PTRAN_INST_NUM
BANCS.OUTPARAM.MTT_PTRAN_VALDATE	MTT_PTRAN_VALDATE
BANCS.OUTPARAM.MTT_PTRAN_REPORTCODE	MTT_PTRAN_REPORTCODE
BANCS.OUTPARAM.MTT_PTRAN_REFNUM	MTT_PTRAN_REFNUM

Return Value:

None

©Copyright Infosys Ltd.

FXBILL_INITTRAN

This userhook Sets fxUsrHkPtr to NULL which is used by FxBill_GetTran.

Syntax:

```
sv_a = urhk_FxBill_InitTran("")
```

Input Arguments:

None

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

None

©Copyright Infosys Ltd.

Get Additional Input Details in MTT

Using this function multiple additional inputs can be obtained in MTT script. Currently this is enabled in FBM / IRM / ORM to get conversion screen details. Using this function total credit amount to customer can be obtained.

Syntax:

```
sv_r = urhk_MTTs_PopulateSubInputDetails("")
```

Input Arguments:

None

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
MTT.PartTranDetails.RefAmount	RefAmount
MTT.PartTranDetails.RefCurrency	RefCurrency
MTT.PartTranDetails.Amount	Amount
MTT.PartTranDetails.TranCurrency	TranCurrency
MTT.PartTranDetails.PtranBusinessType	PtranBusinessType
MTT.PartTranDetails.RefPtranSrlNum	RefPtranSrlNum
MTT.PartTranDetails.ProtectValue	ProtectValue
MTT.PartTranDetails.Rate	Rate
MTT.PartTranDetails. Ratecode	Ratecode
MTT.PartTranDetails.Particulars	Particulars
MTT.PartTranDetails.Remarks	Remarks

MTT.PartTranDetails.ValueDate	diff from bod date
MTT.PartTranDetails.Refnum	Refnum

Return Value:

The fields gets populated in PartTranDetails Class of MTT repository in case of FBM / IRM / ORM menu option.

RefAmount, RefCurrency, Amount, TranCurrency, PtranBusinessType, RefPtranSrlNum, ProtectValue, Rate, Ratecode, Particulars, Remarks, ValueDate (diff from bod date) and Refnum.

See Lodge a Foreign Bill in the Foreign Bills module for FBM menu option details.

See Lodge and Realize Inward Telegraphic Transfer in the Remittances module for IRM menu option details.

See Issue an Outward Telegraphic Transfer in the Remittances module for ORM menu option details.

For Example:

```
sv_r = urhk_MTT_S_PopulateSubInputDetails("")
while(sv_r < 1)
print (MTT.PartTranDetails.RefAmount)
print(MTT.PartTranDetails.PTranBusinessType)
MTT.PartTranDetails.Consolidate="N"
MTT.PartTranDetails.TranIdentifier="FBCharges"
sv_r = urhk_MTT_S_PopulateSubInputDetails("")
do
```

Get Details of RPC Disbursements

Syntax:

```
sv_a = urhk_B2k_getRPCDisbDetails("DISB_ID")
```

Input Arguments:

The input passed is the disb_id.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.DISB_ID	DISB_ID

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.Disbld	Disbld
BANCS.OUTPARAM.RpcAcct	RpcAcct
BANCS.OUTPARAM.ECGCAppFlag	ECGCAppFlag
BANCS.OUTPARAM.LastECGCCalcDate	LastECGCCalcDate
BANCS.OUTPARAM.DueDate	DueDate
BANCS.OUTPARAM.ExtendedDueDate	ExtendedDueDate
BANCS.OUTPARAM.DisbursementDate	DisbursementDate
BANCS.OUTPARAM.DisbursementAmount	DisbursementAmount
BANCS.OUTPARAM.Currency	Currency
BANCS.OUTPARAM.OutstandingAmount	OutstandingAmount
BANCS.OUTPARAM.IntCalcUptoDateFlag	IntCalcUptoDateFlag

Return Value:

DisbId
RpcAcct
ECGCApplFlag
LastECGCCalcDate
DueDate
ExtendedDueDate
DisbursementDate
DisbursementAmount
Currency
OutstandingAmount
IntCalcUpToDateFlag

For Example:

```
<--start
#trace on
sv_e = urhk_B2k_getRPCDisbDetails("AA105")
print(BANCS.OUTPARAM.DisbId)
print(BANCS.OUTPARAM.RpcAcct)
print(BANCS.OUTPARAM.ECGCApplFlag)
print(BANCS.OUTPARAM.LastECGCCalcDate)
print(BANCS.OUTPARAM.DueDate)
print(BANCS.OUTPARAM.ExtendedDueDate)
print(BANCS.OUTPARAM.DisbursementDate)
print(BANCS.OUTPARAM.DisbursementAmount)
print(BANCS.OUTPARAM.Currency)
```

```
print(BANCS.OUTPARAM.OutstandingAmount)
print(BANCS.OUTPARAM.IntCalcUptoDateFlag)
#trace off
end-->
```

©Copyright Infosys Ltd.

Get User Additional Details

This function gets user additional detail for an entity say bill or forward contract.

Syntax:

```
sv_r=urhk_B2k_GetUserAdditionalDetails("")
```

Input Arguments:

The inputs are Module ID and ModuleKey through BANCS.INPARAM class.

Valid values of Module ID are BIL, FEX, BG, FWC, and DC.

Module Key is a field to identify unique entity, for instance, a bill module ID can be sol ID and bill ID concatenated with a slash '/' character.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.ModuleId	ModuleId
BANCS.INPARAM.ModuleKey	ModuleKey

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.AdditionalDetails	AdditionalDetails

Return Value:

This function returns the user additional details into AdditionalDetails field of OUTPARAM class of BANCS repository. Field can comprise of

many values concatenated by pipe characters.

For Example:

Let us say that the bank has captured two remarks for each bill. Bank can get the value of these two fields through following script in BM menu option.

```
BANCS.INPARAM.ModuleId="BIL"
BANCS.INPARAM.ModuleKey="SOL1" + "/" + "BILL125"
sv_r=urhk_B2k_GetUserAdditionalDetails("")
if (sv_r == 0) then
sv_a=BANCS.OUTPARAM.AdditionalDetails
sv_l = getposition(sv_a, "|")
BANCS.FRMVALUE.Remarks1 = mid$(sv_a, 0, sv_l-1)
sv_b = strlen(sv_a)
sv_a = mid$(sv_a, sv_l, sv_b - sv_l)
BANCS.FRMVALUE.Remarks2=sv_a
else
BANCS.FRMVALUE.Remarks1 = ""
BANCS.FRMVALUE.Remarks2 = ""
endif
```

See Lodge Bills in the Inland Bills module for BM menu option details.

©Copyright Infosys Ltd.

Raising Exceptions

Using this function exceptions can be raised at verify time in following menu options BM, FBM, IRM, ORM, IDCM, ODCM, MNTFWC, CNCLFWC, EXTFWC.

See Lodge Bills in the Inland Bills module for BM menu option details.

See Lodge a Foreign Bill in the Foreign Bills module for FBM menu option details.

See Lodge and Realize Inward Telegraphic Transfer in the Remittances module for IRM menu option details.

See Issue an Outward Telegraphic Transfer in the Remittances module for ORM menu option details.

See Issue a Documentary Credit in the Documentary Credits module for ODCM menu option details.

See Advise Inward Documentary Credit in the Documentary Credits module for IDCM menu option details.

See Book Forward Contract in the Forward Contracts module for MNTFWC menu option details.

See Cancel Forward Contract in the Forward Contracts module for CNCLFWC menu option details.

See Extend Forward Contract in the Forward Contracts module for EXTFWC menu option details.

Syntax:

```
sv_r=urhk_B2k_PutUserException (ExceptionCode)
```

Input Arguments:

The input is exception code. Exception code must be a valid exception code setup using HEXCDM.

See Define Exception Codes in the Exception Handling module for

HEXCMD menu option details.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

None

For Example:

In case bank wish to raise an exception whenever forward contract is booked for USD (as from currency)

```
<--start
```

```
sv_a  
urhk_TBAF_InquireFieldValue("datblk1.from_crncy_code") =
```

```
sv_l = B2KTEMP.TEMPSTD.TBAFRESULT
```

```
if (sv_l == "USD") then
```

```
sv_a = urhk_B2k_PutUserException("FWU")
```

```
endif
```

```
exitscript
```

```
end-->
```

©Copyright Infosys Ltd.

Save User Additional Details

This function adds or modifies user additional detail for an entity say bill or forward contract. Currently this function is enabled for the menu options BM, FBM, IRM, ORM, IDCM, ODCM, MNTFWC, CNCLFWC, EXTFWC.

See Lodge Bills in the Inland Bills module for BM menu option details.

See Lodge a Foreign Bill in the Foreign Bills module for FBM menu option details.

See Lodge and Realize Inward Telegraphic Transfer in the Remittances module for IRM menu option details.

See Issue an Outward Telegraphic Transfer in the Remittances module for ORM menu option details.

See Issue a Documentary Credit in the Documentary Credits module for ODCM menu option details.

See Advise Inward Documentary Credit in the Documentary Credits module for IDCM menu option details.

See Book Forward Contract in the Forward Contracts module for MNTFWC menu option details.

See Cancel Forward Contract in the Forward Contracts module for CNCLFWC menu option details.

See Extend Forward Contract in the Forward Contracts module for EXTFWC menu option details.

Syntax:

```
sv_r=urhk_B2k_PutUserAdditionalDetails("")
```

Input Arguments:

The inputs are ModuleId, ModuleKey and AdditionalDetails through BANCS.INPARAM class.

Valid values of Module Id are BIL, FEX, BG, FWC, and DC.

Module Key is a field to identify unique entity e.g. for a bill module id can be sol id and bill id concatenated with a slash '/' character.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.ModuleId	ModuleId
BANCS.INPARAM.ModuleKey	ModuleKey
BANCS.INPARAM.AdditionalDetails	AdditionalDetails

Output Repository Fields:

Output not present.

Return Value:

None.

For Example:

Let us say that the bank wish to put two remarks for each bill. Bank can put the value of these two fields through the script in BM menu option.

BANCS.INPARAM.ModuleId="BIL"

BANCS.INPARAM.ModuleKey="SOL1" + "/" + "BILL125"

BANCS.INPARAM. AdditionalDetails = "REAMRKS1" +
"REMARKS2"

sv_a=urhk_B2k_PutUserAdditionalDetails("")

Userhooks for Connect24

Connect24 is an intermediate component which connects Finacle Core and the Delivery Channels like ATM. Finacle provides userhooks which relate to different functions in Connect24. There is a userhook to determine charge income while another userhook helps in inserting a record to Delivery Channel Transactions table.

This section discusses:

[Determine if Connect24 Online Fees Must be Treated as Charge Income](#)

[Generate Hashed Number for Connect24 Messages](#)

[Create DCTI Record](#)

©Copyright Infosys Ltd.

Determine if Connect24 Online Fees Must be Treated as Charge Income

This function allows the user to specify which of the fee types expected in online Connect24 messages have to be treated as charge income. These charges appears in the charge income report generated through the menu option HCHGIR. By default, the fees is not treated as charges. This function adds a record in the linklist maintained for determining whether an online fee is to be treated as a charge. This decision is based on fee type.

See Generate Customer-wise Income Report in the Administrator module for HCHGIR menu option details.

It must be called from the script 'CDGetFeeInfo'.

Syntax:

```
sv_a = urhk_ B2k_CdciPopulateFeeInfoLL ("" )
```

Input Arguments:

Input through INPARAM repository:

BANCS.INPARAM.FeeType: As expected in the online Connect24 message.

BANCS.INPARAM.CXLInsertReqdFlag: Whether the fees is a charge or not. Valid values are 'Y' (Yes) and 'N' (No).

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.FeeType	As expected in the online Connect24 message.
BANCS.INPARAM.CXLInsertReqdFlag	Whether the fees is a charge or not. Valid values are 'Y' (Yes) and 'N' (No).

Output Repository Fields:

Field Name	Description
BANCS.OUTPUT.successOrFailure	S – Success F - Failure

Return Value:

Output through OUTPARAM repository:

NONE

Function return value:

- 0 - Linklist record addition successful.
- 4 - Linklist record addition unsuccessful.

For Example:

```
BANCS.OUTPUT.successOrFailure = "S"
BANCS.INPARAM.FeeType = "01"
BANCS.INPARAM.CXLInsertReqdFlag = "Y"
sv_r = urhk_B2k_CdciPopulateFeeInfoLL("")
if(sv_r == 1) then
BANCS.OUTPUT.successOrFailure = "F"
exitscript
endif
# Repeat above logic for all fee types
```

Generate Hashed Number for Connect24 Messages

This userhook generates the hash number for the Connect24 messages.

Syntax:

```
sv_a = urhk_B2k_generateHashedNoForMesg('mesg')
```

Input Arguments:

● CDCI message

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.CDCI	CDCI

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.HashValue	HashValue

Return Value:

The userhook returns '0' on the success of the userhook. Output is stored in BANCS.OUTPARAM.HashValue.

For Example:

```
<--start
#trace on
sv_a = "This is a test mesg"
```

```
sv_e = urhk_B2k_generateHashedNoForMesg(sv_a)
print(BANCS.OUTPARAM.HashValue)
#trace off
end-->
```

©Copyright Infosys Ltd.

Create DCTI Record

This userhook is used to create or insert record into DCTI table for Delivery channel transaction Inquiry. Before inserting the record into the table it validates the Acct_id passed. If there is any error in inserting the record into the table the error message is stored in outparam repository.

Syntax:

```
sv_a = urhk_ CreateDCTIRecord(“”)
```

Input Arguments:

- BANCS.INPARAM.ReferenceNo
- BANCS.INPARAM.DelChannelId
- BANCS.INPARAM.DCDeviceId
- BANCS.INPARAM.DCLocation
- BANCS.INPARAM.DCMesgId
- BANCS.INPARAM.TranDate
- BANCS.INPARAM.DCUserId
- BANCS.INPARAM.CustId
- BANCS.INPARAM.SolId
- BANCS.INPARAM.AcctNum
- BANCS.INPARAM.TranType
- BANCS.INPARAM.TranAmt
- BANCS.INPARAM.Currency

BANCS.INPARAM.TranStatus

BANCS.INPARAM.TranRmks

BANCS.INPARAM.FreeCode1

BANCS.INPARAM.FreeCode2

BANCS.INPARAM.FreeText1

BANCS.INPARAM.FreeText2

BANCS.INPARAM.FreeText3

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.ReferenceNo	ReferenceNo
BANCS.INPARAM.DelChannelId	DelChannelId
BANCS.INPARAM.DCDeviceId	DCDeviceId
BANCS.INPARAM.DCLocation	DCLocation
BANCS.INPARAM.DCMesgId	DCMesgId
BANCS.INPARAM.TranDate	TranDate
BANCS.INPARAM.DCUserId	DCUserId
BANCS.INPARAM.CustId	CustId
BANCS.INPARAM.SolId	SolId
BANCS.INPARAM.AcctNum	AcctNum
BANCS.INPARAM.TranType	TranType
BANCS.INPARAM.TranAmt	TranAmt
BANCS.INPARAM.Currency	Currency
BANCS.INPARAM.TranStatus	TranStatus
BANCS.INPARAM.TranRmks	TranRmks
BANCS.INPARAM.FreeCode1	FreeCode1
BANCS.INPARAM.FreeCode2	FreeCode2

BANCS.INPARAM.FreeText1	FreeText1
BANCS.INPARAM.FreeText2	FreeText2
BANCS.INPARAM.FreeText3	FreeText3

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.ErrMsg	ErrMsg

Return Value:

This userhook returns '0' on successful completion and on failure it returns '1' and error message is stored in BANCS.OUTPARAM.ErrMsg.

For Example:

```
<--start
trace on
BANCS.INPARAM.ReferenceNo="ATM12345545403200166556"
BANCS.INPARAM.DelChannelId="BTM"
BANCS.INPARAM.DCDeviceId="DD556"
BANCS.INPARAM.DCLocation="BANGALORE"
BANCS.INPARAM.DCMesgId="MSG6"
BANCS.INPARAM.TranDate="10-12-1998"
BANCS.INPARAM.DCUserId="USER1236"
BANCS.INPARAM.CustId="ACME"
BANCS.INPARAM.SolId="SCGL"
BANCS.INPARAM.AcctNum="SB4"
BANCS.INPARAM.TranType="A6"
BANCS.INPARAM.TranAmt="10000"
```

```
BANCS.INPARAM.Currency="INR"
BANCS.INPARAM.TranStatus="FINE"
BANCS.INPARAM.TranRmks="EXCELLENT"
BANCS.INPARAM.FreeCode1="FREE1"
BANCS.INPARAM.FreeCode2="FREE2"
BANCS.INPARAM.FreeText1="FREETEXT1"
BANCS.INPARAM.FreeText2="FREETEXT2"
BANCS.INPARAM.FreeText3="FREETEXT3"
sv_e = urhk_CreateDCTIRecord("")
print(sv_e)
if(sv_e != 0 ) then
print(BANCS.OUTPARAM.ErrMsg)
trace off
end-->
```

©Copyright Infosys Ltd.

Userhooks used for Conversions

This section discusses:

[Convert Lower String to Upper Case](#)

[B2k_Convert Amount](#)

©Copyright Infosys Ltd.

Convert Lower String to Upper Case

This userhook converts the passed value to upper case. The output is populated in BANCS.OUTPUTPARAM.UpString. The input to this userhook is any string. The passed string is converted into upper case and is populated in BANCS.OUTPUTPARAM.UpString.

Syntax:

```
sv_a = urhk_ UpCase()
```

Input Arguments:

Input is any char string.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The output is available in BANCS.OUTPUTPARAM.UpString.

For Example:

```
sv_a = BANCS.INPARAM.lowString
```

```
sv_b = urhk_ UpCase (sv_a)
```

Output is BANCS.OUTPUTPARAM.UpString field.

B2k_Convert Amount

This userhook is built to accept the customer ID as the input parameter to the userhook . If the customer id is entered then the Preferential Exchange Rate is derived, else the Card Exchange Rate is derived.

Syntax:

```
sv_b = urhk_ B2k_ConvertAmount ("")
```

Input Arguments:

InputAmount

FromCurrency

ToCurrency

RateCode

Rate

Cust ID

The input is not mandatory parameter

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.InputAmount	InputAmount
BANCS.INPARAM.FromCurrency	FromCurrency
BANCS.INPARAM.ToCurrency	ToCurrency
BANCS.INPARAM.RateCode	RateCode
BANCS.INPARAM.Rate	Rate
BANCS.INPARAM.CustId	CustId

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.OutputAmount	OutputAmount
BANCS.OUTPARAM.Rate	Rate

Return value:

Function returns 0 or 1. If 1, then there was failure in conversion. If 0, all output variables is populated in the OUTPARAM class of the BANCS repository. The variables are (all STRINGS):

OutputAmount - The equivalent amount in 'to currency'

Rate - The conversion rate actually used (in case INPARAM.RateCode is used, INPARAM.Rate is zero).

For Example:

```
<--start
sv_a = "USD"
#From Currency
sv_b = "GBP"
#To Currency
sv_c = "TCB"
# Want to get the rate for Travellers Cheque buying
# sv_p contains the amount to be converted
BANCS.INPARAM.InputAmount = sv_p
BANCS.INPARAM.FromCurrency = sv_a
BANCS.INPARAM.ToCurrency = sv_b
Sv_r = urhk_B2k_ConvertAmount("")
```



```
if (sv_r ==1) then
# Error processing
endif
sv_q = BANCS.OUTPARAM.OutputAmount
# sv_q contains the converted amount
sv_r = BANCS.OUTPARAM.Rate
# sv_r contains the TC buying rate
end-->
```

©Copyright Infosys Ltd.

Userhooks used within the Operating System

This section discusses:

[Get PID of the Process](#)

[Get Value of Environment Variable](#)

[Getting the Location of a File in the Path](#)

[Sleep](#)

©Copyright Infosys Ltd.

Get PID of the Process

This userhook returns the PID of the process.

Syntax:

```
sv_a = urhk_B2k_PID("")
```

Input Arguments:

None

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The userhook returns '0' on the success. Output is stored in BANCS.OUTPARAM.PID.

For Example:

```
<--start
#trace on
sv_e = urhk_B2k_PID("")
print(BANCS.OUTPARAM.PID)
#trace off
end-->
```

©Copyright Infosys Ltd.

Finacle Copyright

Get Value of Environment Variable

This userhook returns the value of Environment variable. The output is populated in BANCS.OUTPARAM.envValue. The input to this userhook is the Environment variable name. The value of the Environment variable is obtained and is populated in BANCS.OUTPARAM.envValue.

Syntax:

```
sv_a = urhk_getEnv()
```

Input Arguments:

- Environment variable name

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The output is available in BANCS.OUTPARAM. envValue.

For Example:

```
sv_a = BANCS.INPARAM. EnvName
```

```
sv_b = urhk_getEnv(sv_a)
```

Output will be in this field:

```
BANCS.OUTPARAM. envValue.
```

©Copyright Infosys Ltd.

Finacle Copyright

Getting the Location of a File in the Path

This function is used to get the path of a file by specifying the file name. This function returns the full path of the file name specified as input. It can typically be used in scripts to get the location of a script file, which needs to be called from within another script. The search for the file is carried out according to the site-customization directory search rules.

Syntax:

```
sv_a = urhk_getFileLocation(sv_b)
```

The variable can be a scratch pad variable (sv_b) or a string ("`<file type>|`" + `<filename>`")

Input Arguments:

Input through INPARAM repository: None

Input String as function argument contains two parts separated by a `|` character.

File Type- The type of the file whose path is required. The possible values for this are: "FORM", "EXE", "SCRIPT", "MRT", "SQL","COM"and "MENU".

File Name - The name of the file whose path is required.

Both the input parameters are mandatory.

For Example

```
"SCRIPT|EVTWFS.scr"
```

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Output through OUTPARAM repository: The userhook returns the path of the file in the variable "fileLocation".

This can be accessed as:

```
sv_d = BANCS.OUTPARAM.fileLocation
```

Function return value: Returns '1' if path is successfully got else returns '0'.

For Example:

```
<--start
sv_a = STDWFS.WFSINPUT.CallScriptName
sv_b = "SCRIPT" + "|" + sv_a
sv_c = urhk_getFileLocation(sv_b)
sv_d = BANCS.OUTPARAM.fileLocation
START(sv_d, sv_a)
exitscript
end-->
```

Sleep

This userhook makes the sleep for specified number of seconds.

Syntax:

```
sv_a = urhk_B2k_Sleep("No_of_seconds")
```

Input Arguments:

Number of seconds to sleep.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

None

For Example:

```
<--start
#trace on
sv_e = urhk_B2k_Sleep("5")
#trace off
end-->
```

This will make the process sleep for 5 seconds.

Validation Userhooks

This section discusses:

[Check Pending Verification for Security](#)

[Check Whether Account is Cash Account](#)

[Check Whether Account is FCNR Account](#)

[Check Whether Account is HO Account](#)

[Check Whether Account is System Only Account](#)

[Validate Amount with Currency](#)

[Validate Bank and Branch Code](#)

[Validate Bank Code](#)

[Validate Checksum](#)

[Validate Collateral Code with Report Type](#)

[Validate Currency Code](#)

[Validate Currency for Pool Code](#)

[Validate Date](#)

[Validate Foracid for Mudarabah Pool Product Account](#)

[Validate HO Transaction Category Code](#)

[Validate if Record Already Exists for WMS Type in ASM Table](#)

[Validate Instrument Date](#)

[Validate Instrument for Account](#)

[Validate Job Group](#)

[Validate Mudarabah Pool Product Account](#)

[Validate Mudarabah Pool Product Scheme Code](#)

[Validate Numeric](#)

[Validate Pool Code for Inquiry](#)

[Validate Reference Code](#)

[Validate SOL ID](#)

[Validate Transaction Report Code for Account](#)

[Validating the Account](#)

[Validate GL Subhead Desc Userhooks](#)

[Validate BIC](#)

[Validate Multi Currency Account](#)

[Validate Multi-Currency/Master Account](#)

[Validate Channel Id For Origination](#)

[Validate Channel Id For Services](#)

[Validate Bic For PaysysId](#)

[Validation Originator ID](#)

[Check Whether Account is Repeatedly Overdrawn](#)

©Copyright Infosys Ltd.

Check Whether Account is Repeatedly Overdrawn

This user hook checks if an account is repeatedly overdrawn or not. An account is repeatedly overdrawn if any of the two conditions is satisfied:

- If number of days in the look back duration when the account balance is negative is greater than or equal to “Negative Balance Counter without considering Balance” (HSCFM menu parameter).

Or

- If the number of days in look back period where account balance is less than or equal to “0 -Negative Balance Limit” (HSCFM menu parameter) exceeds or equal to “Negative Balance Counter considering Balance” (HSCFM menu parameter).

In the second scenario mentioned if the Bank level and Account level Currencies are different, the system uses the Bank level Exchange Rate code (Available in the Rate Code field in the Exchange Rate tab in the HSCFM menu) for amount conversion to be used for comparison. The BOD rate for currency combination is used for all the calculations.

Syntax

```
sv_b = urhk_checkRODAccount ("")
```

Input

The following is the syntax for specifying the input fields:

```
sv_a = "any_valid_foracid"
```

```
BANCS.INPARAM.foracid = sv_a
```

The following are the details of the input fields:

Input Field	Description
foracid	The account to be checked for repeatedly overdrawn (ROD) condition.

Output

The following is the syntax for specifying the output fields:

```
sv_c = BANCS.OUTPARAM.repeatedly_overdrawn
sv_d = BANCS.OUTPARAM.successOrFailure
sv_e = BANCS.OUTPARAM.ErrMsg
```

The following are the details of the output fields:

Output Field	Description
repeatedly_overdrawn	The value is "T" if the account is repeatedly overdrawn, "F" if account is not repeatedly overdrawn and blank if there is some error.
successOrFailure	The value is "S" if user hook execution is successful and "F" if there is some error.
ErrMsg	The error message is populated only in case of failure.

Example

A sample script can be like:

```
<--start
trace on
#Get the values
sv_a = "220000000000000000"
BANCS.INPARAM.foracid = sv_a
sv_b = urhk_checkRODAccount("")
```

```
sv_c = BANCS.OUTPUTPARAM.repeatedly_overdrawn  
print(sv_c)  
sv_d = BANCS.OUTPUTPARAM.successOrFailure  
print(sv_d)  
sv_e = BANCS.OUTPUTPARAM.ErrMsg  
print(sv_e)  
trace off  
end-->
```

©Copyright Infosys Ltd.

Check Whether Account is Cash Account

This userhook takes account number as the input, and return failure if the input account is cash account.

Syntax:

```
sv_a=urhk_B2k_valCashAccount("Account")
```

Input Arguments:

Userhook accepts "Account" as the input.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

This function returns '0' if "Account" is not cash account. Otherwise it returns non zero value.

©Copyright Infosys Ltd.

Check Whether Account is FCNR Account

This userhook takes account number as the input and returns failure if the input account is FCNR account.

Syntax:

```
sv_a=urhk_B2k_valFCNRAccount("Account")
```

Input Arguments:

Userhook accepts "Account" as the input.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

This function returns '0' if Account is not FCNR account. Otherwise it returns non zero value.

©Copyright Infosys Ltd.

Check Whether Account is HO Account

This userhook takes account number as the input and returns failure if the input account is not HO account.

Syntax:

```
sv_a=urhk_B2k_valHOAccount("Account")
```

Input Arguments:

Userhook accepts "Account" as the input.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

This function returns '0' if Account is HO account. Otherwise it returns non zero value.

©Copyright Infosys Ltd.

Check Whether Account is System Only Account

This userhook takes account number as the input, and returns failure if the input account is system only account.

Syntax:

```
sv_a=urhk_B2k_valSystemAccount("Account")
```

Input Arguments:

Userhook accepts "Account" as the input.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

This function returns '0' if Account is not system only account. Otherwise it returns non zero value.

©Copyright Infosys Ltd.

Validate Amount with Currency

This userhook takes amount and currency as the input. It validates the amount with currency and returns back the amount in the precision of the currency.

Syntax:

```
sv_a=urhk_B2k_validateAmtWithCrncy(amount| crncy)
```

Input Arguments:

- Amount
- Currency

The amount and currency could be a variable or a value itself. If it is a value it must be enclosed within quotes.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

BANCS.OUTPARAM. OutputAmount

This contains the amount in the precision of the currency.

Validate Bank and Branch Code

The userhook `urhk_B2k_valBankBranchCode(variable)` works as:

This function returns `SUCCESS(0)` if the input bank code, branch code combination is valid, else returns `FAILURE(1)`. If the bank and branch code combination is valid, then some information regarding the branch is given as output. This userhook is provided to validate the given bank code, branch code combination. This also gives the details of the branch in OUTPARAM variables as given in the output section.

Note:

The output variables are available only if the branch and bank code combination is valid and hence the script returns `SUCCESS(0)`.

Syntax:

The syntax for calling the function is

```
sv_r = urhk_B2k_valBankBranchCode (variable)
```

The variable can be a scratch pad variable (`sv_a`) or a string(`"CAB|1098"`).

Input Arguments:

■ Bank Code

■ Branch Code

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description

BANCS.OUTPARAM.BranchName	Name of the branch
BANCS.OUTPARAM.ShortName	Short name of the branch
BANCS.OUTPARAM.Addr1	Address field 1
BANCS.OUTPARAM.Addr2	Address field 2
BANCS.OUTPARAM.CityCode	Branch city code
BANCS.OUTPARAM.StateCode	Branch state code
BANCS.OUTPARAM.TelexNumber	Branch Telex Number
BANCS.OUTPARAM.CountryCode	Branch Country Code
BANCS.OUTPARAM.PhoneNumber	Branch Phone Number
BANCS.OUTPARAM.FaxNumber	Branch Fax Number
BANCS.OUTPARAM.SwiftCode	Swift Code of the branch
BANCS.OUTPARAM.SwiftMsgFlg	Whether swift allowed(Y/N)
BANCS.OUTPARAM.BcbCode	BCB code of the branch
BANCS.OUTPARAM.FbrFlg	Foreign branch flag (Y/N)
BANCS.OUTPARAM.DaysForValueDate	DaysForValueDate
BANCS.OUTPARAM.MicrCenterCode	MicrCenterCode
BANCS.OUTPARAM.MicrBankCode	MicrBankCode
BANCS.OUTPARAM.MicrBranchCode	MicrBranchCode

Return Value:

Return value is '0' if the bank code, branch code combination is valid, else it is '1'. If the bank and branch code is valid the output details are available as the BANCS.OUTPARAM variables.

For Example:

```
<--start
sv_a=Bank Code
sv_b=Branch Code
sv_c=sv_a + "|" + sv_b
```

```
sv_r = urhk_B2k_valBankBranchCode (sv_c)

# If B2k_valBankBranchCode function returns failure exit out of
script. Else use the outparam variables for further processing.

if ( sv_r == 1 ) then
exitscript
else
if (BANCS.OUTPARAM.SwiftMsgFlg == "Y") then
endif
endif
end-->
```

©Copyright Infosys Ltd.

Validate Bank Code

This function returns SUCCESS (0) if the input bank is a valid, else returns FAILURE (1). This userhook is provided to validate the given bank code.

Syntax:

The syntax for calling the function is

```
sv_r = urhk_B2k_valBankCode (bankCode)
```

The variable can be a scratch pad variable (sv_b) or a string("CAB").

Input Arguments:

Bank Code

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value is '0' if the bank code is valid, else it is '1'.

For Example:

```
<--start
sv_a = Bank Code
sv_r = urhk_B2k_valBankCode (sv_a)
# if B2k_valBankCode function returns failure exit out of script.
```

```
if( sv_r != 0 ) then  
  exitscript  
endif  
end-->
```

©Copyright Infosys Ltd.

Validate Checksum

This userhook calculates the checksum of the input string and validates against the value passed as checksum. In the record layouts of all uploads, the check sum is given at the end of each record. The input to this userhook is the entire record and the checksum value separated by a delimiter '|'. This userhook computes the checksum value for the input string, validates with the checksum value passed and returns success or failure.

Syntax:

```
sv_a = urhk_B2k_IsChecksumRight()
```

Input Arguments:

- The input for this userhook is the record layout.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Returns the value '0' if the bank code is valid; otherwise it returns the value '1'.

For Example:

```
# Get the input string and checksum value.  
# Get the input string length
```

```
sv_l = strlen(BANCS.INPUT.Buffer)
# Subtract the input string length from the record length
sv_m = sv_l - 365
# All trim the input string
sv_o = LTRIM(RTRIM(MID$(BANCS.INPUT.Buffer, 0, 365)))
# All trim the checksum part at the end of input string
sv_p = LTRIM(RTRIM(MID$(BANCS.INPUT.Buffer, 365, sv_m)))
# Concatenate the record layout and the checksum with a
delimiter
sv_q = sv_o + "|" + sv_p
#call the user hook for checking the check sum
sv_n = urhk_B2k_IsChecksumRight(sv_q)
IF (sv_n != 0) THEN
BANCS.OUTPUT.successOrFailure="F"
BANCS.OUTPUT.error= "Checksum Mismatch"
ENDIF
```

©Copyright Infosys Ltd.

Validate Collateral Code with Report Type

This user-hook is used to validate the report type with the collateral code. For reports other than 1 and 2, WMS collateral types are not allowed.

Syntax:

```
sv_e=urhk_ExecVal("validateColtrlCodeWithRptType|coltrlCode.code=
rptNumber.code=&r_num|FLD_CHK_NOT_REQ")
```

Input Arguments:

Collateral Code

Report Number

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Collateral Code	ColtrlCode
BANCS.INPARAM.Report Number	rptNumber

Output Repository Fields:

Output not present.

Return Value:

Success or Failure is returned depending on the validation. In case of Failure the error message "WMS Collateral Code cannot be entered for this report" is displayed.

©Copyright Infosys Ltd.

Finacle Copyright

Validate Currency Code

This userhook takes the currency code as the input and validates it.

Syntax:

```
sv_a=urhk_B2k_valCrncy(crncyCode)
```

Input Arguments:

- Currency code

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

This userhook returns '0' if the input is a valid currency code, otherwise it returns a non-zero value.

©Copyright Infosys Ltd.

Validate Currency for Pool Code

This userhook checks if the pool code-currency code combination being passed is a valid or not. This userhook gets executed if pool code validation and currency code validation.

Syntax:

```
sv_b = urhk_validateCrncyForPoolCode (“”)
```

Input Arguments:

Pool Code

Currency

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Pool Code	Pool Code
BANCS.INPARAM.currency	currency

Output Repository Fields:

Output not present.

Return Value:

Error is displayed “Pool Code and CCY Combination does not exist.”

©Copyright Infosys Ltd.

Validate Date

This userhook validates if the string passed to it is a valid date. Only the date component is validated not the time component. However when passing the string either a string with only date or string with date and time may be passed. The userhook truncates the time component and validates the date. This userhook is used to validate if a given string is a valid date. The first 10 characters of the string are taken and validated to check if it is a valid date. The time component is not validated.

Syntax:

```
sv_a = urhk_ B2k_valDate(Variable)
```

sv_a: scratch pad variable to store the return value

Variable: Scratch pad variable containing string to be validate.

Input Arguments:

String to be validated whether it is a valid date or not

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value is success ('0') if string is a valid date else failure.

For Example:

```
sv_r=urhk_B2k_valDate(PASTDUE.PDC.dateofreckoning)
if (sv_r != 0) then
sv_a="Invalid Date given for field Date Of Reckon "
else
sv_a=""
endif
```

©Copyright Infosys Ltd.

Validate Foracid for Mudarabah Pool Product Account

This userhook is used to validate if the foracid is a Mudarabah Pool Product or not.

Syntax:

```
sv_b = urhk_valForacidForMudPoolProdAcct("")
```

Input Arguments:

Account number (FORACID)

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.FORACID	FORACID

Output Repository Fields:

Output not present.

Return Value:

If the validation fails, then error is thrown "Invalid Menu option for Mudarabah Pool based accounts".

©Copyright Infosys Ltd.

Validate HO Transaction Category Code

This function returns SUCCESS(0) if the input Scheme code, Transaction Category Code combination is valid, else returns FAILURE(1). This userhook is provided to validate the given Scheme Code and Transaction Category code combination.

Syntax:

The syntax for calling the function is

```
sv_r = urhk_B2k_valHoTranCode (variable)
```

The variable can be a scratch pad variable (sv_a) or a string("ABSOT|23").

Input Arguments:

- Scheme Code
- Transaction Category Code

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value is '0' if the Scheme Code and Transaction Category Code combination is valid, else it is '1'.

For Example:

```
<--start
sv_a = Scheme Code
sv_b= Transaction Category Code
sv_c=sv_a + "|" + sv_b
sv_r = urhk_B2k_valHoTranCode (sv_c)
# if B2k_valHoTranCode function returns failure exit out of
script.
if( sv_r == 1 ) then
exitscript
endif
end-->
```

©Copyright Infosys Ltd.

Validate if Record Already Exists for WMS Type in ASM Table

This userhook is used to validate that for the same currency two collateral codes are not allowed.

Syntax:

```
sv_a=urhk_ExecVal("validateIfASMRecExistsForWMSType|coltrlCode.
```

Input Arguments:

- Collateral Code
- Collateral Type
- Currency Code

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Collateral Code	coltrlCode
BANCS.INPARAM.Collateral Type	coltrlType
BANCS.INPARAM.Currency Code	crncyCode

Output Repository Fields:

Output not present.

Return Value:

Success or Failure is returned depending on the validation. In case of Failure the error message "Record Already Exist for Collateral Type

and Currency combination” is displayed.

©Copyright Infosys Ltd.

Finacle Copyright

Validate Instrument Date

This userhook does the basic date related validations for a given instrument. The validations done are whether the instrument is post dated or instrument is stale.

Syntax:

```
sv_a = urhk_B2k_validateInstrmntDate(instrmnt_type|instrmnt_date)
```

Input Arguments:

- Instrument Type

- Instrument Date

The date must be in dd-mm-yyyy format only.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The userhook returns '0' if the validation goes through, otherwise it returns a non-zero value.

In case a userhook returns a non zero value it also returns the status related to date in the repository variable, BANCS.OUTPARAM.ReturnCode.

Different status depending on the BANCS.OUTPARAM.ReturnCode value are:

Instrument is post dated, the return code is 2

Instrument is stale then the return value is 3.

Neither of above value - The instrument type passed is wrong.

©Copyright Infosys Ltd.

Validate Instrument for Account

This userhook checks whether a given instrument is issued to passed account, if it is a passed account then returns back the status.

Syntax:

```
sv_a = urhk_ B2k_valInstrumentNumForAcct("")
```

Input Arguments:

acid

instrument number

instrument alpha

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.ReturnCode	.
BANCS.OUTPARAM.ChqStatus	.

Return value:

Return value not available.

For Example:

```
<--start
```

```
trace on
```



```
BANCS.INPARAM.AccountId = "SB4"  
BANCS.INPARAM.InstrumentNumber = "101"  
BANCS.INPARAM.InstrumentAlpha = "SB/"  
sv_e = urhk_B2k_valInstrumentNumForAcct("")  
print(BANCS.OUTPARAM.ReturnCode)  
print(BANCS.OUTPARAM.ChqStatus)  
trace off  
end-->
```

©Copyright Infosys Ltd.

Validate Job Group

This userhook is provided to validate the batch job group ID.

Syntax:

```
sv_a = urhk_validateJobGroup("")
```

Input Arguments:

- Job Group (job_group)
- Invoking process (exec_db_stat_code)

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Output of this userhook is an integer value (retCode) between 1 and 6.

For Example:

```
<--start
#trace on
BANCS.INPARAM.exec_db_stat_code = sv_a
BANCS.INPARAM.job_group = sv_b
if ((sv_a != "") OR (sv_b != "")) then
sv_a = urhk_validateJobGroup("")
```

```
if (fieldexists(BANCS.OUTPARAM.retCode)) then  
sv_g = BANCS.OUTPARAM.retCode  
endif  
endif  
#trace off  
end-->
```

©Copyright Infosys Ltd.

Validate Mudarabah Pool Product Account

This userhook is used to validate whether the account is Mudarabah account or not.

Syntax:

```
sv_b = urhk_valMudPoolProdAcct ("")
```

Input Arguments:

NA

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Error is displayed stating that it is not a Mudarabah account.

©Copyright Infosys Ltd.

Validate Mudarabah Pool Product Scheme Code

This userhook is used to validate if the scheme code belongs to Mudarabah Pool Products.

Syntax:

```
sv_b = urhk_ValMudPoolProdSchmCode (“”)
```

Input Arguments:

NA

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

If the validation fails, then error is thrown “Invalid Scheme Code”.

©Copyright Infosys Ltd.

Validate Numeric

This function returns SUCCESS(0) if the input string contains valid numeric digits, else returns FAILURE(1). This userhook is provided to validate the given string for numeric digits.

Syntax:

The syntax for calling the function is

```
sv_r = urhk_B2k_isStringNumeric (Amount String)
```

The variable can be a scratch pad variable (sv_b) or a string("1234").

Input Arguments:

Amount String

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value is '0' if the input string contains valid digits, else it is '1'.

For Example:

```
<--start
```

```
sv_r = urhk_B2k_isStringNumeric (Amount string)
```

```
# if B2k_isStringNumeric function returns failure exit out of script.
```

```
if( sv_r != 0 ) then  
  exitscript  
endif  
end-->
```

©Copyright Infosys Ltd.

Validate Pool Code for Inquiry

This userhook checks if the pool code being passed is a valid pool code or not for Inquiry Mode.

Syntax:

```
sv_b = urhk_validatePoolCodeForInquiry ("")
```

Input Arguments:

Pool Code

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.Pool Code	Pool Code

Output Repository Fields:

Output not present.

Return Value:

Error is displayed "Invalid pool code".

©Copyright Infosys Ltd.

Validate Reference Code

This userhook validates whether the string passed to it is a valid reference code. The input to the userhook is reference code type and reference code. The userhook validates that the reference code is valid and of type reference code type as passed to it.

Syntax:

```
sv_a = urhk_ B2k_valRefCode(Variable)
```

sv_a: scratch pad variable to store the return value

Variable: Scratch pad variable containing string consisting of reference code type and reference code delimited by |.

Input Arguments:

String consisting of reference code type and reference code to be validated delimited by |.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value is success ('0') if string is a valid reference code of the given reference code type else failure.

For Example:

```
sv_z="12|"+PASTDUE.PDL.authpermwaiver
```

```
sv_r=urhk_B2k_valRefCode(sv_z)
if (sv_r != 0) then
sv_a="Auth. Perm. Waiver"
else
sv_a=""
endif
```

©Copyright Infosys Ltd.

Validate SOL ID

This userhook validates the given sol_id.

Syntax:

```
sv_a = urhk_B2k_valSolId("Sol_id")
```

Input Arguments:

Input is a string that is the sol_id to be validated.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Output value of the userhook is '0' if the given sol_id is a valid sol_id else it returns the value as 1 (not a valid sol_id).

For Example:

```
<--start
#trace on
sv_e = urhk_B2k_valSolId("SCGL")
print(sv_e)
#trace off
end-->
```

©Copyright Infosys Ltd.

Finacle Copyright

Validate Transaction Report Code for Account

This userhook takes the account and transaction report code. Check whether the given transaction report code is valid for the given account.

Syntax:

```
sv_a=urhk_B2k_valTranRptCode(Account|TranReportCode)
```

Input Arguments:

Account

Transaction Report Code

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

This userhook return '0' if the given transaction report code is valid for the account. Otherwise it returns a non-zero value.

©Copyright Infosys Ltd.

Validating the Account

This function validates a given account number in the Finacle database and returns a value depending on whether the account number is valid or not. It can be used in the context of any Script Event or Workflow. This function validates whether a given account number (Foracid) exists in the data center database or not. It simply checks for the presence of the account and does not distinguish between any status of the account (for instance, closed or not, office account or customer account). If more information about an account is required please refer to the function `urhk_getAcctDetailsInRepository` in this document.

Syntax:

The syntax for calling the function is

```
sv_a = urhk_valAcctNumber (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("SBGEN1").

Input Arguments:

- Account number (FORACID).

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The return value is '0' in case a valid account number with the input string is found. Otherwise the return value is '1'.

For Example:

As a result of moving from distributed to a Flex-centralised topology, some banks may desire to change the account numbering format. However, external references to old accounts still exists. Hence there may be situations where a Finacle user may enter an old account number on the screen. Now, if the new account number is derivable from the old account number (either by keeping a mapping file or by applying a simple transformation) we can code the formatAcct.scr as follows to ensure that the user does not get an 'invalid account'message when the old account number is specified.

```
<--start
sv_s = BANCS.INPUT.uAccount
sv_i = urhk_valAcctNumber(sv_s)
if (sv_i = 0) then /* Account exists */
BANCS.OUTPUT.fAccount = sv_s
EXITSCRIPT
else /* Account does not exist */
/* apply new account number derivation logic here */
BANCS.OUTPUT.fAccount = Result of above logic
endif
exitscript
end-->
```

Validate GL Subhead Desc Userhooks

This user hook is created to validate and populates glSubHeadDesc if the input glSubHeadCode exist for any sol and currency.

Syntax:

```
sv_b = urhk_B2k_valGIForAnyCrncyAndSol ("")
```

Input

glSubHeadCode, crncyCode, solid, setId and delFlg are the inputs to the user hook.

Output

BANCS.OUTPARAM.glSubHeadDesc

This user hook will return 0 on successful completion and on failure it will return 1 and error mesg will be stored in BANCS.OUTPARAM.retCode

For Example:

```
<--start
```

```
trace on
```

```
BANCS.INPARAM. glSubHeadCode = "2200"
```

```
BANCS.INPARAM. crncyCode = "INR"
```

```
BANCS.INPARAM. solId = "00000000"
```

```
BANCS.INPARAM. setId = "10000"
```

```
BANCS.INPARAM. delFlg = "N"
```

```
If glSubHeadCode is not valid, return failure.
```

```
sv_z = urhk_B2k_valGIForAnyCrncyAndSol ("")
```



```
print(sv_z)
if(sv_z != 0 ) then
    andprint(BANCS.OUTPARAM.ErrMsg)
trace off
end-->
```

©Copyright Infosys Ltd.

Validate BIC

This user hook selects data from BIC table for the given BIC and paysys_id.

Syntax:

```
sv_q = urhk_validateBicForPaysysId("")
```

Input

BIC

PaysysId

Output

Return value is INF_SUCCESS.

For Example:

```
<--start
trace on
BANCS.INPARAM.bic="123123"
BANCS.INPARAM.paysysId="26354"
sv_d=urhk_validateBicForPaysysId("")
print(sv_d)
trace off
end-->
```

Validate Bic For PaysysId

Syntax:

```
sv_b = urhk_validateBicForPaysysId ("")
```

Functionality

This user hook takes the BIC and Paysys ID from the user and fetches the record from NDT table for that BIC and Paysys Id combination. If record exists, then the BIC is considered as a valid BIC, else it is not a valid BIC.

Input

BANCS.INPPARAM.bic

BANCS.INPPARAM.paysysId

The inputs are mandatory parameters

Output

Returns **SUCCESS** if validations are successful.

Returns **FAILURE** if validations fail.

For Example:

None

Validate Multi Currency Account

This userhook validates if the account is linked with a Multi-currency account or not. It accepts valid foracid and gives a return code as output.

Syntax:

```
sv_a = urhk_validateMultiCcyAAC("")
```

Input:

A valid account number.

Input Repository Fields:

The following input field is available for customizing this user hook

Input Field	Description
BANCS.INPARAM.foracid	Valid Account Number in GAM table

Output Repository Fields:

The following output field is available for customizing this user hook

Output Field	Description
BANCS.OUTPARAM.retCode	Return Code

Output:

Success if the foracid does not belong to the multi-currency account

For Example:

```
<--start
```

```

sv_f = urhk_VAL_GetVal("foracid")
sv_g = BANCS.OUTPARAM.fldValue
BANCS.INPARAM.foracid = sv_g
if (sv_g != "") then
    sv_h = urhk_validateMultiCcyAAC("")
    if(fieldexists(BANCS.OUTPARAM.retCode)) then
        sv_i = BANCS.OUTPARAM.retCode
        if(sv_i == 1) then
            sv_z =
urhk_VAL_SetErr("FLD_NAME=foracid|ERR_TYPE=S|ERR_CODE=E652
        endif
    endif
end if
end..>

```

©Copyright Infosys Ltd.

Validate Multi-Currency/Master Account

This user hook validates if the account is linked with MultiCurrency or master account or not.

Syntax:

```
sv_a = urhk_chkIfMultiOrMaster("")
```

Input:

None

Input Repository Fields:

The following input field is available for customizing this user hook

Input Field	Description
BANCS.INPARAM.foracid	Valid account ID.

Output Repository Fields:

The following output field is available for customizing this user hook

Output Field	Description
BANCS.OUTPARAM.retCode	Return Code

Output:

1, if the account is linked with multicurrency account; 2, if it is linked with master account and 0 if not linked to both.

For Example:

```
<--start
```

```

BANCS.INPARAM.foracid = sv_h
    if (sv_h != "") then
        sv_x = urhk_chkIfMultiOrMaster("")
        if(fieldexists(BANCS.OUTPARAM.retCode)) then
            if(BANCS.OUTPARAM.retCode == 1) then
                if (sv_m != "") then
                    # low_acct_num Field should not be entered
                                                                sv_a =
urhk_VAL_SetErr("FLD_NAME=low_acct_num|ERR_TYPE=M|ERR_CODE=MSG0316658")
                endif
                if (sv_n != "") then
                    # low_acct_num Field should not be entered
                                                                sv_a =
urhk_VAL_SetErr("FLD_NAME=high_acct_num
|ERR_TYPE=M|ERR_CODE=MSG0316658")
                endif
            endif
        endif
    endif
end-->

```

Validate Channel Id For Origination

Syntax:

```
sv_b = urhk_validateChannelIdForOrigination ("")
```

Functionality:

This user hook accepts scheme code as the input parameter and validates the channel id set up at the scheme code level for SB/CA/TD/TU type of accounts.

Input:

BANCS.INPPARAM.schmCode

The input is a mandatory parameter

Output:

Returns SUCCESS if scheme level channel validations are successful.

Returns FAILURE and error message if scheme level channel validations fail.

Example:

NA

Validate Channel Id For Services

Syntax:

```
sv_b = urhk_validateChannelIdForServices (“”)
```

Functionality:

This user hook accepts account id (foracid) as the input parameter and validates the channel id set up at the account level for SB/CA/TD/TU type of accounts.

Input:

BANCS.INPPARAM.foracid

The input is a mandatory parameter

Output:

Returns SUCCESS if account level channel validations are successful.

Returns FAILURE and error message if account level channel validations fail.

Example:

NA

Validation Originator ID

The functionalities of Validation Originator ID userhook are:

- Takes the Originator ID as mandatory input
- Takes the Originator Directory ID also as input, can be blank also
- Validates the Originator ID and checks if it is present in ODT table, If not returns failure
- If the Directory ID is passed, then it checks if the Originator ID and Directory ID combination is present in ODT table, if not present then returns failure
- If the value is present, then system returns success and also populates the Originator Name from ODT Table.

Syntax:

```
sv_b = urhk_validateOrigId("")
```

sv_b: scratch pad variable to store the return value.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.origId	Originator ID (Mandatory)
BANCS.INPARAM.origDirId	Originator Directory ID (Mandatory)

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.origDirId	Originator Directory IDd.
BANCS.OUTPARAM.origName	Originator Name

BANCS.OUTPARAM.retCode

Returns '0', if the passed Originator ID is a valid one and is present in the passed Originator Directory ID.

Returns '1', if the passed Originator ID is either Invalid or is not present in the Directory ID.

©Copyright Infosys Ltd.

SIS Userhooks

This section discusses:

[Get Account Details from SIS DB](#)

[Get Customer Details from SIS DB](#)

[Selection of Fields from CSIS Database using bind variables](#)

[Updating the values in CSIS Database using bind variables](#)

©Copyright Infosys Ltd.

Get Account Details from SIS DB

This userhook is used to get the Account details of the customer from SIS DB.

Syntax:

```
sv_a= urhk_SIS_GetAcctDetails("")
```

Input Arguments:

NA

Input Repository Fields:

Field Name	Description
CDCI.Request.Foracid	Account ID
)CDCI.Request.dclId	DC ID

Output Repository Fields:

Field Name	Description
CDCI.Request.debitDclId	Debit Dc ID
CDCI.Request.debitCifId	Debit Cif ID
CDCI.Request.debitClrBalAmt	Debit Clear Balance Amount
CDCI.Request.debitUnClrBalAmt	Debit Un-Clear Balance Amount
CDCI.Request.debitAvailAmt	Debit Available Amount
CDCI.Request.debitFloatBal	Debit Float Balance
CDCI.Request.debitSanclim	Debit Sanction Limit
CDCI.Request.debitCrLimAmt	Debit Credit Limit Amount
CDCI.Request.debitFfdAvailAmt	Debit Ffd Available Amount
CDCI.Request.debitLienAmt	Debit Lien Amount

CDCI.Request.debitDrwngPwr	Debit Drawing Power
CDCI.Request.debitLastPurgeDate	Debit Last Purge Date
CDCI.Request.debitModeOfOperCode	Debit Mode Of Operation Code
CDCI.Request.debitPbPsCode	Debit Passbook-Pass sheet Code
CDCI.Request.debitFreeText	Debit Free Text
CDCI.Request.debitNomAvailFlg	Debit Nomination Available Flag
CDCI.Request.debitBalOnPurgeDate	Debit Balance On Purge Date
CDCI.Request.debitChqAlwdFlg	Debit Cheque Allowed Flag
CDCI.Request.debitHashedNo	Debit Hashed Number
CDCI.Request.debitNotionalRate	Debit Notional Rate
CDCI.Request.debitNotionalRateCode	Debit Notional Rate Code
CDCI.Request.debitAcctCrncyCode	Debit Account Currency Code
CDCI.Request.debitCrncyCode	Debit Currency Code
CDCI.Request.debitFxClrBalAmt	Debit Forex Clear Balance Amount
CDCI.Request.debitFdRefNum	Debit Fd Reference Number
CDCI.Request.debitFxCumCrAmt	Debit Forex Cum Credit Amount
CDCI.Request.debitFxCumDrAmt	Debit Forex Cum Debit Amount
CDCI.Request.debitSysOnlyAcctFlg	Debit System Only Account Flag
CDCI.Request.debitAcctMgrUserId	Debit Account Manager User ID
CDCI.Request.debitNativeLangName	Debit Native Language Name
CDCI.Request.debitNatLangTitleCode	Debit Native Language Title Code
CDCI.Request.debitLangCode	Debit Language Code
CDCI.Request.debitJointHolderCustId	Debit Joint Holder Customer ID
CDCI.Request.debitPoolId	Debit Pool ID
CDCI.Request.debitAllowSweeps	Debit Allow Sweeps
CDCI.Request.debitPartitionedFlg	Debit Partitioned Flag
CDCI.Request.debitPartitionedType	Debit Partitioned Type
CDCI.Request.debitPbfDownloadFlg	Debit Positive Balance File Download Flag
CDCI.Request.debitContribAmtToPool	Debit Contribution Amount To Pool

CDCI.Request.debitAcctCreationMode	Debit Account Creation Mode
CDCI.Request.debitProvForacid	Debit provision Foracid
CDCI.Request.debitInterSolAccessFlg	Debit Inter Sol Access Flag
CDCI.Request.debitFutureClrBalAmt	Debit Future Clear Balance Amount
CDCI.Request.debitFutureUnClrBalAmt	Debit Future Un-Clear Balance Amount
CDCI.Request.debitUtilFutureBalAmt	Debit Util Future Balance Amount
CDCI.Request.debitFutureOcTodAmt	Debit Future Occurring Tod Amount
CDCI.Request.creditCifId	Credit Cif ID
CDCI.Request.creditClrBalAmt	Credit Clear Balance Amount
CDCI.Request.creditUnClrBalAmt	Credit Un-Clear Balance Amount
CDCI.Request.creditAvailAmt	Credit Available Amount
CDCI.Request.creditFloatBal	Credit Float Balance
CDCI.Request.creditSanclim	Credit Sanction Limit
CDCI.Request.creditCrLimAmt	Credit Credit Limit Amount
CDCI.Request.creditFfdAvailAmt	Credit Ffd Available Amount
CDCI.Request.creditLienAmt	Credit Lien Amount
CDCI.Request.creditDrwngPwr	Credit Drawing Power
CDCI.Request.creditBalOnPurgeDate	Credit Balance On Purge Date
CDCI.Request.creditLastPurgeDate	Credit Last Purge Date
CDCI.Request.creditChqAlwdFlg	Credit Cheque Allowed Flag
CDCI.Request.creditModeOfOperCode	Credit Mode Of Operation Code
CDCI.Request.creditPbPsCode	Credit Passbook-Pass Sheet Code
CDCI.Request.creditFreeText	Credit Free Text
CDCI.Request.creditNomAvailFlg	Credit Nomination Available Flag
CDCI.Request.creditHashedNo	Credit Hashed Number
CDCI.Request.creditNotionalRate	Credit Notional Rate
CDCI.Request.creditNotionalRateCode	Credit Notional Rate Code
CDCI.Request.creditAcctCrncyCode	Credit Account Currency Code
CDCI.Request.creditCrncyCode	Credit Currency Code

CDCI.Request.creditFxClrBalAmt	Credit Forex Clear Balance Amount
CDCI.Request.creditFdRefNum	Credit Fd Reference Number
CDCI.Request.creditFxCumCrAmt	Credit Forex Cum Credit Amount
CDCI.Request.creditFxCumDrAmt	Credit Forex Cum Debit Amount
CDCI.Request.creditSysOnlyAcctFlg	Credit System Only Account Flag
CDCI.Request.creditAcctMgrUserId	Credit Account Manager User ID
CDCI.Request.creditNativeLangName	Credit Native Language Name
CDCI.Request.creditNatLangTitleCode	Credit Native Language Title Code
CDCI.Request.creditLangCode	Credit Language Code
CDCI.Request.creditJointHolderCustId	Credit Joint Holder Customer ID
CDCI.Request.creditPoolId	Credit Pool ID
CDCI.Request.creditAllowSweeps	Credit Allow Sweeps
CDCI.Request.creditPartitionedFlg	Credit Partitioned Flag
CDCI.Request.creditPartitionedType	Credit Partitioned Type
CDCI.Request.creditPbfDownloadFlg	Credit Positive Balance File Download Flag
CDCI.Request.creditContribAmtToPool	Credit Contribution Amount To Pool
CDCI.Request.creditAcctCreationMode	Credit Account Creation Mode
CDCI.Request.creditProvForacid	Credit Provision Foracid
CDCI.Request.creditInterSolAccessFlg	Credit Inter-Sol Access Flag
CDCI.Request.creditFutureClrBalAmt	Credit Future Clear Balance Amount
CDCI.Request.creditFutureUnClrBalAmt	Credit Future Un-Clear Balance Amount
CDCI.Request.creditUtilFutureBalAmt	Credit Util Future Balance Amount
CDCI.Request.creditFutureOcTodAmt	Credit Future Occurring Tod Amount

Return value:

This user hook can be called from the scripts(.scr).

For Example:

PreSis.scr

```
<--start  
trace on  
CDCI.Request.creditCustPermPinCode = ""  
CDCI.Request.creditCustPermCntryCode = ""  
CDCI.Request.creditCustComuPinCode = ""  
CDCI.Request.creditCustComuCntryCode = ""  
CDCI.Request.creditOfflineCumcreditLimit = ""  
CDCI.Request.creditUnClrBalAmt = ""  
CDCI.Request.creditAvailAmt = ""  
CDCI.Request.creditChqAlwdFlg = ""  
CDCI.Request.creditModeOfOperCode = ""  
CDCI.Request.creditLangCode = ""  
CDCI.Request.creditJointHolderCustId = ""  
CDCI.Request.creditPartitionedType = ""  
CDCI.Request.creditPbfDownloadFlg = ""  
CDCI.Request.debitCustCifId = ""  
CDCI.Request.creditCustCifId = ""  
CDCI.Request.foracid = "MATZ-1"  
CDCI.Request.crForacid = "JER1"  
CDCI.Request.dcId = "AB"  
sv_a=urhk_SIS_GetAcctDetails("")  
print(CDCI.Request.debitDcId)
```

```
print(CDCI.Request.debitSolId)
print(CDCI.Request.debitBacid)
print(CDCI.Request.debitAcctName)
print(CDCI.Request.debitAcctShortName)
print(CDCI.Request.debitCifId)
print(CDCI.Request.debitEmpId)
print(CDCI.Request.debitGlSubHeadCode)
print(CDCI.Request.debitAcctOwnership)
print(CDCI.Request.debitSchmCode)
print(CDCI.Request.debitSchmType)
print(CDCI.Request.debitAcctRptCode)
print(CDCI.Request.debitFrezCode)
print(CDCI.Request.debitAcctOpnDate)
print(CDCI.Request.debitFrezReasonCode)
print(CDCI.Request.debitAcctClsFlg)
print(CDCI.Request.debitAcctClsDate)
print(CDCI.Request.debitClrBalAmt)
trace off
end-->
```

Get Customer Details from SIS DB

This userhook is used to fetch the customer details using the foracid as input.

Syntax:

```
sv_a= urhk_SIS_GetCustDetails("")
```

Input arguments:

NA

Input Repository Fields:

Field Name	Description
CDCI.Request.Foracid	Debit Account ID
CDCI.Request.crForacid	Credit Account ID
CDCI.Request.dclid	Data Center ID

Output Repository Fields:

Field Name	Description
CDCI.Request.debitCustShortName	Debit Customer Short Name
CDCI.Request.debitCustTitleCode	Debit Customer Title Code
CDCI.Request.debitCustName	Debit Customer Name
CDCI.Request.debitCustPermAddr1	Debit Customer Permanent Address1
CDCI.Request.debitCustPermAddr2	Debit Customer Permanent Address2
CDCI.Request.debitCustPermCityCode	Debit Customer Permanent City Code
CDCI.Request.debitCustPermStateCode	Debit Customer Permanent State Code
CDCI.Request.debitCustPermPinCode	Debit Customer Permanent Pin Code

CDCI.Request.debitCustPermCntryCode	Debit Customer Permanent Country Code
CDCI.Request.debitCustPermPhoneNum	Debit Customer Permanent Phone Number
CDCI.Request.debitCustPermTelexNum	Debit Customer Permanent Telex Number
CDCI.Request.debitCustSex	Debit Customer Sex
CDCI.Request.debitCustGrp	Debit Customer Group
CDCI.Request.debitCustComuCityCode	debitCust Communication City Code
CDCI.Request.debitCustComuAddr1	Debit Customer Communication Address1
CDCI.Request.debitCustComuAddr2	Debit Customer Communication Address2
CDCI.Request.debitCustComuStateCode	Debit Customer Communication State Code
CDCI.Request.debitCustComuPinCode	Debit Customer Communication Pin Code
CDCI.Request.debitCustComuCntryCode	Debit Customer Communication Country Code
CDCI.Request.debitCustComuPhoneNum1	Debit Customer Communication Phone Number1
CDCI.Request.debitCustComuPhoneNum2	Debit Customer Communication Phone Number2
CDCI.Request.debitCustComuTelexNum	Debit Customer Communication Telex Number
CDCI.Request.debitCustOccpCode	Debit Customer Occupation Code
CDCI.Request.debitCustFirstAcctDate	Debit Customer First Account Date
CDCI.Request.debitCustRatingCode	Debit Customer Rating Code
CDCI.Request.debitCustRatingDate	Debit Customer Rating Date
CDCI.Request.debitCustMgrOpin	Debit Customer Manager Opinion
CDCI.Request.debitTotTodAlwdTimes	Debit Total Tod Allowed Times
CDCI.Request.debitCustTypeCode	Debit Customer Type Code
CDCI.Request.debitCustEmplId	Debit Customer Employee ID
CDCI.Request.debitCustStatCode	Debit Customer Status Code
CDCI.Request.debitCustStatChgDate	Debit Customer Status Change Date
CDCI.Request.debitCustConst	Debit Customer Constitution
CDCI.Request.debitCustMinorFlg	Debit Customer Minor Flag
CDCI.Request.debitCustNreFlg	Debit Customer Nre Flag
CDCI.Request.debitCustHlthCode	Debit Customer Health Code

CDCI.Request.debitNumOfAccounts	Debit Number Of Accounts
CDCI.Request.debitNatIdCardNum	Debit National ID Card Number
CDCI.Request.debitDateOfBirth	Debit Date Of Birth
CDCI.Request.debitPanGirNum	Debit PanGir Number
CDCI.Request.debitPsprtNum	Debit Passport Number
CDCI.Request.debitPsprtIssuDate	Debit Passport Issue Date
CDCI.Request.debitPsprtDet	Debit Passport Details
CDCI.Request.debitPsprtExpDate	Debit Passport Expiry Date
CDCI.Request.debitCustPagerNo	Debit Customer Pager Number
CDCI.Request.debitCustFaxNo	Debit Customer Fax Number
CDCI.Request.debitCustAcctMgrUserId	Debit Customer Account Manager User ID
CDCI.Request.debitPartyFlg	Debit Party Flag
CDCI.Request.debitEmailId	Debit Email Id
CDCI.Request.debitOfflineCumDebitLimit	Debit Offline Cum Debit Limit
CDCI.Request.debitCustCrncyCode	Debit Customer Currency Code
CDCI.Request.debitCustSolId	Debit Customer Sol ID
CDCI.Request.debitCustAllowSweeps	Debit Customer Allow Sweeps
CDCI.Request.debitDespatchMode	Debit Despatch Mode
CDCI.Request.debitCustCreationMode	Debit Customer Creation Mode
CDCI.Request.debitProvCustId	Debit Provision Customer ID
CDCI.Request.debitCustIntrodCustId	Debit Customer Introducer's Customer ID
CDCI.Request.debitIntrodTitleCode	Debit Introducer's Title Code
CDCI.Request.debitCustIntrodName	Debit Customer Introducer's Name
CDCI.Request.debitCustIntrodStatCode	Debit Customer Introducer's Status Code
CDCI.Request.debitCustCifId	Debit Customer Cif ID
CDCI.Request.creditCustShortName	Credit Customer Short Name
CDCI.Request.creditCustTitleCode	Credit Customer Title Code
CDCI.Request.creditCustName	Credit Customer Name

CDCI.Request.creditCustPermAddr1	Credit Customer Permanent Address1
CDCI.Request.creditCustPermAddr2	Credit Customer Permanent Address2
CDCI.Request.creditCustPermCityCode	Credit Customer Permanent City Code
CDCI.Request.creditCustPermStateCode	Credit Customer Permanent State Code
CDCI.Request.creditCustPermPinCode	Credit Customer Permanent Pin Code
CDCI.Request.creditCustPermCntryCode	Credit Customer Permanent Country Code
CDCI.Request.creditCustPermPhoneNum	Credit Customer Permanent Phone Number
CDCI.Request.creditCustPermTelexNum	Credit Customer Permanent Telex Number
CDCI.Request.creditCustSex	Credit Customer Sex
CDCI.Request.creditCustGrp	Credit Customer Group
CDCI.Request.creditCustComuAddr1	Credit Customer Communication Address1
CDCI.Request.creditCustComuAddr2	Credit Customer Communication Address2
CDCI.Request.creditCustComuCityCode	Credit Customer Communication City Code
CDCI.Request.creditCustComuStateCode	Credit Customer Communication State Code
CDCI.Request.creditCustComuPinCode	Credit Customer Communication Pin Code
CDCI.Request.creditCustComuCntryCode	Credit Customer Communication Country Code
CDCI.Request.creditCustComuPhoneNum1	Credit Customer Communication Phone Number1
CDCI.Request.creditCustComuPhoneNum2	Credit Customer Communication Phone Number2
CDCI.Request.creditCustComuTelexNum	Credit Customer Communication Telex Number
CDCI.Request.creditCustOccpCode	Credit Customer Occupation Code
CDCI.Request.creditCustFirstAcctDate	Credit Customer First Account Date
CDCI.Request.creditCustRatingCode	Credit Customer Rating Code
CDCI.Request.creditCustRatingDate	Credit Customer Rating Date
CDCI.Request.creditCustMgrOpin	Credit Customer Manager Opinion
CDCI.Request.creditTotTodAlwdTimes	Credit Total Tod Allowed Times
CDCI.Request.creditCustTypeCode	Credit Customer Type Code
CDCI.Request.creditCustEmpld	Credit Customer Employee ID

CDCI.Request.creditCustStatCode	Credit Customer Status Code
CDCI.Request.creditCustStatChgDate	Credit Customer Status Change Date
CDCI.Request.creditCustConst	Credit Customer Constitution
CDCI.Request.creditCustMinorFlg	Credit Customer Minor Flag
CDCI.Request.creditCustNreFlg	Credit Customer Nre Flag
CDCI.Request.creditCustHlthCode	Credit Customer Health Code
CDCI.Request.creditNumOfAccounts	Credit Number Of Accounts
CDCI.Request.creditNatIdCardNum	Credit National ID Card Number
CDCI.Request.creditDateOfBirth	Credit Date Of Birth
CDCI.Request.creditPanGirNum	Credit PanGir Number
CDCI.Request.creditPsprtNum	Credit Passport Number
CDCI.Request.creditPsprtIssuDate	Credit Passport Issue Date
CDCI.Request.creditPsprtDet	Credit Passport Details
CDCI.Request.creditPsprtExpDate	Credit Passport Expiry Date
CDCI.Request.creditCustPagerNo	Credit Customer Pager Number
CDCI.Request.creditCustFaxNo	Credit Customer Fax Number
CDCI.Request.creditCustAcctMgrUserId	Credit Customer Account Manager User ID
CDCI.Request.creditPartyFlg	Credit Party Flag
CDCI.Request.creditEmailId	Credit Email ID
CDCI.Request.creditOfflineCumdebitLimit	Credit Offline Cum Debit Limit
CDCI.Request.creditCustCrncyCode	Credit Customer Currency Code
CDCI.Request.creditCustSolId	Credit Customer Sol ID
CDCI.Request.creditCustAllowSweeps	Credit Customer Allow Sweeps
CDCI.Request.creditDespatchMode	Credit Despatch Mode
CDCI.Request.creditCustCreationMode	Credit Customer Creation Mode
CDCI.Request.creditProvCustId	Credit Provision Customer ID
CDCI.Request.creditCustIntrodCustId	Customer Introducer's Customer ID
CDCI.Request.creditIntrodTitleCode	Credit Introdocer's Title Code
CDCI.Request.creditCustIntrodName	Credit Customer Introdocer's Name

CDCI.Request.creditCustIntrodStatCode	Credit Customer Introdocer's Status Code
CDCI.Request.creditCustCifId	Credit Customer Cif ID
CDCI.Request.usrhkRet	User Hook Return Value
CDCI.Request.usrhkErr	User Hook Error

Return value:

This user hook can be called from the scripts(.scr).

For Example:

PreSis.scr

```
<--start
```

```
trace on
```

```
CDCI.Request.creditCustPermPinCode = ""
```

```
CDCI.Request.creditCustPermCntryCode = ""
```

```
CDCI.Request.creditCustComuPinCode = ""
```

```
CDCI.Request.creditCustComuCntryCode = ""
```

```
CDCI.Request.creditOfflineCumcreditLimit = ""
```

```
CDCI.Request.creditUnClrBalAmt = ""
```

```
CDCI.Request.creditAvailAmt = ""
```

```
CDCI.Request.creditChqAlwdFlg = ""
```

```
CDCI.Request.creditModeOfOperCode = ""
```

```
CDCI.Request.creditLangCode = ""
```

```
CDCI.Request.creditJointHolderCustId = ""
```

```
CDCI.Request.creditPartitionedType = ""
```

```
CDCI.Request.creditPbfDownloadFlg = ""
```



```
CDCI.Request.debitCustCifId = ""
CDCI.Request.creditCustCifId = ""
CDCI.Request.foracid = "MATZ-1"
CDCI.Request.crForacid = "JER1"
CDCI.Request.dcId = "AB"
sv_a=urhk_SIS_GetCustDetails("")
print(CDCI.Request.debitDcId)
print(CDCI.Request.debitSolId)
print(CDCI.Request.debitBacid)
print(CDCI.Request.debitAcctName)
print(CDCI.Request.debitAcctShortName)
print(CDCI.Request.debitCifId)
print(CDCI.Request.debitEmpId)
print(CDCI.Request.debitGlSubHeadCode)
print(CDCI.Request.debitAcctOwnership)
print(CDCI.Request.debitSchmCode)
print(CDCI.Request.debitSchmType)
print(CDCI.Request.debitAcctRptCode)
print(CDCI.Request.debitFrezCode)
print(CDCI.Request.debitAcctOpnDate)
print(CDCI.Request.debitFrezReasonCode)
print(CDCI.Request.debitAcctClsFlg)
print(CDCI.Request.debitAcctClsDate)
```

```
print(CDCI.Request.debitClrBalAmt)
trace off
end-->
```

©Copyright Infosys Ltd.

Selection of Fields from CSIS Database using Bind Variables

This userhook allows script writer to select data from CSIS database tables. The user specifies the title of each selected field separated by comma before pipe symbol and specifies the complete query after that.

Syntax:

```
sv_r = urhk_SIS_dbSelectWithBind(variable)
```

The variable can be a scratch pad variable (sv_a) or a string (acct | select foracid from gamsis where cust_id = ?SVAR).

Here note that everything before '|' (pipe symbol) is taken as the title value whereas post pipe symbol is taken as query.

Input Arguments:

Comma separated Field titles followed by the Query, separated by pipe symbol. The field symbols must not have spaces.

Input Repository Fields:

BANCS.INPARAM.BINDVARS

The inputs should be passed to the SQL query through BANCS.INPARAM.BINDVARS with delimiter as "|". In the query the following placeholders are supported:

- ❏SVAR: String bind values (including VARCHAR, CHAR, DATE data types)

- ❏NVAR: Numeric bind values (for NUMBER data type)

Output Repository Fields:

Output not present.

Return Value:

The output fields are stored in CDCI.OUTPARAM.<title field>.

For Example:

For selecting scheme and currency for an account and bankid.

```
BANCS.INPARAM.BINDVARS = sv_a + "|" + sv_b
```

```
sv_s = "schmCode, crncyCode | "
```

```
sv_s = sv_s + "SELECT schm_code, acct_crncy_code "
```

```
sv_s = sv_s + "FROM gamsis WHERE foracid = ?SVAR "
```

```
sv_s = sv_s + "AND bank_id = ?SVAR "
```

```
sv_r = urhk_SIS_dbSelectWithBind(sv_b)
```

The above will fetch the value of acct in CDCI.OUTPARAM.acct field.

©Copyright Infosys Ltd.

Updating the values in CSIS Database using Bind Variables

This userhook allows script writer to perform DML Statements on the CSIS database tables.

Syntax:

```
sv_r = urhk_SIS_dbSQLWithBind(variable)
```

The variable can be a scratch pad variable (sv_a) or a string ("update gamsis Set acct_cls_flg = 'N' where foracid = ?SVAR").

Input Arguments:

SQL Statement

Input Repository Fields:

BANCS.INPARAM.BINDAVARS

The inputs should be passed to the SQL query through BANCS.INPARAM.BINDVARS with delimiter as "|". In the query the following placeholders are supported:

- ❏SVAR: String bind values (including VARCHAR, CHAR, DATE data types)
- ❏NVAR: Numeric bind values (for NUMBER data type)

For Example:

For updating the account closure flag for an account.

```
BANCS.INPARAM.BINDVARS = sv_a + "|" + sv_b
```

```
sv_s = "UPDATE gamsis "
```

```
sv_s = sv_s + "SET "  
sv_s = sv_s + "acct_cls_flg = ?SVAR "  
sv_s = sv_s + "WHERE foracid = ?SVAR "  
sv_r = urhk_SIS_dbSQLWithBind(sv_s)
```

Output Repository Fields:

Output not present.

©Copyright Infosys Ltd.

FI Related Userhooks

This section discusses:

- [Validate For Duplicate Transaction](#)
- [Fetching Channel audit information from Finacle Core](#)
- [Make Outbound Calls to FI Services using LIMOC APIs and Connection Caching](#)

©Copyright Infosys Ltd.

Validate For Duplicate Transaction

The user hook `validateForDuplicateTransaction` is used to enable duplicate transaction check and enables Channel User Auditing for `executeFinacleScript` API

This user hook helps the user ascertain whether a particular request is a duplicate request or a new one and enables channel user auditing for a new channel request.

Syntax:

```
sv_a = urhk_ validateForDuplicateTransaction("")  
sv_a: scratch pad variable to store the return value.
```

Input Arguments:

None

Input Repository Fields:

None

Output Repository Fields:

Field Name	Description
CUSTSCR.OutputDetails.insertUTTRec	If the value of <code>insertUTTRec</code> variable in <code>CUSTSCR_REPOSITORY</code> is set to "Y" ie. if <code>CUSTSCR.OutputDetails.insertUTTRec</code> is "Y" insertion of channel data in UTT table will be enabled. If the value of <code>insertUTTRec</code> variable in <code>CUSTSCR_REPOSITORY</code> is set to "N" ie. if <code>CUSTSCR.OutputDetails.insertUTTRec</code> is "N" insertion of channel data in UTT table will be disabled.
BANCS.OUTPARAM.DcclId	Channel Id

BANCS.OUTPARAM.DccReqRefNum	Request Reference Number
BANCS.OUTPARAM.DccReqType	Request Type
BANCS.OUTPARAM.DccReqOriginTime	Request Origination Time
BANCS.OUTPARAM.UserId	Channel User Id
BANCS.OUTPARAM.successOrFailure	If there are no records found in UTT table for the given combination of dcc_id, request_reference_num, request_orig_time, request_type and bank id, the value set in this field will be "F". If records are found in UTT table for the above combination, the value set in successOrFailure field will be "S".

Return Value:

NA

Examples:

For Example:

```
<--start
trace on
sv_q = urhk_b2k_printRepos("CUSTSCR")
print(sv_q)
sv_f = urhk_validateForDuplicateTransaction ("" )
print(sv_f)
print(BANCS.OUTPARAM.successOrFailure)
if(BANCS.OUTPARAM.successOrFailure=="S") then
#{
print(BANCS.OUTPARAM.DccId)
print(BANCS.OUTPARAM.DccReqRefNum)
print(BANCS.OUTPARAM.DccReqType)
```

```

print(BANCS.OUTPARAM.DccReqOriginTime)
#}
endif
if(BANCS.OUTPARAM.successOrFailure=="F") then
#{
sv_a="DBS"
    sv_u= urhk_SetUrhkInp("bankDet.bankCode.bankCode|" +
sv_a)

                                sv_c =
urhk_SetUrhkOut("bankDet.bankCode.bankName|bankName")

                                sv_c =
urhk_SetUrhkOut("bankDet.bankCode.bankCode|bankCode")
    sv_s = urhk_ExecSrv("SRV_InquireBankDtl|retain_all_output
= Y")
    if(sv_s==0) then
        sv_y=BANCS.OUTPARAM.bankName
        sv_c=BANCS.OUTPARAM.bankCode
        BANCS.OUTPUT.successOrFailure="S"
        print(sv_y)
        print(sv_c)
    sv_t = "Bank_Name |" + sv_y
    sv_u = urhk_SetOrbOut(sv_t)
    sv_a = "Bank_Code |" + sv_c
    sv_u = urhk_SetOrbOut(sv_a)
    print(sv_u)
#}
endif

```

```
sv_q = urhk_b2k_printRepos("CUSTSCR")
```

```
print(sv_q)
```

```
trace off
```

```
exitscript
```

```
end-->
```

Note:

If validateForDuplicateTransaction user hook call is made and the channel request is a new one, insertUTTRec variable of CUSTSCR_REPOSITORY will be set to "Y" in the user hook.

If the value of insertUTTRec variable in CUSTSCR_REPOSITORY is set to "Y", the channel data is inserted into UTT table

©Copyright Infosys Ltd.

Fetching Channel audit information from Finacle Core

The user hook `getChannelUserAuditDetails` is used to fetch audit information from Finacle core for a particular channel and channel user Id helps the user ascertain whether a particular request is a duplicate request or a new one. If the request has already been successfully executed, its entry will be present in Finacle Core table (UTT table).

For a particular header information, the user can get back the details like channel user id which executed that particular request, the audit reference number and the transaction id generated by that particular request.

Syntax:

```
sv_a = urhk_getChannelUserAuditDetails ("")
```

sv_a: scratch pad variable to store the return value.

Input Arguments:

None

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.dccld	Channel Id
BANCS.INPARAM.reqRefNum	Request Reference Number
BANCS.INPARAM.reqType	Request Type ie. API name. For eg, XferTrnAdd, executeFinacleScript, AcctLienMod, etc.

Output Repository Fields:

Field Name	Description
------------	-------------

BANCS.OUTPARAM.auditRefnum	Audit Reference Number
BANCS.OUTPARAM.userId	Channel User Id
BANCS.OUTPARAM.NoOfTran	Number of transaction ids present in the tran list..
BANCS.OUTPARAM.errorMesg	Error Message will be populated only in case of failure.
BANCS.OUTPARAM.successOrFailure	Success Or Failure Flag
BANCS.OUTPARAM.tranId_counter	tranId_counter. Here the value of counter can be 1, 2, 3, etc. For eg- The output field will be BANCS.OUTPARAM.tranId_1, BANCS.OUTPARAM.tranId_2, BANCS.OUTPARAM.tranId_3 depending on the number of tran ids fetched from the database.
BANCS.OUTPARAM.additionalInfo	Additional Information

Return Value:

NA

Examples:

For Example:

```
<--start
trace on
BANCS.INPARAM.dccId = "CRM"
BANCS.INPARAM.reqType = "XferTrnAdd"
BANCS.INPARAM.reqRefNum = "REQ_12556222"
#BANCS.INPARAM.reqOrigTime = "10-10-2007 10:07:56"
sv_f = urhk_getChannelUserAuditDetails("")
print(sv_f)
if(BANCS.OUTPARAM.successOrFailure=="S") then
#{
IF(REPEXISTS("TEMPREP") != 1) then
```

```

#{
    CREATEREP("TEMPREP")
#}
ELSE
#{
    DELETEREP("TEMPREP")
    CREATEREP("TEMPREP")
#}
ENDIF

#-----
#Creation of Class
#-----

IF (CLASSEXISTS("TEMPREP","VAR") == 0) THEN
#{
    CREATECLASS("TEMPREP","VAR",5)
#}
ENDIF

if(FIELDEXISTS(BANCS.OUTPARAM.auditRefnum)==1) then
#{
TEMPREP.VAR.auditRefnum=BANCS.OUTPARAM.auditRefnum
#}
endif

if(FIELDEXISTS(BANCS.OUTPARAM.userId)==1) then
#{
TEMPREP.VAR.userId=BANCS.OUTPARAM.userId

```

```
#}  
endif  
if(FIELDEXISTS(BANCS.OUTPARAM.additionalInfo)==1) then  
#{  
TEMPREP.VAR.additionalInfo=BANCS.OUTPARAM.additionalInfo  
#}  
endif  
sv_h=FIELDEXISTS(BANCS.OUTPARAM.NoOfTran)  
print(sv_h)  
if(FIELDEXISTS(BANCS.OUTPARAM.NoOfTran)==1) then  
#{  
sv_k=BANCS.OUTPARAM.NoOfTran  
print(sv_k)  
sv_i=1  
while (sv_i<=sv_k)  
#{  
print("INSIDE WHILE")  
sv_t = "tranId_" + FORMAT$(sv_i,"%d")  
print(sv_t)  
sv_v=("BANCS").("OUTPARAM").(sv_t)  
print(sv_v)  
("TEMPREP").("VAR").(sv_t)=sv_v  
sv_i=sv_i + 1  
#}  
do  
sv_o=1
```

```
while(sv_o<=sv_k)
#{
    sv_t = "tranId_" + FORMAT$(sv_o,"%d")
    print(sv_t)
    sv_a = ("TEMPREP").("VAR").(sv_t)
    print(sv_a)
    sv_j = sv_t + "|" + sv_a
    sv_u = urhk_SetOrbOut(sv_j)
    sv_o= sv_o + 1
#}
dO
#}
endif
if(FIELDEXISTS(TEMPREP.VAR.userId)==1) then
#{
    sv_t=TEMPREP.VAR.userId
    print(sv_t)
    sv_j = "UserId|" + sv_t
    sv_u = urhk_SetOrbOut(sv_j)
#}
endif
if(FIELDEXISTS(TEMPREP.VAR.additionalInfo)==1) then
#{
    sv_t=TEMPREP.VAR.additionalInfo
    print(sv_t)
    sv_j = "Additional_Info|" + sv_t
```



```
sv_u = urhk_SetOrbOut(sv_j)
#}
endif
if(FIELDEXISTS(TEMPREP.VAR.auditRefnum)==1) then
#{
sv_t=TEMPREP.VAR.auditRefnum
print(sv_t)
sv_j = "AuditRefnum |" + sv_t
print(sv_j)
sv_u = urhk_SetOrbOut(sv_j)
print(sv_u)
#}
endif
#}
else
#{
print(BANCS.OUTPUTPARAM.errorMesg)
sv_c=BANCS.OUTPUTPARAM.errorMesg
sv_a = "errorMesg |" + sv_c
sv_u = urhk_SetOrbOut(sv_a)
#}
endif
trace off
exitscript
end-->
```

©Copyright Infosys Ltd.

Finacle Copyright

Make Outbound Calls to FI Services using LIMOC APIs and Connection Caching

This user hook uses the Limo Client APIs to perform following operations:

- Read the client configuration file.
- Open a TCP/IP connection with FI server.
- Send the Message over the connection and receive the response or only send the message w/o waiting for acknowledgement.

This user hook accesses configuration file name from environment variable <Service name>_CONF_FIL_PATH. Configuration File is read. Connection is opened with the FI Server if previous step is successful. Connection Index from previous step is stored in Global repository variable for connection caching, that is, after opening connection it won't be closed. If a request comes to same instance, connection is reused won't need to reopen it, thus saving time. Message is sent using Limo Client API, receiving response depends upon flag sent as input parameter to user hook. In case of error occurs during Limo Client API, corresponding error message is sent back to script in a repository variable.

Note / Constraints / Limitations

This user hook is available to be used in any custom scripts and no Screen / UI and FI API I/F's are provided as a part of this enhancement. Client configuration file for reading server details like IP, Port number is needed. Environment variable for Client Configuration file will need to be exported from Core Services start script.

Syntax:

The syntax of user hook usage is as follows

```
sv_a = urhk_ConnectToFIAndSendMsg(sv_c)
```

sv_a: scratch pad variable to store the return value.

sv_c: message to be sent.

Input Arguments:

■ Service Name

■ Send or Receive Flag

■ Message

Service name and send or receive flag inputs are to be populated into the repositories.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM. SERVICE_NAME	Valid Account Number in GAM table
BANCS.INPARAM. SEND_RECV_FLG	As on Date(in DD-MM-YYYY format) in GAM table

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM. ErrMsg	Account balance in GAM table

Return Value:

If the input values are invalid then the return value is 1.

For Example:

<- start

trace on

While parsing script RHS can hold only 1024 bytes, so RHS is having 1024 characters in below line

```
sv_a="aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
sv_b=sv_a
sv_c=sv_a+sv_b
sv_c=sv_c+sv_a+sv_b+"EOM"
BANCS.INPARAM.SERVICE_NAME="FIJCA"
BANCS.INPARAM.SEND_RECV_FLG="R"
sv_i=urhk_ConnectToFIAndSendMsg(sv_c)
if(sv_i==0) then
    if(BANCS.OUTPARAM.ErrMsg!="") then
        print("Error thrown from the user hook is
["+BANCS.OUTPARAM.ErrMsg+"]")
    endif
endif
trace off
end ->
```

©Copyright Infosys Ltd.

Others

[Enabling Debug Logs](#)

[To move files using scripts](#)

[ProcessOutboundMessage](#)

[FIJCA Connectivity Userhook](#)

©Copyright Infosys Ltd.

To Move Files Using Scripts

This userhook allows script writer to move files from one location to another.

Syntax:

```
sv_e=urhk_moveFile("")
```

Input Arguments:

Input Arguments not required

Input Repository Fields:

BANCS.INPARAM.fromFileName: This should hold the filename along with the path from the file should be picked up.

BANCS.INPARAM.toFileName: This should hold the filename along with the path where the file should be placed.

Output Repository Fields:

Output not present.

Return Value:

A success or failure message is stored in BANCS.OUTPARAM.successOrFailure

For Example:

```
BANCS.INPARAM.fromFileName="/finacle/finadm"+"1.txt"
```

```
print(BANCS.INPARAM.fromFileName)
```

```
BANCS.INPARAM.toFileName="/finacle/finadm"+"1.txt"
```

```
print(BANCS.INPARAM.toFileName)  
sv_e=urhk_moveFile("")  
print(BANCS.OUTPARAM.successOrFailure
```

©Copyright Infosys Ltd.

ProcessOutboundMessage

This user hook is used to send a complete XML message to Finacle Integrator. The XML should have a Header and a Body.

Syntax:

urhk_ProcessOutboundMessage(Request_XML)

Where,

urhk_ProcessOutboundMessage is the name of the user hook

Request_XML contains the XML message to be sent to Finacle Integrator.

Input Arguments:

The following input fields are available for customizing this User Hook:

Input Field	Description
Request XML	Complete XML message that must be sent to FI. This must contain the FI Header and Body.

Request XML must be in the format described in following sections. This must have mandatory fields for routing message to correct service provider:

- Request UUID
- Service Request ID
- Service Request Version
- Channel ID
- Bank ID
- Message creation date and Time

Input Repository Fields:

No input repository fields present

Output Repository Fields:

The output of the user hook is accessed using the following format:

BANCS.OUTPUTPARAM.<Output Field>

The following output fields are available for customizing this user hook:

Output Field	Description
successOrFailure	Returns the status of the user hook. Returns 0 on Success and 1 on Failure.
outputMessage	Response XML message from Finacle Integrator.
addlMessage	Additional response from Finacle Integrator.
errorCode	Error code returned by Finacle Integrator.
errorMesg	Error message returned by Finacle Integrator.

©Copyright Infosys Ltd.

FIJCA Connectivity Userhook

This user hook uses the Limo Client APIs to perform following operations:

- Read the client configuration file.
- Open a TCP/IP connection with FI server.
- Send the Message over the connection and receive the response.

Syntax:

The syntax of user hook usage is as follows

```
sv_a = urhk_ConnectToFIAndSendMsg("message");
```

Input Arguments:

- The input is the message to be sent.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.SERVICE_NAME	Name of FI service to be called
BANCS.INPARAM.SEND_RECV_FLG	Send/receive flag – identifies in which the message is sent

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.errorMsg	Error Message

Return Value:

If error occurs during Limo Client API, corresponding error message is sent back to BANCS.OUTPARAM.errorMesg field.

©Copyright Infosys Ltd.

Online Scripting Userhook

■ [To move data to the frontend](#)

■ [To set the environment variables for use in the com script.](#)

■ [To Execute the com script in background](#)

■ [SetOrbError](#)

■ [SetOrbWarning](#)

©Copyright Infosys Ltd.

To Move the Data to the Frontend

This userhook allows script writer to fetch the data from the database and pass the same to the frontend.

Syntax:

```
sv_a=urhk_SetOrbOut("name of the control in the jsp page | Value for the control from database/script")
```

Input Arguments:

- Name of the control in the jsp page
- Value for the control from database/script

Input Repository Fields:

Input repository fields not required

Output Repository Fields:

Output not present.

Return Value:

No return value

For Example:

To send error message to the frontend

```
sv_a = urhk_SetOrbOut ("Message| Error Message from script")
```

To send result message to the frontend

```
sv_a = urhk_SetOrbOut ("Message| Error Message from script")
```

To fetch the data from the database and send it to the frontend

urhk_SetOrbOut("SolID|" + BANCS.INPUT.Solid)

Where SolID is the fieldname.

©Copyright Infosys Ltd.

To Set the Environment Variables for Use in the Com Script.

This user hook is provided to set the environment variables for use in the com script.

Syntax:

```
sv_a=Urhk_putEnvForCallShellScr("<name|value>")
```

Input Arguments:

<name|value>

Input Repository Fields:

No Input Repository Fields

Output Repository Fields:

No Output repository Fields

Return Value:

This user hook will return 0 on successful completion and 1 on failure.

For Example:

The values of 'com script' name and the 'arguments' have to be explicitly assigned to BANCS.INPARAM.scriptName and BANCS.INPARAM.arguments respectively. An example is shown below

```
sv_a = BANCS.INPUT.scriptName
```

```
BANCS.INPARAM.scriptName = sv_a
```

And


```
sv_b = BANCS.INPUT.arguments.  
BANCS.INPARAM.arguments = sv_b.
```

Note:

If no argument to the com script is required, then assign a blank to BANCS.INPARAM.arguments like:

BANCS.INPARAM.arguments = ""

The user can set the necessary environment variables by calling the user hook `Urhk_putEnvForCallShellScr("<name|value>")`. The com script now can access the value of this environment variable.

For Example:

```
Urhk_putEnvForCallShellScr("MY_VARIABLE| hello")
```

After setting the necessary environment using `putEnvForCallShellScr()`, the User has to call the user hook `CallShellScript()` which takes the value of mode of execution as parameter as foreground or background

For Example:

`urhk_CallShellScript("f")` - for FOREGROUND execution

`urhk_CallShellScript("b")` for BACKGROUND execution.

Note:

The default mode of Execution is BACKGROUND. Using like `urhk_CallShellScript("")` will execute the com script in back ground.

The user hook `urhk_CallShellScript("")` gets the name of com script from `BANCS.INPARAM.scriptName` and arguments from `BANCS.INPARAM.arguments` only. Hence before calling the

urhk_CallShellScript("") these two must be populated properly.

The com script and the batch program which is called in the com script must be written by the user. If the arguments required are more, then the com script can make use of environment variables which were set using `Urhk_putEnvForCallShellScript("name|value")` (provided the value which is required as argument is set into the variable)

Or

If the number of arguments required is more, then a file containing the argument list can be passed as a parameter. The com script must take care of reading the file and use the contents accordingly

The first argument to the com script will always be the job_id, so any user passed arguments to the com script is only after that. In other words, \$1 for the com script is always the job_id (session_id).

Note:

If the userhook `Urhk_putEnvForCallShellScript()` is used to set the environment variables and `urhk_CallShellScript("")` is used to invoke the com script, then the life of those environment variables will end at the end of com script. This works for the combination of these two userhooks.

©Copyright Infosys Ltd.

To Execute the Com Script in Background

To execute the batch programs in the background mode a user hook is provided. This by default executes the com script in background mode.

Syntax:

sv_a=urhk_CallShellScript("f") - for FOREGROUND execution

sv_b=urhk_CallShellScript("b") for BACKGROUND execution

Input Arguments:

f – for FOREBROUND execution

b – for BACKGROUND execution

The first argument to the com script will always be the job_id, so any user passed arguments to the com script is only after that. In other words, \$1 for the com script is always the job_id (session_id).

Input Repository Fields:

BANCS.INPARAM.scriptName

BANCS.INPARAM.arguments

Output Repository Fields:

No Output repository Fields

Return Value:

This user hook will return 0 on successful completion and 1 on failure.

For Example refer the userhook: [urhk_putEnvForCallShellScript](#).

©Copyright Infosys Ltd.

Finacle Copyright

SetOrbError

This user hook sets error when called from the custom script.

Syntax:

```
sv_a = urhk_setOrbErr(<ValidMsgId>)
```

Input:

Valid Message ID.

Output:

Sets error when called from the custom script.

For Example:

```
<--start  
sv_o = "ErrorMesg" + "|" + "System Error Please Contact  
Adminisrator"  
sv_z = urhk_SetOrbErr(sv_o)  
end - ->
```

©Copyright Infosys Ltd.

SetOrbWarning

This user hook sets warning when called from the custom script.

Syntax:

```
sv_a = urhk_setOrbWarning(<ValidMsgId>)
```

Input:

Valid Message ID

Output:

Sets warning when called from the custom script

For Example:

```
<--start  
sv_o = "ErrorMesg" + "|" + "System Error Please Contact  
Adminisrator"  
sv_z = urhk_SetOrbWarning(sv_o)  
end - ->
```

©Copyright Infosys Ltd.

Referential Rate Exchange Userhook

This section discusses:

[Get Tax Rate](#)

[GetFxRate](#)

©Copyright Infosys Ltd.

GetFxRate

Used by the DealetteDetails.scr, urhk_GetFxRate calculates the FX rate based on the currency pair, calendar set of each of the currencies and dates for which the rate is required.

Syntax:

```
sv_a=urhk_GetFxRate(<RegisteredField>|<currency pair>|<ccy1 calendar>|<ccy2 calendar>|<date>)
```

Where:

RegisteredField is the FX dealette.

currency pair is the pair for which the FX rate is required.

ccy1 calendar is the calendar set for Ccy 1 of the currency pair.

ccy2 calendar is the calendar set for Ccy 2 of the currency pair.

date is the date for which the rate is required.

Input Arguments:

The following input field is available for customizing this user hook:

Input Field	Description
String value	A string with the values required to fetch the FX rate.

Input Repository Fields:

Input Repository Fields not present.

Output Repository Fields:

The output of this user hook is available in the following format:

BANCS.OUTPUTPARAM<Output Field>

The following output fields are available for customizing this user hook

Output Field	Description
FxRate	The FX rate calculated

©Copyright Infosys Ltd.

Userhook to Fetch and Update UK_WTR table

The userhook fetchAndUpdateUK_WTR is used for maintenance of fields for Tax reporting menu (BGWTR) . This user hook will fetch the data from UK_WTR table based on the input values and pass it to the customization script and then accepts the modified data and update it back in UK_WTR table.

In case of Fetch:

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctNum	Account number
BANCS.INPARAM.cifId	Cif ID
BANCS.INPARAM.tax_year_end_date	Tax Year End Date
BANCS.INPARAM.fetchOrUpdateFlg	fetchOrUpdateFlg

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.financial_institution	Account number
BANCS.OUTPARAM.branch_name	branch_name
BANCS.OUTPARAM.branch_code	branch_code
BANCS.OUTPARAM.tax_year	tax_year
BANCS.OUTPARAM.sort_code	sort_code
BANCS.OUTPARAM.foracid	foracid
BANCS.OUTPARAM.acct_name	acct_name
BANCS.OUTPARAM.acct_addr1	acct_addr1
BANCS.OUTPARAM.acct_addr2	acct_addr2
BANCS.OUTPARAM.acct_addr3	acct_addr3

BANCS.OUTPARAM.acct_city_code	acct_city_code
BANCS.OUTPARAM.acct_state_code	acct_city_code
BANCS.OUTPARAM.acct_cntry_code	acct_cntry_code
BANCS.OUTPARAM.acct_post_code	acct_post_code
BANCS.OUTPARAM.product_marker_flg	product_marker_flg
BANCS.OUTPARAM.r85_flg	r85_flg
BANCS.OUTPARAM.new_acct_signal_flg	new_acct_signal_flg
BANCS.OUTPARAM.r105_flg	r105_flg
BANCS.OUTPARAM.gross_interest_amt	gross_interest_amt
BANCS.OUTPARAM.tax_deducted_amt	tax_deducted_amt
BANCS.OUTPARAM.crncy_code	crncy_code
BANCS.OUTPARAM.no_of_acct_holders	no_of_acct_holders
BANCS.OUTPARAM.cust_title	cust_title
BANCS.OUTPARAM.cust_first_name	cust_first_name
BANCS.OUTPARAM.cust_middle_name	cust_middle_name
BANCS.OUTPARAM.cust_surname	cust_surname
BANCS.OUTPARAM.cust_pref_addr1	cust_pref_addr1
BANCS.OUTPARAM.cust_pref_addr2	cust_pref_addr2
BANCS.OUTPARAM.cust_pref_addr3	cust_pref_addr3
BANCS.OUTPARAM.cust_pref_city_code	cust_pref_city_code
BANCS.OUTPARAM.cust_pref_state_code	cust_pref_state_code
BANCS.OUTPARAM.cust_pref_cntry_code	cust_pref_cntry_code
BANCS.OUTPARAM.cust_pref_post_code	cust_pref_post_code
BANCS.OUTPARAM.national_insurance_no	national_insurance_no
BANCS.OUTPARAM.date_of_birth	date_of_birth
BANCS.OUTPARAM.tax_identification_no	tax_identification_no
BANCS.OUTPARAM.cntry_code_of_birth	cntry_code_of_birth

BANCS.OUTPARAM.place_of_birth	place_of_birth
BANCS.OUTPARAM.report_category	report_category
BANCS.OUTPARAM.reporting_cntry	reporting_cntry
BANCS.OUTPARAM.cust_residential_addr1	cust_residential_addr1
BANCS.OUTPARAM.cust_residential_addr2	cust_residential_addr2
BANCS.OUTPARAM.cust_residential_addr3	cust_residential_addr3
BANCS.OUTPARAM.cust_residential_city_code	cust_residential_city_code
BANCS.OUTPARAM.cust_residential_state_code	cust_residential_state_code
BANCS.OUTPARAM.cust_residential_cntry_code	cust_residential_cntry_code
BANCS.OUTPARAM.cust_residential_post_code	cust_residential_cntry_code
BANCS.OUTPARAM.cust_nationality	cust_nationality
BANCS.OUTPARAM.passport_issue_cntry	passport_issue_cntry
BANCS.OUTPARAM.tax_cert_num	tax_cert_num
BANCS.OUTPARAM.cust_deceased_flg	cust_deceased_flg
BANCS.OUTPARAM.cust_deceased_date	cust_deceased_date
BANCS.OUTPARAM.date_of_notification	date_of_notification
BANCS.OUTPARAM.tax_marker	tax_marker
BANCS.OUTPARAM.reasonable_xcuse_marker_flg	reasonable_xcuse_marker_flg
BANCS.OUTPARAM.issuer_cntry_of_TIN	issuer_cntry_of_TIN
BANCS.OUTPARAM.residential_cntry	residential_cntry
BANCS.OUTPARAM.saving_income_code	saving_income_code
BANCS.OUTPARAM.saving_income_amt	saving_income_amt
BANCS.OUTPARAM.free_text1	free_text1
BANCS.OUTPARAM.free_text2	free_text2
BANCS.OUTPARAM.free_text3	free_text3
BANCS.OUTPARAM.free_text4	free_text4
BANCS.OUTPARAM.free_text5	free_text5

BANCS.OUTPARAM.reporting_rqd_flg	reporting_rqd_flg
BANCS.OUTPARAM.remarks	remarks
BANCS.OUTPARAM.ods_reported_flg	ods_reported_flg

In case of Update:

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctNum	Account number
BANCS.INPARAM.cifId	Cif ID
BANCS.INPARAM.tax_year_end_date	Tax Year End Date
BANCS.INPARAM.fetchOrUpdateFlg	fetchOrUpdateFlg
BANCS.INPARAM.financial_institution	financial_institution
BANCS.INPARAM.branch_name	branch_name
BANCS.INPARAM.branch_code	branch_code
BANCS.INPARAM.tax_year	tax_year
BANCS.INPARAM.sort_code	sort_code
BANCS.INPARAM.foracid	foracid
BANCS.INPARAM.acct_name	acct_name
BANCS.INPARAM.acct_addr1	acct_addr1
BANCS.INPARAM.acct_addr2	acct_addr2
BANCS.INPARAM.acct_addr3	acct_addr3
BANCS.INPARAM.acct_city_code	acct_city_code
BANCS.INPARAM.acct_state_code	acct_state_code
BANCS.INPARAM.acct_cntry_code	acct_cntry_code
BANCS.INPARAM.acct_post_code	acct_post_code
BANCS.INPARAM.product_marker_flg	product_marker_flg
BANCS.INPARAM.r85_flg	r85_flg

BANCS.INPARAM.new_acct_signal_flg	new_acct_signal_flg
BANCS.INPARAM.r105_flg	r105_flg
BANCS.INPARAM.gross_interest_amt	gross_interest_amt
BANCS.INPARAM.tax_deducted_amt	tax_deducted_amt
BANCS.INPARAM.crncy_code	crncy_code
BANCS.INPARAM.no_of_acct_holders	no_of_acct_holders
BANCS.INPARAM.cust_title	cust_title
BANCS.INPARAM.cust_first_name	cust_first_name
BANCS.INPARAM.cust_middle_name	cust_middle_name
BANCS.INPARAM.cust_surname	cust_surname
BANCS.INPARAM.cust_pref_addr1	cust_pref_addr1
BANCS.INPARAM.cust_pref_addr2	cust_pref_addr2
BANCS.INPARAM.cust_pref_addr3	cust_pref_addr3
BANCS.INPARAM.cust_pref_city_code	cust_pref_city_code
BANCS.INPARAM.cust_pref_state_code	cust_pref_state_code
BANCS.INPARAM.cust_pref_cntry_code	cust_pref_cntry_code
BANCS.INPARAM.cust_pref_post_code	cust_pref_post_code
BANCS.INPARAM.national_insurance_no	national_insurance_no
BANCS.INPARAM.date_of_birth	date_of_birth
BANCS.INPARAM.tax_identification_no	tax_identification_no
BANCS.INPARAM.cntry_code_of_birth	cntry_code_of_birth
BANCS.INPARAM.place_of_birth	place_of_birth
BANCS.INPARAM.report_category	report_category
BANCS.INPARAM.reporting_cntry	reporting_cntry
BANCS.INPARAM.cust_residential_addr1	cust_residential_addr1
BANCS.INPARAM.cust_residential_addr2	cust_residential_addr2
BANCS.INPARAM.cust_residential_addr3	cust_residential_addr3

BANCS.INPARAM.cust_residential_city_code	cust_residential_city_code
BANCS.INPARAM.cust_residential_state_code	cust_residential_state_code
BANCS.INPARAM.cust_residential_cntry_code	cust_residential_cntry_code
BANCS.INPARAM.cust_residential_post_code	cust_residential_cntry_code
BANCS.INPARAM.cust_nationality	cust_nationality
BANCS.INPARAM.passport_issue_cntry	passport_issue_cntry
BANCS.INPARAM.tax_cert_num	tax_cert_num
BANCS.INPARAM.cust_deceased_flg	cust_deceased_flg
BANCS.INPARAM.cust_deceased_date	cust_deceased_date
BANCS.INPARAM.date_of_notification	date_of_notification
BANCS.INPARAM.tax_marker	tax_marker
BANCS.INPARAM.reasonable_xcuse_marker_flg	reasonable_xcuse_marker_flg
BANCS.INPARAM.issuer_cntry_of_TIN	issuer_cntry_of_TIN
BANCS.INPARAM.residential_cntry	residential_cntry
BANCS.INPARAM.saving_income_code	saving_income_code
BANCS.INPARAM.saving_income_amt	saving_income_amt
BANCS.INPARAM.free_text1	free_text1
BANCS.INPARAM.free_text2	free_text2
BANCS.INPARAM.free_text3	free_text3
BANCS.INPARAM.free_text4	free_text4
BANCS.INPARAM.free_text5	free_text5
BANCS.INPARAM.reporting_rqd_flg	reporting_rqd_flg
BANCS.INPARAM.remarks	remarks
BANCS.INPARAM.ods_reported_flg	ods_reported_flg

Output Repository Fields:

Field Name	Description

BANCS.OUTPARAM.successOrFailure	Success or Failure
BANCS.OUTPARAM.errorMesg	Error Message

©Copyright Infosys Ltd.

Userhook for Select Queries

This userhook is used for select queries. Instead of directly using variables in the query, bind variables would be used. It takes input from CDCI.Request.bindvars repository variable. The bind variables would be pipe separated.

For Example:

CDCI.Request.bindvars= acid + "|" + contextBankId

The query would be of the following format:

sv_k = "acctClsFlg | select acct_cls_flg from GAM where acid = ?BVAR and BANK_ID = ?BVAR"

sv_a=urhk_SIS_dbSelectWithBind(sv_k)

where BVAR/SVAR stands for the string bind values. NVAR would indicate numerical value.

Note:

There are no quotes before and after BVAR/SVAR/NVAR.

Output:

The userhook returns 0 on success and 1 for failure.

©Copyright Infosys Ltd.

Update Statements

This userhook is used for insert, delete and update queries. Instead of directly using variables in the query, bind variables would be used. It takes input from CDCI.Request.bindvars repository variable. The bind variables would be pipe separated.

For Example:

For a delete statement the query format will be

```
CDCI.Request.bindvars="kishlay|02"
```

```
sv_k="delete from test_user where name=?BVAR and  
experience=?BVAR"
```

```
sv_a=urhk_SIS_dbSQLWithBind(sv_k)
```

where BVAR/SVAR stands for the string bind values. NVAR would indicate numerical value.

Note:

There are no quotes before and after BVAR/SVAR/NVAR.

Output:

The userhook returns 0 on success and 1 for failure.

©Copyright Infosys Ltd.

Get APY Earned with Compounding

This user hook will do the following:

- If only a/c number is passed as input then, System will calculate APY earned for account from account opening date to till date.
- If start date and end date is passed the APY earned will be calculated for the given start and end date.
- If Interest Compounding flag is 'Yes' then, System will use special formula to calculate APY earned by the account for the period between start date and end date.
- If Interest Compounding flag is 'No' then. System will use general formula to calculate APY earned by the account for the period between start date and end date.
- It will be governed by last interest booking date which is less than or equal to end date. It will not consider the accrual data.

Syntax:

The syntax of user hook usage is as follows

```
sv_b = urhk_ getAPYEWWithCompounding ("")
```

Input Arguments:

- Account Number
- Start Date
- End date
- Compound date

Input Repository Fields:

Field Name	Description
BANCS.INPARAM. acctId	Valid Account Number in GAM table
BANCS.INPARAM. startDate	APY calculation start date
BANCS.INPARAM. endDate	APY calculation end date
BANCS.INPARAM. compoundFlg	Compound flag (Y/N)

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM. APYE	APY calculated amount.
BANCS.OUTPARAM. ErrMsg	Error message if any.

Userhook Output:

Returns Success in case all userhook validations are through else returns failure and an error message stating the reason for failure.

©Copyright Infosys Ltd.

Insert Record into DTD

This user hook will have the following functionality:

- Insert records in DTD, with debit leg and Credit Leg for a value date which is passed to this userhook.
- Both the records will have posted flag as YES.
- Inserts record into DTH
- No multicurrency transaction allowed.
- This userHook will only insert record in DTD/DTH if `gst_based_on_val_dt_flg` is 'N' in GST (HSCFM set up). If the said flag is 'Y', FAILURE will be returned with error message and no processing will take place.
- Validation checks (for account/scheme) similar to the ones done before making an online transaction using menus like HTM is out of scope of this userhook. No such checks are performed before making entries in DTD/DTH.
- Entered transaction would be masked while taking statements mini/full, pass sheet, ledger balance. To mask it from full/mini statements, ATD table entries will be made for the said transactions. To mask it from HACLI/HPSP tran remarks handle has been provided which will be used as it is used in TACBSH batch. Setup should be done in HSRGPM and CustTranMask.scr need to be customized to achieve the same.

Syntax:

The syntax of user hook usage is as follows

```
sv_b = urhk_insertRecIntoDTD ("")
```

Input Arguments:

The below said four fields are mandatory:

Valid Account ID

Debit value date

Credit value date

Amount

Input Repository Fields:

Field Name	Description
BANCS.INPARAM. acctId	Valid Account Number in GAM table
BANCS.INPARAM. drValDate	Debit value date
BANCS.INPARAM. crValDate	Credit value date
BANCS.INPARAM. tranParticular	Transaction particular
BANCS.INPARAM. tranParticular2	Transaction particular
BANCS.INPARAM. tranParticularCode	Transaction particular code
BANCS.INPARAM. amount	Amount
BANCS.INPARAM. srlNum	Serial Number
BANCS.INPARAM. tranRemarks	Transaction remarks

Return Value:

Returns Success in case all record inserted successfully else returns failure and an error message stating the reason for failure.

©Copyright Infosys Ltd.

Insert into CFATbIs

This userhook is used to insert records in OCASHT and OCASDT tables with the details related to schedule which is passed to this userhook. The tables are updated based on the value of updateIndicator(updateInd) passed from script. If the value of updateInd is Yes, then both the tables get updated with the values passed.

Depending on the action passed, modification or insertion into the tables happens for the records. The default logic of bucket calculation is applied here. Required logic can be written in the script and update of tables can be called according to the requirement.

Syntax:

```
sv_b = urhk_insertIntoCFATbIs(“”)
```

Input Arguments:

Input arguments not available.

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.ruleId	rule_id
BANCS.INPARAM.zoneCode	zone_code
BANCS.INPARAM.solId	zone_sol_id
BANCS.INPARAM.zoneDate	zone_date
BANCS.INPARAM.setNum	set_num
BANCS.INPARAM.endInstr	endorsed_instrument
BANCS.INPARAM.redInstr	redeposited_instrument
BANCS.INPARAM.doubtInCol	doubt_in_collectability
BANCS.INPARAM.emrgncyCond	emergency_condition
BANCS.INPARAM.tranCode	tran_code
BANCS.INPARAM.ociCrncyCode	oci_crncy_code

BANCS.INPARAM.instrmntAmt	instrmnt_amt
BANCS.INPARAM.ociRcreTime	oci_rcre_time
BANCS.INPARAM.valueDate	value_date
BANCS.INPARAM.instrSrlNum	instrmnt_srl_num
BANCS.INPARAM.instrId	instrmnt_id
BANCS.INPARAM.instrSchdlSrlNum	instrSchdlSrlNum
BANCS.INPARAM.instrLocType	instrument_location_type
BANCS.INPARAM.depLocType	deposit_location_type
BANCS.INPARAM.bankCode	bank_code
BANCS.INPARAM.brCode	br_code
BANCS.INPARAM.tranValueDate	tran_value_date
BANCS.INPARAM.shdBalCode	shd_bal_code
BANCS.INPARAM.repCode	rep_code
BANCS.INPARAM.sortCode	sort_code
BANCS.INPARAM.acctSolId	acct_sol_id
BANCS.INPARAM.payingAcctId	paying_acct_id
BANCS.INPARAM.futCrFlagAtTranPost	fut_cr_flag_at_tran_post
BANCS.INPARAM.issExtnCntr	iss_extn_cntr
BANCS.INPARAM.issBankCode	iss_bank_code
BANCS.INPARAM.issBrCode	iss_br_code
BANCS.INPARAM.extnCode	extn_code
BANCS.INPARAM.extnNumOfDays	extn_num_of_days_flg
BANCS.INPARAM.acid	acid
BANCS.INPARAM.ocpCrncyCode	ocp_crncy_code
BANCS.INPARAM.partTranAmt	part_tran_amt_in_inst_crncy
BANCS.INPARAM.acctCrncyCode	acct_crncy_code
BANCS.INPARAM.ocpRcreTime	ocp_rcre_time
BANCS.INPARAM.partTranSrlNum	part_tran_srl_num

BANCS.INPARAM.MRHTMatchRuleCond	MRHTMatchRuleCond
BANCS.INPARAM.GAMSchmCode	GAMSchmCode
BANCS.INPARAM.GAMAcctOpnDate	GAMAcctOpnDate
BANCS.INPARAM.ocpRecCount	numOCPreC
BANCS.INPARAM.ociRecCount	numOCIRec
BANCS.INPARAM.updOCTbl	updateInd
BANCS.INPARAM.action	action

Output Repository Fields:

Output not present.

Return Value:

Returns Success in case all records inserted successfully.

©Copyright Infosys Ltd.

urhk_changeAcctOpnDate.uhk

Note:

This topic describes the enhancements delivered for 10.2.15 release.

This userhook will change the account open date to BOD date of the user during account activating process.

```
sv_a = urhk_changeAcctOpnDate ("" )
```

For the inactive accounts, the account open date of the might be backdated. So while activating the account open date of the accounts gets updated to BOD date and then the account becomes active or activated.

The following is the syntax for specifying the input fields:

BANCS.INPARAM.acid

The following are the details of the input fields for the Userhook

Input Field	Description
BANCS.INPARAM.acid	Account ID for which the account open date should get changed.

The following is the syntax for specifying the output fields:

The userhook returns 0 if the update was successful and 1 on failure.

BANCS.INPARAM.acid = INDLOC.WKIT.acid

```
sv_a = urhk_changeAcctOpnDate ("" )
```

©Copyright Infosys Ltd.

Finacle Copyright

urhk_getSSAMiscDtlsForAcct

Note:

This topic describes the enhancements delivered for 10.2.15 release.

```
sv_a = urhk_getSSAMiscDtlsForAcct("")
```

This userhook is introduced internally for populating ISSATD menu option fields (loan related and PPF account details to fetch MRPM menu option sub category details information).

Userhooks for India Localization

©Copyright Infosys Ltd.

urhk_performCleanUp

Note:

This topic describes the enhancements delivered for 10.2.15 release.

A new userhook has been added to perform cleanup .The same would close all open cursors and file pointers.

Usage:

```
sv_k=urhk_performCleanUp("")
```

This user hook performs a cleanup by closing all the open cursors and file pointers.

NA.

The following is the syntax for specifying the output fields:

```
sv_k=urhk_performCleanUp("")
```

Returns a 0 on success.

sv_k will be set to 0 on success.

Example

```
test.scr
```

```
<--start
```

```
trace on
```

```
BANCS.INPARAM.BINDVARS="SYSTEM_DEF"
```

```
print(BANCS.INPARAM.BINDVARS)
```

```
sv_a=urhk_dbCursorOpenWithBind("user_id0,term_id0|select  
user_id,DEFAULT_TERM_ID      from      upr      where  
DEFAULT_TERM_ID=?NVAR")
```

```
sv_a=urhk_dbCursorOpenWithBind("user_id0,term_id0|select  
user_id,DEFAULT_TERM_ID      from      upr      where  
DEFAULT_TERM_ID=?NVAR")
```

```
sv_a=urhk_dbCursorOpenWithBind("user_id0,term_id0|select  
user_id,DEFAULT_TERM_ID      from      upr      where  
DEFAULT_TERM_ID=?NVAR")
```

```
sv_a=urhk_dbCursorOpenWithBind("user_id0,term_id0|select  
user_id,DEFAULT_TERM_ID      from      upr      where  
DEFAULT_TERM_ID=?NVAR")
```

```
sv_a=urhk_dbCursorOpenWithBind("user_id0,term_id0|select  
user_id,DEFAULT_TERM_ID      from      upr      where  
DEFAULT_TERM_ID=?NVAR")
```

```
sv_k=urhk_performCleanUp("")
```

```
end→
```

Comment out `sv_k=urhk_performCleanUp("")` from test.scr.
And execute the script using HSCRIPT menu.

User hooks Added

Note:

This topic describes the enhancements delivered for 10.2.16 release.

Core

Srl. No.	User Hook	Remarks
1	insertSystemTranPartTranCode	Using this user hook, user can add, modify or delete entries for transaction and part transaction codes into the database. Inputs: Function code, mode, system transaction code, system part transaction code, message id, module id and locale code. Output: Error Message
2	validateUserPartTranCode	This user hook validates the user part transaction code specified through the HTTUM upload file, it would not allow the transaction to go through and will give an error message in case the validation fails. Inputs: User Part Transaction Code Output: User Part Transaction Code Description, Error Message.

Get Derived in Rate

This user hook is used to derive the interest rate applicable based on the inputs provided.

Syntax

```
sv_b = urhk_getDerivedIntRate ("")
```

Input Arguments:

Account Number

As of Date

Amount

Pref Rate Flag

Interest Rate Code

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.acctNum	This is a mandatory field which specifies the account for which the interest rate needs to be derived
BANCS.INPARAM.asOfDate	This is a mandatory field which specifies the date on which the rate needs to be determined.
BANCS.INPARAM.Amt	This is a mandatory filed specifies the balance amount
BANCS.INPARAM.prefRateFlg	This is a mandatory field specifies the type of rate i.e. debit or credit which needs to be fetched.
BANCS.INPARAM. interestRateCode	This field specifies the rate code given as input

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM. interestRate	The derived interest rate

BANCS.OUTPARAM. The account preference rate
acctPrefRate

BANCS.OUTPARAM. The customer preference rate
custPrefRate

BANCS.OUTPARAM. The channel preference rate
channelPrefRate

BANCS.OUTPARAM. The analytical preference rate
analyticalPrefRate

©Copyright Infosys Ltd.

User Hooks Modified

Note:

This topic describes the enhancements delivered for 10.2.16 release.

None

©Copyright Infosys Ltd.

Userhooks

Note:

This topic describes the enhancements delivered for 10.2.15 release.

This section describes scripting userhooks related to database access as part of the enhancement for this release.

Userhooks for Core

©Copyright Infosys Ltd.

Userhooks

Note:

This topic describes the enhancements delivered for 10.2.16 release.

This section describes scripting userhooks related to database access as part of the enhancement for this release.

©Copyright Infosys Ltd.

Set Default Attribute of Field in Form

This function provides a facility to change the attributes of a field in the form. It can be used in the context of WorkFlow or in the script executed during the Pre Form Load event (PreLoad.scr) only.

See [Scripting Events](#) module for more details about Preload scripts.

For setting the attribute of a field in any other online event refer to urhk_TBAF_ChangeFieldAttrib in this document.

This function can be used to change the attribute of any field in a form that is currently being loaded or about to be called for loading (in the case of WorkFlow). By default, all fields in a form that is loaded carry the attribute assigned to it by Infosys. However, a Bank may wish to customize the attributes of certain fields in certain forms which is different from the Infosys setting.

For Example

A Bank may decide that the Communication address line 1 field in the Customer master maintenance screen is to be made mandatory (default setting is non-mandatory). Then this function can be used in the form load event of the form baff0009 to set the attribute of the field cust_comu_addr1 as mandatory.

Note:

1. when a field has a default setting of Mandatory, setting Protect or Entry allowed attributes to that field is not allowed.
2. Also, once an attribute for a field has been set, setting the attribute of that field again (even with a different attribute) before the form is unloaded, does not have any effect.

Syntax:

```
sv_a = urhk_TBAF_SetAttrib(Variable)
```

Variable needs to be formatted as "<formname>.<blockname>.<fieldname>|Attribute"

A Attribute M Mandatory

P Protect (Cursor and data entry not allowed)

H Hide

E called as Entry Allowed Cursor entry allowed, Data Entry not allowed

The variable can be a scratch pad variable (sv_b) or a string ("baff0009.datablk2.cust_comu_addr1|M")

Note:

Use dispfields utility for finding this information. This utility is documented in Workflow manual.

Input Arguments:

Input String contains two parts separated by a |

Field name

Attribute

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Always returns '0' as return value

The form displays the new attribute after it is loaded if the specified

field exists in the form and the attribute does not conflict with the default setting. Otherwise, nothing happens.

For Example:

1) baff0009.datablk2.cust_comu_addr1|M

Field name should be given as
FormName.BlockName.FieldName.

In the above example the attribute is given as M to cust_comu_addr1 field which makes it mandatory to enter a value in the field.

Valid values for Attribute are :

P " Protect " The field is made non-enterable and tab will skip past this field

M " Mandatory " The field is made mandatory and something needs to be entered in this field before proceeding

H " Hide " The field value is not displayed

E ' Entry allowed " The cursor can be positioned in the field but no data can be entered. Useful if function keys need to be used in a protected field

2) The logic discussed in the functionality section can be implemented by coding baff0009PreLoad.scr as follows:

```
<--start
```

```
sv_a                                     =  
urhk_TBAF_SetAttrib(baff0009.datablk2.cust_comu_addr1|M)
```

```
exitscript
```

```
end-->
```

Populate Reason Code for ACH

This userhook is created to Populate Reason Code and Reason ID for ACH instructions.

Syntax:

```
sv_b = urhk_PopulateReasonCodeForACH ("")
```

Input Arguments:

entrySrlNum

paysysId

reason ID

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

No output from the userhook

For Example:

```
<--start
```

```
#trace on
```

```
sv_a = "10486"
```

```
sv_b = "ACH9"
```



```
sv_c = "0009"  
BANCS.INPARAM.entrySrINum=sv_a  
BANCS.INPARAM.paysysId=sv_b  
BANCS.INPARAM.reasonId=sv_c  
sv_z = urhk_PopulateReasonCodeForACH("")  
#trace off  
end-->
```

©Copyright Infosys Ltd.

Insert into SMH

This userhook is used to insert a record into SMH table based on the given input parameters.

Syntax:

```
sv_b = urhk_insertIntoSMH (“”)
```

Input arguments:

NA

Input Repository Fields:

Field Name	Description
BANCS.INPARAM.paysysId	Paysys ID
BANCS.INPARAM.mtNo	Mt Number
BANCS.INPARAM.senderBic	Sender Bank Identification Code
BANCS.INPARAM.recvrBic	Receiver Bank Identification Code
BANCS.INPARAM.solId	Sol ID
BANCS.INPARAM.reqExecDate	Required Execution Date
BANCS.INPARAM.valueDate	Value Date
BANCS.INPARAM.tranDate	Transaction Date
BANCS.INPARAM.tranId	Transaction ID
BANCS.INPARAM.senderRefNum	Sender Reference Number
BANCS.INPARAM.routingRefNum	Routing Reference Number
BANCS.INPARAM.remitCrncy	Remit Currency
BANCS.INPARAM.instructedCrncy	Instructed Currency
BANCS.INPARAM.remitAmt	Remit Amount
BANCS.INPARAM.instructedAmt	Instructed Amount

BANCS.INPARAM.mesgPriority	Message Priority
BANCS.INPARAM.deptNum	Department Number
BANCS.INPARAM.paymentMessage	Payment Message
BANCS.INPARAM.inOutInd	In Out Indicator
BANCS.INPARAM.validateScriptName	Validate Script Name
BANCS.INPARAM.utrNumber	Unique Transaction Reference Number

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.errorMesg	Error Message
BANCS.OUTPARAM.routedPaysysRefNum	Routed Paysys Reference Number

Return value:

Example:

NA

©Copyright Infosys Ltd.

Inquire Field Value in Form

This function gets the value of a field in the current form. This function gets the value of a field in the form in the repository field B2KTEMP.TEMPSTD.TBAFRESULT. This function should not be used during Pre Form load or Post Form Load events.

Syntax:

```
sv_a = urhk_TBAF_InquireFieldValue(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string (optionblk.option_code)

Input Arguments:

Input string has one part

Field Name

EXAMPLE - optionblk.option_code

Field name must be given as BlockName.FieldName.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Always returns '0' as return value.

Required value is available in B2KTEMP.TEMPSTD.TBAFRESULT if

the field is found in the current form, else the required value is available in B2KTEMP.TEMPSTD.TBAFRESULT.

For Example:

In this example, a sample script has been written which can be used to invoke all the mandatory forms to be visited while opening a deposit account automatically.

First we must code the PreLoad script of the deposit form to execute an "accept-key" when the option block is visited.

Then we must code the pre-replay script of the above key to populate the appropriate option code into the option field.

Then we must code the post-replay script of the above key to reset the replay key so that when the option block is visited again the next option is chosen.

Script Name :- tdf3201PreLoad.scr

```
<--start
```

```
sv_a      =      urhk_TBAF_SetReplayKey("optnblk.key-  
f2|tdoptpre.scr|0|tdoptpost.scr|0")
```

```
exitscript
```

```
end-->
```

Script Name :- tdoptpre.scr

```
<--start
```

```
sv_b = cint(INTBAF.INTBAFC.TbafEventStep)
```

```
if (sv_b == 0) then
```

```
GOTO STEP0
```

```
endif
```

```
if (sv_b == 1) then
```

```
GOTO STEP1
```

```
endif
```

```
if (sv_b == 2) then
```

```
GOTO STEP2
```

```
endif
```

```
if (sv_b == 3) then
```

```
GOTO STEP3
```

```
endif
```

```
exitscript
```

```
# Populate option as G to visit general details.
```

```
STEP0:
```

```
sv_a =  
urhk_TBFAF_ChangeFieldValue("optnblk.acct_opn_option|G")
```

```
exitscript
```

```
# Populate option as N to visit Nominee details
```

```
STEP1:
```

```
sv_a =  
urhk_TBFAF_ChangeFieldValue("optnblk.acct_opn_option|N")
```

```
exitscript
```

```
# Populate option as F to visit Flow details.
```

```
STEP2:
```

```
sv_a =  
urhk_TBFAF_ChangeFieldValue("optnblk.acct_opn_option|F")
```

```
exitscript
```

```
# Populate option as V to visit Advance details.
```

```
STEP3:
```

```
sv_a =  
urhk_TBFAF_ChangeFieldValue("optnblk.acct_opn_option|V")
```

```
exitscript
```

```
end-->
Script Name :- tdoptpost.scr
<--start
sv_b = cint(INTBAF.INTBAFC.TbafEventStep)
if (sv_b == 0) then
GOTO STEP0
endif
if (sv_b == 1) then
GOTO STEP1
endif
if (sv_b == 2) then
GOTO STEP2
endif
exitscript
# Populate Replay key to execute Step 1 or Step 2 of pre-script
STEP0:
sv_a      =      urhk_TBAF_InquireFieldValue
(datablk1.nom_available_flg)
if (B2KTEMP.TEMPSTD.TBAFRESULT = "Y")
sv_a      =      urhk_TBAF_SetReplayKey("optnblk.key-
f2|tdoptpre.scr|1|tdoptpost.scr|1")
else
sv_a      =      urhk_TBAF_SetReplayKey("optnblk.key-
f2|tdoptpre.scr|2|tdoptpost.scr|2")
exitscript
# Populate Replay key to execute Step 2 of pre-script
STEP1:
```

```
sv_a      =      urhk_TBAF_SetReplayKey("optnblk.key-  
f2|tdoptpre.scr|2|tdoptpost.scr|2")
```

```
exitscript
```

```
# Populate Replay key to execute Step 3 of pre-script
```

```
STEP2:
```

```
sv_a      =      urhk_TBAF_SetReplayKey("optnblk.key-  
f2|tdoptpre.scr|3")
```

```
exitscript
```

```
end-->
```

©Copyright Infosys Ltd.

Get Past Due Display or Print String

This userhook accepts a string, which is either displayed through PDADI menu option or dumped for report generation. This userhook is specific to PDADI menu option. A formatted string is accepted and the same is displayed on multiple record screen or dump for report generation based on 'Display or Report' flag specified in PDADI menu option. This userhook may be called any number of times. The maximum length of the string can be 2500 characters.

The string passed for display is cut to 150 characters and display the string in two lines of 75 characters each. The string passed for display must be a formatted one.

For generating report, string passed must contain individual fields separated by a '|' character. The number of fields in string must match with the record layout of the Maha Report Template being used for report generation.

Syntax:

`sv_a = urhk_putPastDueDispPrintString (Variable)`

`sv_a`: scratch pad variable to store the return value

Variable: Scratch pad variable containing formatted string.

Input Arguments:

Formatted string.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value is always success ('0').

For Example:

Example to build and pass a display string

#Get past due record and populate all required values into

#scratch pad variables.

```
sv_b = urhk_getPastDueRecord("")
```

```
while (sv_b == 0)
```

```
# Pad string with spaces
```

```
sv_r = sv_r + 1
```

```
sv_a = BANCS.OUTPARAM.setId
```

```
sv_a = rpad(sv_a, 8)
```

```
sv_c = BANCS.OUTPARAM.acctId
```

```
sv_d = BANCS.OUTPARAM.solId
```

```
sv_d = rpad(sv_d, 8)
```

```
sv_e = BANCS.OUTPARAM.schmCode
```

```
sv_e = rpad(sv_e, 5)
```

```
sv_f = BANCS.OUTPARAM.glSubHeadCode
```

```
sv_f = rpad(sv_f, 5)
```

```
sv_h = BANCS.OUTPARAM.acctCrncyCode
```

```
sv_h = rpad(sv_h, 3)
```

```
sv_i = BANCS.OUTPARAM.acctMgrId
```

```
sv_i = rpad(sv_i, 15)
```

```
sv_j = BANCS.OUTPARAM.custId
```

```
sv_j = rpad(sv_j, 9)
sv_k = rpad(sv_c, 16)
sv_l = urhk_getAcctDetailsInRepository(sv_c)
sv_m = BANCS.OUTPUTPARAM.acctName
if (strlen(sv_m) >= 10) then
sv_m = left$(sv_m, 10)
else
sv_m = rpad(sv_m, 10)
endif
sv_p = BANCS.OUTPUTPARAM.ClearBalance
sv_p = CDOUBLE(sv_p)
sv_x = sv_x + sv_p
sv_p = format$(sv_p, "%17.02f")
sv_l = urhk_getAdvanceDetailsForAccount(sv_c)
if (sv_l == 0) then
sv_n = BANCS.OUTPUTPARAM.PastDueXferDate
else
sv_n = ""
endif
if (strlen(sv_n) >= 10) then
sv_n = left$(sv_n, 10)
else
sv_n = rpad(sv_n, 10)
endif
# Build the final string.
sv_t = " "
```

```
sv_g = sv_k + sv_t + sv_d + sv_t + sv_m + sv_t + sv_n
```

```
sv_z = urhk_putPastDueDispPrintString(sv_g)
```

```
sv_b = urhk_getPastDueRecord("")
```

```
do
```

Example to build and pass string for report generation

```
sv_b = urhk_getPastDueRecord("")
```

```
while (sv_b == 0)
```

```
# Pad string with spaces
```

```
sv_r = sv_r + 1
```

```
sv_a = BANCS.OUTPARAM.setId
```

```
sv_c = BANCS.OUTPARAM.acctId
```

```
sv_d = BANCS.OUTPARAM.solId
```

```
sv_e = BANCS.OUTPARAM.schmCode
```

```
sv_f = BANCS.OUTPARAM.glSubHeadCode
```

```
sv_h = BANCS.OUTPARAM.acctCrncyCode
```

```
sv_i = BANCS.OUTPARAM.acctMgrId
```

```
sv_j = BANCS.OUTPARAM.custId
```

```
sv_l = urhk_getAcctDetailsInRepository(sv_c)
```

```
sv_m = BANCS.OUTPARAM.acctName
```

```
sv_p = BANCS.OUTPARAM.ClearBalance
```

```
sv_l = urhk_getAdvanceDetailsForAccount(sv_c)
```

```
if (sv_l == 0) then
```

```
sv_n = BANCS.OUTPARAM.PastDueXferDate
```

```
else
```

```
sv_n = ""
```

```
endif
```

Build the final string.

sv_t = "|"

sv_g = sv_a + sv_t + sv_c + sv_t + sv_d + sv_t + sv_e sv_t +
sv_m + sv_t + sv_n

sv_z = urhk_putPastDueDispPrintString(sv_g)

sv_b = urhk_getPastDueRecord("")

do

end

©Copyright Infosys Ltd.

Get Name of Current Field in Current Form

This function gives the name of the current field in the current form. Whenever invoked, this user hook gives the name of the current field on which the cursor is lying in the current form in 'formname.blockname.fieldname' format. This value is available in B2KTEMP.TEMPSTD.TBAFRESULT field.

Syntax:

```
sv_a = urhk_TBAF_GetCurFldName("")
```

Input Arguments:

Input string is null.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Always returns '0' as return value.

The current field name (field on which the cursor is lying) is returned as output in the B2KTEMP.TEMPSTD.TBAFRESULT field in <formname>.<blockname>.<fieldname> format.

For Example:

```
<--start
```

```
sv_a = usrhk_TBAF_GetCurFldName("")
```

```
if (B2KTEMP.TEMPSTD.TBAFRESULT =  
"baff0009.datablk6.cust_id") then  
# some actions  
endif  
exitscript  
end-->
```

©Copyright Infosys Ltd.

Get Default Value for Field in Form during Form Load

This function populates the given value for a field in a form that is currently being loaded. It can be used in the context of WorkFlow or in the script executed during the Form Load event (PreLoad.scr) only.

See [Scripting Events](#) module for more details about Preload scripts.

For changing the value of a field in any other online event refer to urhk_TBAF_ChangeFieldValue in this document.

This function can be used to set the default value of any field in a form that is currently being loaded or about to be called for loading(in the case of WorkFlow). By default, many fields in a form that is loaded carry a NULL value assigned to it by Infosys. However, a Bank may wish to assign a default values of certain fields in certain forms. For example, the Customer master maintenance function assigns no value to the "Customer Non Resident", "Customer staff" and "Customer minor" fields. A Bank may decide that a default value of "N" needs to be assigned to these 3 fields. Then this function can be used in the form load event of the form baff0009 to set the default value as "N" for these three fields.

It is also used in conjunction with urhk_TBAF_GetValue to get back values from a form after "Commit". In order to do that, this function should be used to specify all the form fields whose values (as they exist after commit processing) is to be accessed, and then after commit processing use urhk_TBAF_GetValue to access their values.

Note:

Once a value for a field has been set, setting the value of that field again (even with a different value) before the form is unloaded, does not have any effect.

Syntax:

sv_a = urhk_TBAF_SetValue (Variable)

As discussed for TBAF_SetAttrib above.

The variable can be a scratch pad variable (sv_b) or a string ("tdfe3201.datablk1.ledg_num|1")

Repository name is meant for advanced use and typically should not be specified as shown in the example above. In this case the repository INTBAF is assumed. However, when we intend to specify the list of field names whose value we need to access after "commit" processing, this repository should be specified as OUTTBAF.

Input Arguments:

Input String contains three parts separated by a "|"

Field name

Value - Max. size is 255 characters

Repository [Optional]

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Always returns 0 as return value.

When the repository is not specified then the form displays the new value after it is loaded if the specified field exists in the form, otherwise, nothing happens.

When the repository OUTTBAF is specified, nothing happens when

the form is loaded, but you can then use `urhk_TBFAF_GetValue` in post-commit events to access the value of the field.

For Example:

```
"tdfe3201.datablk1.ledg_num|1"
```

Field name must be given as `FormName.BlockName.FieldName`.

Value is the data to be populated in the field. It can have a NULL value to set the value of a field to NULL. Ex.
"tdfe3201.datablk1.ledg_num|"

The logic discussed in the "functionality section" can be implemented by coding `baff0009PreLoad.scr` as:

```
<--start

sv_a      =      urhk_TBFAF_SetValue("baff0009.datablk1.
cust_nre_flg|N")

sv_a      =      urhk_TBFAF_SetValue("baff0009.datablk1.
cust_emp_flg|N")

sv_a      =      urhk_TBFAF_SetValue("baff0009.datablk1.
cust_minor_flg|N")

exitscript

end-->
```

The logic for getting the assigned Customer Id (after commit) can be implemented by coding a Workflow script which calls CUMM menu Option, as:

```
<--start

sv_a      =      urhk_TBFAF_SetValue("baff0009.datablk6.cust_id|
|OUTTBFAF")

sv_b = urhk_WFS_CallMenuOption("CUMM|0|3")

if(sv_b == 1) then

exitscript

endif
```

```
sv_d = CLASSEXISTS("OUTTBAF","baff0009")
if (sv_d == 1) then
sv_a = urhk_TBAF_GetValue("baff0009.datablk6.cust_id ")
STDWFS.GLBDATA.cust_id = B2KTEMP.TEMPSTD.TBAFRESULT
DELETECLASS("OUTTBAF","baff0009")
endif
exitscript
end-->
```

©Copyright Infosys Ltd.

Generate Past Due Report

This userhook generates the past due report using the specified report template. This userhook is specific to PDADI menu option in print mode. This userhook may be used to generate additional reports in specified formats. This userhook is applicable only in print mode of PDADI menu option.

Before calling this function a few strings need to be passed to the system through the userhook `urhk_putPastDueDispPrintString`. These strings are dumped into a spool file and report generation routine is called. Report generated by this userhook is in addition to default report generated in any case. The generated report is put into print queue.

Syntax:

```
sv_a = urhk_generatePastDueReport(Variable)
```

sv_a: scratch pad variable to store the return value

Variable: Scratch pad variable containing formatted string or maha report template name.

Input Arguments:

Maha report template

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Return value is success ('0') if reported is generated successfully.

For Example:

To build and pass string for report generation

```
sv_b = urhk_getPastDueRecord("")
while (sv_b == 0)
# Pad string with spaces
sv_r = sv_r + 1
sv_a = BANCS.OUTPARAM.setId
sv_c = BANCS.OUTPARAM.acctId
sv_d = BANCS.OUTPARAM.solId
sv_e = BANCS.OUTPARAM.schmCode
sv_f = BANCS.OUTPARAM.glSubHeadCode
sv_h = BANCS.OUTPARAM.acctCrncyCode
sv_i = BANCS.OUTPARAM.acctMgrId
sv_j = BANCS.OUTPARAM.custId
sv_l = urhk_getAcctDetailsInRepository(sv_c)
sv_m = BANCS.OUTPARAM.acctName
sv_p = BANCS.OUTPARAM.ClearBalance
sv_l = urhk_getAdvanceDetailsForAccount(sv_c)
if (sv_l == 0) then
sv_n = BANCS.OUTPARAM.PastDueXferDate
else
sv_n = ""
endif
# Build the final string.
```

```
sv_t = "|"
sv_g = sv_a + sv_t + sv_c + sv_t + sv_d + sv_t + sv_e sv_t +
sv_m + sv_t + sv_n
sv_z = urhk_putPastDueDispPrintString(sv_g)
sv_b = urhk_getPastDueRecord("")
do
sv_r = urhk_generatePastDueReport("pastDue.mrt")
# If urhk_generatePastDueReport function returns failure,
# populate error message and exit out of script.
if ( sv_r == 1 ) then
BANCS.OUTPUT. errorMessage='Could not print Report'
exitscript
endif
end
```

©Copyright Infosys Ltd.

Display Repository Fields

This userhook allows script writer to display repository values on the screen.

Syntax:

```
sv_r = urhk_SE_Dispositories(variable)
```

The variable can be a scratch pad variable (sv_a) or a string specifying repository variable.

Input Arguments:

The name of the repository.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

The output is shown on the screen through form.

For Example:

```
sv_r = urhk_SE_Dispositories("BANCS")
```

Check Pending Verification for Security

This userhook is provided to check whether any verification is pending for the given security identified by the security code.

Syntax:

```
sv_a = urhk_Check_PendVerificationForSecurity("")
```

Input Arguments:

Input for this userhook is the security code (secu_code).

Input Repository Fields:

Input not present.

Output Repository Fields:

Field Name	Description
BANCS.OUTPARAM.retCode	retCode

Return Value:

Output is a return code (retCode) whose value is either '1' or '2'. If any verification is pending at the limit level then the return code is '1'. If any verification is pending at the account level of the given security code then the return code is '2'.

For Example:

```
<--start
#trace on
sv_b = "TEST"
BANCS.INPARAM.secu_code = sv_b
```



```
sv_a = urhk_Check_PendVerifcationForSecurity("")  
if (fieldexists(BANCS.OUTPARAM.retCode)) then  
sv_z = BANCS.OUTPARAM.retCode  
endif  
#trace off  
end-->
```

©Copyright Infosys Ltd.

Change Field Value in Form

This function changes the value of a field in the current form to the specified value. This userhook changes the value of a field in the form. This userhook must not be used in Pre Form load or Post Form Load scripts. In the Pre Form load scripts use the function `urhk_TBAF_SetValue`.

Syntax:

```
sv_a = urhk_TBAF_ChangeFieldValue(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string (optionblk.option_code|P).

Input Arguments:

Input string has two parts separated by a |

Field name.

Value - Max. size is 255 characters

For Example

```
optionblk.option_code|P
```

Field name must be given as BlockName.FieldName.

Value is the data to be populated in the field. It can have a NULL value to change the value of a field to NULL. Ex.datablk1.ledg_num|.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Always returns '0' as return value.

The form displays the new value immediately, if the field is found in the current form, else nothing happens.

For Example:

Please refer to Example section of urhk_TBAF_SetReplayKey.

In tm1.scr this function is used to populate the option code as 'P' to request for posting the transaction.

©Copyright Infosys Ltd.

Change Attribute of Field in Form

This function changes the attribute of a field in the form to the specified value. This userhook changes the attribute of a field in the form. This userhook must not be used in Pre Form Load and Post Form Load scripts. In the Pre Form Load scripts use the function `urhk_TBAF_SetAttrib`.

Syntax:

```
sv_a = urhk_TBAF_ChangeFieldAttrib(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string (dispblk2.acct_bal|H)

Input Arguments:

Input string has two parts separated by a |

Field Name

Attribute

For Example

```
dispblk2.acct_bal|H
```

Field name must be given as BlockName.FieldName.

Valid values for Attribute are:

P " Protect " The field is made non-enterable and tab skips past this field.

M " Mandatory " The field is made mandatory and something needs to be specified in this field before proceeding.

H " Hide " The field value is not displayed.

U - "UnHide " The field value is displayed.

E " Entry allowed " The cursor can be positioned in the field but no data can be specified. Useful if function keys need to be used in a protected field.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Always returns 0 as return value.

The form displays the changed attribute immediately if the field is found in the current form, else nothing happens.

For Example:

Let us assume that a Bank wants to hide the account balance field in the Transaction maintenance screen for Customer accounts. This is achieved by coding the script as under.

Script Name - CheckAndDispAcctBal.scr

```
<--start
```

```
sv_a = BANCS.INPUT.AcctId
```

```
sv_b = urhk_getAcctDetailsInRepository(sv_a)
```

```
# if getAcctDetailsInRepository function returns failure exit out  
of script.
```

```
if( sv_b == 1 ) then
```

```
  exitscript
```

```
endif
sv_c="E"
# -----
# If the acctOwnerShip flag of ACCOUNT class is equal to "E"
# then hide the dispblk2.acct_bal field
# -----

if (BANCS.OUTPARAM.acctOwnerShip == sv_c) then
sv_z = "dispblk2.acct_bal|H"
sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)
else
sv_z = "dispblk2.acct_bal|U"
sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)
endif
exitscript
end-->
```

©Copyright Infosys Ltd.

Call General Parameter Acceptance Form

This function invokes a specified form. The form must be script-enabled in the sense that it must operate only of repository variables, both for input and output. bafi2028, aafe0003, aafe0004 is an example of such a form. This function brings up the form specified in the argument. As of now only the forms bafi2028, aafe0003, aafe0004 are supported. Each supported form has its own inputs and outputs.

Syntax:

```
sv_a = urhk_B2K_ShowParamAcptFrm("bafi2028").
```

Input Arguments:

The call to the function requires a form name which is to be brought up for accepting data from the user. As of now 'bafi2028, aafe0003, aafe0004' are supported.

Inputs for bafi2028

The form 'bafi2028' requires certain inputs for bringing up the form, displaying the literals and accepting data. The inputs are to be given by way of a repository field in FRMATTRIB class of BANCS repository, for each field that needs to be displayed or accepted. Each field requires:

- The name of the field - assigned by referring to this name while populating the rest of the information for the field.
- The literal for the field - first part of the value assigned to the above variable. Maximum length for literal is 30 characters.
- Length of the value of the field - second part of the value assigned to the above variable separated by "|". Maximum length for value is 100 characters.

Field to be protected or not (P/N) - third part of the value assigned to the above variable separated by "|" Protected field.

For Example:

BANCS.FRMATTRIB.senderSwiftCode="Sender Swift Code|12|P"

	Field name	Literal
Field length		

If a default value needs to be specified for any field then that value needs to be assigned to the corresponding output field. For ex. To assign a default value of 'XXX' for the senderSwiftCode, we should do the following before calling this function :

BANCS.FRMVALUE.senderSwiftCode = 'XXX'

A well known field BANCS.INPARAM.ErrorMesg can be used to force the form to display an Error message.

A well known field BANCS.FRMVALUE.FormTitle can be used to display the title in the form.

INPUTS for aafe0003

There are 4 enterable fields in this form, one of which is the function code field to accept the character input of the function to be carried out by the calling workflow script. Three other fields are free fields to accept any criteria required for the function to be carried out.

Enterable here means the field in which a value can be given on screen by the end user. The additional functionality is:

When funcblk.pg0_exe_name is specified by the user before calling the form - either in the formload script or in the calling workflow script, on pressing <key-f2> the Print Parameter form will be

brought up and the executable (com script or any other executable) specified in the above field is executed. the executable should be available in the appropriate system area.

INPUTS for aafe0004

There are 40 enterable fields in this form, spread in total 4 pages, each page containing 10 fields. These fields can be used to accept data from the end user on the screen and then to call appropriate script to build a workflow.

The additional functionality is:

When funcblk.exe_name is specified by the user before calling the form - either in the formload script or in the calling workflow script, on pressing <key-f2> the Print Parameter form will be brought up and the executable (com script or any other executable) specified in the above field will be executed. the executable should be available in the appropriate system area. If funcblk.pg0_mode is populated with the value 'I' then, input will not be allowed in any of the fields (fields will still be enterable). Also, the listing will not be allowed in any of the above fields even if the pg0_field-<x>_type is specified. Here also type can be specified for each field and listing will be available based on the type.

Input Repository Fields:

Input not present.

Output Repository Fields:

Output not present.

Return Value:

Always returns 0 as return value.

The values specified in the screen for the specified fields is available in fields of FRMVALUE class of BANCS repository, after the user hits the <ACCEPT> or <COMMIT> key in the form. The name of the field is the same as the name assigned in the input described above. That is, the value that the user specified for the 'senderSwiftCode' field is available in BANCS.FRMVALUE.senderSwiftCode field.

For Example:

Example for bafi2028

The following is a sample script which is used to manage a Swift message(mt410) in Finacle. Among other things it requires user input for certain fields like sender swift code, Sender to receiver info etc. These user input are taken by using the urhk_B2K_ShowParamAcptFrm("bafi2028") function.

```
<--start
```

```
trace on
```

```
#*****:
```

```
# Script for generation of MT410 swift message
```

```
#*****:
```

```
#
```

```
# While forming the message, even if the optional fields are not
```

```
# concatenated, a new line separator"|" (pipe character)
```

```
# should be concatenated.
```

```
#
```

```
# In Generate mode, apart from other INPUT repository variables
```

```
# (addOrMod) will be populated by the calling program.
```

```
#
```

```
# In Modify mode, INPUT repository variables (status, addOrMod)
```

```

# will be populated by the calling program. Also, all the field[n]
# INPUT variables will be populated.
#
#*****:
#
*****

# Populate the FRMVALUE fields for display in bafi2028
# in modify mode.
#
*****

if (BANCS.INPUT.addOrMod == "M") then
BANCS.FRMVALUE.senderSwiftCode =BANCS.INPUT.field1
BANCS.FRMVALUE.mtNumber =BANCS.INPUT.field2
BANCS.FRMVALUE.receiverSwiftCode =BANCS.INPUT.field3
BANCS.FRMVALUE.billId =BANCS.INPUT.field4
BANCS.FRMVALUE.referenceNum =BANCS.INPUT.field5
BANCS.FRMVALUE.amountAcknowledged =BANCS.INPUT.field6
BANCS.FRMVALUE.senderToReceiverInfo1=BANCS.INPUT.field7
BANCS.FRMVALUE.senderToReceiverInfo2=BANCS.INPUT.field8
BANCS.FRMVALUE.senderToReceiverInfo3=BANCS.INPUT.field9
BANCS.FRMVALUE.senderToReceiverInfo4=BANCS.INPUT.field10
BANCS.FRMVALUE.senderToReceiverInfo5=BANCS.INPUT.field11
BANCS.FRMVALUE.senderToReceiverInfo6=BANCS.INPUT.field12
endif
#
*****

# Massage the field values present in OUTPARAM

```

when in ADD mode

#

if (BANCS.INPUT.addOrMod == "A") then

populate MT number

BANCS.FRMVALUE.mtNumber="410"

Populate amount Acknowledged

sv_s=BANCS.FRMVALUE.billAmount

call("swiftAmount.scr")

BANCS.FRMVALUE.billAmount= sv_t

sv_w=BANCS.FRMVALUE.dueDate

call("swiftDate.scr")

BANCS.FRMVALUE.dueDate = sv_x

sv_a = sv_x + BANCS.FRMVALUE.billCrncy

sv_a = sv_a + BANCS.FRMVALUE.billAmount

BANCS.FRMVALUE.amountAcknowledged = sv_a

Sender to Receiver information

BANCS.FRMVALUE.senderToReceiverInfo1=""

BANCS.FRMVALUE.senderToReceiverInfo2=""

BANCS.FRMVALUE.senderToReceiverInfo3=""

```

BANCS.FRMVALUE.senderToReceiverInfo4=""
BANCS.FRMVALUE.senderToReceiverInfo5=""
BANCS.FRMVALUE.senderToReceiverInfo6=""
# -----
# default value for status of message
# -----
BANCS.FRMVALUE.status="N"
endif

# Massaging of data in ADD mode complete
#
*****

# Populate the FRMATTRIB fields - Provide definition for the
bafi2028 form
#
*****

BANCS.FRMVALUE.FormTitle="MT410      SWIFT      Message
Generation"
BANCS.FRMATTRIB.senderSwiftCode="Sender Swift Code|12|P"
BANCS.FRMATTRIB.mtNumber="MT|3|P"
BANCS.FRMATTRIB.receiverSwiftCode="Receiver      Swift
Code|12|P"
BANCS.FRMATTRIB.billId="Sending Bank's TRN|16|P"
BANCS.FRMATTRIB.referenceNum="Related reference|16|P"
BANCS.FRMATTRIB.amountAcknowledged="Amount
Acknowledged|24|P"
BANCS.FRMATTRIB.senderToReceiverInfo1="Sender to Receiver
Info|35|N"
BANCS.FRMATTRIB.senderToReceiverInfo2=" |35|N"
BANCS.FRMATTRIB.senderToReceiverInfo3=" |35|N"

```

```

BANCS.FRMATTRIB.senderToReceiverInfo4=" |35|N"
BANCS.FRMATTRIB.senderToReceiverInfo5=" |35|N"
BANCS.FRMATTRIB.senderToReceiverInfo6=" |35|N"
# -----
# status field
# -----
BANCS.FRMATTRIB.status="Status of Message|1|N"
#
*****

# Call general acceptance form
# Conditional validations can be done here.
#
*****

BANCS.INPARAM.ErrorMesg = " "
while(BANCS.INPARAM.ErrorMesg != "")
sv_a = urhk_B2K_ShowParamAcptFrm("bafi2028")
sv_b = BANCS.FRMVALUE.status
if ((sv_b == "N") or (sv_b == "R") or (sv_b == "D")) then
BANCS.INPARAM.ErrorMesg = ""
else
BANCS.INPARAM.ErrorMesg = "Status can have only the
following values. [N-Not ready, R -Ready, D-Delete]"
endif
do
#
*****

# Form the swift message

```

```

# For optional tags check if the value is null or not

#
*****

# -----
# Form MT410 message
# -----

sv_a = BANCS.FRMVALUE.senderSwiftCode
sv_a = sv_a + "|" + BANCS.FRMVALUE.mtNumber
sv_a = sv_a + "|" + BANCS.FRMVALUE.receiverSwiftCode
sv_a = sv_a + "|" + ":20:" + BANCS.FRMVALUE.billId
sv_a = sv_a + "|" + ":21:" + BANCS.FRMVALUE.referenceNum
sv_a      =      sv_a      +      "|"      +      ":32A:"      +
BANCS.FRMVALUE.amountAcknowledged

if ((BANCS.FRMVALUE.senderToReceiverInfo1  !=  "") or
(BANCS.FRMVALUE.senderToReceiverInfo2 !=
"")) or (BANCS.FRMVALUE.senderToReceiverInfo3  !=  "") or
(BANCS.FRMVALUE.senderToReceiverInfo4
!=  "")or (BANCS.FRMVALUE.senderToReceiverInfo5 !=  "") or
(BANCS.FRMVALUE.senderToReceiverInfo
6 !=  "")) then

sv_a      =      sv_a      +      "|"      +      ":72:"      +
BANCS.FRMVALUE.senderToReceiverInfo1
sv_a = sv_a + "|" + BANCS.FRMVALUE.senderToReceiverInfo2
sv_a = sv_a + "|" + BANCS.FRMVALUE.senderToReceiverInfo3
sv_a = sv_a + "|" + BANCS.FRMVALUE.senderToReceiverInfo4
sv_a = sv_a + "|" + BANCS.FRMVALUE.senderToReceiverInfo5
sv_a = sv_a + "|" + BANCS.FRMVALUE.senderToReceiverInfo6
else

```

```
sv_a = sv_a + "|||||"  
endif  
BANCS.OUTPUT.swiftMessage = sv_a  
exitscript  
end-->
```

©Copyright Infosys Ltd.

About Infosys Finacle

Finacle is the industry-leading digital banking solution suite from EdgeVerve Systems, a wholly owned product subsidiary of Infosys. Finacle helps traditional and emerging financial institutions drive truly digital transformation to achieve frictionless customer experiences, larger ecosystem play, insights-driven interactions and ubiquitous automation. Today, banks in over 100 countries rely on Finacle to service more than a billion consumers and 1.3 billion accounts.

Finacle solutions address the core banking, omnichannel banking, payments, treasury, origination, liquidity management, Islamic banking, wealth management, analytics, artificial intelligence, and blockchain requirements of financial institutions to drive business excellence. An assessment of the top 1250 banks in the world reveals that institutions powered by the Finacle Core Banking solution, on average, enjoy 7.2% points lower costs-to-income ratio than others.



For more information, contact finacle@edgeverve.com

www.finacle.com

©2019 EdgeVerve Systems Limited, a wholly owned subsidiary of Infosys, Bangalore, India. All Rights Reserved. This documentation is the sole property of EdgeVerve Systems Limited ("EdgeVerve"). EdgeVerve believes the information in this document or page is accurate as of its publication date; such information is subject to change without notice. EdgeVerve acknowledges the proprietary rights of other companies to the trademarks, product names and such other intellectual property rights mentioned in this document. This document is not for general distribution and is meant for use solely by the person or entity that it has been specifically issued to and can be used for the sole purpose it is intended to be used for as communicated by EdgeVerve in writing. Except as expressly permitted by EdgeVerve in writing, neither this documentation nor any part of it may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, printing, photocopying, recording or otherwise, without the prior written permission of EdgeVerve and/ or any named intellectual property rights holders under this document.